Advanced Machine Learning

Comprehensive Study Guide & Theoretical Companion

Master's Degree in Computer Engineering — Politecnico di Torino
Course Codes: 01URWOV / 01URWYG (6 CFU)

Course Instructor: Prof.ssa Tatiana Tommasi
DAUIN, VANDAL Lab, ELLIS Unit Turin

Study Guide Engineering: Antigravity AML Skill Team

Academic Year 2026/2027

Contents

1 Foundations and learning paradigms **1**
- 1.1 Course architecture and evaluation . . . 1
- 1.2 Artificial intelligence paradigms . . . 2
- 1.3 Defining machine learning . . . 8
- 1.4 Learning paradigms . . . 9
- 1.5 Data uncertainty . . . 12
- 1.6 Methodological data pillars . . . 13
- 1.7 Algorithmic bias and generalization limits . . . 20

2 Probability and decision theory **25**
- 2.1 Probability foundations . . . 25
- 2.2 Multivariable distributions and inference . . . 30
- 2.3 Bayes' rule and independence . . . 37
- 2.4 Naïve Bayes and estimation . . . 41
- 2.5 Statistical decision theory . . . 49

3 Linear predictors and gradient optimization **65**
- 3.1 Linear regression and capacity . . . 65
- 3.2 Regularization for linear models . . . 73
- 3.3 Logistic regression . . . 77
- 3.4 Loss functions and geometry . . . 82
- 3.5 Gradient descent . . . 88
- 3.6 The learning rate . . . 96
- 3.7 Accelerated gradient methods . . . 98
- 3.8 Critical points and generalization . . . 104
- 3.9 Stochastic gradient descent . . . 107

4 Deep feedforward networks and backpropagation **111**
- 4.1 The perceptron . . . 112
- 4.2 Multilayer perceptrons and approximation . . . 118
- 4.3 Computational graphs and backpropagation . . . 124

5 Convolutional neural networks **133**
- 5.1 Failure of dense networks on vision . . . 133
- 5.2 Convolution mechanics and geometry . . . 138
- 5.3 CNN priors and structural design . . . 143
- 5.4 Architectural design principles . . . 149
- 5.5 Computer vision tasks . . . 155

6 Deep network training and regularization **159**
6.1 Activation dynamics and pathologies . . . 159
6.2 Input preprocessing and conditioning . . . 172
6.3 Weight initialization . . . 177
6.4 Batch Normalization . . . 181
6.5 Deep regularization . . . 185
6.6 Training diagnostics and learning rates . . . 194
6.7 Hyperparameter optimization . . . 199
6.8 Model ensembling and weight averaging . . . 201

7 Transfer learning and adaptation **205**
7.1 Transfer learning principles . . . 205
7.2 Linear probing versus fine-tuning . . . 208
7.3 Rethinking pre-training . . . 211
7.4 Distribution shifts and the LP-FT protocol . . . 212
7.5 Knowledge distillation . . . 214
7.6 Transfer, multitask, and meta-learning . . . 218

8 Landmark CNN architectures **219**
8.1 The ImageNet challenge . . . 219
8.2 AlexNet . . . 221
8.3 ZFNet . . . 224
8.4 VGGNet . . . 225
8.5 GoogLeNet . . . 227
8.6 ResNet . . . 230
8.7 Complexity and design guidelines . . . 234

Chapter 1

Foundations and learning paradigms

1.1 Course architecture and evaluation

The graduate curriculum in *Advanced Machine Learning* at Politecnico di Torino (Course Codes: `01URWOV` / `01URWYG`, 6 CFU) is structured to transition students from passive consumers of machine learning APIs into rigorous theoretical researchers and deep learning engineers. The curriculum operates under the academic direction of the DAUIN, VANDAL Lab, and ELLIS Unit Turin teams.

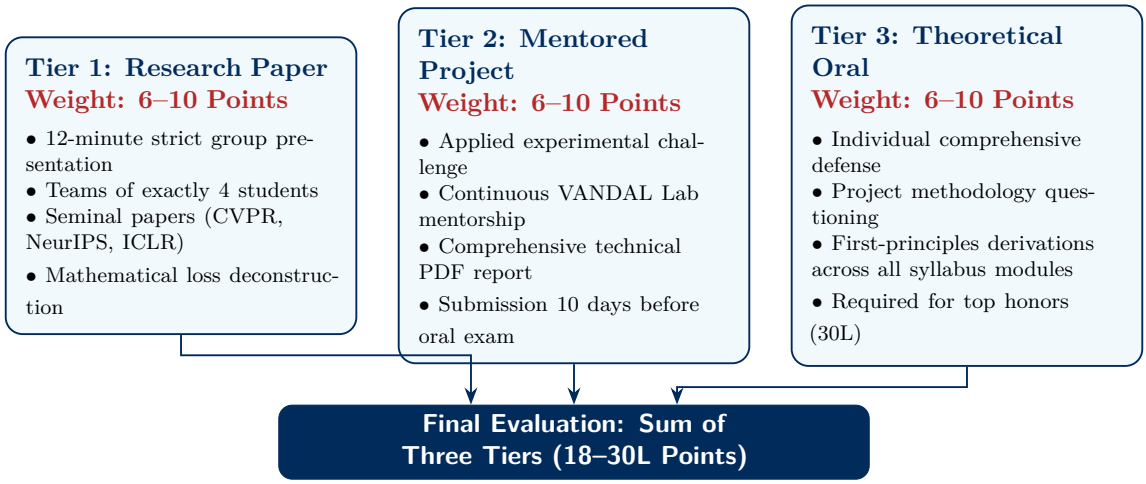


Figure 1.1: The 3-tier examination and evaluation architecture of the Advanced Machine Learning course at Politecnico di Torino. The final grade combines three complementary assessments: a 12-minute group research paper presentation (6–10 points), a mentored experimental lab project (6–10 points), and a rigorous theoretical oral examination (6–10 points).

1.1.1 Evaluation tiers

As formalized in Figure 1.1, successful completion of the course requires mastering three complementary evaluation tiers:

1. **Seminal Research Paper Presentation (6–10 Points)**:
    - Teams of exactly 4 students deliver a strictly timed 12-minute presentation breaking down a seminal research paper published within the last 3–7 years in top computer vision and machine learning venues (CVPR, ICCV, ECCV, NeurIPS, ICLR, ICML, TPAMI).

- Evaluation criteria focus on problem formulation, architectural mechanisms, mathematical loss deconstructions, ablation analysis, and unstated assumptions.

2. **Mentored Group Project (6–10 Points)**:

   - Student teams tackle applied research challenges under the continuous mentorship of VANDAL Lab teaching assistants.
   - Teams submit a comprehensive technical experimental report in PDF format strictly 10 days prior to the examination date.

3. **Theoretical Oral Examination (6–10 Points)**:

   - A comprehensive individual defense covering both the experimental project and **mandatory theoretical questioning spanning all course syllabus modules**.
   - To achieve top honors (30L), students must articulate first-principles derivations, declare explicit tensor dimensions, state foundational statistical assumptions, and navigate loss landscape geometries. Hand-waving or purely qualitative answers are insufficient.

—

1.2 Artificial intelligence paradigms

To understand modern machine learning, one must first deconstruct the historical evolution and foundational definitions of artificial intelligence. In their seminal treatise, Russell and Norvig [81] observe that artificial intelligence has historically been defined along two primary orthogonal axes:

1. **Thinking vs. Acting**: Does intelligence concern internal cognitive and inferential processes (how an agent reasons, plans, and structures beliefs), or does it concern external, observable behavior and agency (how an agent acts in and interacts with its environment)?

2. **Human-Centric vs. Rationality-Centric**: Is the benchmark of success measured by fidelity to human performance (including human biological idiosyncrasies, memory limitations, and psychological cognitive biases), or is it measured by adherence to an idealized, normative standard of rationality (doing the mathematically "right thing")?

These two orthogonal dimensions define a 2×2 taxonomy spanning four distinct historical paradigms of artificial intelligence, formalized structurally in Figure 1.2.

A rigorous inspection of each of the four quadrants reveals their distinct foundational assumptions, methodologies, and limitations:

1. **Acting Humanly: The Turing Test Approach (1950)**. In his landmark paper *Computing Machinery and Intelligence*, Alan Turing [93] proposed an operational definition of machine intelligence designed to sidestep the metaphysical vagueness of the question "Can machines think?". Under the *Turing Test* (or Imitation Game), a computer communicates via text terminal with a human interrogator. The machine passes if the interrogator cannot reliably distinguish its responses from those of a human subject.

   To pass a rigorously administered Turing Test, an artificial system cannot rely on simple keyword matching (such as Weizenbaum's 1966 ELIZA program). It must possess four primary computational capabilities:

   - **Natural Language Processing (NLP)**: To communicate fluently, interpret idiom, and navigate syntactic ambiguity in human discourse.

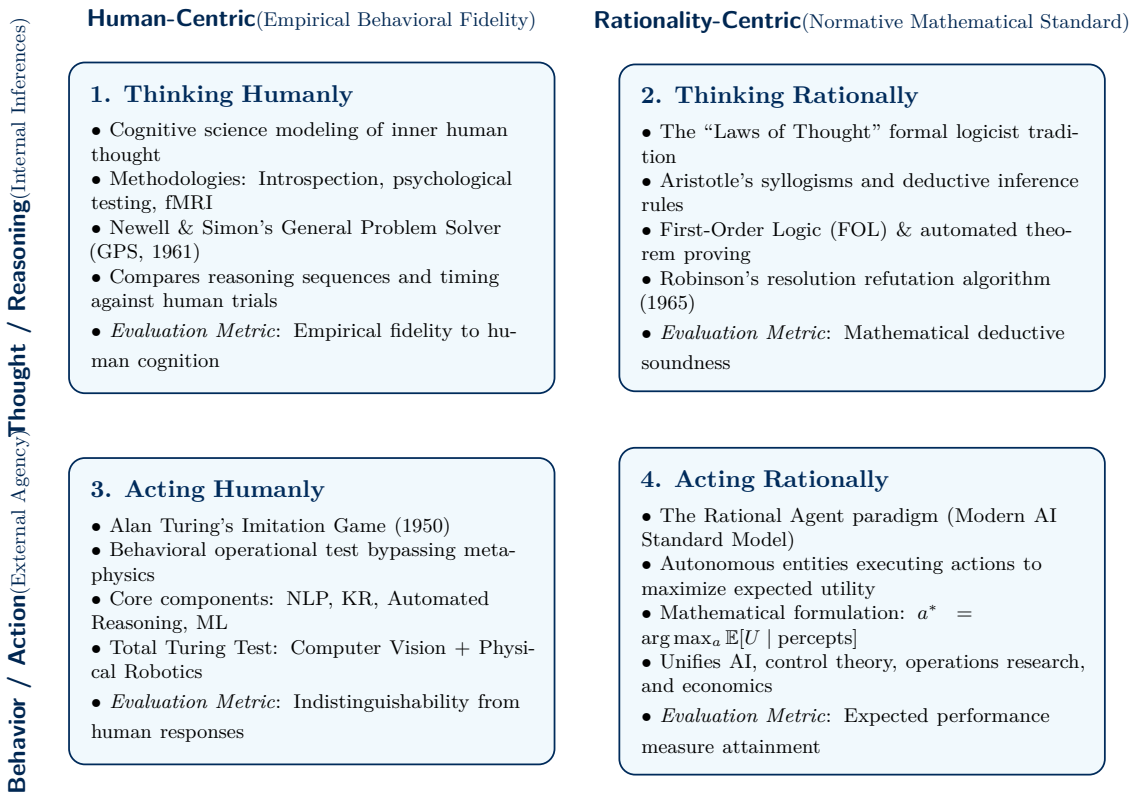


Figure 1.2: The 2×2 taxonomy of artificial intelligence paradigms (adapted from Russell & Norvig [81]). The horizontal dimension distinguishes empirical human fidelity from normative mathematical rationality; the vertical dimension distinguishes internal inferential processes from external behavioral agency.

- **Knowledge Representation**: To store, organize, and retrieve what it knows or hears about physical and social worlds.
- **Automated Reasoning**: To use stored knowledge to answer queries, draw deductive conclusions, and identify logical fallacies.
- **Machine Learning**: To adapt to novel circumstances, extrapolate inductive patterns from experience, and correct internal errors.

Turing deliberately excluded physical interaction to isolate intellectual capacity from biological simulation. However, later researchers formulated the **Total Turing Test**, which requires the agent to interact directly with physical objects and individuals in the physical environment. Passing the Total Turing Test requires two additional sensory-motor disciplines:

- **Computer Vision and Speech Processing**: To perceive physical objects, recognize gestures, and parse acoustic wave-trains.
- **Robotics**: To manipulate tools, manipulate physical objects, and navigate complex 3D physical terrain.

Together, these six disciplines constitute the foundational scope of modern artificial intelligence.

Despite its historical brilliance, modern AI researchers have largely abandoned the Turing Test as a primary research goal. Russell and Norvig draw an insightful analogy to **aeronautical engineering**: aeronautical texts do not define the goal of their discipline as building "machines that fly so exactly like pigeons that they can fool even other pigeons." The field of aviation succeeded only when engineers ceased attempting to flap feathers and instead developed wind tunnels to understand the fundamental physics of **aerodynamics**. Similarly, artificial intelligence focuses on discovering the underlying mathematical principles of rational agency, rather than copying human cognitive flaws, reaction delays, or biological deceptions.

2. **Thinking Humanly: The Cognitive Modeling Approach**. If the goal is to design a program that thinks like a human, we must understand the precise causal mechanics of human cognition. Historically, cognitive scientists infer the workings of the human mind through three complementary empirical avenues:

   - **Introspection**: Observing and tracking one's own internal train of thoughts as they unfold during problem solving.
   - **Psychological Experimentation**: Observing the behavior, error patterns, and reaction latencies of human subjects performing structured cognitive tasks.
   - **Brain Imaging / Neurophysiology**: Monitoring neural activation directly in real time via functional Magnetic Resonance Imaging (fMRI), electroencephalography (EEG), and multi-electrode arrays.

   Once an empirical theory of human thought is articulated, it can be formalized as an algorithmic computer program. A celebrated early milestone was Allen Newell and Herbert Simon's **General Problem Solver (GPS)**, developed in 1961 [69]. Newell and Simon were not content merely to demonstrate that GPS could solve a given logic puzzle or cryptarithmetic task; they meticulously compared the chronological sequence of intermediate sub-goals, trace steps, and hesitation times generated by GPS against the verbal think-aloud protocols of human subjects solving identical puzzles.

   This line of inquiry crystallized into the interdisciplinary field of **Cognitive Science**, uniting computational AI models with experimental psychology and neuroscience. While cognitive science provides vital biological inspirations (most famously, Hubel and Wiesel's

discovery of orientation-selective simple and complex cells in the mammalian visual cortex directly inspiring the neocognitron and modern Convolutional Neural Networks), it remains an empirical descriptive science of biological creatures. Engineering AI, conversely, seeks normative computational mechanisms that perform optimally regardless of biological mimicry.

3. **Thinking Rationally: The "Laws of Thought" Approach**. The philosopher Aristotle was among the first to attempt to codify "right thinking"—that is, irrefutable, truth-preserving reasoning processes. His classical **syllogisms** provided argument templates that invariably yield correct conclusions whenever the premises are true. For example:

$$\begin{aligned} &\text{Premise 1: All humans are mortal.} \\ &\text{Premise 2: Socrates is a human.} \\ &\therefore \text{Conclusion: Socrates is mortal.} \end{aligned} \tag{1.1}$$

   These "laws of thought" were presumed to govern the ideal operation of the rational mind, launching the formal discipline of **Logic**. In the 19th and early 20th centuries, logicians including George Boole, Gottlob Frege, and Giuseppe Peano developed symbolic notations to express assertions about objects and relations. Whitehead and Russell attempted to ground all mathematics in symbolic logic in their *Principia Mathematica*.

   By 1965, the invention of the **resolution refutation principle** by Alan Robinson [77] enabled computer programs to systematically prove any valid theorem expressible in first-order logic. This ignited the **logicist tradition** in artificial intelligence: the ambition that intelligent systems could be constructed by encoding expert human knowledge into formal logic sentences and running deductive inference engines.

4. **Acting Rationally: The Rational Agent Approach**. An **agent** is an entity that acts (from the Latin *agere*, to do). While all computer programs execute code, an intelligent agent is characterized by autonomous operation: perceiving its surrounding environment through sensors, persisting across extended time horizons, adapting dynamically to environmental variations, and formulating and pursuing goals.

   A **rational agent** is formally defined as one that acts so as to achieve the best outcome or, when operating under uncertainty, the **best expected outcome**.

   In the "laws of thought" approach, the central focus was on computing correct deductive inferences. Making correct inferences is often an integral part of being a rational agent: an agent may deduce that action a leads to catastrophic failure while action b satisfies its objectives, and choose b accordingly. However, rational action is strictly broader than rational inference:

   - There are situations where correct logical inference is completely absent, yet the action taken is profoundly rational. For example, recoiling one's hand instantaneously from a red-hot stove burner is a purely defensive **reflex action**. If the agent were forced to pause, formulate premises in first-order logic, and run a deductive resolution theorem prover to deduce that burning flesh reduces utility, third-degree burns would occur long before deduction terminated.
   - Rationality concerns the **consequences** of actions, not the purity of internal deductions.

   The rational agent paradigm provides two profound advantages over the other three approaches:

   (a) **Generality**: It encompasses correct logical deduction as one possible mechanism among many, while seamlessly accommodating heuristic search, reflex loops, and statistical learning.

(b) **Mathematical Amenability**: Rationality is grounded in a mathematically rigorous standard of expected performance. We can specify a objective utility or loss metric and mathematically optimize agent designs to provably achieve it.

Consequently, the rational agent approach has become the **Standard Model** of modern artificial intelligence. It unifies AI with control engineering (where a controller minimizes an integral cost function), operations research (where a policy maximizes cumulative expected rewards in a Markov decision process), statistical decision theory (where a decision rule minimizes conditional risk), and microeconomics (where an actor maximizes expected utility).

1.2.1 Limitations of logicism

Why did the classical logicist tradition—despite its deductive elegance—fail as a universal architecture for artificial intelligence? When researchers in the 1970s and 1980s attempted to scale symbolic logic systems to real-world tasks such as medical diagnosis, visual recognition, and robotic navigation, the logicist program encountered five insurmountable structural bottlenecks:

1. **Epistemic Uncertainty and the Binary Truth Trap**: Classical deductive logic operates strictly over binary truth values: a proposition ϕ is either definitively true ($\top$) or false ($\bot$). In closed, deterministic synthetic worlds (such as chess or group theory), binary assertions hold without ambiguity. However, in real-world physical and perceptual environments, assertions are almost never 100% certain. Consider diagnosing a patient: the proposition "The patient has bacterial pneumonia" cannot be assigned a definitive truth value based on a single fever reading or cough. Perceptual data is corrupted by sensor thermal noise, ambient lighting variations, occlusions, and incomplete physiological indicators. Logic lacks any native mechanism to represent degrees of belief, partial evidence, or graded confidence without resorting to probability theory.

2. **The Qualification Problem (McCarthy, 1969)**: To write down valid first-order logic rules for actions in the physical world, an agent must state all the preconditions required for the action to succeed. Consider the rule for starting an automobile:

$$\forall x \Big(\text{TurnKey}(x) \wedge \text{HasBattery}(x) \wedge \text{HasFuel}(x) \implies \text{StartsEngine}(x) \Big). \tag{1.2}$$

   In reality, this rule is false: the engine will fail to start if the starter motor solenoid is burned out, if the fuel line is frozen, if the electrical wiring is severed by rodents, if there is a potato lodged in the exhaust tailpipe, or if a localized electromagnetic pulse just discharged overhead. Attempting to enumerate all possible caveats, exceptions, and background conditions requires an infinite or computationally intractable premise expansion. The qualification problem demonstrates that deterministic logical rules cannot capture the messiness of open environments.

3. **Computational Complexity and Combinatorial Explosion**: Even if one could formalize real-world knowledge into symbolic logic, automated inference is computationally intractable. By the Cook-Levin Theorem, propositional satisfiability (SAT) is NP-complete. For First-Order Logic, Alan Turing and Alonzo Church proved in 1936 that the decision problem is **semi-decidable**: if a sentence is a logical consequence of a knowledge base, an automated theorem prover (such as resolution refutation) is guaranteed to eventually find a proof; but if the sentence is **not** entailed, the search algorithm may run indefinitely without ever halting. In practical knowledge bases containing tens of thousands of assertions, the branching factor of resolution inference explodes combinatorially, exhausting all available RAM and CPU cycles within seconds.

4. **The Frame Problem (McCarthy & Hayes, 1969)**: In a dynamic world, executing an action changes certain properties of the environment while leaving the overwhelming majority of properties completely unchanged. If an agent moves a coffee cup from a desk to a shelf, the cup's coordinates change; but the color of the walls, the temperature in Tokyo, the president of the university, and the laws of gravity remain strictly identical. In classical logic, an inference engine cannot assume that something remains unchanged unless it is explicitly told so. Specifying explicit "frame axioms" for every action and every invariant property requires $\mathcal{O}(|\mathcal{A}| \cdot |\mathcal{P}|)$ axioms, creating massive computational overhead that paralyzes symbolic reasoning engines.

5. **The Thought-vs-Action Gap and Real-Time Deadlines**: Pure logic is concerned with **truth**, whereas an agent situated in an environment must produce **timely actions**. In high-stakes real-time systems (such as autonomous driving, high-frequency trading, or robotic collision avoidance), an agent does not have the luxury of waiting minutes or hours for an exhaustive deductive proof to terminate. A bounded rational agent must act within an execution deadline τ. An algorithm that computes the mathematically perfect deduction two seconds after the vehicle has collided with a concrete barrier has an effective performance of zero.

These fundamental limitations forced artificial intelligence to undergo a paradigm shift: from brittle symbolic deduction over handcrafted rules to **probabilistic reasoning** and **data-driven statistical machine learning**.

1.2.2 Rationality and bounded optimality

To resolve the limitations of logicism, modern AI redefines intelligence as **computational rationality**:

Definition 1.1 (Rational agent)**.** An agent is an autonomous entity that perceives its environment through sensors, processes percept sequences via an internal program, and executes actions $a \in \mathcal{A}$ via actuators to maximize its expected utility:

$$a^* = \arg\max_{a \in \mathcal{A}} \mathbb{E}_{s' \sim P(\cdot|s,a)} \left[U(s') \right], \tag{1.3}$$

where $\mathcal{A}$ denotes the action space, $s \in \mathcal{S}$ is the current environment state, $P(s' \mid s, a)$ is the transition dynamics probability distribution, and $U : \mathcal{S} \to \mathbb{R}$ is a scalar utility function encoding objective performance.

However, evaluating the exact optimal action in Equation (1.3) is physically impossible in complex worlds: evaluating all future branching state-action trajectories in continuous or high-dimensional spaces is computationally intractable. Real computational agents are strictly bounded by finite execution time τ, limited memory capacity $\mathcal{M}$, and finite energy budgets.

We therefore formalize **bounded optimality** [81]. Let $\mathcal{P}(M)$ denote the set of all programs that can execute on a physical hardware architecture M within resource constraints. An agent program $p^* \in \mathcal{P}(M)$ is bounded optimal if it maximizes expected utility averaged over operational environments:

$$p^* = \arg\max_{p \in \mathcal{P}(M)} \mathbb{E}_{e \sim \mathcal{E}} \left[U(\text{Run}(p, M, e)) \right], \tag{1.4}$$

where $\mathcal{E}$ represents the probability distribution over operational task environments. Bounded rationality provides the primary mathematical and engineering motivation for **machine learning**: rather than attempting intractable symbolic deductions from first principles, agents learn compact, parameterized inductive models that produce near-optimal heuristic decisions within available compute budgets.

Consequently, bounded agents operating in high-dimensional perceptual environments must abandon brittle symbolic logic in favor of empirical learning, balancing inductive efficiency against the hazard of shortcut learning and spurious correlations.

—

1.3 Defining machine learning

Rather than hand-crafting explicit rule-based systems ($C = A + B$), machine learning inverts the traditional computer science programming paradigm:

- **Classical Software Engineering**: Data + Rules (Code) $\longrightarrow$ Outputs (Answers).
- **Machine Learning Engineering**: Data+Outputs (Answers) $\longrightarrow$ Rules (Hypothesis Model $h \in \mathcal{H}$).

1.3.1 Mitchell's formulation

The canonical mathematical definition of learning is formulated by Tom Mitchell:

Definition 1.2 (Machine learning [6, 32])**.** A computer program is said to learn from experience $\mathcal{E}$ with respect to some class of tasks $\mathcal{T}$ and performance measure $\mathcal{P}$, if its performance at tasks in $\mathcal{T}$, as measured by $\mathcal{P}$, improves with experience $\mathcal{E}$.

We formalize each component of this fundamental triad:

1. **The Task $\mathcal{T}$**: A functional mapping $h : \mathcal{X} \to \mathcal{Y}$ where $\mathcal{X} \subseteq \mathbb{R}^d$ is the feature space and $\mathcal{Y}$ is the output space. Common tasks include:
   - *Classification*: $\mathcal{Y} = \{1, \dots, K\}$ (discrete categorical prediction).
   - *Regression*: $\mathcal{Y} = \mathbb{R}$ or $\mathbb{R}^m$ (continuous value prediction).
   - *Density Estimation*: $h(\mathbf{x}) = p(\mathbf{x})$ (unsupervised probability modeling).
2. **The Experience $\mathcal{E}$**: A discrete dataset $\mathcal{D}_N = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ sampled identically and independently (i.i.d.) from an unknown, stationary data-generating distribution p_{data}:
$$(\mathbf{x}_i, y_i) \overset{\text{i.i.d.}}{\sim} p_{\text{data}}(\mathbf{x}, y) = p(\mathbf{x})P(y \mid \mathbf{x}). \tag{1.5}$$
3. **The Performance Measure $\mathcal{P}$**: Quantified via a scalar loss function $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_{\geq 0}$. The ultimate objective is the **true statistical risk** $\mathcal{R}(h)$:
$$\mathcal{R}(h) \triangleq \mathbb{E}_{(\mathbf{x},y)\sim p_{\text{data}}} \left[\ell(h(\mathbf{x}), y)\right] = \int_{\mathcal{X}\times\mathcal{Y}} \ell(h(\mathbf{x}), y)\, dp_{\text{data}}(\mathbf{x}, y). \tag{1.6}$$

Critical oral exam insight: Statistical risk vs. empirical risk. A fatal misconception in oral examinations is confusing training error with the true learning objective. Because the true distribution $p_{\text{data}}(\mathbf{x}, y)$ is strictly unknown, the statistical risk $\mathcal{R}(h)$ cannot be evaluated directly. Instead, the learning algorithm can only compute the **empirical risk** evaluated across the observed training set $\mathcal{D}_N$:

$$\hat{\mathcal{R}}_N(h) \triangleq \frac{1}{N} \sum_{i=1}^{N} \ell(h(\mathbf{x}_i), y_i). \tag{1.7}$$

By the Law of Large Numbers, $\hat{\mathcal{R}}_N(h) \to \mathcal{R}(h)$ as $N \to \infty$ for any fixed hypothesis h. However, because optimization actively selects $h \in \mathcal{H}$ to minimize $\hat{\mathcal{R}}_N(h)$, this minimum is statistically biased downwards:

$$\mathbb{E}\left[\min_{h\in\mathcal{H}} \hat{\mathcal{R}}_N(h)\right] \leq \min_{h\in\mathcal{H}} \mathcal{R}(h). \tag{1.8}$$

This gap constitutes the core challenge of **generalization and overfitting**.

—

1.4 Learning paradigms

The nature of the supervisory feedback embedded within experience $\mathcal{E}$ dictates the structure of the learning algorithm. Machine learning is organized into five primary paradigms:

1.4.1 Supervised learning

Motivation and Problem Setting: In supervised learning, the agent is provided with an explicit "teacher" that annotates each input vector $\mathbf{x} \in \mathcal{X} \subseteq \mathbb{R}^d$ with a verified ground-truth target $y \in \mathcal{Y}$. The training experience is a discrete sample of N input-output tuples:

$$\mathcal{D}_{\text{train}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N} \overset{\text{i.i.d.}}{\sim} p_{\text{data}}(\mathbf{x}, y). \tag{1.9}$$

The operational objective is to learn a parameterized hypothesis function $h(\mathbf{x}; \mathbf{w})$ that generalizes accurately to unseen test samples $\mathbf{x}^* \sim p_{\text{data}}(\mathbf{x})$.

Supervised learning bifurcates according to the topological nature of the target space $\mathcal{Y}$:

1. **Classification ($\mathcal{Y}$ is discrete)**: When targets represent categorical classes $\mathcal{Y} = \{1, 2, \ldots, K\}$ (or $\mathcal{Y} = \{-1, +1\}$ for binary classification), the task is to partition the feature space $\mathcal{X}$ into K decision regions separated by decision boundaries. The model outputs posterior class probabilities $\hat{p}_k = P(y = k \mid \mathbf{x})$, typically parameterized via a softmax output layer:

$$P(y = k \mid \mathbf{x}; \mathbf{w}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x})}{\sum_{j=1}^{K} \exp(\mathbf{w}_j^\top \mathbf{x})}. \tag{1.10}$$

   The canonical loss is the **Cross-Entropy (Negative Log-Likelihood)** loss:

$$\ell_{\text{CE}}(h(\mathbf{x}; \mathbf{w}), y) = -\sum_{k=1}^{K} \mathbb{I}(y = k) \log P(y = k \mid \mathbf{x}; \mathbf{w}). \tag{1.11}$$

2. **Regression ($\mathcal{Y}$ is continuous)**: When targets represent continuous scalar or vector values $\mathcal{Y} \subseteq \mathbb{R}^m$ (e.g., predicting temperature, financial asset prices, or continuous joint coordinates of a robotic arm), the goal is to approximate the conditional expectation surface:

$$h^*(\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}]. \tag{1.12}$$

   Under the assumption of additive Gaussian observation noise $y = f(\mathbf{x}) + \epsilon$ with $\epsilon \sim \mathcal{N}(0, \sigma^2)$, the maximum likelihood estimator corresponds to minimizing the **Mean Squared Error (MSE)** loss:

$$\ell_{\text{MSE}}(h(\mathbf{x}; \mathbf{w}), y) = \frac{1}{2} \|h(\mathbf{x}; \mathbf{w}) - y\|_2^2. \tag{1.13}$$

Core Bottleneck: The fundamental vulnerability of supervised learning is the **annotation bottleneck**. Acquiring millions of high-quality human-annotated labels (e.g., pixel-level semantic segmentation masks or expert radiologist diagnoses) is economically exorbitant, slow, and prone to inter-annotator disagreement and label noise.

1.4.2 Unsupervised learning

Motivation and Problem Setting: In biological learning, humans do not receive labeled input-output pairs for every visual scene or tactile sensation. The overwhelming majority of data generated across the world is completely unlabeled. Unsupervised learning operates on raw feature observations without any supervisory feedback:

$$\mathcal{D}_{\text{train}} = \{\mathbf{x}_i\}_{i=1}^{N} \overset{\text{i.i.d.}}{\sim} p_{\text{data}}(\mathbf{x}). \tag{1.14}$$

The mathematical objective is to discover the intrinsic structure, statistical regularities, and underlying probability density $p(\mathbf{x})$ that generated the observations.

Unsupervised learning is structured into three primary sub-paradigms:

1. **Clustering**: Partitioning the unlabelled samples $\{\mathbf{x}_i\}_{i=1}^N$ into K discrete, cohesive clusters such that intra-cluster distance is minimized while inter-cluster distance is maximized. Classic formulations include:

   - **Hard clustering (k-means)**: Minimizing the sum of squared Euclidean distances to cluster centroids $\boldsymbol{\mu}_k$:

   $$J_{\text{kmeans}} = \sum_{i=1}^{N} \sum_{k=1}^{K} r_{ik} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2, \quad r_{ik} \in \{0,1\}, \quad \sum_k r_{ik} = 1. \tag{1.15}$$

   - **Soft probabilistic clustering (Gaussian Mixture Models)**: Modeling $p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ and estimating parameters via the Expectation-Maximization (EM) algorithm.

2. **Dimensionality Reduction and Manifold Learning**: High-dimensional real-world data (such as 1024×1024 images residing in $\mathbb{R}^{10^6}$) typically lie on or near a smooth, low-dimensional non-linear sub-manifold $\mathcal{M} \subset \mathbb{R}^d$ of intrinsic dimension $d_{\text{int}} \ll d$ (the **Manifold Hypothesis**).

   - **Linear methods (Principal Component Analysis - PCA)**: Finds orthogonal projection directions $\mathbf{W} \in \mathbb{R}^{d \times k}$ ($k \ll d$) that maximize data variance or minimize reconstruction error $\|\mathbf{x} - \mathbf{W}\mathbf{W}^\top \mathbf{x}\|_2^2$.
   - **Non-linear autoencoders**: Neural networks containing an architectural information bottleneck (encoder $f_\theta : \mathbb{R}^d \to \mathbb{R}^k$ and decoder $g_\phi : \mathbb{R}^k \to \mathbb{R}^d$) trained to minimize reconstruction loss $\|\mathbf{x} - g_\phi(f_\theta(\mathbf{x}))\|_2^2$.
   - **Topological embeddings (t-SNE, UMAP)**: Preserve local neighborhood affinities in low-dimensional visualizable spaces ($\mathbb{R}^2$ or $\mathbb{R}^3$).

3. **Density Estimation and Generative Modeling**: Explicitly estimating the continuous probability density function $p(\mathbf{x})$ (via Normalizing Flows or Kernel Density Estimation) or learning an implicit stochastic sampler $G_\theta : \mathcal{Z} \to \mathcal{X}$ that generates synthetic data indistinguishable from real data (Variational Autoencoders - VAEs, Generative Adversarial Networks - GANs, and Denoising Diffusion Probabilistic Models - DDPMs).

1.4.3 Self-supervised learning

Motivation and Problem Setting: Self-supervised learning represents the modern bridge that unlocked foundation models (such as GPT and BERT in NLP, and DINO and CLIP in vision). Rather than requiring external human annotations, the learning algorithm constructs an internal **pretext task** derived entirely from the raw unlabeled data itself:

1. **Masked Modeling (Denoising Autoencoding)**: A portion of the input data is artificially corrupted, masked, or removed, and the model is tasked with reconstructing the missing components. In NLP, Masked Language Modeling (BERT) masks 15% of input tokens and minimizes cross-entropy on the masked tokens:

   $$\ell_{\text{MLM}}(\theta) = -\sum_{i \in \text{masked}} \log P(x_i \mid \mathbf{x}_{\setminus \text{masked}}; \theta). \tag{1.16}$$

   In computer vision, Masked Autoencoders (MAE) mask up to 75% of image patches, forcing the network to learn rich semantic representations of 3D geometry and semantics to predict missing pixels.

2. **Autoregressive Next-Token Prediction**: Given a sequence of tokens $(x_1, x_2, \ldots, x_{t-1})$, the model predicts the categorical distribution of the subsequent token x_t:

$$\ell_{\text{AR}}(\theta) = -\sum_{t=1}^{T} \log P(x_t \mid x_{<t}; \theta). \tag{1.17}$$

   This simple self-supervised objective powers modern Large Language Models (LLMs).

3. **Contrastive Representation Learning**: Given an anchor sample $\mathbf{x}$, stochastic data augmentations generate a positive sample $\mathbf{x}^+$ (e.g., random crop, color jitter) alongside unrelated negative samples $\{\mathbf{x}_j^-\}_{j=1}^K$. The model is trained using the **InfoNCE loss** to pull positive pairs close together in latent feature space while repelling negative pairs:

$$\ell_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(\mathbf{z}, \mathbf{z}^+)/\tau)}{\exp(\text{sim}(\mathbf{z}, \mathbf{z}^+)/\tau) + \sum_{j=1}^{K} \exp(\text{sim}(\mathbf{z}, \mathbf{z}_j^-)/\tau)}, \tag{1.18}$$

   where $\mathbf{z} = h(\mathbf{x})/\|h(\mathbf{x})\|_2$ and $\tau > 0$ is a temperature hyperparameter.

1.4.4 Reinforcement learning

Motivation and Problem Setting: In both supervised and unsupervised learning, the dataset is static and passive: the model's predictions have no causal effect on the data it observes next. In reinforcement learning (RL), the agent is situated inside a dynamic environment where its actions actively alter future states. Experience is acquired through sequential, closed-loop interaction.

Reinforcement learning is formally modeled as a **Markov Decision Process (MDP)**:

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle, \tag{1.19}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}(s' \mid s, a)$ is the transition probability, $\mathcal{R}(s, a, s')$ is the reward function, and $\gamma \in [0, 1)$ is the temporal discount factor.

At each discrete time step t, the agent observes state $s_t \in \mathcal{S}$, selects action $a_t \sim \pi(\cdot \mid s_t)$ according to its policy π, receives scalar feedback reward $r_{t+1} \in \mathbb{R}$, and transitions to next state $s_{t+1} \sim \mathcal{P}(\cdot \mid s_t, a_t)$. The objective is to learn an optimal policy π^* that maximizes the expected cumulative discounted return:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \right]. \tag{1.20}$$

Reinforcement learning introduces three unique foundational challenges absent from supervised learning:

1. **The Credit Assignment Problem**: Rewards in RL are often delayed and sparse. If an agent plays a 50-move game of chess and receives a single reward of +1 (win) at the final move, which specific move among the 50 caused the victory? Temporal credit assignment requires propagating reward signals backwards across extended time horizons using the **Bellman equation**:

$$Q^\pi(s, a) = \mathcal{R}(s, a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}(s' \mid s, a) \sum_{a' \in \mathcal{A}} \pi(a' \mid s') Q^\pi(s', a'). \tag{1.21}$$

2. **The Exploration vs. Exploitation Dilemma**: The agent only observes rewards for actions it actively chooses to execute. It faces a perpetual dilemma: should it **exploit** actions that currently yield known positive rewards, or should it **explore** unknown state-action trajectories that might yield vastly superior returns (at the risk of immediate failure)?

3. **Non-i.i.d. Data Stream**: The agent's own behavioral policy π determines the distribution of collected transitions (s, a, r, s'). As the policy improves, the data distribution shifts dynamically, violating the independent and identically distributed (i.i.d.) assumption that underpins standard empirical risk minimization.

1.4.5 Semi-supervised and active learning

In practical machine learning systems, two intermediate hybrid paradigms resolve resource constraints:

- **Semi-Supervised Learning**: Operates on a small labeled dataset $\mathcal{D}_L = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N_L}$ alongside a massive unlabeled dataset $\mathcal{D}_U = \{\mathbf{x}_j\}_{j=1}^{N_U}$ (where $N_U \gg N_L$). By enforcing the **smoothness assumption** (if two inputs $\mathbf{x}_1, \mathbf{x}_2$ are close in high-density regions of feature space, their labels must be identical) and the **cluster assumption** (decision boundaries should pass through low-density regions, not cross dense clusters), semi-supervised methods achieve high classification accuracy with minimal human labeling.

- **Active Learning**: Instead of passively accepting a fixed labeled set, the algorithm actively queries a human expert (an oracle) to label only the most informative unlabeled samples $\mathbf{x}^* \in \mathcal{D}_U$—typically those where model prediction entropy $-\sum \hat{p}_k \log \hat{p}_k$ is maximized (uncertainty sampling) or where labeling the sample maximizes expected model gradient change.

—

1.5 Data uncertainty

In statistical machine learning, every model is only as sound as the assumptions it makes regarding data uncertainty. Real-world observations $\mathbf{x}$ and targets y are never pristine mathematical constants; they are contaminated by stochastic uncertainty. A rigorous machine learning engineer must decompose uncertainty into its two fundamental constituents:

1.5.1 Aleatoric uncertainty

Aleatoric uncertainty (from the Latin *alea*, meaning dice) refers to the inherent, irreducible randomness present in the data-generating process itself. It captures stochastic variations that cannot be eliminated regardless of how many training examples are collected.

Sources of aleatoric uncertainty include:

- Thermal noise and photon shot noise in optical camera sensors.
- Quantum measurement noise in scientific instruments.
- Unobserved confounding variables (omitted features): for instance, predicting patient blood pressure based on age and weight, while omitting unobserved genetic factors, stress levels, and salt intake.
- Overlapping class-conditional distributions in feature space (e.g., two distinct tumor types exhibiting identical morphological appearance under a microscope).

Aleatoric uncertainty is further subdivided into two mathematical forms:

1. **Homoscedastic Uncertainty**: The noise variance is constant across the entire input feature space $\mathcal{X}$:

$$y = f(\mathbf{x}) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2), \quad \sigma^2 \text{ is constant } \forall \mathbf{x} \in \mathcal{X}. \tag{1.22}$$

2. **Heteroscedastic Uncertainty**: The noise variance is an explicit function of the input location $\mathbf{x}$:
$$y = f(\mathbf{x}) + \epsilon(\mathbf{x}), \quad \epsilon(\mathbf{x}) \sim \mathcal{N}(0, \sigma^2(\mathbf{x})). \tag{1.23}$$
For example, in autonomous driving, depth estimation from stereo cameras exhibits low variance for nearby objects ($\sigma^2(\mathbf{x})$ is small), but variance explodes quadratically as distance increases ($\sigma^2(\mathbf{x}) \propto d^2$).

Aleatoric uncertainty establishes the fundamental theoretical ceiling on classification performance: the **irreducible Bayes error rate** $\mathcal{R}^*$.

1.5.2 Epistemic uncertainty

Epistemic uncertainty (from the Greek *episteme*, meaning knowledge) refers to the uncertainty arising from a **lack of knowledge** or data in specific regions of the feature space.

Sources of epistemic uncertainty include:

- Sparse training samples in low-density regions of $p(\mathbf{x})$.
- Model parameter uncertainty resulting from fitting a high-capacity model to a finite dataset $\mathcal{D}_N$.
- Architectural model misspecification (inductive bias mismatch).

Crucially, **epistemic uncertainty is reducible**: as the sample size $N \to \infty$ with support across the entire input domain $\mathcal{X}$, parameter uncertainty converges to zero by the asymptotic consistency of estimators:
$$\lim_{N\to\infty} \mathrm{Var}_{\mathcal{D}_N}(\hat{\mathbf{w}}_N) = \mathbf{0}. \tag{1.24}$$
In safety-critical deep learning (such as medical diagnosis and autonomous robotics), quantifying epistemic uncertainty is vital: when an autonomous vehicle encounters an unfamiliar obstacle outside its training distribution, its epistemic uncertainty spikes, signaling the system to disengage autonomous mode and transfer control to a human driver. Epistemic uncertainty is mathematically quantified using **Bayesian Neural Networks**, Gaussian Processes, deep ensembles, or Monte Carlo Dropout.

—

1.6 Methodological data pillars

In modern statistical machine learning, data is not merely a static numerical array to be fed uncritically into black-box algorithms; it is the physical empirical substrate from which all intelligence is distilled. As established in the introductory lecture, becoming a genuine machine learner requires mastering an explicit, disciplined seven-step protocol for interacting with data before, during, and after model training:

1. **Observe the data**: Perform exhaustive exploratory visualization across native and projected feature spaces.
2. **Summarize their uncertainty**: Account for aleatoric and epistemic variations via probability distributions rather than scalar point summaries.
3. **Search for constraints and invariances**: Exploit domain prior knowledge to drastically prune the hypothesis search space.
4. **Find a model to explain the data**: Select hypothesis families that match the physical underlying phenomenon and balance capacity.
5. **Represent the data**: Construct, lift, or compress feature spaces to expose class separability.

6. **Learn representation and model jointly**: Exploit modern high-performance GPU computing to train end-to-end deep feature hierarchies optimized for the target task.
7. **Avoid data bias**: Guard against spurious correlations, historical demographic skew, and out-of-distribution domain shift.

1.6.1 The Datasaurus fallacy

The fundamental catalyst driving this seven-pillar protocol is the mathematical failure of low-order summary statistics. A pervasive trap among novice practitioners is summarizing datasets exclusively by their sample means, sample variances, and Pearson correlation coefficients, assuming these scalars adequately characterize the underlying phenomenon.

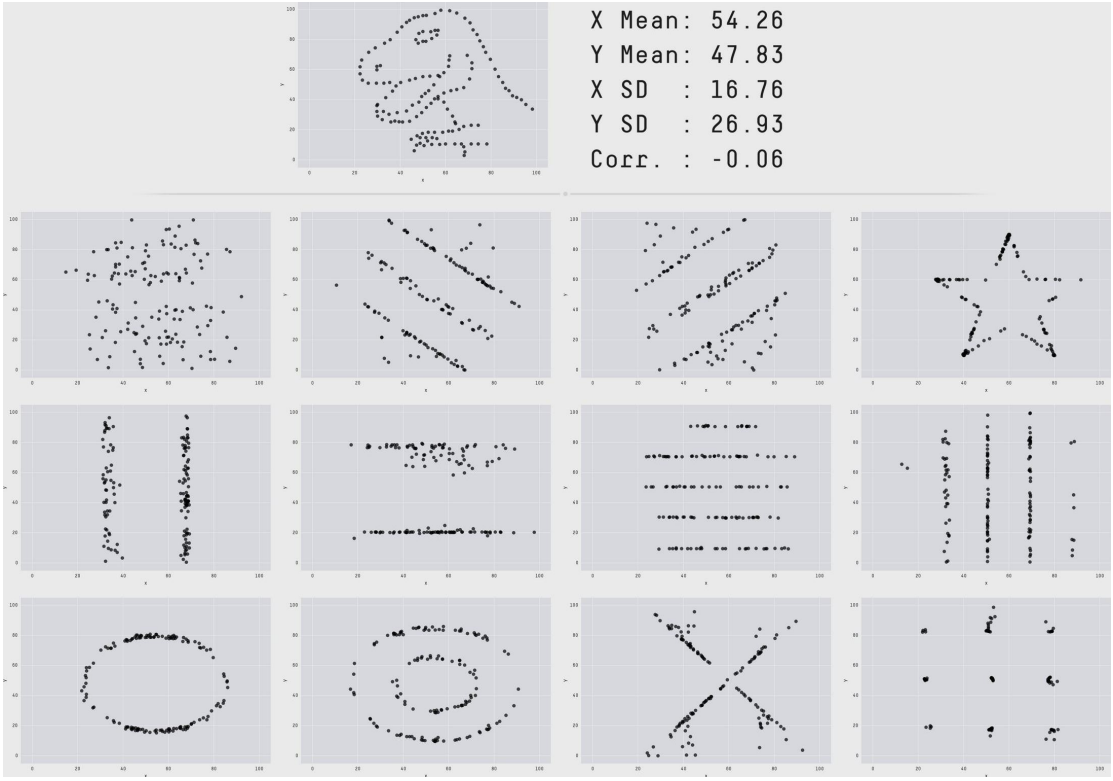


Figure 1.3: The Datasaurus Dozen (Matejka & Fitzmaurice, 2017) [63]. All 13 distinct bivariate distributions share identical summary statistics up to two decimal places: horizontal mean $\bar{x} = 54.26$, vertical mean $\bar{y} = 47.83$, standard deviations $s_x = 16.76, s_y = 26.93$, and Pearson correlation coefficient $r = -0.06$. Yet their true empirical geometries range from a Tyrannosaurus Rex to concentric rings, starbursts, and parallel lines.

Figure 1.3 illustrates the **Datasaurus Dozen** experiment [63], an eloquent modern generalization of Anscombe's 1973 quartet. Each of the 13 bivariate datasets consists of 142 points and exhibits identical numerical properties to two decimal places:

$$\bar{x} = 54.26, \quad \bar{y} = 47.83, \quad s_x = 16.76, \quad s_y = 26.93, \quad r_{xy} = -0.06. \tag{1.25}$$

An algorithm or practitioner relying solely on these metrics would conclude that all 13 datasets represent identical physical mechanisms. Yet inspecting the geometric plots reveals entirely distinct realities: one is a dinosaur, another is a high-density cluster surrounded by an outer ring, and another is a sequence of discrete vertical stripes.

To understand mathematically why summary statistics fail so catastrophically, consider the

analytical formula for the **sample Pearson correlation coefficient** r_{xy}:

$$r_{xy} = \frac{\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{N}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{N}(y_i - \bar{y})^2}}. \tag{1.26}$$

Equation (1.26) measures exclusively **first-order linear co-variation**. When the underlying structural relationship is non-linear (such as a circle $x^2 + y^2 = R^2$, a parabola $y = x^2$, or a complex contour), deviations above and below the mean cancel out symmetrically in the numerator $\sum(x_i - \bar{x})(y_i - \bar{y}) \approx 0$.

Furthermore, summary statistics completely discard:

- High-order statistical moments: skewness ($\mathbb{E}[(X - \mu)^3]/\sigma^3$) and kurtosis ($\mathbb{E}[(X - \mu)^4]/\sigma^4$).
- Multi-modal cluster topology and manifold connectivity.
- Heavy-tailed extreme values and catastrophic outlier distributions.

Hence, the golden imperative of machine learning: **Never trust summary statistics alone; always visualize and deeply inspect the data before modeling.**

1.6.2 Pillar 1: Data observation

The first rule of machine learning is to open the dataset directory and directly examine the individual instances. In tabular domains or low-dimensional problems ($d \leq 3$), scatter plots, histograms, and pair-wise correlation grids allow immediate visual diagnosis.

However, modern deep learning typically operates on extremely high-dimensional spaces: an uncompressed 256×256 RGB image lives in $\mathbb{R}^{196,608}$, and feature embeddings extracted from foundation models live in $\mathbb{R}^{1,024}$ or $\mathbb{R}^{4,096}$. Humans cannot directly visualize geometries beyond three dimensions. To observe such data, practitioners deploy mathematical projection and dimensionality reduction techniques:

- **Principal Component Analysis (PCA)**: Identifies the orthogonal directions of maximal variance by computing the eigenvectors of the empirical covariance matrix $\boldsymbol{\Sigma} = \frac{1}{N}\mathbf{X}^\top\mathbf{X}$, projecting d-dimensional representations onto the leading 2D or 3D principal sub-space.
- **Non-linear Manifold Learning**: Methods such as t-Distributed Stochastic Neighbor Embedding (t-SNE) and Uniform Manifold Approximation and Projection (UMAP) preserve local neighborhood topologies, unrolling complex non-linear manifolds into interpretable 2D diagnostic visual maps.

Critical oral exam insight: Knowing your dataset. In the oral examination, a fundamental line of questioning focuses on dataset mastery: *Which benchmark did you use in your project or paper analysis? What semantic object categories does it contain? How many total instances are present, and what is the exact sample distribution across classes?* A candidate who views data as an interchangeable numeric matrix without knowing its class taxonomy, sensor resolution, or subgroup composition demonstrates a shallow understanding. A true machine learner understands the physical reality generating each feature coordinate.

1.6.3 Pillar 2: Uncertainty modeling

Rather than condensing a feature coordinate into a single scalar average $\bar{x}$, rigorous data analysis models uncertainty through full probability distributions:

$$p(\mathbf{x}) \quad \text{or} \quad p(\mathbf{x} \mid y). \tag{1.27}$$

As derived in Section 1.5, uncertainty comprises both irreducible sensor stochasticity (aleatoric) and epistemic parameter ignorance arising from data sparsity. In safety-critical applications—such

as autonomous navigation or medical oncology—a point prediction $\hat{y} = f(\mathbf{x})$ without an accompanying confidence interval or posterior distribution $p(y \mid \mathbf{x})$ is dangerous. Modeling the full distribution provides the foundational calculus for Bayes decision theory, hypothesis testing, and risk-aware decision boundaries.

1.6.4 Pillar 3: Constraints and invariances

Without prior knowledge, an algorithm facing a finite dataset $\mathcal{D}_N$ must search through the uncountably infinite universe of mathematical functions that interpolate the points. Prior domain knowledge acts as a powerful mathematical filter, restricting the hypothesis space $\mathcal{H}$ to functions that obey known physical laws, geometric symmetries, and environmental constraints.

Historical case study: Alan Turing and the Enigma machine. The power of prior constraints to collapse search complexity is historically illustrated by Alan Turing's codebreaking triumph at Bletchley Park during World War II (dramatized in *The Imitation Game*). The German military Enigma cipher machine possessed an astronomical keyspace: with multiple rotating rotors, ring settings, and a plugboard (*Steckerbrett*), the machine offered approximately 1.5×10^{20} possible daily wiring configurations. A brute-force search over this space was physically impossible, even with electromechanical computing machines (the *Bombes*), within the daily 24-hour window before settings changed.

Turing's decisive insight was discovering an empirical invariance—a deterministic constraint in the communication protocol. German telegraph operators routinely began their first daily transmissions at exactly 06:00 with standardized meteorological reports containing invariant phrasing:

$$\textit{Wetterbericht} \quad (\text{"Weather Report"}), \tag{1.28}$$

and frequently concluded messages with the obligatory military salutation:

$$\textit{Heil Hitler}. \tag{1.29}$$

These predictable, recurring plaintext sequences (known as **cribs**) provided deterministic anchors between known plaintext and observed ciphertext. By enforcing these known linguistic constraints, Turing's Bombe did not have to search the entire combinatorial space of billions of permutations; the search space collapsed to a tiny feasible subset that the machines could eliminate in minutes.

Machine learning formulation. In statistical machine learning, prior knowledge functions in precisely the same manner: it dramatically reduces the volume of the hypothesis space $|\mathcal{H}|$ that must be searched during optimization.

- If domain physics indicates that a signal originates from a continuous periodic industrial process (e.g., turbine vibration or seasonal temperature oscillation), searching over arbitrary high-degree polynomials is wasteful and leads to catastrophic boundary oscillations. Instead, the practitioner restricts $\mathcal{H}$ to a sinusoidal basis family:

$$f(x; \mathbf{w}) = A \sin(\omega x + \phi) + b. \tag{1.30}$$

- In computer vision, our prior knowledge that physical objects retain their semantic identity regardless of where they appear in the visual field directly justifies imposing **translation equivariance** via Convolutional Neural Networks ($f(T_g(\mathbf{x})) = T_g(f(\mathbf{x}))$), eliminating billions of redundant parameters compared to unconstrained fully connected layers.

Prior knowledge is the most effective regularizer: it saves optimization time, prevents overfitting, and minimizes the sample complexity required to achieve high generalization.

1.6.5 Pillar 4: Model selection and capacity

Once data has been observed and prior constraints identified, the machine learner selects an appropriate mathematical model family $f(\mathbf{x}; \boldsymbol{\theta})$ to explain the relationship between input $\mathbf{x}$ and target y.

Model family complexity trade-offs. Consider fitting a two-dimensional dataset $\{(x_i, y_i)\}_{i=1}^{N}$:

- **Degree 1 Polynomial (Linear)**: $y = w_1 x + w_0$. Imposes high inductive bias; requires fitting only 2 scalar parameters. Fast to optimize, but suffers from underfitting if the true physical relationship exhibits curvature.
- **Degree 2 Polynomial (Quadratic)**: $y = w_2 x^2 + w_1 x + w_0$. Requires fitting 3 parameters; models parabolic trajectories.
- **Degree 3 Polynomial (Cubic)**: $y = w_3 x^3 + w_2 x^2 + w_1 x + w_0$. Requires fitting 4 parameters; captures inflection points.
- **Parametric 2D Curves**: Expressing coordinates parametrically as $(x(t), y(t)) = (a_x t + b_x, a_y t + b_y)$ introduces 4 parameters. While more flexible for trajectories, it doubles optimization requirements relative to a standard Cartesian linear model.

Coordinate transformations and the power of linearization. An expert machine learner does not merely increase model capacity when faced with geometrically complex data; they look for coordinate transformations that simplify the problem.

Consider a dataset whose points form an Archimedean spiral or circular ring centered at (x_c, y_c). In standard Cartesian coordinates (x, y), describing this trajectory with an explicit algebraic function $y = f(x)$ is impossible because the curve fails the vertical line test (a single x maps to multiple values of y). Attempting to fit a high-degree Cartesian polynomial results in severe optimization instabilities.

Instead of adopting an unwieldy high-capacity black box, the practitioner shifts the origin to the empirical data centroid (x_c, y_c) and transforms the coordinates into polar space:

$$r = \sqrt{(x - x_c)^2 + (y - y_c)^2}, \quad \theta = \text{atan2}(y - y_c, x - x_c). \tag{1.31}$$

In polar coordinates (r, θ), the complex spiral trajectory unrolls into an elementary linear relationship:

$$r(\theta) = a\theta. \tag{1.32}$$

By simply changing the geometric frame of reference, the learning problem reduces from fitting an intractable non-linear model to fitting a **single linear parameter** a! Mathematical sophistication in representation simplifies computational optimization.

1.6.6 Pillar 5: Data representation

Learning algorithms do not ingest raw physical reality; they process mathematical representations. In classical machine learning (from the 1980s through 2010), representations were constructed via manual feature engineering guided by human domain experts.

The Fisher Iris paradigm. Consider the classical problem of classifying botanical species of Iris flowers into *Iris setosa* and *Iris versicolor*. Feeding raw, unprocessed botanical descriptions to a classifier yields poor results. Instead, a human botanical expert identifies the two morphological measurements that carry discriminative power:

$$x_1 = \text{Petal Length}, \quad x_2 = \text{Sepal Length}. \tag{1.33}$$

Projecting each flower instance into this engineered 2D feature space $\mathbf{x} = [x_1, x_2]^\top \in \mathbb{R}^2$ separates the two species cleanly, allowing a simple Support Vector Machine (SVM) or linear hyperplane to achieve near-perfect classification.

Resolving non-separable distributions: Two foundational strategies. When classes overlap in a chosen feature space and cannot be separated linearly, the machine learner faces two distinct design choices:

1. **Invest in Classifier Complexity**: Retain the current low-dimensional feature space, but replace the linear classifier with an intricate, highly non-linear decision boundary (e.g., high-degree polynomial kernels, deep decision trees, or multi-layer neural networks).

2. **Invest in Representation (Lifting the Dimension)**: Consult domain experts or extract additional feature measurements (e.g., adding $x_3 =$ Petal Width and $x_4 =$ Sepal Width). By embedding the data into a higher-dimensional space $\mathbb{R}^d$ ($d > 2$), entangled clusters disentangle naturally, allowing an elementary linear classifier to separate them with high margin.

Dimensionality reduction and manifold unrolling. Conversely, when raw data is unnecessarily high-dimensional (e.g., 100 sensor readings that are heavily correlated), the data may lie on a low-dimensional intrinsic manifold (e.g., a 2D sheet embedded in $\mathbb{R}^3$). In such cases, applying linear PCA or non-linear manifold unrolling reduces dimensionality, eliminates multicollinearity, suppresses measurement noise, and circumvents the curse of dimensionality.

1.6.7 Pillar 6: End-to-end learning

The decisive paradigm shift of the 2010 Deep Learning revolution was abandoning decoupled manual feature engineering in favor of **end-to-end joint representation learning**.

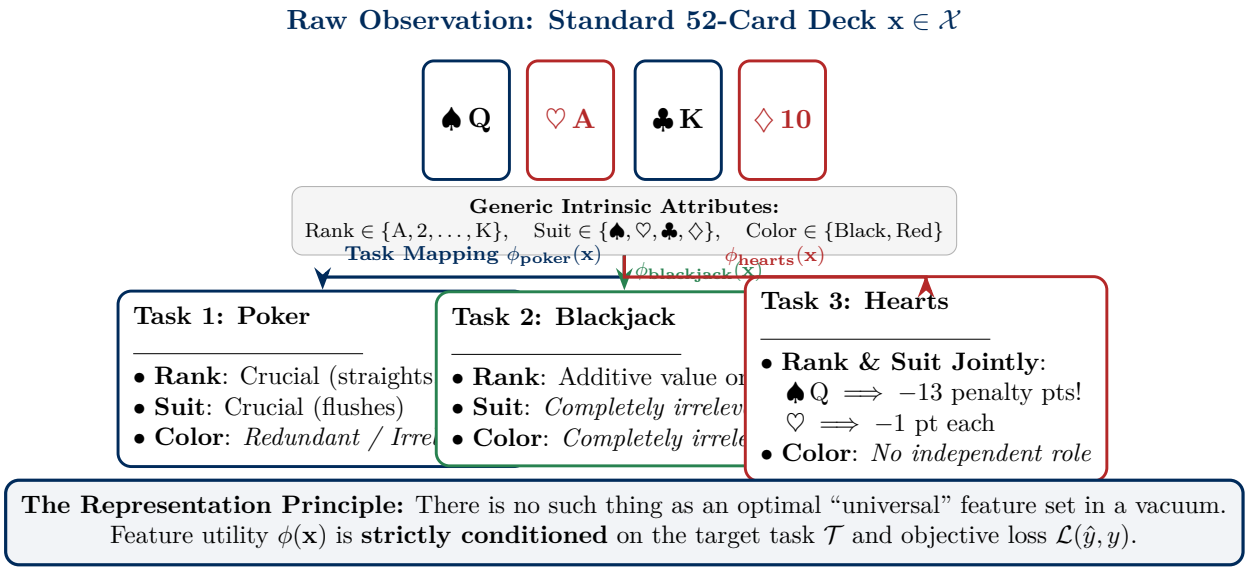


Figure 1.4: The playing card representation analogy: context-dependent, task-driven features. In a standard 52-card deck, cards possess invariant intrinsic properties (rank, suit, and color). However, the utility of these features depends entirely on the game being played: in Hearts, the Queen of Spades carries devastating negative penalty points; in Poker, flushes require tracking suits while rank determines straights; in Blackjack, suits and colors are completely irrelevant and only numeric card values matter.

The playing cards analogy: Features are strictly task-driven. Why must representation learning be coupled directly to the learning task? The answer is illustrated by the card-game analogy depicted in Figure 1.4.

A standard deck of playing cards contains 52 elements, each possessing fixed intrinsic attributes:

$$\text{Rank} \in \{\text{Ace}, 2, \ldots, \text{King}\}, \quad \text{Suit} \in \{\heartsuit, \diamondsuit, \clubsuit, \spadesuit\}, \quad \text{Color} \in \{\text{Red}, \text{Black}\}. \tag{1.34}$$

If you ask an expert to build a "universal feature representation" for a playing card, they cannot answer without knowing the game to be played:

- In **Poker**, suit is vital for detecting flushes, and rank order is vital for straights, whereas color is redundant.
- In **Blackjack**, suit and color are completely irrelevant; only the additive numeric value of the rank matters.
- In **Hearts**, the Queen of Spades (Q$\spadesuit$) carries a severe 13-point penalty, making that specific combination of suit and rank uniquely critical, while diamond cards carry zero penalty.

This pedagogical example reveals a core axiom of modern machine learning: **There is no universally optimal feature representation; features only make sense relative to the specific task $\mathcal{T}$ being solved!**

Shallow vs. Deep learning architectures. This principle explains why shallow machine learning frequently hit performance ceilings:

$$\text{Raw Data } \mathbf{x} \xrightarrow[\text{Human Expert}]{\text{Fixed Handcrafted Features}} \boldsymbol{\phi}(\mathbf{x}) \xrightarrow[\text{Learned}]{\text{Shallow Model}} \hat{y}. \tag{1.35}$$

In shallow ML, the feature extractor $\boldsymbol{\phi}(\mathbf{x})$ is fixed and decoupled from the classifier. If the human expert's handcrafted descriptors omit subtle correlations essential for the downstream task, the classifier can never recover them.

In contrast, Deep Learning unifies representation extraction and task optimization into a single differentiable pipeline:

$$\text{Raw Data } \mathbf{x} \xrightarrow[\text{Differentiable Layers}]{\mathbf{W}_1, \mathbf{W}_2, \ldots, \mathbf{W}_{L-1}} \mathbf{h}_{L-1}(\mathbf{x}) \xrightarrow[\text{Classification Head}]{\mathbf{W}_L} \hat{y}. \tag{1.36}$$

Driven by the backpropagation of the task-specific loss gradient $\nabla_{\mathbf{W}} \ell(\hat{y}, y)$, the internal layers dynamically mold their intermediate representations $\mathbf{h}_l$ specifically to solve task $\mathcal{T}$.

ImageNet and hierarchical feature emergence. In the seminal 2012 AlexNet architecture trained on the 1,000-class ImageNet benchmark, inspecting the learned convolutional filter weights revealed a structured hierarchy of representation:

- **Early Layers (Layers 1–2)**: Extract generic low-level visual primitives: oriented Gabor-like edge filters across multiple angles, spatial frequency bars, and localized color blobs.
- **Intermediate Layers (Layers 3–4)**: Group low-level edges into composite textures, corners, repetitive grids, and structural object parts (e.g., circular wheel rims, mesh grilles, and animal eyes).
- **Deep Layers (Layer 5 and fully connected heads)**: Assemble object parts into holistic class-specific semantic representations (e.g., complete dog faces, bird beaks, and bicycle frames).

This hierarchical decomposition was not programmed by humans; it emerged autonomously from raw pixels driven entirely by gradient optimization on the cross-entropy task loss.

1.6.8 Pillar 7: Fairness and bias mitigation

The final pillar governs the ethical and epistemological reliability of deployed models. A pervasive myth in society is that mathematical algorithms are inherently objective and immune to human prejudice.

In reality, **machine learning models are objective only in the sense that they faithfully optimize their loss function on whatever empirical data they are fed.** If the training data contains historical societal inequities, unrepresentative demographic sampling, or spurious statistical artifacts, the model will not correct them; it will exploit them as optimal predictive shortcuts.

—

1.7 Algorithmic bias and generalization limits

When machine learning systems are deployed in real-world institutions—such as criminal justice, credit scoring, medical triage, and hiring—data bias ceases to be a theoretical optimization nuisance and becomes an active source of systemic harm.

1.7.1 Spurious correlations

Machine learning models optimize purely for accuracy under their training distribution. When presented with training datasets that reflect historical gender imbalances in professional occupations, computer vision models learn to rely on spurious demographic shortcuts rather than true semantic features.

In benchmark experiments on occupation classification from photographs:

- A human expect the classifier to distinguish a doctor from a nurse by examining functional attire: stethoscopes, surgical masks, white laboratory coats, or specialized instruments.
- Instead, feature attribution maps (such as Grad-CAM) revealed that the deep networks were focusing directly on the subject's face to classify **gender**: if the subject was male, the network predicted "Doctor"; if the subject was female, it predicted "Nurse".

Because the scraped web training data had a high proportion of male doctors and female nurses, the model discovered that facial gender recognition was an easy statistical shortcut to achieve high training accuracy with minimal gradient effort.

When deployed in clinical environments where female doctors and male nurses are present, the model failed catastrophically. This failure illustrates the fundamental machine learning adage: **"Garbage in, garbage out"**—if an algorithm is trained on biased data, it produces biased, untrustworthy predictions.

1.7.2 The COMPAS case study

The most consequential and widely analyzed real-world failure of algorithmic decision-making occurred in the United States criminal justice system with the **COMPAS** (Correctional Offender Management Profiling for Alternative Sanctions) algorithm, developed by Northpointe (now Equivant) [1].

Algorithmic deployment in pretrial detention. COMPAS is a commercial risk-assessment instrument used by judges in multiple US jurisdictions to predict a criminal defendant's likelihood of reoffending (recidivism) within two years. The algorithm outputs a decile risk score from 1 (lowest risk) to 10 (highest risk). Judges routinely consult this scalar score when deciding whether a defendant should be granted pretrial release on bail or remanded to prison pending trial.

COMPAS was trained on historical criminal records and arrest histories spanning decades—a historical record heavily shaped by racially disparate policing practices, stop-and-frisk policies, and socio-economic profiling.

The 2016 ProPublica investigative audit. In a landmark 2016 investigative analysis, journalists Julia Angwin, Jeff Larson, Surya Mattu, and Lauren Kirchner from *ProPublica* audited the COMPAS risk scores and subsequent two-year arrest records of over 7,000 individuals arrested in Broward County, Florida [1].

While the algorithm did not explicitly receive race as an input feature coordinate, it took 137 questionnaire responses and historical arrest counts that served as close proxies for socio-economic background and race. When ProPublica evaluated the accuracy of COMPAS across demographic groups, they discovered a severe, systematic racial asymmetry in prediction errors:

- **Disparate False Positive Rate (FPR)**: Among defendants who **did not reoffend** during the subsequent two-year follow-up window, Black defendants were mistakenly classified as medium or high risk at almost **twice the rate** of White non-reoffenders:

$$\text{FPR}_{\text{Black}} = 44.9\% \quad \text{vs.} \quad \text{FPR}_{\text{White}} = 23.5\%. \tag{1.37}$$

- **Disparate False Negative Rate (FNR)**: Conversely, among defendants who **did reoffend** within two years, White defendants were mistakenly classified as low risk at almost **twice the rate** of Black reoffenders:

$$\text{FNR}_{\text{White}} = 47.7\% \quad \text{vs.} \quad \text{FNR}_{\text{Black}} = 28.0\%. \tag{1.38}$$

In human terms, the algorithm exhibited a severe asymmetric bias: it systematically projected future criminality onto innocent Black individuals while extending undue leniency and benefit of the doubt to White individuals who subsequently committed crimes.

The Impossibility Theorem of Algorithmic Fairness. The COMPAS controversy sparked an intense mathematical debate between Northpointe and academic computer scientists. Northpointe defended the tool by proving that COMPAS satisfied **Predictive Parity (Calibration within Groups)**: for any given risk score s (e.g., score $= 7$), the probability that a defendant reoffends is roughly equal regardless of race:

$$P(Y = 1 \mid S = s, A = \text{Black}) \approx P(Y = 1 \mid S = s, A = \text{White}). \tag{1.39}$$

ProPublica, by contrast, evaluated the algorithm according to **Error Rate Parity (Equalized Odds / Separation)**: requiring equal false positive rates and false negative rates across groups:

$$P(\hat{Y} = 1 \mid Y = 0, A = \text{Black}) \stackrel{?}{=} P(\hat{Y} = 1 \mid Y = 0, A = \text{White}). \tag{1.40}$$

In 2016, computer scientists Jon Kleinberg, Sendhil Mullainathan, and Manish Raghavan [52] (and independently Alexandra Chouldechova) proved a foundational mathematical theorem known as the **Impossibility Theorem of Fairness**:

Theorem 1.1 (Fairness Impossibility Theorem [52])**.** *Let $Y \in \{0, 1\}$ be the true binary outcome, $\hat{Y} \in \{0, 1\}$ be the binary decision, and $A \in \{a, b\}$ be a sensitive demographic attribute. Suppose the base rates of the two groups are unequal:*

$$P(Y = 1 \mid A = a) \neq P(Y = 1 \mid A = b), \tag{1.41}$$

*and the classifier is not a perfect predictor (i.e., its error rate is non-zero). Then, it is **mathematically impossible** for any algorithm to simultaneously satisfy:*

1. ***Calibration (Predictive Parity):*** $P(Y = 1 \mid \hat{Y} = 1, A = a) = P(Y = 1 \mid \hat{Y} = 1, A = b)$.
2. ***Equal False Positive Rates:*** $P(\hat{Y} = 1 \mid Y = 0, A = a) = P(\hat{Y} = 1 \mid Y = 0, A = b)$.
3. ***Equal False Negative Rates:*** $P(\hat{Y} = 0 \mid Y = 1, A = a) = P(\hat{Y} = 0 \mid Y = 1, A = b)$.

Because historical arrest base rates differed between demographic groups due to structural socio-economic inequalities, satisfying calibration mathematically forced COMPAS to produce asymmetric false positive rates!

This mathematical reality underscores why algorithmic fairness cannot be resolved purely by code: practitioners must consciously choose which fairness definition to prioritize based on legal, ethical, and societal values.

1.7.3 The EU AI Act

In response to scandals such as COMPAS and predatory biometric profiling, governments have instituted binding legal regulations. In **August 2024**, the European Union enacted into law the **Artificial Intelligence Act (EU AI Act)**, establishing the world's first comprehensive horizontal legal framework for AI governance.

The EU AI Act establishes a four-tiered risk-based regulatory pyramid:

1. **Unacceptable Risk (Prohibited)**: AI systems that pose an intolerable threat to human safety and fundamental rights are banned outright. This includes government social credit scoring, cognitive behavioral manipulation targeting vulnerable populations, biometric categorization based on political/philosophical beliefs, and real-time biometric identification in public spaces by law enforcement (with narrow judicial exceptions).

2. **High-Risk (Strictly Regulated)**: Systems deployed in critical sectors that directly impact human livelihoods, health, safety, and civil rights. Crucially, the EU AI Act classifies AI used in **the administration of justice, law enforcement, judicial bail decisions, credit scoring, employment recruitment, education grading, and critical infrastructure** as **High-Risk**.

3. **General Purpose AI (GPAI)**: Large foundation models (e.g., frontier LLMs) must comply with transparency mandates, copyright documentation, and systemic risk assessments for frontier capabilities exceeding 10^{25} FLOPs.

4. **Minimal / Low Risk**: Systems like spam filters, AI recommendation engines, or video games, which remain largely unconstrained.

Mandatory compliance for High-Risk AI. Under the EU AI Act, systems operating in the justice and law enforcement domains (such as COMPAS-like recidivism tools) face stringent legal obligations before they can enter the EU market:

- **Mandatory Data Governance**: Training, validation, and testing datasets must be vetted for representative demographic balance, data quality, and statistical bias mitigation prior to training.
- **Mandatory Human Oversight**: AI models cannot automate judicial sentences or legal determinations autonomously. A qualified human magistrate must retain effective supervisory control (*human-in-the-loop*), with full authority to override, reject, or ignore algorithmic outputs.
- **Traceability and Audit Logs**: High-risk systems must automatically log their operating states, input distributions, and decision paths to facilitate post-hoc judicial audits.

While the EU AI Act legally applies only within European jurisdiction, its extraterritorial scope binds any international entity or US provider deploying AI systems whose outputs affect individuals residing within the European Union.

1.7.4 Domain shift and generalization limits

Statistical learning theory rests entirely on the foundational assumption that training samples and future deployment test samples are drawn **independently and identically distributed (i.i.d.)** from the exact same stationary probability distribution:

$$p_{\text{train}}(\mathbf{x}, y) \equiv p_{\text{test}}(\mathbf{x}, y) = p_{\text{data}}(\mathbf{x}, y). \tag{1.42}$$

In real-world deployment, this assumption is universally violated.

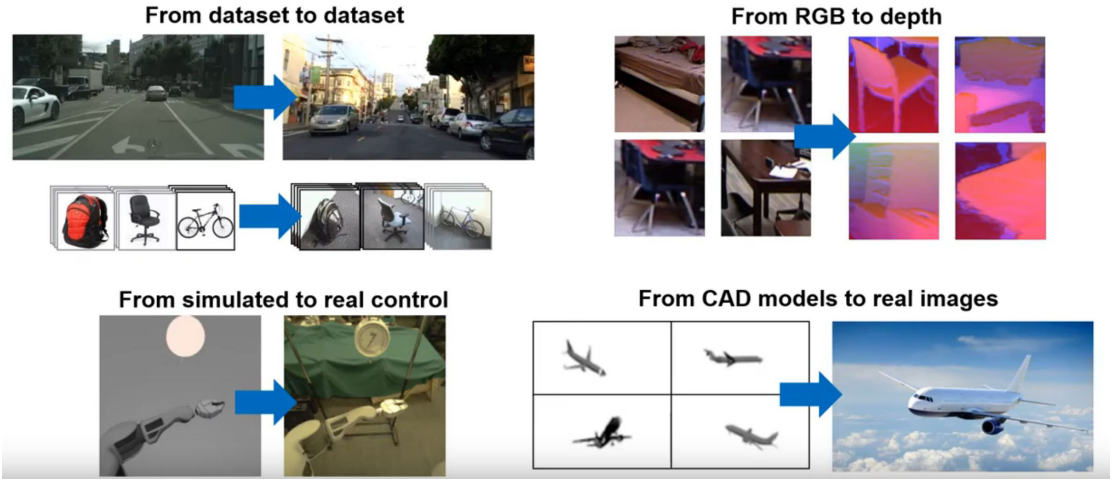


Figure 1.5: Dataset bias and domain shift (Torralba & Efros, 2011; Saenko et al., 2010) [82, 92]. When an image classifier trained on high-quality product images (e.g., Amazon catalogue photos) is deployed on real-world webcam photos or mobile phone snapshots, accuracy drops precipitously due to distribution shift across lighting, background clutter, and camera angles.

As shown in Figure 1.5, when models encounter **domain shift**, test accuracy drops precipitously. The two primary mathematical forms of domain shift are:

- **Covariate Shift**: The marginal input feature distribution shifts ($p_{\text{train}}(\mathbf{x}) \neq p_{\text{test}}(\mathbf{x})$), while the posterior conditional label boundary remains invariant ($P(y \mid \mathbf{x})$ is constant).
- **Concept Drift**: The conditional label distribution itself changes ($P_{\text{train}}(y \mid \mathbf{x}) \neq P_{\text{test}}(y \mid \mathbf{x})$), meaning identical input observations map to different semantic classes over time.

Autonomous driving and the Sim-to-Real gap. A vivid empirical example occurs in autonomous robotics and self-driving vehicles:

- **Geographic and Environmental Domain Shift**: An autonomous driving perception network trained exclusively on camera footage collected in sunny daylight conditions in Rome will fail catastrophically when deployed during a heavy snowstorm at night in Beijing. Lighting conditions, road textures, precipitation scatter, and pedestrian behaviors change drastically ($p_{\text{train}}(\mathbf{x}) \neq p_{\text{test}}(\mathbf{x})$).
- **The Sim-to-Real Gap**: Autonomous robots cannot be trained from scratch in physical reality because random exploration would cause physical collisions, vehicle crashes, and immense financial cost. Consequently, roboticists and autonomous vehicle developers train perception systems inside high-fidelity 3D synthetic simulators or game engines like **Grand Theft Auto V (GTA V)** and CARLA.

While synthetic video game data is infinitely cheap, fully scalable, and comes with free, perfect pixel-level semantic segmentation annotations, real-world deployment reveals a severe **Sim-to-Real domain gap**. Synthetic graphics rendering engines lack the high-frequency physical noise, optical aberrations, dynamic specular reflections, and chaotic imperfections of the physical world. A model that achieves 99% semantic segmentation accuracy inside GTA V drops precipitously when tested on physical highway cameras.

Overcoming this gap requires modern techniques beyond classical i.i.d. learning:

- **Unsupervised Domain Adaptation (UDA)**: Aligning source and target feature representations via adversarial minimax objectives (e.g., Domain-Adversarial Neural Networks, DANN [28]).
- **Domain Randomization**: Randomizing simulation lighting, camera angles, textures, and weather dynamics during synthetic training so that physical reality appears to the network as merely another synthetic variation.
- **Continual / Lifelong Learning**: Updating network parameters online in response to non-stationary environments without catastrophically forgetting previously mastered domains.

The study of dataset bias, domain adaptation, and statistical robustness represents the true frontier of modern machine learning, bridging laboratory benchmarks to robust physical reality.

Chapter 2

Probability and decision theory

Machine learning is fundamentally the science of making principled inferences and optimal decisions under uncertainty. Every real-world sensor reading is perturbed by noise, every dataset represents an incomplete sample of reality, and every prediction about future test instances carries inherent ambiguity. To build algorithms that learn from data, we must ground our reasoning in the mathematical framework of **probability theory** and **statistical decision theory**.

In this chapter, we develop the foundations of probabilistic machine learning. We begin from the axiomatic formulation of probability spaces, Cox's plausibility postulates, and de Finetti's Dutch Book theorem, progress through continuous density transformations and multivariable joint distributions, establish the core foundations of information theory (entropy, KL divergence, and cross-entropy), analyze Bayes' rule with exact geometric area models, resolve probabilistic paradoxes, and explore causal confounding topologies.

We then construct generative classifiers: exploring the complete continuum from discrete and continuous Gaussian Naïve Bayes to Linear Discriminant Analysis (LDA) and Quadratic Discriminant Analysis (QDA), followed by an exhaustive examination of Maximum Likelihood Estimation (MLE) versus Maximum A Posteriori (MAP) parameter learning. Finally, we establish the foundations of statistical learning theory and decision theory: formalizing task-dependent loss functions via parametric distributions, True Risk, the Bayes Optimal Classifier under asymmetric loss matrices, the reject option, regression decision theory (conditional mean and median), the information-theoretic equivalence of MLE to KL divergence minimization, and point estimator statistics with Bessel's correction.

—

2.1 Probability foundations

Why are basic statistics and probability indispensable for machine learning? As established in lecture, machine learning models do not possess omniscience; they process finite empirical observations to infer unobserved truths. Probability provides the formal mathematical calculus to:

- Quantify our degree of belief in competing hypotheses given partial, noisy evidence.
- Formulate principled classification rules that minimize expected misclassification costs.
- Bridge observed finite datasets to unobserved continuous population distributions.

2.1.1 Sample spaces and probability measures

To formalize uncertainty mathematically, we construct a probability space as a measure-theoretic triplet $(\Omega, \mathcal{F}, P)$:

1. **Sample Space** Ω: The set of all conceivable, mutually exclusive outcomes of a physical or conceptual experiment:

   - *Discrete Sample Spaces*: When outcomes can be enumerated. For a single coin flip, $\Omega = \{\text{Heads}, \text{Tails}\}$; for a 6-sided die, $\Omega = \{1, 2, 3, 4, 5, 6\}$; for a standard deck of cards, $|\Omega| = 52$.
   - *Continuous Sample Spaces*: When outcomes represent uncountably infinite measurements, such as sensor voltages, physical temperatures, or spatial coordinates: $\Omega = \mathbb{R}$ or $\Omega = \mathbb{R}^d$.

2. **Event** $A \subseteq \Omega$: An event is any subset of the sample space whose occurrence we wish to evaluate. For example, rolling an even number on a die corresponds to the event subset $A = \{2, 4, 6\} \subset \Omega$.

3. **Probability Measure** $P(A)$: A real-valued function $P : \mathcal{F} \to [0, 1]$ that assigns a scalar plausibility to event A. Visually, if the entire sample space Ω is represented as a geometric region with unit area ($P(\Omega) = 1$), the probability of event A is the fraction of total area covered by subset A.

2.1.2 Kolmogorov axioms

In 1933, Andrey Kolmogorov established that any valid mathematical calculus of probability must satisfy three non-negotiable axioms:

Definition 2.1 (Kolmogorov Probability Axioms [6])**.** Let Ω be a sample space and $\mathcal{F}$ be the collection of valid events. A function $P : \mathcal{F} \to [0, 1]$ is a probability measure if and only if:

1. **Axiom 1 (Non-Negativity)**: For every event $A \subseteq \Omega$,

$$P(A) \geq 0. \tag{2.1}$$

2. **Axiom 2 (Total Measure / Normalization)**: The probability of the entire sample space is unity:

$$P(\Omega) = 1. \tag{2.2}$$

3. **Axiom 3 (Countable Additivity / σ-Additivity)**: For any sequence of pairwise mutually exclusive events $\{A_1, A_2, \dots\}$ (where $A_i \cap A_j = \emptyset$ for all $i \neq j$):

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i). \tag{2.3}$$

From these three minimal axioms, the foundational properties of probability calculus follow directly:

- **Empty Set**: $P(\emptyset) = 0$.
- **Complementation Rule**: Since $A \cup A^c = \Omega$ and $A \cap A^c = \emptyset$, Axioms 2 and 3 imply:

$$P(A^c) = 1 - P(A). \tag{2.4}$$

- **Monotonicity**: If event A is a subset of event B ($A \subseteq B$), then $P(A) \leq P(B)$.
- **Inclusion-Exclusion Principle**: For any two events A and B (not necessarily disjoint):

$$P(A \cup B) = P(A) + P(B) - P(A \cap B). \tag{2.5}$$

2.1.3 Cox's postulates

A profound foundational question in machine learning is: *Why must we quantify uncertainty using Kolmogorov probability calculus rather than inventing arbitrary scoring heuristics, fuzzy logics, or ad-hoc confidence metrics?*

In 1946, physicist Richard Cox [14,48] proved that any mathematical framework for plausible reasoning that satisfies basic qualitative desiderata of human common sense is **isomorphic to standard probability theory**. Cox defined three commonsense postulates:

1. **Real-Valued Representation**: Degrees of plausibility are represented by real scalar numbers: if hypothesis A is more plausible than hypothesis B, and B is more plausible than C, then $p(A) > p(B) > p(C)$.
2. **Qualitative Correspondence**: As the plausibility of hypothesis A increases given background information, the plausibility of its negation $\neg A$ must strictly decrease.
3. **Consistency**: If a conclusion can be reasoned out in multiple mathematically valid ways, every equivalent line of reasoning must yield the identical numerical plausibility value.

From these three qualitative postulates alone, Cox proved through functional equations that the product rule $P(A \cap B) = P(A \mid B)P(B)$ and the sum rule $P(A) + P(\neg A) = 1$ are mathematically inevitable. **Probability theory is not an arbitrary convention; it is the unique mathematical extension of Boolean logic to conditions of uncertainty.**

2.1.4 de Finetti's Dutch Book theorem

While Richard Cox approached the uniqueness of probability from the perspective of qualitative logic, Italian mathematician Bruno de Finetti [18, 81] provided an independent, operational justification grounded in **decision theory and rational economics**: the celebrated **Dutch Book Theorem**.

Suppose an intelligent agent asserts subjective degrees of belief in propositions and is willing to accept bets at betting odds corresponding to those numbers. Let $p \in [0, 1]$ be the agent's stated probability for event A. The agent agrees to accept a bet with positive or negative stake S:

- The agent receives $p \cdot S$ upfront.
- If event A occurs, the agent pays the opponent S.
- The agent's net payoff is:

$$\text{Payoff} = p \cdot S - S \cdot \mathbb{I}(A) = \begin{cases} (p-1)S & \text{if } A \text{ occurs,} \\ pS & \text{if } A \text{ does not occur.} \end{cases} \tag{2.6}$$

Theorem 2.1 (de Finetti's Dutch Book Theorem [18, 81])**.** *If an agent's subjective probabilities violate any of the Kolmogorov probability axioms, then there exists a set of simultaneous bets (a "Dutch book") that guarantees the agent incurs a strictly negative net payoff regardless of the actual state of the world.*

Proof for Violation of the Complementation / Additivity Axiom. Suppose the agent's subjective beliefs violate the sum rule for an event A and its complement A^c:

$$p(A) + p(A^c) = 1 + \epsilon, \quad \text{with } \epsilon > 0. \tag{2.7}$$

An astute bookmaker observes these irrational odds and places two simultaneous bets with the agent:

1. Bet 1 on event A with stake $S_1 = 1$. The bookmaker pays the agent $p(A)$.

2. Bet 2 on event A^c with stake $S_2 = 1$. The bookmaker pays the agent $p(A^c)$.

The total money received upfront by the agent is $p(A) + p(A^c) = 1 + \epsilon$. Now examine the two possible physical states of nature:

- **Case 1: Event A occurs.** The agent must pay the bookmaker $S_1 = 1$ on Bet 1, while Bet 2 expires worthless. The agent's net payoff is:

  $$\text{Net Profit} = (1+\epsilon) - 1 = +\epsilon \quad \text{(so the agent loses } \epsilon \text{ relative to the bookmaker's net gain of } \epsilon\text{)}. \tag{2.8}$$

- **Case 2: Event A^c occurs.** The agent must pay the bookmaker $S_2 = 1$ on Bet 2, while Bet 1 expires worthless. The agent's net payoff is again:

  $$\text{Net Profit} = (1 + \epsilon) - 1 = +\epsilon. \tag{2.9}$$

In every possible state of reality, the bookmaker is guaranteed to extract exactly $\epsilon > 0$ from the agent. A symmetric Dutch book forces an unavoidable loss if $p(A) + p(A^c) < 1$. Therefore, an agent who deviates from Kolmogorov's axioms is mathematically irrational in an operational sense: their beliefs permit an opponent to systematically drain their resources with zero risk. □

2.1.5 Random variables and transformations

A **random variable** X is a formal mathematical function that maps outcomes from the sample space to real numbers:

$$X : \Omega \to \mathbb{R}. \tag{2.10}$$

Random variables allow us to translate physical experimental outcomes into numerical quantities amenable to algebra and calculus:

- **Discrete Random Variables**: Take values from a countable set $\{x_1, x_2, \dots\}$. Their distribution is specified by a **probability mass function (PMF)** $P(X = x)$, satisfying $\sum_x P(X = x) = 1$.
- **Continuous Random Variables**: Take values in continuous intervals of $\mathbb{R}$. Because the probability of hitting an exact real number is zero ($P(X = x) = 0$), continuous uncertainty is specified by a **probability density function (PDF)** $p(x) \geq 0$, where probabilities correspond to integrals over intervals:

  $$P(a \leq X \leq b) = \int_a^b p(x)\, dx, \quad \text{with} \quad \int_{-\infty}^{\infty} p(x)\, dx = 1. \tag{2.11}$$

Transformation of continuous densities and the Jacobian determinant. In machine learning, continuous random variables are routinely transformed through deterministic non-linear mappings (such as neural network layers, coordinate changes, or normalizing flows).

Let x be a continuous random variable with density $p_x(x)$, and let $y = g(x)$ be a strictly monotonic, invertible transformation. Because probability mass must be conserved across corresponding infinitesimal intervals $[x, x + \delta x]$ and $[y, y + \delta y]$:

$$p_y(y)|\delta y| \approx p_x(x)|\delta x|. \tag{2.12}$$

Taking the infinitesimal limit $\delta x \to 0$ yields the continuous **change-of-variables formula** [6]:

$$p_y(y) = p_x(x) \left| \frac{dx}{dy} \right| = p_x(g^{-1}(y)) \left| \frac{d}{dy} g^{-1}(y) \right|. \tag{2.13}$$

For multi-dimensional random vectors $\mathbf{x} \in \mathbb{R}^d$ mapped via an invertible vector-valued function $\mathbf{y} = \mathbf{g}(\mathbf{x}) \in \mathbb{R}^d$, the scalar derivative generalizes to the absolute value of the **Jacobian determinant**:

$$p_{\mathbf{y}}(\mathbf{y}) = p_{\mathbf{x}}(\mathbf{x}) \left| \det \left(\frac{\partial \mathbf{x}}{\partial \mathbf{y}} \right) \right| = p_{\mathbf{x}}(\mathbf{g}^{-1}(\mathbf{y})) \left| \det \left(\mathbf{J}_{\mathbf{g}}(\mathbf{x}) \right) \right|^{-1}, \tag{2.14}$$

where $\mathbf{J}_{\mathbf{g}}(\mathbf{x}) \in \mathbb{R}^{d \times d}$ is the Jacobian matrix with entries $(\mathbf{J}_{\mathbf{g}})_{ij} = \frac{\partial g_i}{\partial x_j}$.

A fundamental concept frequently examined in oral exams is that **the mode (maximum) of a continuous probability density is not invariant under non-linear parameter transformations!** For a discrete random variable, if x^* maximizes $P_X(x)$, then $y^* = g(x^*)$ uniquely maximizes $P_Y(y)$. However, for continuous densities, maximizing $p_y(y)$ is not equivalent to evaluating $g(x^*)$ because the Jacobian term $\left|\frac{dx}{dy}\right|$ in Equation (2.13) varies with y. Differentiating $p_y(y)$ with respect to y:

$$\frac{dp_y(y)}{dy} = \frac{d}{dy}\left[p_x(x) \left| \frac{dx}{dy} \right| \right] = \frac{dp_x(x)}{dx} \left(\frac{dx}{dy} \right)^2 + p_x(x) \frac{d^2x}{dy^2}. \tag{2.15}$$

Setting this to zero reveals that the stationary point of $p_y(y)$ depends on the curvature $\frac{d^2x}{dy^2}$ of the transformation. Consequently, Maximum A Posteriori (MAP) point estimates depend on the chosen coordinate parameterization!

2.1.6 Expectation, variance, and covariance

To summarize probability distributions compactly, we compute their statistical moments:

- **Mathematical Expectation** $\mathbb{E}[X]$: The probability-weighted average value of a random variable:

$$\mathbb{E}[X] = \begin{cases} \sum_x x P(X = x) & \text{(discrete)}, \\ \int_{-\infty}^{\infty} x p(x)\, dx & \text{(continuous)}. \end{cases} \tag{2.16}$$

  Expectation is a strictly linear operator: $\mathbb{E}[aX + bY + c] = a\mathbb{E}[X] + b\mathbb{E}[Y] + c$ for any scalars $a, b, c \in \mathbb{R}$.

- **Variance** $\mathrm{Var}(X)$: Measures the expected squared deviation from the mean:

$$\mathrm{Var}(X) \triangleq \mathbb{E}\left[(X - \mathbb{E}[X])^2 \right] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2. \tag{2.17}$$

  The standard deviation is $\sigma = \sqrt{\mathrm{Var}(X)}$, which retains the original physical measurement units.

- **Covariance Matrix** $\boldsymbol{\Sigma}$: For a multi-dimensional random vector $\mathbf{x} = [x_1, \ldots, x_d]^\top \in \mathbb{R}^d$ with mean vector $\boldsymbol{\mu} = \mathbb{E}[\mathbf{x}] \in \mathbb{R}^d$, the pairwise interactions are captured by the $(d \times d)$ covariance matrix:

$$\boldsymbol{\Sigma} \triangleq \mathbb{E}\left[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top \right] \in \mathbb{R}^{d \times d}. \tag{2.18}$$

  The diagonal entry $\Sigma_{jj} = \mathrm{Var}(x_j)$ is the variance of feature j, while the off-diagonal entry $\Sigma_{ij} = \mathrm{Cov}(x_i, x_j) = \mathbb{E}[(x_i - \mu_i)(x_j - \mu_j)]$ measures the linear co-variation between features i and j.

Properties of the Covariance Matrix. The covariance matrix satisfies two non-negotiable algebraic properties:

1. **Symmetry**: $\boldsymbol{\Sigma} = \boldsymbol{\Sigma}^\top$, because $\text{Cov}(x_i, x_j) = \text{Cov}(x_j, x_i)$.
2. **Positive Semi-Definiteness**: For any arbitrary non-zero real vector $\mathbf{z} \in \mathbb{R}^d$:

$$\mathbf{z}^\top \boldsymbol{\Sigma} \mathbf{z} = \mathbf{z}^\top \mathbb{E}\left[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top\right] \mathbf{z} = \mathbb{E}\left[\mathbf{z}^\top (\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top \mathbf{z}\right] = \mathbb{E}\left[(\mathbf{z}^\top (\mathbf{x} - \boldsymbol{\mu}))^2\right] \geq 0. \tag{2.19}$$

   Because it is the expected value of a squared scalar quantity, $\mathbf{z}^\top \boldsymbol{\Sigma} \mathbf{z} \geq 0$. Hence, all eigenvalues of $\boldsymbol{\Sigma}$ are non-negative ($\lambda_i \geq 0$).

Independence versus Zero Covariance. In oral exams, students must clearly articulate that while statistical independence implies zero covariance, the converse is **strictly false**:

$$X \perp Y \implies \text{Cov}(X, Y) = 0, \qquad \text{Cov}(X, Y) = 0 \not\implies X \perp Y. \tag{2.20}$$

To prove this counterexample, let $X \sim \mathcal{U}[-1, 1]$ be uniformly distributed on the interval $[-1, 1]$, and let $Y = X^2$. Here, Y is completely deterministic given X (they are maximally dependent). However, the covariance evaluates to:

$$\text{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y] = \mathbb{E}[X^3] - (0)\mathbb{E}[Y] = \int_{-1}^{1} \frac{x^3}{2}\, dx = 0. \tag{2.21}$$

The covariance is identically zero because covariance measures only *linear* relationships. Non-linear dependencies escape detection by covariance alone!

Notation distinction in AML literature. Throughout this text, we adhere strictly to canonical probabilistic notation:

- Capital letter X denotes the **random variable** (the abstract mapping).
- Lowercase letter x denotes a **specific scalar value** realization ($X = x$).
- Bold lowercase $\mathbf{x} = [x_1, x_2, \ldots, x_d]^\top \in \mathbb{R}^d$ denotes a multi-dimensional **feature vector**.
- $P(X = x)$ denotes the probability of the event that random variable X takes value x, frequently abbreviated as $P(x)$ when the variable context is unambiguous.

—

2.2 Multivariable distributions and inference

Real-world machine learning systems rarely observe a single isolated random variable; they observe dozens, hundreds, or millions of interacting features simultaneously.

2.2.1 Joint distributions

When two or more random variables interact, the probability of one variable taking a specific value depends on the values assumed by the others. We define a **joint ensemble** (X, Y) governed by the **joint probability distribution**:

$$P(X = x, Y = y) \quad \text{or} \quad P(X = x \cap Y = y). \tag{2.22}$$

The joint distribution provides a complete, exhaustive description of uncertainty across all variables. The sum across all possible combinations of assignments must equal unity:

$$\sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} P(X = x, Y = y) = 1. \tag{2.23}$$

Table 2.1: Joint probability distribution $P(T, W)$ for Temperature and Weather.

	Weather $W =$ **sun**	**Weather** $W =$ **rain**
Temperature $T =$ **hot**	0.40	0.10
Temperature $T =$ **cold**	0.10	0.40

Consider a discrete environmental domain with two binary variables: Temperature $T \in \{\text{hot}, \text{cold}\}$ and Weather $W \in \{\text{sun}, \text{rain}\}$. Their interaction is completely defined by a 2×2 joint distribution table, as shown in Table 2.1.

From this joint distribution table, the probability of **any** compound event can be evaluated by summing the probabilities of the atomic outcomes satisfying the event condition:

- **Probability that it is hot AND sunny**: Directly looked up from the table intersection:

$$P(T = \text{hot}, W = \text{sun}) = 0.40. \tag{2.24}$$

- **Probability that it is hot OR sunny**: Evaluated via the inclusion-exclusion principle or by summing the three disjoint matching cells:

$$\begin{aligned} P(T = \text{hot} \cup W = \text{sun}) &= P(T = \text{hot}, W = \text{sun}) + P(T = \text{hot}, W = \text{rain}) + P(T = \text{cold}, W = \text{sun}) \\ &= 0.40 + 0.10 + 0.10 = 0.60. \end{aligned} \tag{2.25}$$

2.2.2 Marginalization

In many practical scenarios, we possess a high-dimensional joint distribution over multiple variables, but we are interested in the probability distribution over a smaller subset of variables, ignoring the others.

A **marginal distribution** is a sub-table obtained by eliminating (collapsing) irrelevant variables through summation. This fundamental operation is known as **marginalization** or **"summing out"**:

$$P(X = x) = \sum_{y \in \mathcal{Y}} P(X = x, Y = y). \tag{2.26}$$

For continuous distributions, summation is replaced by integration over the domain of the omitted variable:

$$p(x) = \int_{\mathcal{Y}} p(x, y)\, dy. \tag{2.27}$$

Applying Equation (2.26) to Table 2.1:

$$P(T = \text{hot}) = P(T = \text{hot}, W = \text{sun}) + P(T = \text{hot}, W = \text{rain}) = 0.40 + 0.10 = 0.50, \tag{2.28}$$

$$P(T = \text{cold}) = P(T = \text{cold}, W = \text{sun}) + P(T = \text{cold}, W = \text{rain}) = 0.10 + 0.40 = 0.50. \tag{2.29}$$

Similarly, summing the columns yields the marginal distribution of Weather:

$$P(W = \text{sun}) = 0.40 + 0.10 = 0.50, \tag{2.30}$$

$$P(W = \text{rain}) = 0.10 + 0.40 = 0.50. \tag{2.31}$$

Because weather has only two possible mutually exclusive values, observing $P(W = \text{sun}) = 0.50$ immediately allows us to express $P(W = \text{rain}) = 1 - P(W = \text{sun}) = 0.50$.

2.2.3 Conditional probability

Conditional probability quantifies our updated belief in an event $X = x$ given that we have observed deterministic evidence that another event $Y = y$ has occurred. Mathematically, conditional probability is defined as the fraction of worlds in which X is true among those worlds where Y is true:

$$P(X = x \mid Y = y) = \frac{P(X = x, Y = y)}{P(Y = y)} = \frac{P(X = x, Y = y)}{\sum_{x'} P(X = x', Y = y)}. \tag{2.32}$$

The intuitive geometric meaning of Equation (2.32) is **sample space contraction**: observing evidence $Y = y$ collapses our universe of possibilities from the entire sample space Ω down to the conditioning slice $Y = y$. The denominator $P(Y = y)$ acts as a local normalizing constant that rescales the probabilities within the slice so that their sum equals 1:

$$\sum_{x \in \mathcal{X}} P(X = x \mid Y = y) = 1. \tag{2.33}$$

Applying Equation (2.32) to our weather joint table:

$$P(W = \text{sun} \mid T = \text{hot}) = \frac{P(W = \text{sun}, T = \text{hot})}{P(T = \text{hot})} = \frac{0.40}{0.50} = 0.80, \tag{2.34}$$

$$P(W = \text{rain} \mid T = \text{hot}) = \frac{P(W = \text{rain}, T = \text{hot})}{P(T = \text{hot})} = \frac{0.10}{0.50} = 0.20. \tag{2.35}$$

Notice how our belief in sunshine increased from its prior marginal probability $P(W = \text{sun}) = 0.50$ to 0.80 once we observed the evidence that temperature is hot.

2.2.4 Probabilistic belief updating

Probabilistic inference is the computational process of calculating a desired probability distribution from a known model and observed evidence. In machine learning, an intelligent agent maintains beliefs about unobserved state variables (e.g., traffic conditions, disease status, or semantic object labels) and dynamically updates these beliefs as new sensor measurements arrive:

- Baseline belief: $P(\text{On Time} \mid \text{No Reported Accidents}) = 0.90$.
- Incorporating time of day: $P(\text{On Time} \mid \text{No Accidents}, 5 \text{ a.m.}) = 0.95$.
- Incorporating weather: $P(\text{On Time} \mid \text{No Accidents}, 5 \text{ a.m.}, \text{Heavy Rain}) = 0.80$.

As additional evidence is observed, the conditioning set expands, and beliefs are continuously revised according to the laws of probability.

2.2.5 Inference by enumeration

When we possess the full joint distribution table across all variables in a domain, any probabilistic query can be answered using the general algorithm of **Inference by Enumeration** (pioneered in the UC Berkeley CS188 artificial intelligence curriculum [81]).

Consider three discrete random variables: Weather $W \in \{\text{sun}, \text{rain}\}$, Temperature $T \in \{\text{hot}, \text{cold}\}$, and Season $S \in \{\text{summer}, \text{fall}, \text{winter}, \text{spring}\}$. The full joint distribution contains $2 \times 2 \times 4 = 16$ atomic states, as tabulated in Table 2.2.

Table 2.2: Full joint distribution $P(S, T, W)$ for Season, Temperature, and Weather (CS188 benchmark).

Season S	**Temperature** T	**Weather** W	**Joint Probability** $P(S, T, W)$
summer	hot	sun	0.150
summer	hot	rain	0.010
summer	cold	sun	0.020
summer	cold	rain	0.020
fall	hot	sun	0.050
fall	hot	rain	0.020
fall	cold	sun	0.060
fall	cold	rain	0.070
winter	hot	sun	0.010
winter	hot	rain	0.005
winter	cold	sun	0.085
winter	cold	rain	0.150
spring	hot	sun	0.100
spring	hot	rain	0.020
spring	cold	sun	0.120
spring	cold	rain	0.100

Inference query 1: Unconditional marginal $P(W)$. To compute the marginal distribution of Weather $P(W)$, we sum out the unobserved hidden variables (Season S and Temperature T):

$$P(W = \text{sun}) = \sum_{s \in \mathcal{S}} \sum_{t \in \mathcal{T}} P(S = s, T = t, W = \text{sun}). \tag{2.36}$$

Summing the eight corresponding rows from Table 2.2:

$$P(W = \text{sun}) = 0.150 + 0.020 + 0.050 + 0.060 + 0.010 + 0.085 + 0.100 + 0.120 = 0.595, \tag{2.37}$$

$$P(W = \text{rain}) = 1 - P(W = \text{sun}) = 1 - 0.595 = 0.405. \tag{2.38}$$

Inference query 2: Conditional distribution with evidence $P(W \mid S = \textbf{winter}, T = \textbf{hot})$. Now suppose we observe two pieces of evidence: the season is winter, and the temperature is hot. We wish to infer the distribution of Weather W.

1. **Step 1: Select matching rows**. Filter Table 2.2 for rows where $S = \text{winter}$ and $T = \text{hot}$:

$$P(W = \text{sun}, S = \text{winter}, T = \text{hot}) = 0.010, \tag{2.39}$$

$$P(W = \text{rain}, S = \text{winter}, T = \text{hot}) = 0.005. \tag{2.40}$$

2. **Step 2: Compute marginal probability of the evidence (normalizer)**. Sum the filtered rows:

$$P(S = \text{winter}, T = \text{hot}) = 0.010 + 0.005 = 0.015. \tag{2.41}$$

3. **Step 3: Normalize to obtain posterior probabilities**. Define the normalization factor $\alpha = \frac{1}{0.015} \approx 66.67$:

$$P(W = \text{sun} \mid \text{winter}, \text{hot}) = \frac{0.010}{0.015} = \frac{2}{3} \approx 0.667, \tag{2.42}$$

$$P(W = \text{rain} \mid \text{winter}, \text{hot}) = \frac{0.005}{0.015} = \frac{1}{3} \approx 0.333. \tag{2.43}$$

Inference query 3: Conditional distribution with hidden variable $P(W \mid S = \textbf{winter})$. Suppose we observe only that $S = \text{winter}$, while Temperature T is unobserved (hidden):

1. **Step 1: Select matching rows and marginalize out hidden variable T**:

$$P(W = \text{sun}, \text{winter}) = P(\text{winter}, \text{hot}, \text{sun}) + P(\text{winter}, \text{cold}, \text{sun}) = 0.010 + 0.085 = 0.095, \tag{2.44}$$

$$P(W = \text{rain}, \text{winter}) = P(\text{winter}, \text{hot}, \text{rain}) + P(\text{winter}, \text{cold}, \text{rain}) = 0.005 + 0.150 = 0.155. \tag{2.45}$$

2. **Step 2: Compute total probability of winter**:

$$P(\text{winter}) = 0.095 + 0.155 = 0.250. \tag{2.46}$$

3. **Step 3: Normalize**:

$$P(W = \text{sun} \mid \text{winter}) = \frac{0.095}{0.250} = 0.380, \tag{2.47}$$

$$P(W = \text{rain} \mid \text{winter}) = \frac{0.155}{0.250} = 0.620. \tag{2.48}$$

Inference by enumeration is exact and universal. However, if a problem has d discrete variables each taking v values, the joint table has size v^d, making inference by enumeration computationally intractable ($O(v^d)$ time and space) for large-scale machine learning problems.

2.2.6 Product and chain rules

Rearranging the definition of conditional probability in Equation (2.32) yields the **Product Rule**:

$$P(X, Y) = P(X \mid Y)P(Y) = P(Y \mid X)P(X). \tag{2.49}$$

By repeatedly applying the product rule to an ensemble of d random variables $X_1, X_2, \ldots, X_d$, we obtain the fundamental **Chain Rule of Probability**:

$$\begin{aligned} P(X_1, X_2, \ldots, X_d) &= P(X_1)P(X_2 \mid X_1)P(X_3 \mid X_1, X_2) \ldots P(X_d \mid X_1, \ldots, X_{d-1}) \\ &= \prod_{i=1}^{d} P(X_i \mid X_1, \ldots, X_{i-1}). \end{aligned} \tag{2.50}$$

Equation (2.50) is an exact identity that holds for **any** joint probability distribution without requiring any independence assumptions. It proves that any joint distribution can be decomposed into an incremental product of conditional distributions.

2.2.7 Information-theoretic foundations

In modern statistical machine learning and deep learning, quantifying information content, uncertainty, and divergence between probability distributions is essential [6, 32]. Information theory, founded by Claude Shannon in 1948 [84], provides the formal mathematical language for this quantification.

Shannon Self-Information. How much information is conveyed when an event x occurs? Shannon observed that the information content of an event should be inversely related to its probability:

- Highly probable events (e.g., "the sun rose this morning") convey virtually zero surprise or information.
- Extremely rare events (e.g., "a severe magnitude 8 earthquake just struck") carry immense surprise and high information content.

Furthermore, if two independent events A and B occur simultaneously ($P(A \cap B) = P(A)P(B)$), their total information content must be strictly additive: $I(A \cap B) = I(A) + I(B)$. The unique continuous mathematical function satisfying $f(x \cdot y) = f(x) + f(y)$ is the logarithm. Hence, the **Shannon self-information** of an event x is defined as:

$$I(x) \triangleq -\log_2 P(x) = \log_2 \frac{1}{P(x)}. \tag{2.51}$$

When base-2 logarithm is used, self-information is measured in **bits** (or *shannons*). When natural logarithm ln is used, it is measured in **nats**.

Shannon Entropy. The **Shannon Entropy** $H(X)$ measures the expected information content, or average uncertainty, inherent in a random variable X:

$$H(X) \triangleq \mathbb{E}_{x\sim P}[I(x)] = -\sum_{x\in\mathcal{X}} P(x)\log_2 P(x), \tag{2.52}$$

with the continuous limit convention that $0\log_2 0 = \lim_{p\to 0^+} p\log_2 p = 0$. Shannon's Source Coding Theorem establishes that $H(X)$ is the fundamental mathematical lower bound on the average number of bits required to losslessly compress and transmit symbols drawn from distribution P.

- For a deterministic variable ($P(x_1) = 1$): $H(X) = 0$ bits (zero uncertainty).
- For a uniform distribution over K outcomes ($P(x_k) = \frac{1}{K}$): $H(X) = \log_2 K$ bits, achieving maximal possible entropy.

Kullback-Leibler (KL) Divergence and Relative Entropy. Suppose a physical process generates data according to a true target distribution $P(x)$, but we approximate or model it using an empirical distribution $Q(x)$. How much extra information (or coding inefficiency) is introduced by using Q instead of P?

This statistical distance is quantified by the **Kullback-Leibler (KL) Divergence** (or **Relative Entropy**) [54]:

$$D_{\mathrm{KL}}(P \parallel Q) \triangleq \sum_{x\in\mathcal{X}} P(x)\log\frac{P(x)}{Q(x)} = \mathbb{E}_{x\sim P}\left[\log P(x) - \log Q(x)\right]. \tag{2.53}$$

Theorem 2.2 (Gibbs' Inequality: Non-Negativity of KL Divergence)**.** *For any two discrete probability distributions P and Q on $\mathcal{X}$:*

$$D_{KL}(P \parallel Q) \geq 0, \tag{2.54}$$

with equality $D_{KL}(P \parallel Q) = 0$ if and only if $P(x) = Q(x)$ for all $x \in \mathcal{X}$.

Proof via Jensen's Inequality. Recall Jensen's inequality: for any strictly convex function f and random variable Z, $\mathbb{E}[f(Z)] \geq f(\mathbb{E}[Z])$. Because $f(t) = -\ln t$ is strictly convex on $(0, \infty)$, we write:

$$\begin{aligned} D_{\mathrm{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \left(-\ln \frac{Q(x)}{P(x)} \right) = \mathbb{E}_{x \sim P} \left[-\ln \frac{Q(x)}{P(x)} \right] \\ &\geq -\ln \left(\mathbb{E}_{x \sim P} \left[\frac{Q(x)}{P(x)} \right] \right) = -\ln \left(\sum_{x \in \mathcal{X}} P(x) \frac{Q(x)}{P(x)} \right) \\ &= -\ln \left(\sum_{x \in \mathcal{X}} Q(x) \right) = -\ln(1) = 0. \end{aligned} \tag{2.55}$$

Because $-\ln t$ is strictly convex, equality holds if and only if the ratio $\frac{Q(x)}{P(x)}$ is constant almost everywhere. Since $\sum_x P(x) = \sum_x Q(x) = 1$, the constant must be 1, proving $P(x) = Q(x)$ for all x. □

In oral exams, professors frequently test whether students understand why KL divergence is **not a mathematical distance metric**:

1. **Asymmetry**: $D_{\mathrm{KL}}(P \parallel Q) \neq D_{\mathrm{KL}}(Q \parallel P)$ in general. Optimizing $D_{\mathrm{KL}}(P \parallel Q)$ forces Q to cover all modes of P (zero-avoiding / mean-seeking), whereas optimizing $D_{\mathrm{KL}}(Q \parallel P)$ forces Q to fit inside a single mode of P (zero-forcing / mode-seeking).
2. **Triangle Inequality Failure**: KL divergence does not satisfy $D_{\mathrm{KL}}(P \parallel R) \leq D_{\mathrm{KL}}(P \parallel Q) + D_{\mathrm{KL}}(Q \parallel R)$.

Cross-Entropy and Its Decomposition. The **Cross-Entropy** between target distribution P and model distribution Q is defined as:

$$H(P, Q) \triangleq -\sum_{x \in \mathcal{X}} P(x) \log Q(x) = \mathbb{E}_{x \sim P}[-\log Q(x)]. \tag{2.56}$$

Expanding Equation (2.53) yields the canonical decomposition:

$$\begin{aligned} D_{\mathrm{KL}}(P \parallel Q) &= \sum_x P(x) \log P(x) - \sum_x P(x) \log Q(x) = -H(P) + H(P, Q) \\ \implies H(P, Q) &= H(P) + D_{\mathrm{KL}}(P \parallel Q). \end{aligned} \tag{2.57}$$

When training neural networks with empirical data distribution $P = \tilde{p}_{\mathrm{data}}$, the target data entropy $H(P)$ is a constant independent of network weights $\boldsymbol{\theta}$. Therefore, **minimizing cross-entropy loss is mathematically identical to minimizing the KL divergence $D_{\mathrm{KL}}(P \parallel Q)$ from the empirical data to the model!**

Joint Entropy, Conditional Entropy, and Mutual Information. For a joint ensemble (X, Y), we define:

- **Joint Entropy**: $H(X, Y) \triangleq -\sum_{x,y} P(x, y) \log_2 P(x, y)$.
- **Conditional Entropy**: $H(Y \mid X) \triangleq -\sum_{x,y} P(x, y) \log_2 P(y \mid x) = \mathbb{E}_{x \sim P}[H(Y \mid X = x)]$.
- **Mutual Information**: Quantifies the reduction in uncertainty of X due to observing Y:

$$I(X; Y) \triangleq H(X) - H(X \mid Y) = H(Y) - H(Y \mid X) = D_{\mathrm{KL}}(P(X, Y) \parallel P(X)P(Y)). \tag{2.58}$$

These set-theoretic relationships are visualized geometrically in Figure 2.1.

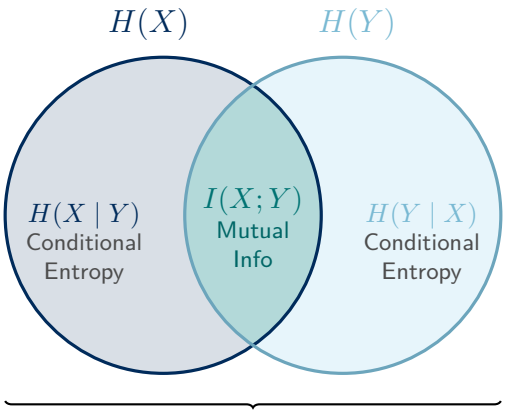


Figure 2.1: The information-theoretic relationship between entropy, conditional entropy, joint entropy, and mutual information. The left circle represents the total entropy $H(X)$, the right circle represents $H(Y)$, their intersection is the mutual information $I(X;Y) = D_{\mathrm{KL}}(P(X,Y) \parallel P(X)P(Y))$, and the combined union represents the joint entropy $H(X,Y)$.

—

2.3 Bayes' rule and independence

2.3.1 Derivation of Bayes' rule

Equating the two symmetric formulations of the product rule from Equation (2.49):

$$P(Y \mid X)P(X) = P(X \mid Y)P(Y), \tag{2.59}$$

and dividing both sides by $P(X)$ produces **Bayes' Rule**:

$$P(Y \mid X) = \frac{P(X \mid Y)P(Y)}{P(X)}. \tag{2.60}$$

Applying the law of total probability to expand the evidence denominator $P(X)$ over all mutually exclusive hypotheses $\{y_1, y_2, \ldots, y_K\} \in \mathcal{Y}$:

$$P(Y = y_k \mid X) = \frac{P(X \mid Y = y_k)P(Y = y_k)}{\sum_{j=1}^{K} P(X \mid Y = y_j)P(Y = y_j)}. \tag{2.61}$$

The four constituents of Bayes' rule define the epistemological vocabulary of statistical inference:

1. **Prior Probability** $P(Y)$: The baseline degree of belief in hypothesis Y **before** observing feature evidence X.
2. **Likelihood** $P(X \mid Y)$: The generative probability that hypothesis Y would produce the observed evidence X.
3. **Evidence (Marginal Likelihood)** $P(X)$: The total probability of observing evidence X, computed by summing the joint probability across all possible hypotheses. It acts as the normalizing constant ensuring the posterior distribution sums to 1.
4. **Posterior Probability** $P(Y \mid X)$: The updated, rational degree of belief in hypothesis Y **after** incorporating evidence X.

2.3.2 Geometric interpretation of Bayes' rule

To develop an intuitive geometric understanding of Bayes' rule, probability updates can be visualized as proportional area slicing within a bounded sample space box [48], as depicted in Figure 2.2. Rather than manipulating abstract algebraic fractions, conditioning on evidence corresponds to restricting our attention to the surviving subset of the sample space and computing relative geometric area ratios.

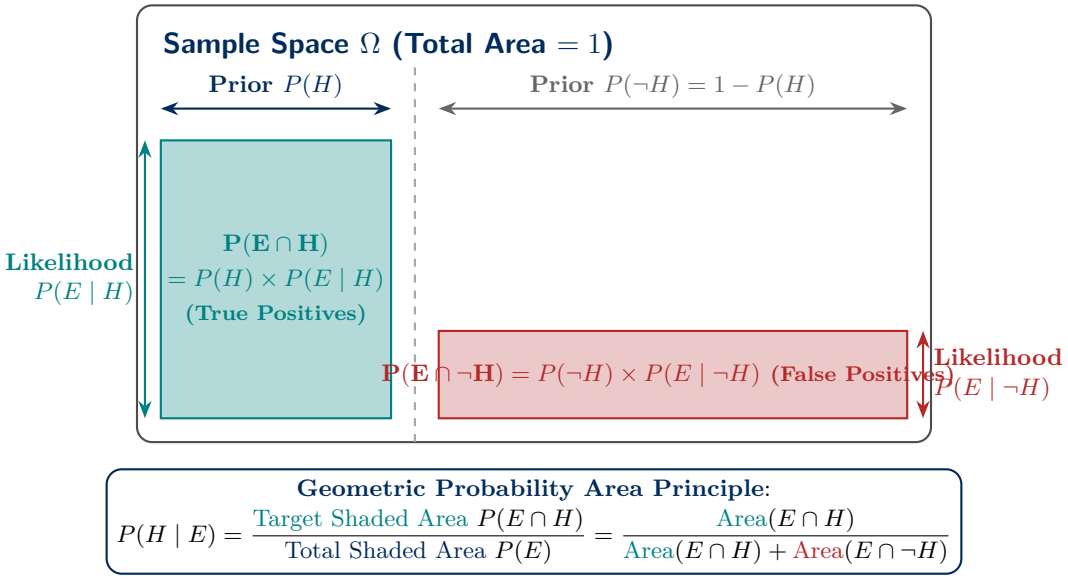


Figure 2.2: The geometric probability area representation of Bayes' Theorem. The sample space Ω is horizontally partitioned into prior degrees of belief $P(H)$ and $P(\neg H)$. Conditioning on evidence E extracts vertical probability bars whose 2D areas equal the joint probabilities $P(E \cap H)$ and $P(E \cap \neg H)$. The posterior probability $P(H \mid E)$ is simply the geometric ratio of the target shaded rectangle to the total shaded area across the entire box.

The geometric mechanics proceed as follows:

1. **Prior Horizontal Slicing**: The width of the box represents the prior probabilities: the left column has width $P(H)$, and the right column has width $P(\neg H) = 1 - P(H)$.

2. **Likelihood Vertical Bars**: Observing evidence E carves out vertical slices whose heights correspond to the conditional likelihoods $P(E \mid H)$ and $P(E \mid \neg H)$.

3. **Joint Areas**: The physical 2D area of each shaded rectangle represents joint probability:

$$\text{Area}_{\text{left}} = P(H) \times P(E \mid H) = P(E \cap H), \quad \text{Area}_{\text{right}} = P(\neg H) \times P(E \mid \neg H) = P(E \cap \neg H). \tag{2.62}$$

4. **Evidence Normalization**: The total shaded area across both columns is the marginal evidence $P(E) = \text{Area}_{\text{left}} + \text{Area}_{\text{right}}$.

5. **Posterior Ratio**: The posterior probability $P(H \mid E)$ is the fraction of the **total shaded area** that lies in the left (H) column. Even if the likelihood $P(E \mid H)$ is high (tall bar), if the prior $P(H)$ is minuscule (very narrow column), the right shaded area (false positives) can easily dwarf the left shaded area, explaining why rare disease screenings often produce low posterior true-positive rates.

2.3.3 The Monty Hall problem

To demonstrate how counter-intuitive probabilistic reasoning can be when intuition conflicts with rigorous Bayes conditioning, consider the famous **Monty Hall Problem** (dramatized in the movie *21*, where MIT blackjack students exploit basic statistics to win in Las Vegas).

Problem setup. A game show contestant is presented with three closed doors: A, B, and C.

- Behind one door is a valuable sports car (the prize).
- Behind the other two doors are goats (no prize).

The game proceeds in three stages:

1. The contestant chooses a door (without loss of generality, let the contestant choose **Door A**).
2. The host (Monty Hall), who knows what is behind every door, opens one of the two remaining doors (Door B or Door C) to reveal a goat. Suppose the host opens **Door C**, showing a goat.
3. The host then offers the contestant a choice: *"Do you want to stick with your original Door A, or do you want to switch to Door B?"*

Should the contestant stick, switch, or does it make no difference? Most people assume that since two closed doors remain, the probability is 50/50. This intuition is mathematically false: **switching doubles your probability of winning from** $1/3$ **to** $2/3$**!**

Rigorous Bayesian derivation. Let random variable $C_{\text{prize}} \in \{A, B, C\}$ denote the door hiding the car. The prior probabilities are uniform:

$$P(A) = P(B) = P(C) = \frac{1}{3}. \tag{2.63}$$

Suppose the contestant chooses Door A. Let the observed evidence event be H_C: *"The host opens Door C"*.

We now compute the likelihood $P(H_C \mid C_{\text{prize}})$ under each possible location of the car:

- If the car is behind **Door A** ($C_{\text{prize}} = A$): The host can open either Door B or Door C without revealing the car. Assuming the host picks randomly between them:

$$P(H_C \mid A) = \frac{1}{2}. \tag{2.64}$$

- If the car is behind **Door B** ($C_{\text{prize}} = B$): The host cannot open Door A (chosen by contestant) and cannot open Door B (car is there). The host is **forced** to open Door C:

$$P(H_C \mid B) = 1. \tag{2.65}$$

- If the car is behind **Door C** ($C_{\text{prize}} = C$): The host cannot open Door C because the car is behind it:

$$P(H_C \mid C) = 0. \tag{2.66}$$

Now, compute the total marginal probability of evidence H_C using the law of total probability:

$$\begin{aligned} P(H_C) &= P(H_C \mid A)P(A) + P(H_C \mid B)P(B) + P(H_C \mid C)P(C) \\ &= \left(\frac{1}{2} \times \frac{1}{3}\right) + \left(1 \times \frac{1}{3}\right) + \left(0 \times \frac{1}{3}\right) = \frac{1}{6} + \frac{1}{3} = \frac{3}{6} = \frac{1}{2}. \end{aligned} \tag{2.67}$$

Finally, apply Bayes' rule to compute the posterior probabilities:

$$P(\text{Stick}) = P(A \mid H_C) = \frac{P(H_C \mid A)P(A)}{P(H_C)} = \frac{\frac{1}{2} \times \frac{1}{3}}{\frac{1}{2}} = \frac{1}{3}, \tag{2.68}$$

$$P(\text{Switch}) = P(B \mid H_C) = \frac{P(H_C \mid B)P(B)}{P(H_C)} = \frac{1 \times \frac{1}{3}}{\frac{1}{2}} = \frac{2}{3}. \tag{2.69}$$

Because $P(B \mid H_C) = \frac{2}{3} = 2 \times P(A \mid H_C)$, the optimal rational strategy is **always to switch**. The host's action transfers all the unallocated probability mass from the unchosen doors into Door B.

2.3.4 Conditional independence and confounding

Unconditional statistical independence. Two random variables X and Y are **statistically independent** ($X \perp Y$) if and only if their joint distribution factorizes into the product of their marginals:

$$P(X, Y) = P(X)P(Y) \iff P(X \mid Y) = P(X). \tag{2.70}$$

- *Independent Example*: Winning the national lottery this week versus winning it next week. The two draws are physically uncorrelated; knowing the result of one provides zero information about the other.
- *Dependent Example*: Russian roulette. In a revolver cylinder with 6 chambers and 1 bullet, the probability of firing a blank on trigger pull 1 is 5/6. If you survive pull 1, the probability of firing the bullet on pull 2 increases to 1/5, because the sample space of remaining chambers contracted from 6 to 5 without spinning. The trials are statistically dependent.

Conditional independence. Two random variables X and Y are **conditionally independent** given a third variable Z ($X \perp Y \mid Z$) if and only if:

$$P(X, Y \mid Z) = P(X \mid Z)P(Y \mid Z) \iff P(X \mid Y, Z) = P(X \mid Z). \tag{2.71}$$

Once Z is known, learning Y provides no additional information about X.

The Russell & Norvig dental clinic domain. Consider three variables: Toothache (T), Cavity (C), and the dentist's metal probe Catch (M) [81]. Toothache and Catch are strongly correlated: patients with toothaches are much more likely to have the probe catch on a tooth.

However, they are **conditionally independent given Cavity**:

$$P(T, M \mid C) = P(T \mid C)P(M \mid C). \tag{2.72}$$

If the dentist directly inspects the tooth and confirms a cavity is present ($C = \text{true}$), whether the probe catches depends strictly on the physical shape of the cavity, not on whether the patient feels pain. The latent cause C completely explains the correlation between the two symptoms.

The London taxi driver confounding mystery. The societal danger of ignoring conditional independence is illustrated by an empirical study in London:

- **Observed Correlation**: Traffic safety analysts observed a statistically significant positive correlation between London taxi drivers wearing heavy winter coats (C) and the occurrence of vehicle traffic accidents (A): $P(A \mid C) > P(A \mid \neg C)$.
- **Naive Policy Reaction**: Transport regulators considered a municipal decree banning taxi drivers from wearing coats while driving, hypothesizing that coats restricted arm mobility and caused accidents!

- **The Confounding Reality**: The true underlying causal variable was **Rain / Sleet Weather** (R). On rainy days, roads become slick and braking distances double (causing accidents), while temperatures drop (causing drivers to wear coats).

Wearing a coat and having an accident are **conditionally independent given rain**:

$$P(A, C \mid R) = P(A \mid R)P(C \mid R). \tag{2.73}$$

Conditioning on the true confounding variable R dissolves the spurious correlation, proving that banning coats would have had zero effect on road safety.

Graphical Causal Topologies and d-Separation. In probabilistic graphical models and causal machine learning [6, 81], conditional independence relationships are formalized through three fundamental network motifs:

1. **The Fork (Common Cause / Confounder)**: $X \leftarrow Z \rightarrow Y$. Variable Z is a common cause of both X and Y. In the absence of knowledge about Z, the path is open: X and Y are correlated ($X \not\perp Y$). However, conditioning on the common cause Z **blocks the causal path**, rendering X and Y conditionally independent ($X \perp Y \mid Z$). This directly formalizes the London taxi driver phenomenon (where Z is rainy weather).

2. **The Chain (Causal Mediation)**: $X \rightarrow Z \rightarrow Y$. Variable X causally influences Z, which in turn influences Y. The marginal path is open ($X \not\perp Y$). However, once intermediate mediator Z is fully observed, all information transmitted from X to Y is mediated through Z. Hence, conditioning on Z blocks the path: $X \perp Y \mid Z$.

3. **The Collider (V-structure / Berkson's Paradox)**: $X \rightarrow Z \leftarrow Y$. Both X and Y independently influence common effect Z. Marginally, X and Y are completely independent ($X \perp Y$). However, **conditioning on the collider Z opens the path**, creating an artificial dependence between X and Y ($X \not\perp Y \mid Z$). This counter-intuitive phenomenon is known as **explaining away** or *Berkson's paradox*: observing that a student was accepted into an elite university ($Z = 1$) given that their grades are low ($X = 0$) heavily implies their athletic skill must be extraordinarily high ($Y = 1$).

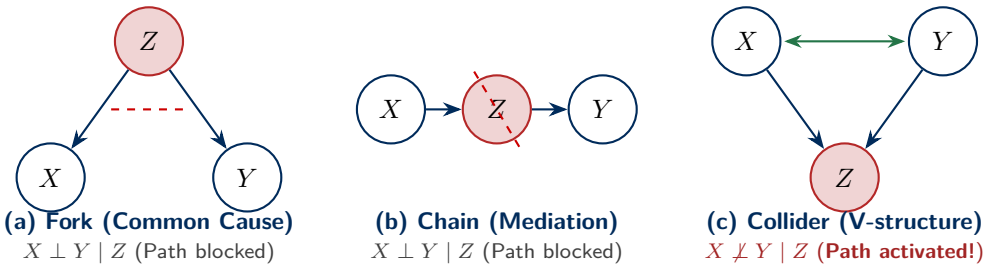


Figure 2.3: The three canonical causal graphical topologies governing conditional independence and d-separation. Shaded red nodes indicate observed conditioning evidence Z. In (a) the Fork and (b) the Chain, conditioning on Z blocks the path, producing conditional independence $X \perp Y \mid Z$. In (c) the Collider, the path is marginally blocked, but conditioning on Z unblocks the path, inducing dependence $X \not\perp Y \mid Z$ (explaining away / Berkson's paradox).

—

2.4 Naïve Bayes and estimation

We now construct our first complete, formal probabilistic machine learning classifier: the **Naïve Bayes Classifier**.

2.4.1 Model-based classification

Let each training instance be represented by a feature vector of d attributes:

$$\mathbf{x} = [x_1, x_2, \ldots, x_d]^\top \in \mathcal{X}, \tag{2.74}$$

and let the target label be a discrete categorical variable $y \in \mathcal{Y} = \{c_1, c_2, \ldots, c_K\}$.

Given an observed feature vector $\mathbf{x}$, the optimal probabilistic decision assigns $\mathbf{x}$ to the class that maximizes the posterior probability (the Maximum A Posteriori, or MAP, decision rule):

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} P(y \mid \mathbf{x}) = \arg\max_{y \in \mathcal{Y}} \frac{P(\mathbf{x} \mid y)P(y)}{P(\mathbf{x})}. \tag{2.75}$$

Because the evidence denominator $P(\mathbf{x}) = \sum_y P(\mathbf{x} \mid y)P(y)$ is identical for all candidate classes y, it does not alter the arg max ranking. We can safely discard it:

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} P(\mathbf{x} \mid y)P(y). \tag{2.76}$$

2.4.2 The Naïve Bayes assumption

Estimating the prior $P(y)$ from training data is trivial: we simply count class frequencies $P(y = c) \approx \frac{N_c}{N}$.

However, estimating the class-conditional joint likelihood $P(\mathbf{x} \mid y) = P(x_1, x_2, \ldots, x_d \mid y)$ directly is computationally and statistically impossible in high dimensions:

- If all d features are binary ($x_j \in \{0, 1\}$), there are 2^d possible feature combinations for each of the K classes.
- To populate this joint likelihood table directly without assumptions requires estimating:

$$K \times (2^d - 1) = O(K \cdot 2^d) \quad \text{parameters!} \tag{2.77}$$

  For $d = 30$ features and $K = 2$ classes, $2 \times 2^{30} \approx 2.14 \times 10^9$ parameters—vastly exceeding the number of training samples in any realistic dataset.

The **Naïve Bayes assumption** resolves this exponential bottleneck by making a radical, simplifying architectural assumption: **All features $x_1, x_2, \ldots, x_d$ are conditionally independent given the class label y**:

$$P(\mathbf{x} \mid y) = P(x_1, x_2, \ldots, x_d \mid y) = \prod_{j=1}^{d} P(x_j \mid y). \tag{2.78}$$

Substituting Equation (2.78) into the MAP decision rule yields the **Naïve Bayes Classifier**:

$$\hat{y}_{\text{NB}} = \arg\max_{y \in \mathcal{Y}} P(y) \prod_{j=1}^{d} P(x_j \mid y). \tag{2.79}$$

Exponential parameter reduction. Under the Naïve Bayes assumption:

- Instead of estimating full joint distributions, we estimate only d one-dimensional marginal conditionals $P(x_j \mid y)$ for each class.
- For binary features, each 1D distribution requires only 1 parameter, collapsing the total parameter count from exponential to linear:

$$O(K \cdot 2^d) \xrightarrow{\text{Naïve Bayes Assumption}} O(K \cdot d). \tag{2.80}$$

  For $d = 30$ and $K = 2$, parameter complexity drops from 2×10^9 down to just $2 \times 30 = 60$ parameters!

2.4.3 EnjoySport dataset walkthrough

To understand how Naïve Bayes trains and evaluates in practice, consider the classic benchmark from Tom Mitchell's curriculum [65]: predicting whether a weekend picnic spot is enjoyable ($y \in \{\text{Yes}, \text{No}\}$) based on $d = 6$ environmental features:

$$\mathbf{x} = [\text{Sky}, \text{Temp}, \text{Humidity}, \text{Wind}, \text{Water}, \text{Forecast}]^\top. \tag{2.81}$$

Table 2.3 presents a representative training dataset of $N = 14$ instances.

Table 2.3: The EnjoySport / Picnic training dataset ($N = 14$).

Day	**Sky**	**AirTemp**	**Humidity**	**Wind**	**Water**	**Forecast**	**EnjoySport (y)**
D1	Sunny	Warm	Normal	Strong	Warm	Same	**Yes**
D2	Sunny	Warm	High	Strong	Warm	Same	**Yes**
D3	Rainy	Cold	High	Strong	Warm	Change	**No**
D4	Sunny	Warm	High	Strong	Cool	Change	**Yes**
D5	Sunny	Warm	Normal	Weak	Warm	Same	**No**
D6	Rainy	Warm	Normal	Strong	Cool	Same	**No**
D7	Overcast	Cold	High	Strong	Cool	Change	**Yes**
D8	Sunny	Cold	High	Strong	Warm	Same	**No**
D9	Sunny	Warm	Normal	Strong	Cool	Same	**Yes**
D10	Rainy	Warm	High	Strong	Warm	Change	**No**
D11	Overcast	Warm	High	Strong	Warm	Same	**Yes**
D12	Overcast	Cold	Normal	Strong	Cool	Same	**Yes**
D13	Overcast	Warm	High	Weak	Warm	Same	**Yes**
D14	Rainy	Cold	High	Strong	Warm	Change	**No**

Phase 1: Estimating prior probabilities. From the $N = 14$ days, we count $N_{\text{Yes}} = 8$ and $N_{\text{No}} = 6$:

$$P(y = \text{Yes}) = \frac{8}{14} \approx 0.571, \quad P(y = \text{No}) = \frac{6}{14} \approx 0.429. \tag{2.82}$$

Phase 2: Estimating class-conditional likelihoods. We count attribute occurrences within each class subset:

- Sky attribute:

$$P(\text{Sky} = \text{Sunny} \mid \text{Yes}) = 3/8, \quad P(\text{Sky} = \text{Sunny} \mid \text{No}) = 2/6, \tag{2.83}$$
$$P(\text{Sky} = \text{Overcast} \mid \text{Yes}) = 4/8, \quad P(\text{Sky} = \text{Overcast} \mid \text{No}) = 0/6, \tag{2.84}$$
$$P(\text{Sky} = \text{Rainy} \mid \text{Yes}) = 1/8, \quad P(\text{Sky} = \text{Rainy} \mid \text{No}) = 4/6. \tag{2.85}$$

- Wind attribute:

$$P(\text{Wind} = \text{Strong} \mid \text{Yes}) = 7/8, \quad P(\text{Wind} = \text{Strong} \mid \text{No}) = 5/6, \tag{2.86}$$
$$P(\text{Wind} = \text{Weak} \mid \text{Yes}) = 1/8, \quad P(\text{Wind} = \text{Weak} \mid \text{No}) = 1/6. \tag{2.87}$$

Phase 3: Classifying a novel test instance. Suppose a new weekend arrives with conditions:

$$\mathbf{x}^* = [\text{Sky} = \text{Sunny}, \text{AirTemp} = \text{Cold}, \text{Humidity} = \text{High}, \text{Wind} = \text{Strong}, \text{Water} = \text{Warm}, \text{Forecast} = \text{Same}]^\top. \tag{2.88}$$

We compute the unnormalized posterior scores:

$$\text{Score}(\text{Yes}) = P(\text{Yes}) \prod_{j=1}^{6} P(x_j^* \mid \text{Yes}) = \frac{8}{14} \times \left(\frac{3}{8} \times \frac{2}{8} \times \frac{5}{8} \times \frac{7}{8} \times \frac{5}{8} \times \frac{6}{8}\right) \approx 0.00618, \tag{2.89}$$

$$\text{Score}(\text{No}) = P(\text{No}) \prod_{j=1}^{6} P(x_j^* \mid \text{No}) = \frac{6}{14} \times \left(\frac{2}{6} \times \frac{3}{6} \times \frac{5}{6} \times \frac{5}{6} \times \frac{4}{6} \times \frac{3}{6}\right) \approx 0.01029. \tag{2.90}$$

Because Score(No) > Score(Yes), Naïve Bayes predicts $\hat{y} =$ **No** (the picnic spot will not be enjoyable). Normalizing yields $P(\text{No} \mid \mathbf{x}^*) = \frac{0.01029}{0.01029+0.00618} \approx 62.5\%$.

2.4.4 Laplace smoothing

The catastrophic zero-frequency flaw. Consider what happens if we attempt to classify a test day where Sky = Overcast under the class $y =$ No. Looking at Table 2.3, an overcast sky was never observed among the days where $y =$ No. Thus, the maximum likelihood estimate evaluates to:

$$P(\text{Sky} = \text{Overcast} \mid y = \text{No}) = \frac{0}{6} = 0. \tag{2.91}$$

Because the Naïve Bayes score is a multiplicative product across all attributes, a single zero probability completely destroys all other evidence:

$$\text{Score}(\text{No}) = P(\text{No}) \times 0 \times \prod_{j=2}^{d} P(x_j^* \mid \text{No}) \equiv 0! \tag{2.92}$$

Even if all five other attributes overwhelmingly indicate $y =$ No, the model is mathematically prohibited from predicting No, solely because a specific attribute value was missing in the finite training sample.

The mathematical fix: Laplace (Add-1) smoothing. To eliminate the zero-frequency catastrophe, we apply **Laplace Smoothing** (also known as Add-1 smoothing). We add a pseudo-count of 1 to every possible attribute outcome, adjusting the denominator by the cardinality $|V_j|$ (the number of distinct discrete values attribute j can assume):

$$\hat{P}_{\text{Laplace}}(x_j = v \mid y = c) = \frac{\text{count}(x_j = v, y = c) + 1}{\text{count}(y = c) + |V_j|}. \tag{2.93}$$

Notice that the probabilities remain strictly normalized:

$$\sum_{v \in V_j} \hat{P}_{\text{Laplace}}(x_j = v \mid y = c) = \frac{\sum_v \text{count}(x_j = v, y = c) + |V_j|}{\text{count}(y = c) + |V_j|} = \frac{\text{count}(y = c) + |V_j|}{\text{count}(y = c) + |V_j|} = 1. \tag{2.94}$$

For the overcast attribute ($|V_{\text{Sky}}| = 3$ for Sunny, Overcast, Rainy):

$$\hat{P}_{\text{Laplace}}(\text{Sky} = \text{Overcast} \mid \text{No}) = \frac{0+1}{6+3} = \frac{1}{9} > 0. \tag{2.95}$$

The probability is no longer zero, preserving the influence of all other feature evidence.

A generalized version is **Lidstone Smoothing** (Add-k smoothing), where pseudo-count $\alpha > 0$ is a tunable hyperparameter:

$$\hat{P}_{\alpha}(x_j = v \mid y = c) = \frac{\text{count}(x_j = v, y = c) + \alpha}{\text{count}(y = c) + \alpha|V_j|}. \tag{2.96}$$

2.4.5 Generative versus discriminative models

Naïve Bayes exemplifies the **generative learning paradigm**:

- **Generative Classifiers** (e.g., Naïve Bayes, Gaussian Mixture Models, Variational Autoencoders): Model the joint distribution:

$$P(\mathbf{x}, y) = P(y)P(\mathbf{x} \mid y). \tag{2.97}$$

  A generative model learns how each class generates its observed features. It can not only classify inputs, but can also generate synthetic samples $\mathbf{x} \sim P(\mathbf{x} \mid y)$.

- **Discriminative Classifiers** (e.g., Logistic Regression, Support Vector Machines, Deep Neural Networks): Model the posterior class boundary directly:

$$P(y \mid \mathbf{x}). \tag{2.98}$$

  A discriminative model bypasses the difficult task of modeling the feature distribution $P(\mathbf{x})$, focusing all its parameter capacity on learning the decision boundary separating classes.

2.4.6 MLE and MAP estimation

When training probabilistic classifiers such as Naïve Bayes, how do we estimate the numerical parameters $\boldsymbol{\theta}$ (e.g., class priors $P(y)$ and feature probabilities $P(x_j \mid y)$) from an observed finite training dataset $\mathcal{D} = \{\mathbf{x}_i, y_i\}_{i=1}^N$?

In statistical learning, two competing philosophical and mathematical frameworks exist: **Maximum Likelihood Estimation (MLE)** and **Maximum A Posteriori (MAP)**.

1. The Philosophical Foundations: Frequentist vs. Bayesian.

- **The Frequentist / MLE Paradigm**: In the frequentist framework, the true parameter vector $\boldsymbol{\theta}^*$ is a **fixed, deterministic, but unknown constant of nature**. It is not a random variable, and writing a probability distribution over parameters ($p(\boldsymbol{\theta})$) is considered philosophically meaningless. Probability is interpreted strictly as the long-run empirical frequency of repeatable events.

  The objective of Maximum Likelihood Estimation is to find the single parameter vector $\hat{\boldsymbol{\theta}}_{\text{MLE}}$ that makes the observed training sample $\mathcal{D}$ most probable under the model:

$$\hat{\boldsymbol{\theta}}_{\text{MLE}} \triangleq \arg\max_{\boldsymbol{\theta}} p(\mathcal{D} \mid \boldsymbol{\theta}) = \arg\max_{\boldsymbol{\theta}} \ln p(\mathcal{D} \mid \boldsymbol{\theta}). \tag{2.99}$$

- **The Bayesian / MAP Paradigm**: In the Bayesian framework, parameters $\boldsymbol{\theta}$ are treated as **random variables** endowed with epistemic uncertainty. Before observing any data, the practitioner specifies a **prior distribution** $p(\boldsymbol{\theta})$ reflecting background knowledge, physical constraints, or inductive biases.

  As training data $\mathcal{D}$ arrives, Bayes' rule is applied to compute the full **posterior distribution**:

$$p(\boldsymbol{\theta} \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})} = \frac{p(\mathcal{D} \mid \boldsymbol{\theta})p(\boldsymbol{\theta})}{\int p(\mathcal{D} \mid \boldsymbol{\theta}')p(\boldsymbol{\theta}')\, d\boldsymbol{\theta}'}. \tag{2.100}$$

  Because computing the marginal evidence integral in the denominator is often analytically intractable, Maximum A Posteriori (MAP) simplifies full Bayesian inference by seeking the **mode** (the single most probable point) of the posterior distribution:

$$\hat{\boldsymbol{\theta}}_{\text{MAP}} \triangleq \arg\max_{\boldsymbol{\theta}} p(\boldsymbol{\theta} \mid \mathcal{D}) = \arg\max_{\boldsymbol{\theta}} \left[\ln p(\mathcal{D} \mid \boldsymbol{\theta}) + \ln p(\boldsymbol{\theta})\right]. \tag{2.101}$$

2. How and Why the Likelihood and Parameter Formulas Change. To understand the exact mathematical mechanism distinguishing MLE from MAP, consider a concrete binary feature (e.g., $x_j \in \{0, 1\}$ or coin flipping) governed by a Bernoulli distribution with parameter $\theta = P(x_j = 1 \mid y)$. Suppose in our training data for class y, we observe N instances, of which k have $x_j = 1$ and $N - k$ have $x_j = 0$.

1. **The MLE Derivation**: Under the i.i.d. assumption, the data likelihood is:

$$L(\theta) = p(\mathcal{D} \mid \theta) = \theta^k (1 - \theta)^{N-k}. \tag{2.102}$$

Taking the natural logarithm yields the log-likelihood:

$$\ln L(\theta) = k \ln \theta + (N - k) \ln(1 - \theta). \tag{2.103}$$

To find the maximum, we compute the first derivative with respect to θ and set it to zero:

$$\frac{d \ln L(\theta)}{d\theta} = \frac{k}{\theta} - \frac{N - k}{1 - \theta} = 0 \implies k(1 - \theta) = (N - k)\theta \implies k = N\theta. \tag{2.104}$$

Solving for θ yields the standard **Maximum Likelihood Estimator**:

$$\hat{\theta}_{\text{MLE}} = \frac{k}{N}. \tag{2.105}$$

Notice the severe failure mode: if an event is rare and happens not to appear in the sample ($k = 0$), $\hat{\theta}_{\text{MLE}} = 0$. This triggers the zero-frequency catastrophe in Naïve Bayes.

2. **The MAP Derivation with a Conjugate Beta Prior**: In the MAP framework, we choose a conjugate prior for the Bernoulli likelihood: the **Beta distribution**:

$$p(\theta) = \text{Beta}(\theta; \alpha, \beta) = \frac{1}{\text{B}(\alpha, \beta)} \theta^{\alpha-1} (1 - \theta)^{\beta-1}, \quad \alpha > 0,\ \beta > 0, \tag{2.106}$$

where α and β are hyperparameters representing **virtual pseudocounts** (prior belief that we have previously observed $\alpha - 1$ positive outcomes and $\beta - 1$ negative outcomes before seeing the data).

The posterior distribution is proportional to likelihood times prior:

$$p(\theta \mid \mathcal{D}) \propto \left[\theta^k (1 - \theta)^{N-k}\right] \times \left[\theta^{\alpha-1} (1 - \theta)^{\beta-1}\right] = \theta^{k+\alpha-1} (1 - \theta)^{N-k+\beta-1}. \tag{2.107}$$

Notice that the posterior is *another Beta distribution*: $\text{Beta}(\theta; k + \alpha, N - k + \beta)$. This closure property defines a **conjugate prior**.

Taking the log-posterior objective $\mathcal{J}_{\text{MAP}}(\theta)$:

$$\mathcal{J}_{\text{MAP}}(\theta) = (k + \alpha - 1) \ln \theta + (N - k + \beta - 1) \ln(1 - \theta) + \text{constant}. \tag{2.108}$$

Differentiating with respect to θ and setting to zero:

$$\frac{d\mathcal{J}_{\text{MAP}}(\theta)}{d\theta} = \frac{k + \alpha - 1}{\theta} - \frac{N - k + \beta - 1}{1 - \theta} = 0. \tag{2.109}$$

Cross-multiplying:

$$(k + \alpha - 1)(1 - \theta) = (N - k + \beta - 1)\theta \implies k + \alpha - 1 = (N + \alpha + \beta - 2)\theta. \tag{2.110}$$

Solving for θ yields the **Maximum A Posteriori Estimator**:

$$\hat{\theta}_{\text{MAP}} = \frac{k + \alpha - 1}{N + \alpha + \beta - 2}. \tag{2.111}$$

Why the Formula Changes: The Prior as an Additive Pseudocount Regularizer. Comparing Equation (2.105) with Equation (2.111) reveals the exact mathematical origin of smoothing:

- If we select a uniform, completely flat prior ($\alpha = 1, \beta = 1$), the prior density is constant $p(\theta) = 1$. The MAP formula reduces exactly to the MLE formula:

$$\hat{\theta}_{\text{MAP}} = \frac{k + 1 - 1}{N + 1 + 1 - 2} = \frac{k}{N} = \hat{\theta}_{\text{MLE}}. \tag{2.112}$$

- If we select Laplace's symmetric prior $\alpha = 2, \beta = 2$ (representing the assumption of having observed 1 virtual positive and 1 virtual negative sample):

$$\hat{\theta}_{\text{MAP}} = \frac{k + 2 - 1}{N + 2 + 2 - 2} = \frac{k + 1}{N + 2}. \tag{2.113}$$

  Laplace smoothing is mathematically identical to MAP estimation under a $\text{Beta}(2, 2)$ **prior!**

Generalization to Categorical Variables and Dirichlet Priors. For general multi-valued categorical features $x_j \in \{v_1, \ldots, v_M\}$ with counts $N_1, \ldots, N_M$ ($\sum_{m=1}^{M} N_m = N$), the conjugate prior to the Multinomial / Categorical likelihood is the **Dirichlet distribution** $\text{Dir}(\theta_1, \ldots, \theta_M; \alpha_1, \ldots, \alpha_M) \propto \prod_{m=1}^{M} \theta_m^{\alpha_m - 1}$.

Following the identical derivation yields:

$$\hat{\theta}_m^{\text{MLE}} = \frac{N_m}{N}, \qquad \hat{\theta}_m^{\text{MAP}} = \frac{N_m + \alpha_m - 1}{N + \sum_{j=1}^{M} \alpha_j - M}. \tag{2.114}$$

Setting symmetric pseudocounts $\alpha_m = 2$ strictly recovers the general Laplace estimator in Equation (2.93):

$$\hat{\theta}_m^{\text{MAP}} = \frac{N_m + 1}{N + M}. \tag{2.115}$$

Asymptotic Convergence: The Domination of Data. Notice what happens to the MAP estimator as the training dataset size $N \to \infty$:

$$\lim_{N \to \infty} \hat{\theta}_{\text{MAP}} = \lim_{N \to \infty} \frac{k + \alpha - 1}{N + \alpha + \beta - 2} = \lim_{N \to \infty} \frac{k/N + (\alpha - 1)/N}{1 + (\alpha + \beta - 2)/N} = \frac{k}{N} = \hat{\theta}_{\text{MLE}}. \tag{2.116}$$

This establishes a universal principle of statistical estimation: **as sample size grows, the empirical data likelihood** $\mathcal{O}(N)$ **overwhelms the static prior** $\mathcal{O}(1)$**, and the MAP estimate converges asymptotically to the MLE estimate.** The prior protects against variance in small samples, while allowing empirical reality to dictate parameters in large samples.

2.4.7 Gaussian Naïve Bayes

When input features are continuous real numbers $\mathbf{x} \in \mathbb{R}^d$ (e.g., patient blood pressure, temperatures, acoustic frequency spectra), the discrete frequency counting of Section 2.4.3 cannot be applied directly because the probability of any exact real number is zero ($P(X = x) = 0$).

To handle continuous attributes, **Gaussian Naïve Bayes** assumes that within each class $y = c$, each feature x_j is distributed according to a one-dimensional Gaussian (Normal) density:

$$p(x_j \mid y = c) = \frac{1}{\sqrt{2\pi\sigma_{cj}^2}} \exp\left(-\frac{(x_j - \mu_{cj})^2}{2\sigma_{cj}^2} \right), \tag{2.117}$$

parameterized by class-conditional mean μ_{cj} and variance σ_{cj}^2.

Under the Naïve Bayes conditional independence assumption, the joint class likelihood is:

$$p(\mathbf{x} \mid y = c) = \prod_{j=1}^{d} p(x_j \mid y = c) = \prod_{j=1}^{d} \frac{1}{\sqrt{2\pi\sigma_{cj}^2}} \exp\left(-\frac{(x_j - \mu_{cj})^2}{2\sigma_{cj}^2}\right). \tag{2.118}$$

Taking the natural logarithm:

$$\ln p(\mathbf{x} \mid y = c) = -\frac{1}{2}\sum_{j=1}^{d} \ln(2\pi\sigma_{cj}^2) - \sum_{j=1}^{d} \frac{(x_j - \mu_{cj})^2}{2\sigma_{cj}^2}. \tag{2.119}$$

Given training instances $\mathcal{D}_c = \{\mathbf{x}_i : y_i = c\}$ of size N_c, the Maximum Likelihood parameters are computed in closed form:

$$\hat{\mu}_{cj} = \frac{1}{N_c}\sum_{i:y_i=c} x_{ij}, \qquad \hat{\sigma}_{cj}^2 = \frac{1}{N_c}\sum_{i:y_i=c} (x_{ij} - \hat{\mu}_{cj})^2. \tag{2.120}$$

The decision rule assigns $\mathbf{x}$ to the class maximizing $\ln P(y = c) + \ln p(\mathbf{x} \mid y = c)$.

2.4.8 From Naïve Bayes to LDA and QDA

Gaussian Naïve Bayes models each feature independently. What happens if we remove the conditional independence assumption and allow full multivariate interactions between continuous features?

Consider a generative classifier modeling each class $y = c$ via a full multivariate Gaussian distribution [6]:

$$p(\mathbf{x} \mid y = c) = \frac{1}{(2\pi)^{d/2}|\mathbf{\Sigma}_c|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_c)^\top \mathbf{\Sigma}_c^{-1}(\mathbf{x} - \boldsymbol{\mu}_c)\right), \tag{2.121}$$

where $\boldsymbol{\mu}_c \in \mathbb{R}^d$ is the class mean vector and $\mathbf{\Sigma}_c \in \mathbb{R}^{d\times d}$ is the class covariance matrix.

The shape and geometry of the resulting decision boundary depend entirely on the structural constraints imposed on the covariance matrices $\mathbf{\Sigma}_c$:

1. **Gaussian Naïve Bayes (GNB)**: Assumes all features are conditionally independent. This constrains each $\mathbf{\Sigma}_c$ to be a strictly **diagonal matrix**:

   $$\mathbf{\Sigma}_c = \mathrm{diag}(\sigma_{c1}^2, \sigma_{c2}^2, \ldots, \sigma_{cd}^2). \tag{2.122}$$

   Geometrically, the contours of equal probability density form **axis-aligned orthogonal ellipsoids**. Feature correlations are completely ignored.

2. **Linear Discriminant Analysis (LDA)**: Allows features to be correlated, but assumes that **all classes share an identical, pooled covariance matrix**:

   $$\mathbf{\Sigma}_c = \mathbf{\Sigma} \quad \text{for all } c \in \{1, \ldots, K\}. \tag{2.123}$$

   Now consider the log-posterior odds ratio between class 1 and class 0 in binary classification:

   $$a(\mathbf{x}) \triangleq \ln \frac{P(y = 1 \mid \mathbf{x})}{P(y = 0 \mid \mathbf{x})} = \ln \frac{p(\mathbf{x} \mid y = 1)P(y = 1)}{p(\mathbf{x} \mid y = 0)P(y = 0)}. \tag{2.124}$$

   Substituting Equation (2.121) with shared $\mathbf{\Sigma}$:

   $$\begin{aligned} a(\mathbf{x}) &= -\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_1)^\top \mathbf{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}_1) + \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_0)^\top \mathbf{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}_0) + \ln \frac{P(y = 1)}{P(y = 0)} \\ &= -\frac{1}{2}\left[\mathbf{x}^\top \mathbf{\Sigma}^{-1}\mathbf{x} - 2\boldsymbol{\mu}_1^\top \mathbf{\Sigma}^{-1}\mathbf{x} + \boldsymbol{\mu}_1^\top \mathbf{\Sigma}^{-1}\boldsymbol{\mu}_1\right] \\ &\quad + \frac{1}{2}\left[\mathbf{x}^\top \mathbf{\Sigma}^{-1}\mathbf{x} - 2\boldsymbol{\mu}_0^\top \mathbf{\Sigma}^{-1}\mathbf{x} + \boldsymbol{\mu}_0^\top \mathbf{\Sigma}^{-1}\boldsymbol{\mu}_0\right] + \ln \frac{P(y = 1)}{P(y = 0)}. \end{aligned} \tag{2.125}$$

Notice the remarkable cancellation: **the quadratic terms $\mathbf{x}^\top \boldsymbol{\Sigma}^{-1}\mathbf{x}$ cancel out identically!**

The log-odds reduces to a strictly **linear affine function of $\mathbf{x}$**:

$$a(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + w_0, \tag{2.126}$$

where:

$$\mathbf{w} \triangleq \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0), \qquad w_0 \triangleq -\frac{1}{2}\boldsymbol{\mu}_1^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{\mu}_0^\top \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_0 + \ln \frac{P(y=1)}{P(y=0)}. \tag{2.127}$$

Inverting the log-odds ratio $a = \ln \frac{P}{1-P}$ yields the posterior probability:

$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-a(\mathbf{x})}} = \sigma(\mathbf{w}^\top \mathbf{x} + w_0). \tag{2.128}$$

Linear Discriminant Analysis strictly produces a planar linear decision boundary ($\mathbf{w}^\top \mathbf{x} + w_0 = 0$) and directly recovers the logistic sigmoid functional form of Logistic Regression!

3. **Quadratic Discriminant Analysis (QDA)**: If we permit each class to possess its own distinct, unconstrained covariance matrix $\boldsymbol{\Sigma}_c$, the quadratic terms $\mathbf{x}^\top \boldsymbol{\Sigma}_c^{-1}\mathbf{x}$ do not cancel.

   The decision boundary equation $a(\mathbf{x}) = 0$ contains quadratic forms in $\mathbf{x}$, resulting in a **quadratic hypersurface boundary** (parabolic, hyperbolic, or ellipsoidal). While QDA is much more expressive than LDA, it requires estimating $\mathcal{O}(K \cdot d^2)$ parameters, risking severe overfitting in high dimensions.

Figure 2.4 visualizes the geometric progression from Gaussian Naïve Bayes to LDA and QDA.

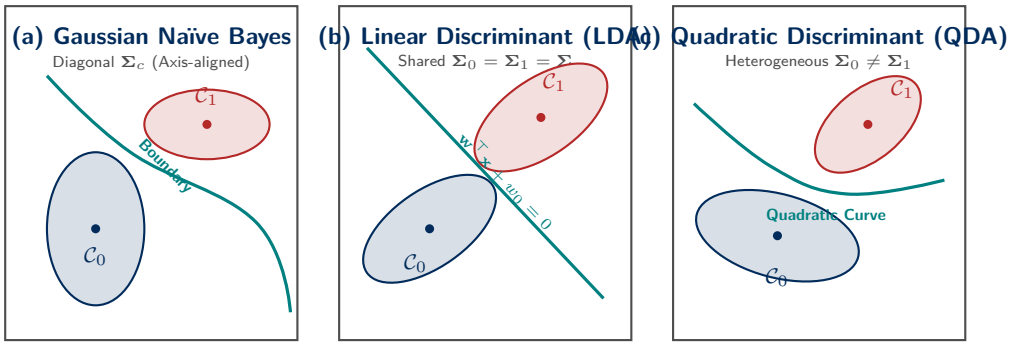


Figure 2.4: The geometric continuum of Gaussian generative classifiers. (a) Gaussian Naïve Bayes constrains covariance matrices to be diagonal, producing axis-aligned orthogonal density ellipses. (b) Linear Discriminant Analysis (LDA) allows correlations but enforces a shared covariance matrix $\boldsymbol{\Sigma}$, causing quadratic terms to cancel and yielding a strictly linear decision hyperplane $\mathbf{w}^\top \mathbf{x} + w_0 = 0$. (c) Quadratic Discriminant Analysis (QDA) permits independent class covariance matrices $\boldsymbol{\Sigma}_c$, generating a non-linear quadratic hypersurface boundary.

—

2.5 Statistical decision theory

We now take a decisive forward step: transitioning from discrete probability models (Naïve Bayes) to **Statistical Learning Theory**, which provides the theoretical foundations for Deep Learning and Empirical Risk Minimization.

2.5.1 Formal setup and notation

Let the learning framework be defined by the following mathematical entities:

- **Input Space** $\mathcal{X} \subseteq \mathbb{R}^d$: The domain of feature vectors $\mathbf{x}$.
- **Output Space** $\mathcal{Y}$: The domain of target labels (e.g., $\mathcal{Y} = \{0, 1\}$ for binary classification, $\mathcal{Y} = \{1, \dots, K\}$ for multi-class classification, or $\mathcal{Y} = \mathbb{R}$ for continuous regression).
- **Data-Generating Distribution** $\mathcal{D}$: A fixed, unknown joint probability distribution $p(\mathbf{x}, y)$ over $\mathcal{X} \times \mathcal{Y}$.
- **Hypothesis Class** $\mathcal{H}$: A family of candidate predictive functions $h : \mathcal{X} \to \mathcal{Y}$.
- **Predictor / Model** $h \in \mathcal{H}$: A specific hypothesis parameterized by weights $\boldsymbol{\theta}$.

2.5.2 Task-dependent loss functions

How do we measure the quality of a model's prediction? A model outputs a prediction $\hat{y} = h(\mathbf{x})$, while reality provides the true ground-truth target y.

A **loss function** $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}^+$ quantifies the penalty or cost incurred by predicting $h(\mathbf{x})$ when the true target is y:

$$\ell(h(\mathbf{x}), y) \geq 0. \tag{2.129}$$

Core Machine Learning Principle: Loss selection depends on the task. A critical concept emphasized throughout the lectures is that **the mathematical formulation of the loss function strictly depends on the target task being solved.** Choosing a loss function that does not match the physical nature of the target output invalidates the optimization:

1. **Continuous Regression Tasks** ($\mathcal{Y} = \mathbb{R}$): When targets are continuous physical quantities (e.g., house prices, temperatures, object coordinates):
   - **Mean Squared Error (MSE) / Quadratic Loss**:
   $$\ell_{\text{MSE}}(h(\mathbf{x}), y) = (h(\mathbf{x}) - y)^2. \tag{2.130}$$
   Penalizes large errors quadratically; its expectation is minimized by the conditional mean $\mathbb{E}[Y \mid \mathbf{x}]$.
   - **Mean Absolute Error (MAE) / L_1 Loss**:
   $$\ell_{\text{MAE}}(h(\mathbf{x}), y) = |h(\mathbf{x}) - y|. \tag{2.131}$$
   Linear penalty; robust to extreme outliers; minimized by the conditional median.
2. **Discrete Categorical Classification Tasks** ($\mathcal{Y} = \{0, 1\}$ or $\mathcal{Y} = \{1, \dots, K\}$):
   - **The Ideal 0-1 Loss**:
   $$\ell_{0-1}(h(\mathbf{x}), y) = \mathbb{I}(h(\mathbf{x}) \neq y) = \begin{cases} 0 & \text{if } h(\mathbf{x}) = y, \\ 1 & \text{if } h(\mathbf{x}) \neq y. \end{cases} \tag{2.132}$$
   Directly measures classification error rate.
   - **Cross-Entropy / Negative Log-Likelihood Loss**: When classifiers output a normalized probability distribution over K classes $\hat{\mathbf{p}} = \text{softmax}(\mathbf{z}) \in [0, 1]^K$, the standard loss is Categorical Cross-Entropy:
   $$\ell_{\text{CE}}(\hat{\mathbf{p}}, y) = -\sum_{k=1}^{K} y_k \log \hat{p}_k = -\log \hat{p}_y, \tag{2.133}$$
   where $\mathbf{y}$ is the one-hot encoded ground-truth vector ($y_k = 1$ if class is k, else 0).

Why surrogate continuous losses are mandatory. If our ultimate goal in classification is to minimize the 0-1 loss, why do we not optimize ℓ_{0-1} directly?

- The 0-1 loss is a non-convex step function: its gradient with respect to model weights is **identically zero almost everywhere** ($\nabla_{\mathbf{w}}\ell_{0-1} = \mathbf{0}$), and undefined at the step threshold.
- A gradient of zero provides zero directional guidance for gradient descent: backpropagation cannot update weights.
- Minimizing 0-1 loss on a dataset is an NP-hard combinatorial optimization problem.
- Therefore, machine learning relies on **continuous, convex surrogate losses** (such as Cross-Entropy for neural networks, Logistic loss for logistic regression, and Hinge loss for Support Vector Machines) that provide smooth, informative non-zero gradients everywhere.

2.5.3 Risk and Bayes optimality

The foundational objective of statistical learning theory is to evaluate and guarantee how a machine learning hypothesis $h \in \mathcal{H}$ will perform when deployed in the real world. To formalize this performance, we must distinguish between the performance on the observed training sample and the performance across the entire universe of data.

True Risk: The universal dataset evaluation. Let $\mathcal{D}$ denote the true, underlying joint probability distribution over the feature-label space $\mathcal{X} \times \mathcal{Y}$. The theoretical benchmark of performance is the **True Risk** (also known as **Expected Risk**, **Population Risk**, or **Generalization Risk**), denoted $R(h)$:

$$R(h) \triangleq \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}[\ell(h(\mathbf{x}), y)] = \int_{\mathcal{X}\times\mathcal{Y}} \ell(h(\mathbf{x}), y)\, p(\mathbf{x}, y)\, d\mathbf{x}\, dy. \tag{2.134}$$

Crucially, $R(h)$ is evaluated over the theoretical **universal dataset**—an infinite population consisting of all possible input-output pairs that could ever be drawn from $\mathcal{D}$.

Two key properties of True Risk must be kept in mind:

1. $R(h)$ **is model-dependent:** Notice that the hypothesis h appears inside the expectation: $R(h)$ is a scorecard for a *specific* model h. If we train a linear classifier h_1 and a deep neural network h_2, each possesses its own distinct True Risk: $R(h_1) \neq R(h_2)$. It quantifies the lifetime expected loss of that exact model across all future, unseen data.
2. $R(h)$ **is fundamentally incomputable:** To evaluate the integral in (2.134), one must possess full analytical knowledge of the true joint data-generating distribution $p(\mathbf{x}, y)$. In practice, this distribution is unknown—if we already possessed $p(\mathbf{x}, y)$, machine learning would be entirely unnecessary because the optimal prediction could be calculated directly via Bayes' theorem.

The Bayes Optimal Classifier h^*. Given that every model has a True Risk $R(h)$, what is the absolute best conceivable predictor that could ever exist in the universe?

The theoretical gold standard is the **Bayes Optimal Classifier** h^*, defined as the function that achieves the infimum of the True Risk over the space of *all possible measurable functions* $\mathcal{F} = \{h : \mathcal{X} \to \mathcal{Y}\}$, unconstrained by any hypothesis family $\mathcal{H}$:

$$h^* \triangleq \arg\min_{h\in\mathcal{F}} R(h). \tag{2.135}$$

For classification under the standard 0-1 loss $\ell_{0\text{-}1}(\hat{y}, y) = \mathbb{I}(\hat{y} \neq y)$, we can decompose the True Risk pointwise via the law of total expectation:

$$R(h) = \mathbb{E}_{\mathbf{x}}\left[\mathbb{E}_{y|\mathbf{x}}[\ell_{0\text{-}1}(h(\mathbf{x}), y)]\right] = \int_{\mathcal{X}} \left(\sum_{y\in\mathcal{Y}} \mathbb{I}(h(\mathbf{x}) \neq y)\, P(Y = y \mid X = \mathbf{x})\right) p(\mathbf{x})\, d\mathbf{x}. \tag{2.136}$$

Notice that for each individual feature vector $\mathbf{x}$, the inner sum is simply the probability of making an error:

$$\sum_{y \in \mathcal{Y}} \mathbb{I}(h(\mathbf{x}) \neq y)\, P(Y = y \mid X = \mathbf{x}) = 1 - P(Y = h(\mathbf{x}) \mid X = \mathbf{x}). \tag{2.137}$$

Because $p(\mathbf{x}) \geq 0$, minimizing the overall integral $R(h)$ is equivalent to minimizing the inner error probability independently for each single coordinate $\mathbf{x} \in \mathcal{X}$. To minimize $1 - P(Y = h(\mathbf{x}) \mid X = \mathbf{x})$, we must maximize $P(Y = h(\mathbf{x}) \mid X = \mathbf{x})$.

Therefore, the Bayes optimal decision rule assigns $\mathbf{x}$ to the class with the highest posterior probability:

$$h^*(\mathbf{x}) = \arg\max_{y \in \mathcal{Y}} P(Y = y \mid X = \mathbf{x}). \tag{2.138}$$

Why we cannot achieve a zero error rate: The Bayes Risk $\mathcal{R}^*$. A natural question arises: *If the Bayes Optimal Classifier has access to the exact true posterior distribution $P(Y \mid X)$ and possesses infinite functional capacity, why does it still fail to achieve an error rate of zero?*

The impossibility of achieving zero error stems from **irreducible aleatoric uncertainty** (intrinsic data ambiguity and overlapping class distributions). In real-world phenomena:

- The relationship between observed features $\mathbf{x}$ and the ground-truth target y is almost never a deterministic mathematical bijection.
- Features $\mathbf{x}$ represent an incomplete, lossy measurement of the physical world. As a consequence, two distinct physical entities can have the **exact same observed feature vector $\mathbf{x}$** while possessing **different true labels** y.
- Because any classifier h must output a single deterministic prediction $\hat{y} \in \mathcal{Y}$ for a given query $\mathbf{x}$, it cannot predict both classes simultaneously.

When the Bayes optimal classifier selects $h^*(\mathbf{x}) = \arg\max_{y \in \mathcal{Y}} P(Y = y \mid X = \mathbf{x})$, its pointwise error rate on that specific input coordinate $\mathbf{x}$ is:

$$\mathbb{P}(h^*(\mathbf{x}) \neq Y \mid X = \mathbf{x}) = 1 - \max_{y \in \mathcal{Y}} P(Y = y \mid X = \mathbf{x}). \tag{2.139}$$

Unless the posterior distribution is strictly degenerate (i.e., $\max_y P(Y = y \mid X = \mathbf{x}) = 1$ almost everywhere, meaning complete determinism), this pointwise error is strictly positive.

Integrating this unavoidable pointwise ambiguity over the entire feature space yields the **Bayes Risk $\mathcal{R}^*$**:

$$\mathcal{R}^* \triangleq R(h^*) = \mathbb{E}_{\mathbf{x}} \left[1 - \max_{y \in \mathcal{Y}} P(Y = y \mid X = \mathbf{x}) \right]. \tag{2.140}$$

The Bayes Risk represents the **aleatoric floor of the universe**. No machine learning algorithm, no deep neural network with billions of parameters, and no quantum computer can ever achieve a True Risk lower than $\mathcal{R}^*$:

$$R(h) \geq \mathcal{R}^*, \quad \forall h \in \mathcal{F}. \tag{2.141}$$

Example 2.1 (Irreducible Aleatoric Error: Clinical Diagnosis and the Biased Coin)**.** To build an indelible mental picture of the distinction between True Risk and Bayes Risk, consider the following two canonical settings:

1. **Clinical Diagnostic Screening (Overlapping Feature Profiles):** Suppose two patients walk into a hospital and present with the *identical* clinical measurement vector:

$$\mathbf{x} = [\text{Age: } 50,\ \text{Resting BP: } 135/85,\ \text{Fasting Glucose: } 105,\ \text{Total Cholesterol: } 220].$$

Due to unobserved genetic variations and environmental factors not captured in $\mathbf{x}$, the true biological ground-truth distribution is:

$$P(\text{Cardiovascular Disease} \mid \mathbf{x}) = 0.80, \qquad P(\text{Healthy} \mid \mathbf{x}) = 0.20.$$

An omniscient Bayes classifier h^*, possessing perfect knowledge of these probabilities, must choose a single diagnosis. It rationally predicts **Cardiovascular Disease**, because $0.80 > 0.20$.

What happens across 100 patients presenting with this exact feature profile $\mathbf{x}$?

- 80 patients truly have the disease $\implies$ h^* is correct.
- 20 patients are healthy $\implies$ h^* misdiagnoses them!

Even with perfect omniscience, the classifier makes an unavoidable error on 20% of these instances. That 20% error is the **Bayes Risk** at $\mathbf{x}$. It cannot be eliminated without measuring new, orthogonal features (such as genetic sequencing) that separate the overlapping distributions.

2. **The Biased Coin Game (Strategy vs. Intrinsic Randomness):** Consider predicting the outcome of repeated coin flips where the coin is biased to land **Heads 70% of the time** ($P(\text{Heads}) = 0.70$) and **Tails 30% of the time** ($P(\text{Tails}) = 0.30$).

   - *The Bayes Optimal Predictor h^*:* The optimal strategy is to always predict Heads.
   - *The Bayes Risk $\mathcal{R}^*$:* Because Tails occurs 30% of the time, even the optimal strategy suffers an error rate of $\mathcal{R}^* = 0.30$. No player can achieve an error rate below 30%.
   - *True Risk of Candidate Models $R(h)$:*
     - A random guessing model h_{random} that predicts Heads with probability 0.5 and Tails with probability 0.5 achieves a True Risk of:

$$R(h_{\text{random}}) = 0.5 \times 0.30 + 0.5 \times 0.70 = 0.50.$$

     - A superstitious model $h_{\text{pessimist}}$ that always predicts Tails achieves a True Risk of $R(h_{\text{pessimist}}) = 0.70$.
     - An optimal learner h_{opt} that converges to always predicting Heads achieves $R(h_{\text{opt}}) = 0.30 = \mathcal{R}^*$.

Systematic Comparison: True Risk vs. Bayes Risk. Table 2.4 provides a direct, side-by-side comparison to consolidate these two fundamental concepts.

Conceptual & Geometric Intuition: The Universal Risk Decomposition: Dissecting Generalization Er

To understand how learning algorithms bridge the gap between empirical data and optimal performance, consider any hypothesis $h \in \mathcal{H}$ trained from a finite sample $\mathcal{S}_N$. Its total True Risk $R(h)$ can be decomposed into three fundamental, orthogonal components:

$$R(h) - \mathcal{R}^* = \underbrace{\left(\inf_{f \in \mathcal{H}} R(f) - \mathcal{R}^*\right)}_{\textbf{Approximation Error (Bias)}} + \underbrace{\left(R(h) - \inf_{f \in \mathcal{H}} R(f)\right)}_{\textbf{Estimation Error (Variance)}}. \tag{2.142}$$

Adding the irreducible noise $\mathcal{R}^*$ gives the full hierarchy of risk:

$$R(h) = \underbrace{\mathcal{R}^*}_{\substack{\textbf{Bayes Risk} \\ \text{(Irreducible aleatoric noise)}}} + \underbrace{\mathcal{E}_{\text{approx}}(\mathcal{H})}_{\substack{\textbf{Approximation Error} \\ \text{(Hypothesis space capacity)}}} + \underbrace{\mathcal{E}_{\text{est}}(\mathcal{H}, N)}_{\substack{\textbf{Estimation Error} \\ \text{(Finite sample penalty)}}} + \underbrace{\mathcal{E}_{\text{opt}}}_{\substack{\textbf{Optimization Error} \\ \text{(Algorithm sub-optimality)}}}. \tag{2.143}$$

Dimension	True Risk $R(h)$	Bayes Risk $\mathcal{R}^*$
Mathematical Nature	A functional evaluating a *specific* candidate hypothesis h: $R(h) = \mathbb{E}_{\mathcal{D}}[\ell(h(\mathbf{x}), y)]$.	A *universal constant* for the task: $\mathcal{R}^* = \inf_{h \in \mathcal{F}} R(h) = R(h^*)$.
Domain of Evaluation	The theoretical **universal dataset** (all infinite samples drawn from $\mathcal{D}$).	The theoretical **universal dataset** under the optimal predictor h^*.
Governing Source	Determined by the model architecture, training algorithm, and learned parameters $\mathbf{w}$.	Determined exclusively by the physics of the data generation process $\mathcal{D}$ (aleatoric noise).
Can it change?	**Yes.** Changing the model from h_1 to h_2 alters the True Risk: $R(h_1) \neq R(h_2)$.	**No.** It is strictly invariant to the learning algorithm or model architecture.
Fundamental Bound	Upper-bounded by worst-case loss; lower-bounded by Bayes Risk: $R(h) \geq \mathcal{R}^*$.	The absolute theoretical lower bound (the infimum): $\mathcal{R}^* = \inf_h R(h)$.
Practical Status	Incomputable (approximated via empirical test/validation sample risk).	Incomputable theoretical constant (the irreducible noise floor).

Table 2.4: Systematic comparison between True Risk $R(h)$ and Bayes Risk $\mathcal{R}^*$.

- **Bayes Risk $\mathcal{R}^*$:** The floor set by nature. Cannot be reduced by collecting more data or expanding the model.
- **Approximation Error $\mathcal{E}_{\text{approx}}$:** The penalty for restricting our search to a specific model family $\mathcal{H}$ (e.g., linear models cannot fit a quadratic boundary). Decreases as the capacity of $\mathcal{H}$ increases.
- **Estimation Error $\mathcal{E}_{\text{est}}$:** The penalty for optimizing on a finite training set of size N rather than the universal dataset. Increases with model complexity but decreases towards zero as sample size $N \to \infty$.
- **Optimization Error $\mathcal{E}_{\text{opt}}$:** The penalty if the numerical optimization algorithm (e.g., gradient descent) fails to find the empirical empirical risk minimizer due to non-convexity, saddle points, or early stopping.

2.5.4 Empirical risk minimization

Because the true distribution $\mathcal{D}$ is unknown, machine learning operates in the **Batch Learning** regime:

- We are provided with a finite training dataset of N instances drawn i.i.d. from $\mathcal{D}$:

$$\mathcal{S}_N = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)\} \overset{\text{i.i.d.}}{\sim} \mathcal{D}. \tag{2.144}$$

- By the **Law of Large Numbers**, the expected value over the true distribution can be approximated by the sample average over the observed training set.

This sample average is the **Empirical Risk** $\hat{R}_N(h)$:

$$\hat{R}_N(h) \triangleq \frac{1}{N} \sum_{i=1}^{N} \ell(h(\mathbf{x}_i), y_i). \tag{2.145}$$

The foundational learning paradigm introduced by Vladimir Vapnik is the **Empirical Risk Minimization (ERM)** principle:

$$h_{\text{ERM}} \triangleq \arg\min_{h \in \mathcal{H}} \hat{R}_N(h) = \arg\min_{h \in \mathcal{H}} \frac{1}{N} \sum_{i=1}^{N} \ell(h(\mathbf{x}_i), y_i). \tag{2.146}$$

ERM replaces the impossible problem of minimizing True Risk $R(h)$ with the computationally feasible problem of minimizing the average loss on the observed training sample.

2.5.5 PAC learning theory

Can we guarantee that a hypothesis h_{ERM} that achieves low empirical risk $\hat{R}_N(h)$ will also achieve low true risk $R(h)$ on unobserved test data?

Leslie Valiant's framework of **Probably Approximately Correct (PAC) Learning** establishes the mathematical conditions under which empirical learning is guaranteed to succeed [83]:

- **Approximately Correct**: We cannot demand that an algorithm finds a hypothesis with exactly zero error ($R(h) = 0$). Instead, we demand that the true generalization error is bounded by an accuracy tolerance parameter $\epsilon > 0$:

$$R(h) - R(h^*) \leq \epsilon. \tag{2.147}$$

- **Probably Correct**: Because our training dataset $\mathcal{S}_N$ is drawn stochastically from $\mathcal{D}$, there is always a non-zero probability that the random sample happens to be completely unrepresentative (e.g., drawing 100 straight coin flips of heads from a fair coin). We cannot guarantee success with 100% certainty. Instead, we demand that the learning algorithm succeeds with high probability:

$$P_{\mathcal{S}_N \sim \mathcal{D}^N}\Big(R(h) - R(h^*) \leq \epsilon\Big) \geq 1 - \delta, \tag{2.148}$$

  where $\delta \in (0, 1)$ is the failure confidence parameter.

Theorem 2.3 (PAC Sample Complexity for Finite Hypothesis Spaces [83])**.** *Let $\mathcal{H}$ be a finite hypothesis space. For any distribution $\mathcal{D}$ and loss bounded in $[0, 1]$, to guarantee that with probability at least $1 - \delta$, the ERM hypothesis satisfies $R(h_{ERM}) \leq \min_{h \in \mathcal{H}} R(h) + \epsilon$, it suffices to draw a training sample size N satisfying:*

$$N \geq \frac{1}{\epsilon}\left(\ln|\mathcal{H}| + \ln\frac{1}{\delta}\right). \tag{2.149}$$

Equation (2.149) reveals the fundamental price of model capacity: as the size of the hypothesis space $|\mathcal{H}|$ grows, the sample complexity N required to guarantee generalization increases logarithmically.

2.5.6 Asymmetric loss matrices

In Section 2.5.3, we derived the Bayes Optimal Classifier under the symmetric 0-1 loss function, proving that the risk-minimizing rule assigns $\mathbf{x}$ to the class with highest posterior probability ($P(Y = 1 \mid \mathbf{x}) \gtrless 0.5$).

However, in real-world engineering, clinical, and safety-critical domains, **the costs of different misclassification errors are fundamentally asymmetric** [6]:

- *Medical Oncology*: Classifying a malignant cancer as benign (**False Negative**) delays life-saving chemotherapy, typically leading to patient death. In contrast, classifying a benign lesion as malignant (**False Positive**) results in an unnecessary secondary biopsy—anxiety-inducing and costly, but rarely fatal. Missing cancer is vastly worse than a false alarm.

- *Autonomous Vehicles*: Classifying an empty shadow as a pedestrian causes an uncomfortable braking event. Classifying a real pedestrian as an empty shadow causes fatal collision.
- *Cybersecurity / Fraud Detection*: Flagging a legitimate transaction as fraudulent causes minor customer inconvenience; missing a multi-million-dollar cyberattack causes catastrophic systemic insolvency.

The General Loss Matrix Formulation. To formalize asymmetric costs mathematically, we introduce a **loss matrix** $\mathbf{L} \in \mathbb{R}^{K \times K}$ where entry $L_{k,j}$ defines the exact penalty incurred for predicting class $\mathcal{C}_j$ when the ground-truth state of nature is class $\mathcal{C}_k$:

$$L_{k,j} \triangleq \text{Loss}\left(\text{predict } \mathcal{C}_j \mid \text{true } \mathcal{C}_k\right) \geq 0. \tag{2.150}$$

Typically, correct classifications incur zero loss ($L_{k,k} = 0$), while incorrect classifications carry strictly positive penalties ($L_{k,j} > 0$ for $j \neq k$).

When presented with an observed feature vector $\mathbf{x}$, the system does not know the true class with absolute certainty, but possesses the posterior distribution $P(\mathcal{C}_k \mid \mathbf{x})$. The **expected conditional loss (conditional risk)** of selecting decision $\mathcal{C}_j$ is the expectation over all possible true classes:

$$R(j \mid \mathbf{x}) \triangleq \mathbb{E}_{\mathcal{C}_k \mid \mathbf{x}}\left[L_{k,j}\right] = \sum_{k=1}^{K} L_{k,j} P(\mathcal{C}_k \mid \mathbf{x}). \tag{2.151}$$

By the fundamental principle of statistical decision theory, the optimal decision rule partitions the input space into decision regions $\mathcal{R}_1, \ldots, \mathcal{R}_K$ such that each coordinate $\mathbf{x}$ is assigned to the class that minimizes expected risk:

$$\hat{y}(\mathbf{x}) = \underset{j \in \{1,\ldots,K\}}{\arg\min} \sum_{k=1}^{K} L_{k,j} P(\mathcal{C}_k \mid \mathbf{x}). \tag{2.152}$$

Analytical Derivation of the Optimal Threshold for Binary Classification. Consider binary classification with classes $\mathcal{C}_1$ (e.g., disease present) and $\mathcal{C}_2$ (disease absent). The loss matrix is:

$$\mathbf{L} = \begin{pmatrix} L_{1,1} & L_{1,2} \\ L_{2,1} & L_{2,2} \end{pmatrix}, \tag{2.153}$$

where $L_{1,2}$ is the cost of a False Negative and $L_{2,1}$ is the cost of a False Positive.

The decision rule assigns $\mathbf{x}$ to class $\mathcal{C}_1$ if and only if the expected risk of choosing $\mathcal{C}_1$ is strictly less than choosing $\mathcal{C}_2$:

$$R(1 \mid \mathbf{x}) < R(2 \mid \mathbf{x}). \tag{2.154}$$

Expanding using Equation (2.151):

$$L_{1,1} P(\mathcal{C}_1 \mid \mathbf{x}) + L_{2,1} P(\mathcal{C}_2 \mid \mathbf{x}) < L_{1,2} P(\mathcal{C}_1 \mid \mathbf{x}) + L_{2,2} P(\mathcal{C}_2 \mid \mathbf{x}). \tag{2.155}$$

Grouping identical posterior probabilities:

$$(L_{1,2} - L_{1,1}) P(\mathcal{C}_1 \mid \mathbf{x}) > (L_{2,1} - L_{2,2}) P(\mathcal{C}_2 \mid \mathbf{x}). \tag{2.156}$$

Since $P(\mathcal{C}_2 \mid \mathbf{x}) = 1 - P(\mathcal{C}_1 \mid \mathbf{x})$:

$$(L_{1,2} - L_{1,1}) P(\mathcal{C}_1 \mid \mathbf{x}) > (L_{2,1} - L_{2,2})\Big(1 - P(\mathcal{C}_1 \mid \mathbf{x})\Big). \tag{2.157}$$

Expanding and collecting $P(\mathcal{C}_1 \mid \mathbf{x})$ on the left-hand side:

$$\Big[(L_{1,2} - L_{1,1}) + (L_{2,1} - L_{2,2})\Big] P(\mathcal{C}_1 \mid \mathbf{x}) > L_{2,1} - L_{2,2}. \tag{2.158}$$

Assuming that misclassifications are strictly more costly than correct classifications (so the bracketed term is strictly positive), dividing yields the exact optimal decision threshold:

$$\boxed{P(\mathcal{C}_1 \mid \mathbf{x}) > \frac{L_{2,1} - L_{2,2}}{(L_{1,2} - L_{1,1}) + (L_{2,1} - L_{2,2})}} \tag{2.159}$$

When correct decisions incur zero penalty ($L_{1,1} = L_{2,2} = 0$), this fundamental equation simplifies to:

$$P(\mathcal{C}_1 \mid \mathbf{x}) > \frac{L_{2,1}}{L_{1,2} + L_{2,1}} = \frac{1}{1 + \frac{L_{1,2}}{L_{2,1}}}. \tag{2.160}$$

In oral examinations, students are often asked: *"If missing a cancer diagnosis is 100 times more severe than a false alarm, at what posterior probability should we classify a patient as positive?"*

- Let False Negative cost $L_{1,2} = 100$ and False Positive cost $L_{2,1} = 1$.
- Correct diagnosis costs $L_{1,1} = L_{2,2} = 0$.
- Substituting into Equation (2.160):

$$P(\text{Cancer} \mid \mathbf{x}) > \frac{1}{100 + 1} = \frac{1}{101} \approx 0.0099 \quad (0.99\%). \tag{2.161}$$

The optimal decision threshold does not reside at 0.5; it drops to **less than** 1%! If the model estimates just a 1% chance of malignancy, the risk-minimizing rational action is to classify the patient as positive and immediately order follow-up diagnostic procedures.

2.5.7 The reject option

In many automated decision systems, forced binary or categorical classification is inappropriate when the model is uncertain. For regions of the feature space where class posteriors are nearly balanced ($P(\mathcal{C}_1 \mid \mathbf{x}) \approx P(\mathcal{C}_2 \mid \mathbf{x}) \approx 0.5$), any automated decision incurs a substantial probability of error.

Statistical decision theory resolves this through the **Reject Option** (or *Doubt Option*) [6]: permitting the model to abstain from making an autonomous decision, instead routing the instance to a human specialist (e.g., radiologist or senior fraud investigator).

Let $\theta \in [0, 1]$ be the fixed cost of choosing the reject action (e.g., the labor cost of human review).

- If an automated decision is made, the expected misclassification loss is $1 - \max_k P(\mathcal{C}_k \mid \mathbf{x})$.
- If the system abstains, it incurs the fixed rejection penalty θ.

Therefore, the system chooses to execute an automated prediction if and only if the error risk is strictly less than the cost of rejection:

$$1 - \max_k P(\mathcal{C}_k \mid \mathbf{x}) < \theta \iff \max_k P(\mathcal{C}_k \mid \mathbf{x}) \geq 1 - \theta. \tag{2.162}$$

If $\max_k P(\mathcal{C}_k \mid \mathbf{x}) < 1 - \theta$, the instance falls into the **Rejection Region**, triggering an alert for human expert intervention. Figure 2.5 illustrates both asymmetric threshold shifting and the reject option band.

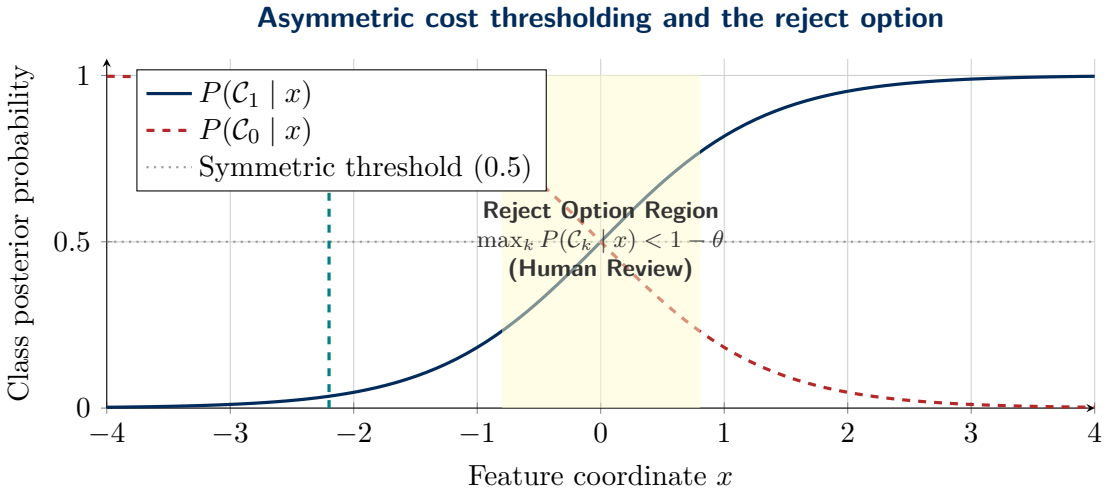


Figure 2.5: Decision boundaries under asymmetric misclassification costs and the reject option. The solid navy curve is $P(\mathcal{C}_1 \mid x)$ and the dashed red curve is $P(\mathcal{C}_0 \mid x)$. Under symmetric 0-1 loss, the decision boundary lies at $P = 0.5$. When the cost of missing class 1 (e.g., cancer) is severe, the threshold shifts to the asymmetric threshold, drastically expanding the acceptance region for $\mathcal{C}_1$. The shaded central band denotes the reject option: where posterior confidence is ambiguous ($\max_k P(\mathcal{C}_k \mid x) < 1 - \theta$), the system abstains from automated diagnosis and alerts a human specialist.

2.5.8 Decision theory for regression

How does statistical decision theory operate when target outputs are continuous real numbers ($y \in \mathbb{R}$)?

Let $h(\mathbf{x})$ be our regression predictor, and let $\ell(h(\mathbf{x}), y)$ be a continuous loss function. The expected True Risk is:

$$R(h) = \int_{\mathcal{X}} \int_{\mathcal{Y}} \ell(h(\mathbf{x}), y)\, p(\mathbf{x}, y)\, dy\, d\mathbf{x} = \int_{\mathcal{X}} \left[\int_{\mathcal{Y}} \ell(h(\mathbf{x}), y)\, p(y \mid \mathbf{x})\, dy \right] p(\mathbf{x})\, d\mathbf{x}. \tag{2.163}$$

Because $p(\mathbf{x}) \geq 0$, minimizing overall risk $R(h)$ is achieved by independently minimizing the inner conditional expectation for each single coordinate $\mathbf{x} \in \mathcal{X}$:

$$h^*(\mathbf{x}) = \arg\min_{a \in \mathbb{R}} \int_{\mathcal{Y}} \ell(a, y)\, p(y \mid \mathbf{x})\, dy. \tag{2.164}$$

1. The Squared Error Loss: Optimal Predictor is Conditional Expectation. Let the loss be quadratic: $\ell(h(\mathbf{x}), y) = (h(\mathbf{x}) - y)^2$. Using the calculus of variations, we differentiate the inner integral with respect to $h(\mathbf{x})$:

$$\frac{\partial}{\partial h(\mathbf{x})} \int_{\mathcal{Y}} (h(\mathbf{x}) - y)^2 p(y \mid \mathbf{x})\, dy = 2 \int_{\mathcal{Y}} (h(\mathbf{x}) - y) p(y \mid \mathbf{x})\, dy = 0. \tag{2.165}$$

Distributing the integral:

$$h(\mathbf{x}) \underbrace{\int_{\mathcal{Y}} p(y \mid \mathbf{x})\, dy}_{=1} - \int_{\mathcal{Y}} y\, p(y \mid \mathbf{x})\, dy = 0. \tag{2.166}$$

Solving for $h(\mathbf{x})$ yields the fundamental theorem of regression:

$$\boxed{h^*(\mathbf{x}) = \int_{\mathcal{Y}} y\, p(y \mid \mathbf{x})\, dy = \mathbb{E}[Y \mid \mathbf{x}]} \tag{2.167}$$

Under squared error loss, the theoretically optimal predictor is the conditional expectation (conditional mean) of the target given the features!

Exact Decomposition into Reducible Loss and Irreducible Bayes Noise. To analyze why a model cannot achieve zero regression risk, we decompose $(h(\mathbf{x}) - y)^2$ around the optimal predictor $\mathbb{E}[y \mid \mathbf{x}]$ [6]:

$$\begin{aligned}(h(\mathbf{x}) - y)^2 &= \Big((h(\mathbf{x}) - \mathbb{E}[y \mid \mathbf{x}]) + (\mathbb{E}[y \mid \mathbf{x}] - y)\Big)^2 \\ &= (h(\mathbf{x}) - \mathbb{E}[y \mid \mathbf{x}])^2 + (\mathbb{E}[y \mid \mathbf{x}] - y)^2 + 2(h(\mathbf{x}) - \mathbb{E}[y \mid \mathbf{x}])(\mathbb{E}[y \mid \mathbf{x}] - y). \quad (2.168)\end{aligned}$$

Now, take the conditional expectation over $y \mid \mathbf{x}$. Notice what happens to the cross-term:

$$\mathbb{E}_{y|\mathbf{x}}\left[2(h(\mathbf{x}) - \mathbb{E}[y \mid \mathbf{x}])(\mathbb{E}[y \mid \mathbf{x}] - y)\right] = 2(h(\mathbf{x}) - \mathbb{E}[y \mid \mathbf{x}]) \underbrace{\mathbb{E}_{y|\mathbf{x}}\left[\mathbb{E}[y \mid \mathbf{x}] - y\right]}_{=\mathbb{E}[y|\mathbf{x}]-\mathbb{E}[y|\mathbf{x}]=0} = 0. \quad (2.169)$$

The cross-term vanishes identically! Substituting back into the full True Risk:

$$\boxed{R(h) = \int_{\mathcal{X}} (h(\mathbf{x}) - \mathbb{E}[y \mid \mathbf{x}])^2 p(\mathbf{x})\, d\mathbf{x} + \int_{\mathcal{X}} \mathrm{Var}(Y \mid \mathbf{x}) p(\mathbf{x})\, d\mathbf{x}} \quad (2.170)$$

Equation (2.170) is a celebrated result in machine learning:

1. **First Term (Reducible Model Error)**: Depends on our choice of hypothesis $h(\mathbf{x})$. It is non-negative and reaches its minimum of zero if and only if our model perfectly learns the true conditional expectation $h(\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}]$.
2. **Second Term (Irreducible Bayes Noise)**: Represents the intrinsic variance $\mathrm{Var}(Y \mid \mathbf{x})$ of the data generation process. It is completely independent of the model $h(\mathbf{x})$. No matter how much compute, data, or parameters an algorithm possesses, expected test error can never fall below this irreducible noise floor.

2. The Absolute Error (L_1) Loss: Optimal Predictor is Conditional Median. What if we choose the L_1 loss $\ell(h(\mathbf{x}), y) = |h(\mathbf{x}) - y|$? Differentiating with respect to $h(\mathbf{x})$:

$$\frac{\partial}{\partial h(\mathbf{x})} \int_{\mathcal{Y}} |h(\mathbf{x}) - y| p(y \mid \mathbf{x})\, dy = \int_{\mathcal{Y}} \mathrm{sign}(h(\mathbf{x}) - y) p(y \mid \mathbf{x})\, dy = 0. \quad (2.171)$$

Splitting the integral into regions where $y < h(\mathbf{x})$ and $y > h(\mathbf{x})$:

$$\int_{-\infty}^{h(\mathbf{x})} p(y \mid \mathbf{x})\, dy - \int_{h(\mathbf{x})}^{\infty} p(y \mid \mathbf{x})\, dy = 0 \implies P(Y \le h^*(\mathbf{x}) \mid \mathbf{x}) = P(Y \ge h^*(\mathbf{x}) \mid \mathbf{x}) = \frac{1}{2}. \quad (2.172)$$

Under L_1 absolute loss, the optimal predictor is the conditional median $\mathrm{median}(Y \mid \mathbf{x})$**!** The median is inherently robust to extreme outlier perturbations, making L_1 loss the principled choice for datasets with heavy-tailed sensor anomalies.

2.5.9 Losses from probability models

In Section 2.5.2, we observed that loss functions depend on the target task. A profound unifying principle established in modern deep learning literature [32, 73] is that **loss functions are not heuristic inventions; they are derived directly as the Negative Log-Likelihood (NLL) of parametric probability distributions**:

$$\ell(\hat{y}, y) \triangleq -\ln p(y \mid \phi(\mathbf{x})), \quad (2.173)$$

where $\phi(\mathbf{x})$ represents the parameters of a probability distribution predicted by the machine learning model.

1. Gaussian Distribution Yields Mean Squared Error (MSE). Suppose the continuous target $y \in \mathbb{R}$ is perturbed by homoscedastic Gaussian noise:

$$p(y \mid \mathbf{x}) = \mathcal{N}(y; \mu(\mathbf{x}), \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \mu(\mathbf{x}))^2}{2\sigma^2}\right). \tag{2.174}$$

Let our model predict the mean: $\mu(\mathbf{x}) = h_{\boldsymbol{\theta}}(\mathbf{x})$. The negative log-likelihood of a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ is:

$$\mathcal{L}(\boldsymbol{\theta}) = -\sum_{i=1}^{N} \ln p(y_i \mid \mathbf{x}_i) = \frac{1}{2\sigma^2} \sum_{i=1}^{N} (y_i - h_{\boldsymbol{\theta}}(\mathbf{x}_i))^2 + \frac{N}{2} \ln(2\pi\sigma^2). \tag{2.175}$$

Assuming constant variance σ^2, the second term is constant and $\frac{1}{2\sigma^2}$ is a positive scaling factor. Minimizing NLL is **identically equivalent to minimizing the Mean Squared Error (MSE)**:

$$\arg\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) \equiv \arg\min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^{N} (h_{\boldsymbol{\theta}}(\mathbf{x}_i) - y_i)^2. \tag{2.176}$$

Heteroscedastic Regression: Predicting Uncertainty. In real-world data, noise is rarely uniform (homoscedastic); some input regions have high measurement ambiguity while others are precise. Prince [73] emphasizes that neural networks can predict **both** the mean $\mu(\mathbf{x})$ and the input-dependent variance $\sigma^2(\mathbf{x})$ (heteroscedastic regression) using a dual-head architecture:

$$\mathcal{L}_{\text{hetero}}(\boldsymbol{\theta}) = \frac{1}{2} \sum_{i=1}^{N} \left[\frac{(y_i - \mu_{\boldsymbol{\theta}}(\mathbf{x}_i))^2}{\sigma^2_{\boldsymbol{\theta}}(\mathbf{x}_i)} + \ln \sigma^2_{\boldsymbol{\theta}}(\mathbf{x}_i) \right]. \tag{2.177}$$

The network automatically learns to predict high variance $\sigma^2(\mathbf{x})$ for noisy, ambiguous inputs to avoid massive squared-error penalties, while the second log-variance term prevents the trivial collapse $\sigma^2 \to \infty$.

2. Laplace Distribution Yields Mean Absolute Error (MAE / L_1). Suppose target deviations follow a Laplace distribution:

$$p(y \mid \mathbf{x}) = \text{Laplace}(y; \mu(\mathbf{x}), b) = \frac{1}{2b} \exp\left(-\frac{|y - \mu(\mathbf{x})|}{b}\right). \tag{2.178}$$

The negative log-likelihood is:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{b} \sum_{i=1}^{N} |h_{\boldsymbol{\theta}}(\mathbf{x}_i) - y_i| + N \ln(2b). \tag{2.179}$$

Minimizing this NLL is **identically equivalent to minimizing Mean Absolute Error (MAE / L_1 loss)**.

3. Bernoulli Distribution Yields Binary Cross-Entropy (BCE). For binary classification $y \in \{0, 1\}$, let the model predict the Bernoulli success probability $\hat{p} = \sigma(z(\mathbf{x})) \in [0, 1]$:

$$p(y \mid \mathbf{x}) = \hat{p}^y (1 - \hat{p})^{1-y}. \tag{2.180}$$

The negative log-likelihood is:

$$\mathcal{L}_{\text{BCE}}(\boldsymbol{\theta}) = -\sum_{i=1}^{N} \Big[y_i \ln \hat{p}_i + (1 - y_i) \ln(1 - \hat{p}_i) \Big], \tag{2.181}$$

which is the celebrated **Binary Cross-Entropy Loss**.

4. Categorical Distribution Yields Multiclass Cross-Entropy. For multi-class classification $y \in \{1, \dots, K\}$, the output follows a Categorical distribution parameterized by softmax probabilities $\hat{\mathbf{p}} = \text{softmax}(\mathbf{z}) \in [0,1]^K$:

$$p(\mathbf{y} \mid \mathbf{x}) = \prod_{k=1}^{K} \hat{p}_k^{y_k}. \tag{2.182}$$

The negative log-likelihood evaluates to:

$$\mathcal{L}_{\text{CE}}(\boldsymbol{\theta}) = -\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik} \ln \hat{p}_{ik} = -\sum_{i=1}^{N} \ln \hat{p}_{i,y_i}, \tag{2.183}$$

which is the standard **Categorical Cross-Entropy Loss** ubiquitous in deep learning.

2.5.10 Maximum likelihood and KL divergence

We can now establish the profound unification connecting statistical estimation to information theory [6, 32]:

Theorem 2.4 (Equivalence of Maximum Likelihood and KL Divergence Minimization)**.** *Let $\tilde{p}_{data}(\mathbf{x}, y) = \frac{1}{N}\sum_{i=1}^{N} \delta(\mathbf{x} - \mathbf{x}_i, y - y_i)$ be the empirical data distribution defined by a training sample $\mathcal{D}$. Maximizing the sample log-likelihood under a parametric model $p_{model}(\mathbf{x}, y; \boldsymbol{\theta})$ is mathematically identical to minimizing the Kullback-Leibler divergence between the empirical data distribution and the model distribution:*

$$\arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{N} \ln p_{model}(\mathbf{x}_i, y_i; \boldsymbol{\theta}) \equiv \arg\min_{\boldsymbol{\theta}} D_{KL}\Big(\tilde{p}_{data} \,\Big\|\, p_{model}(\cdot; \boldsymbol{\theta})\Big). \tag{2.184}$$

Proof. Expanding the definition of KL divergence in Equation (2.53):

$$\begin{aligned} D_{\text{KL}}\Big(\tilde{p}_{\text{data}} \,\Big\|\, p_{\text{model}}\Big) &= \mathbb{E}_{(\mathbf{x},y)\sim\tilde{p}_{\text{data}}}\left[\ln \frac{\tilde{p}_{\text{data}}(\mathbf{x}, y)}{p_{\text{model}}(\mathbf{x}, y; \boldsymbol{\theta})}\right] \\ &= \mathbb{E}_{(\mathbf{x},y)\sim\tilde{p}_{\text{data}}}\left[\ln \tilde{p}_{\text{data}}(\mathbf{x}, y)\right] - \mathbb{E}_{(\mathbf{x},y)\sim\tilde{p}_{\text{data}}}\left[\ln p_{\text{model}}(\mathbf{x}, y; \boldsymbol{\theta})\right]. \end{aligned} \tag{2.185}$$

Notice that the first term $\mathbb{E}_{\tilde{p}_{\text{data}}}[\ln \tilde{p}_{\text{data}}]$ is the negative empirical entropy of the dataset, which depends strictly on the fixed training data and is completely independent of the model parameters $\boldsymbol{\theta}$.

Evaluating the expectation in the second term over the empirical distribution $\tilde{p}_{\text{data}}$:

$$\mathbb{E}_{(\mathbf{x},y)\sim\tilde{p}_{\text{data}}}\left[\ln p_{\text{model}}(\mathbf{x}, y; \boldsymbol{\theta})\right] = \frac{1}{N}\sum_{i=1}^{N} \ln p_{\text{model}}(\mathbf{x}_i, y_i; \boldsymbol{\theta}). \tag{2.186}$$

Therefore:

$$\arg\min_{\boldsymbol{\theta}} D_{\text{KL}}\Big(\tilde{p}_{\text{data}} \,\Big\|\, p_{\text{model}}\Big) = \arg\min_{\boldsymbol{\theta}} \left(-\frac{1}{N}\sum_{i=1}^{N} \ln p_{\text{model}}(\mathbf{x}_i, y_i; \boldsymbol{\theta})\right) = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{N} \ln p_{\text{model}}(\mathbf{x}_i, y_i; \boldsymbol{\theta}). \tag{2.187}$$

□

This fundamental equivalence proves that Maximum Likelihood Estimation is not an arbitrary optimization heuristic; it is the unique statistical procedure that minimizes information-theoretic divergence from reality to model.

2.5.11 Point estimator bias and Bessel's correction

To complete our foundational statistical grounding, we formalize the properties of **point estimators** [6, 32].

A **point estimator** $\hat{\boldsymbol{\theta}}_N = g(\mathbf{x}_1, \ldots, \mathbf{x}_N)$ is a mathematical function mapping an observed sample of N data points to an estimate of the true underlying population parameter $\boldsymbol{\theta}^*$. Because the training sample is stochastically drawn from $\mathcal{D}$, the estimator $\hat{\boldsymbol{\theta}}_N$ is itself a **random variable** possessing its own sampling distribution, mean, and variance.

Estimator Bias and Standard Error.

- **Estimator Bias**: The difference between the expected value of the estimator across infinite hypothetical samples and the true parameter:

$$\text{Bias}(\hat{\boldsymbol{\theta}}_N) \triangleq \mathbb{E}_{\mathcal{D}^N}[\hat{\boldsymbol{\theta}}_N] - \boldsymbol{\theta}^*. \tag{2.188}$$

  An estimator is **unbiased** if $\text{Bias}(\hat{\boldsymbol{\theta}}_N) = \mathbf{0}$, meaning $\mathbb{E}[\hat{\boldsymbol{\theta}}_N] = \boldsymbol{\theta}^*$. It is **asymptotically unbiased** if $\lim_{N\to\infty} \text{Bias}(\hat{\boldsymbol{\theta}}_N) = \mathbf{0}$.
- **Estimator Variance and Standard Error**: The variance of the estimator across sample realizations is $\text{Var}(\hat{\boldsymbol{\theta}}_N)$. The **Standard Error (SE)** is the standard deviation of the sampling distribution. For the sample mean $\bar{x} = \frac{1}{N}\sum_{i=1}^N x_i$ of i.i.d. variables with variance σ^2:

$$\text{SE}(\bar{x}) = \sqrt{\text{Var}\left(\frac{1}{N}\sum_{i=1}^N x_i\right)} = \sqrt{\frac{1}{N^2}\sum_{i=1}^N \sigma^2} = \frac{\sigma}{\sqrt{N}}. \tag{2.189}$$

  This establishes the classic $\mathcal{O}(1/\sqrt{N})$ rate of statistical convergence.

Analytical Proof of Bias in Sample Variance and Bessel's Correction. Consider drawing N independent observations $x_1, \ldots, x_N \sim \mathcal{N}(\mu, \sigma^2)$. The sample mean $\bar{x} = \frac{1}{N}\sum_{i=1}^N x_i$ is an unbiased estimator of μ:

$$\mathbb{E}[\bar{x}] = \frac{1}{N}\sum_{i=1}^N \mathbb{E}[x_i] = \mu. \tag{2.190}$$

The Maximum Likelihood Estimator for the variance evaluates to:

$$\hat{\sigma}^2_{\text{ML}} = \frac{1}{N}\sum_{i=1}^N (x_i - \bar{x})^2. \tag{2.191}$$

What is the expected value of $\hat{\sigma}^2_{\text{ML}}$? Does it equal the true variance σ^2?

Proof of Bias in Sample Variance [6]. Expand the deviation $x_i - \bar{x}$ by centering around the true population mean μ:

$$x_i - \bar{x} = (x_i - \mu) - (\bar{x} - \mu). \tag{2.192}$$

Squaring both sides:

$$(x_i - \bar{x})^2 = (x_i - \mu)^2 - 2(x_i - \mu)(\bar{x} - \mu) + (\bar{x} - \mu)^2. \tag{2.193}$$

Summing over all N data points:

$$\begin{aligned}\sum_{i=1}^{N}(x_i-\bar{x})^2 &= \sum_{i=1}^{N}(x_i-\mu)^2 - 2(\bar{x}-\mu)\underbrace{\sum_{i=1}^{N}(x_i-\mu)}_{=N(\bar{x}-\mu)} + N(\bar{x}-\mu)^2 \\ &= \sum_{i=1}^{N}(x_i-\mu)^2 - 2N(\bar{x}-\mu)^2 + N(\bar{x}-\mu)^2 \\ &= \sum_{i=1}^{N}(x_i-\mu)^2 - N(\bar{x}-\mu)^2. \end{aligned} \tag{2.194}$$

Now, take the mathematical expectation of Equation (2.194):

$$\mathbb{E}\left[\sum_{i=1}^{N}(x_i-\bar{x})^2\right] = \sum_{i=1}^{N}\mathbb{E}\left[(x_i-\mu)^2\right] - N\mathbb{E}\left[(\bar{x}-\mu)^2\right]. \tag{2.195}$$

Notice the two expectations:

1. By definition of population variance: $\mathbb{E}[(x_i-\mu)^2] = \sigma^2$, so the sum evaluates to $N\sigma^2$.
2. By definition of the variance of the sample mean: $\mathbb{E}[(\bar{x}-\mu)^2] = \mathrm{Var}(\bar{x}) = \frac{\sigma^2}{N}$.

Substituting these two expectations:

$$\mathbb{E}\left[\sum_{i=1}^{N}(x_i-\bar{x})^2\right] = N\sigma^2 - N\left(\frac{\sigma^2}{N}\right) = N\sigma^2 - \sigma^2 = (N-1)\sigma^2. \tag{2.196}$$

Dividing by N to evaluate the expectation of the Maximum Likelihood estimator:

$$\boxed{\mathbb{E}[\hat{\sigma}^2_{\mathrm{ML}}] = \frac{N-1}{N}\sigma^2} \tag{2.197}$$

□

Equation (2.197) is a foundational theorem: **the Maximum Likelihood estimator of variance is systematically biased downward by a factor of $\frac{N-1}{N}$!**

Because the sample mean $\bar{x}$ is calculated from the exact same data points, the deviations $(x_i-\bar{x})$ are measured relative to the empirical sample center rather than the true center μ. The sample points are mathematically closer to their own average $\bar{x}$ than to the true mean μ, systematically underestimating the spread.

To construct an **unbiased estimator of variance**, we multiply by $\frac{N}{N-1}$, yielding the celebrated **Bessel's Correction**:

$$S^2 \triangleq \frac{1}{N-1}\sum_{i=1}^{N}(x_i-\bar{x})^2 \implies \mathbb{E}[S^2] = \sigma^2. \tag{2.198}$$

The denominator $N-1$ reflects the loss of one **degree of freedom**: once the sample mean $\bar{x}$ is computed, only $N-1$ deviations are freely variable because $\sum_{i=1}^{N}(x_i-\bar{x}) = 0$.

The Bridge to Chapter 3: From Probabilistic Risk to Iterative Optimization. With the complete foundations of probability theory, information theory, generative modeling, statistical decision theory, and empirical risk minimization established, a fundamental computational challenge emerges.

In classical generative models (Naïve Bayes, LDA), parameter estimation admits closed-form analytical formulas (counting frequencies or inverting sample covariance matrices). However,

when we transition to discriminative models parameterized by non-linear compositions (such as Logistic Regression and Deep Neural Networks), setting the empirical risk gradient to zero $\nabla_{\boldsymbol{\theta}} \hat{R}_N(\boldsymbol{\theta}) = \mathbf{0}$ yields transcendental, non-linear algebraic systems where closed-form solutions do not exist.

To navigate continuous loss landscapes, minimize empirical risk, and optimize non-linear classifiers, machine learning must abandon closed-form algebraic inversions in favor of first-order numerical optimization: **Gradient Descent**. How to formulate loss geometries, escape saddle points, and navigate non-convex surfaces forms the foundational core of Chapter 3.

Chapter 3

Linear predictors and gradient optimization

In the quest to construct predictive machines from empirical observations, the most natural starting point is the **linear model**—a hypothesis class that models outputs as affine combinations of input features. Linear models possess an extraordinary mathematical elegance: under the classic squared error criterion, their optimal parameters admit a closed-form, globally optimal analytical solution via the celebrated **Normal Equations**.

However, as statistical learning scales to real-world complexities, this apparent simplicity fractures along three profound theoretical and computational fault lines:

1. **Generalization and capacity**: When linear models are enriched with non-linear basis expansions (such as high-degree polynomials), they acquire immense expressive capacity. Without principled constraints, they succumb to catastrophic **overfitting**—memorizing sample noise while losing all predictive power on unseen data.
2. **High-dimensional breakdown of closed-form algebra**: Computing analytical matrix inversions requires $\mathcal{O}(D^3)$ computational steps. When the feature dimension D exceeds the number of observations N ($D > N$), the sample covariance matrix becomes strictly singular, causing classical least squares to collapse into an ill-posed system with infinitely many solutions.
3. **The regression-to-classification barrier**: When discrete labels replace continuous targets, least squares outputs uncalibrated, unbounded values that violate probability axioms, and remote outliers exert quadratic leverage that destabilizes decision boundaries. Replacing squared error with probabilistic cross-entropy eliminates these pathologies, but destroys the existence of closed-form solutions: setting the gradient to zero yields a transcendental system that **cannot be inverted algebraically**.

To conquer these barriers, machine learning must abandon closed-form linear algebra and embrace two foundational pillars: **regularization** (to tame capacity and restore matrix invertibility) and **iterative numerical optimization** (gradient descent, momentum, and stochastic mini-batch dynamics). This chapter charts this complete intellectual journey from closed-form least squares to modern large-scale optimization.

3.1 Linear regression and capacity

In the study of supervised learning, linear models occupy a foundational position as both an intuitive baseline and an analytically tractable laboratory for exploring core statistical principles. By modeling the relationship between input features and continuous targets as an affine transformation, linear regression provides an interpretable mapping whose parameters directly reflect feature importances. Under the classic squared error criterion, the parameter

estimation problem achieves a rare and remarkable property in machine learning: it admits an exact, globally optimal analytical closed-form solution via the celebrated Normal Equations, bypassing the need for exploratory numerical search.

3.1.1 Ordinary least squares and projection

Consider a supervised regression setting where each input instance is represented as a real-valued feature vector $\mathbf{x} = [x_1, x_2, \ldots, x_D]^\top \in \mathbb{R}^D$, and the associated target variable is continuous $y \in \mathbb{R}$. The **linear predictor** posits an affine relationship between inputs and targets:

$$h_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b = \sum_{j=1}^{D} w_j x_j + b, \tag{3.1}$$

where $\mathbf{w} = [w_1, w_2, \ldots, w_D]^\top \in \mathbb{R}^D$ denotes the weight vector (governing the slope or orientation of the predictive hyperplane) and $b \in \mathbb{R}$ denotes the scalar bias (governing the intercept, offsetting the hyperplane from the origin).

To streamline algebraic manipulations and eliminate separate treatment of the bias term, we adopt the standard **homogeneous coordinate augmentation**. We append a constant 1 as a dummy feature to the input vector and absorb the bias directly into the parameter vector:

$$\tilde{\mathbf{x}} \triangleq \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} \in \mathbb{R}^{D+1}, \quad \tilde{\mathbf{w}} \triangleq \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix} \in \mathbb{R}^{D+1} \implies h_{\tilde{\mathbf{w}}}(\tilde{\mathbf{x}}) = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}. \tag{3.2}$$

Given an empirical training dataset consisting of N independently and identically distributed (i.i.d.) observations $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, we assemble the inputs into the **augmented design matrix** $\tilde{\mathbf{X}} \in \mathbb{R}^{N \times (D+1)}$ and the targets into the target vector $\mathbf{y} \in \mathbb{R}^N$:

$$\tilde{\mathbf{X}} = \begin{bmatrix} \tilde{\mathbf{x}}_1^\top \\ \tilde{\mathbf{x}}_2^\top \\ \vdots \\ \tilde{\mathbf{x}}_N^\top \end{bmatrix} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,D} & 1 \\ x_{2,1} & x_{2,2} & \cdots & x_{2,D} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{N,1} & x_{N,2} & \cdots & x_{N,D} & 1 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}. \tag{3.3}$$

In this vectorized matrix notation, the predictions across all N training samples are expressed compactly as the matrix-vector product:

$$\hat{\mathbf{y}} \triangleq \tilde{\mathbf{X}} \tilde{\mathbf{w}} \in \mathbb{R}^N. \tag{3.4}$$

To determine the optimal parameter vector $\tilde{\mathbf{w}}^*$, we define an empirical loss metric penalizing discrepancies between model predictions $\hat{y}_i$ and ground-truth targets y_i. Under the **Ordinary Least Squares (OLS)** framework, we adopt the squared error loss $\ell(h(\mathbf{x}), y) = \frac{1}{2}(h(\mathbf{x}) - y)^2$. The empirical risk (mean squared error objective) over the training set is:

$$J(\tilde{\mathbf{w}}) \triangleq \frac{1}{2N} \sum_{i=1}^{N} \left(\tilde{\mathbf{x}}_i^\top \tilde{\mathbf{w}} - y_i \right)^2 = \frac{1}{2N} \|\tilde{\mathbf{X}} \tilde{\mathbf{w}} - \mathbf{y}\|_2^2. \tag{3.5}$$

Theoretical Deep Dive: Probabilistic Derivation of Least Squares from Maximum Likelihood

Why squared error? In Chapter 2, we proved that minimizing empirical risk under squared loss corresponds to Maximum Likelihood Estimation (MLE) under a conditional Gaussian noise model. Specifically, assume the continuous target y is generated by a deterministic linear mapping corrupted by additive zero-mean Gaussian noise:

$$y = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} + \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, \sigma^2). \tag{3.6}$$

The conditional probability density of observing target y given input $\tilde{\mathbf{x}}$ is therefore:

$$p(y \mid \tilde{\mathbf{x}}; \tilde{\mathbf{w}}, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}})^2}{2\sigma^2}\right). \tag{3.7}$$

Assuming observations are drawn i.i.d., the total conditional log-likelihood of the dataset $\mathcal{D}$ evaluates to:

$$\begin{aligned}
\log p(\mathbf{y} \mid \tilde{\mathbf{X}}; \tilde{\mathbf{w}}, \sigma^2) &= \sum_{i=1}^{N} \log p(y_i \mid \tilde{\mathbf{x}}_i; \tilde{\mathbf{w}}, \sigma^2) \\
&= -\frac{N}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{N}\left(y_i - \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_i\right)^2 \\
&= -\frac{N}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\|\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y}\|_2^2.
\end{aligned} \tag{3.8}$$

Because the first term is independent of $\tilde{\mathbf{w}}$, maximizing the log-likelihood with respect to parameters $\tilde{\mathbf{w}}$ is mathematically equivalent to minimizing the squared Euclidean norm $\|\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y}\|_2^2$:

$$\hat{\tilde{\mathbf{w}}}_{\text{MLE}} = \arg\max_{\tilde{\mathbf{w}}} \log p(\mathbf{y} \mid \tilde{\mathbf{X}}; \tilde{\mathbf{w}}, \sigma^2) \equiv \arg\min_{\tilde{\mathbf{w}}} \frac{1}{2N}\|\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y}\|_2^2 = \arg\min_{\tilde{\mathbf{w}}} J(\tilde{\mathbf{w}}). \tag{3.9}$$

Thus, ordinary least squares is not merely an arbitrary heuristic: it is the optimal maximum likelihood estimator under homoscedastic Gaussian observation noise.

Because the empirical risk $J(\tilde{\mathbf{w}})$ is a quadratic function of $\tilde{\mathbf{w}}$ and its Hessian $\nabla^2_{\tilde{\mathbf{w}}} J(\tilde{\mathbf{w}}) = \frac{1}{N}\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}$ is positive semi-definite (strictly positive definite when $\tilde{\mathbf{X}}$ has full column rank), the objective is convex and any stationary point is a global minimizer. We can therefore find its exact global minimum by expanding the objective in matrix form and setting its vector gradient to zero.

Derivation of the Normal Equations. Expanding the squared L_2 norm:

$$\begin{aligned}
J(\tilde{\mathbf{w}}) &= \frac{1}{2N}(\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y})^\top(\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y}) \\
&= \frac{1}{2N}\left((\tilde{\mathbf{w}}^\top\tilde{\mathbf{X}}^\top - \mathbf{y}^\top)(\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \mathbf{y})\right) \\
&= \frac{1}{2N}\left(\tilde{\mathbf{w}}^\top\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \tilde{\mathbf{w}}^\top\tilde{\mathbf{X}}^\top\mathbf{y} - \mathbf{y}^\top\tilde{\mathbf{X}}\tilde{\mathbf{w}} + \mathbf{y}^\top\mathbf{y}\right).
\end{aligned} \tag{3.10}$$

Since $\tilde{\mathbf{w}}^\top\tilde{\mathbf{X}}^\top\mathbf{y}$ is a scalar, it equals its transpose $\mathbf{y}^\top\tilde{\mathbf{X}}\tilde{\mathbf{w}}$. Combining these identical bilinear terms yields:

$$J(\tilde{\mathbf{w}}) = \frac{1}{2N}\left(\tilde{\mathbf{w}}^\top\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}\tilde{\mathbf{w}} - 2\mathbf{y}^\top\tilde{\mathbf{X}}\tilde{\mathbf{w}} + \mathbf{y}^\top\mathbf{y}\right). \tag{3.11}$$

We now take the vector derivative with respect to $\tilde{\mathbf{w}}$. We invoke the two canonical matrix calculus identities:

$$\nabla_{\mathbf{z}}(\mathbf{z}^\top\mathbf{A}\mathbf{z}) = (\mathbf{A} + \mathbf{A}^\top)\mathbf{z} = 2\mathbf{A}\mathbf{z} \quad \text{(for symmetric } \mathbf{A}\text{)}, \qquad \nabla_{\mathbf{z}}(\mathbf{c}^\top\mathbf{z}) = \mathbf{c}. \tag{3.12}$$

Applying these identities to Eq. (3.11) (where $\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}$ is symmetric):

$$\nabla_{\tilde{\mathbf{w}}} J(\tilde{\mathbf{w}}) = \frac{1}{2N}\left(2\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}\tilde{\mathbf{w}} - 2\tilde{\mathbf{X}}^\top\mathbf{y}\right) = \frac{1}{N}\left(\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}\tilde{\mathbf{w}} - \tilde{\mathbf{X}}^\top\mathbf{y}\right). \tag{3.13}$$

Setting the gradient to the zero vector $\nabla_{\tilde{\mathbf{w}}} J(\tilde{\mathbf{w}}) = \mathbf{0}$ yields the celebrated **Normal Equations**:

$$\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}} = \tilde{\mathbf{X}}^\top \mathbf{y}. \tag{3.14}$$

If the $(D+1) \times (D+1)$ Gram matrix $\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$ is invertible (non-singular), multiplying both sides by $(\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1}$ delivers the unique analytical closed-form solution:

$$\tilde{\mathbf{w}}^* = (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top \mathbf{y}. \tag{3.15}$$

The matrix $(\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top \triangleq \tilde{\mathbf{X}}^\dagger \in \mathbb{R}^{(D+1)\times N}$ is the classical **Moore-Penrose pseudoinverse** of $\tilde{\mathbf{X}}$ for full column-rank matrices. □

The Normal Equations possess an illuminating geometric interpretation in the N-dimensional sample space $\mathbb{R}^N$ (Bishop, 2024; Strang, 2016).

The target vector $\mathbf{y} = [y_1, \ldots, y_N]^\top$ is an arbitrary point in $\mathbb{R}^N$. Any prediction vector $\hat{\mathbf{y}} = \tilde{\mathbf{X}} \tilde{\mathbf{w}}$ is a linear combination of the columns of the design matrix $\tilde{\mathbf{X}}$:

$$\hat{\mathbf{y}} = \tilde{\mathbf{w}}_1 \tilde{\mathbf{x}}_{*,1} + \tilde{\mathbf{w}}_2 \tilde{\mathbf{x}}_{*,2} + \cdots + \tilde{\mathbf{w}}_{D+1} \tilde{\mathbf{x}}_{*,D+1} \in \mathrm{col}(\tilde{\mathbf{X}}), \tag{3.16}$$

where $\mathrm{col}(\tilde{\mathbf{X}}) \subset \mathbb{R}^N$ is the $(D+1)$-dimensional subspace spanned by the column vectors of $\tilde{\mathbf{X}}$.

Minimizing the squared Euclidean distance $\|\hat{\mathbf{y}} - \mathbf{y}\|_2^2$ means finding the point $\hat{\mathbf{y}}$ within the subspace $\mathrm{col}(\tilde{\mathbf{X}})$ that lies closest to the target vector $\mathbf{y}$. By elementary Euclidean geometry, the shortest distance from an external point to a linear subspace is achieved when the error (residual) vector $\mathbf{e} \triangleq \mathbf{y} - \hat{\mathbf{y}}$ is strictly **orthogonal** to the subspace:

$$\mathbf{e} \perp \mathrm{col}(\tilde{\mathbf{X}}) \iff \tilde{\mathbf{x}}_{*,j}^\top \mathbf{e} = 0 \quad \forall j \in \{1, \ldots, D+1\} \iff \tilde{\mathbf{X}}^\top (\mathbf{y} - \tilde{\mathbf{X}} \tilde{\mathbf{w}}^*) = \mathbf{0}, \tag{3.17}$$

which is precisely the Normal Equations $\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \tilde{\mathbf{w}}^* = \tilde{\mathbf{X}}^\top \mathbf{y}$!

Substituting the optimal weights $\tilde{\mathbf{w}}^*$ into the prediction vector:

$$\hat{\mathbf{y}} = \tilde{\mathbf{X}} \tilde{\mathbf{w}}^* = \tilde{\mathbf{X}} (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top \mathbf{y} \triangleq \mathbf{P}_{\tilde{\mathbf{X}}} \mathbf{y}, \tag{3.18}$$

where $\mathbf{P}_{\tilde{\mathbf{X}}} \triangleq \tilde{\mathbf{X}} (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top \in \mathbb{R}^{N \times N}$ is the **orthogonal projection operator** (or *hat matrix* $\mathbf{H}$, because it puts the "hat" on $\mathbf{y}$). The projection matrix satisfies:

1. **Symmetry**: $\mathbf{P}_{\tilde{\mathbf{X}}}^\top = (\tilde{\mathbf{X}} (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top)^\top = \tilde{\mathbf{X}} (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top = \mathbf{P}_{\tilde{\mathbf{X}}}$.
2. **Idempotency**: $\mathbf{P}_{\tilde{\mathbf{X}}}^2 = \tilde{\mathbf{X}} (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top = \tilde{\mathbf{X}} (\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top = \mathbf{P}_{\tilde{\mathbf{X}}}$. Projecting an already-projected vector leaves it unchanged.

Figure 3.1 visually demonstrates how Ordinary Least Squares computes the unique point $\hat{\mathbf{y}}$ within the subspace $\mathrm{col}(\tilde{\mathbf{X}})$ closest to $\mathbf{y}$.

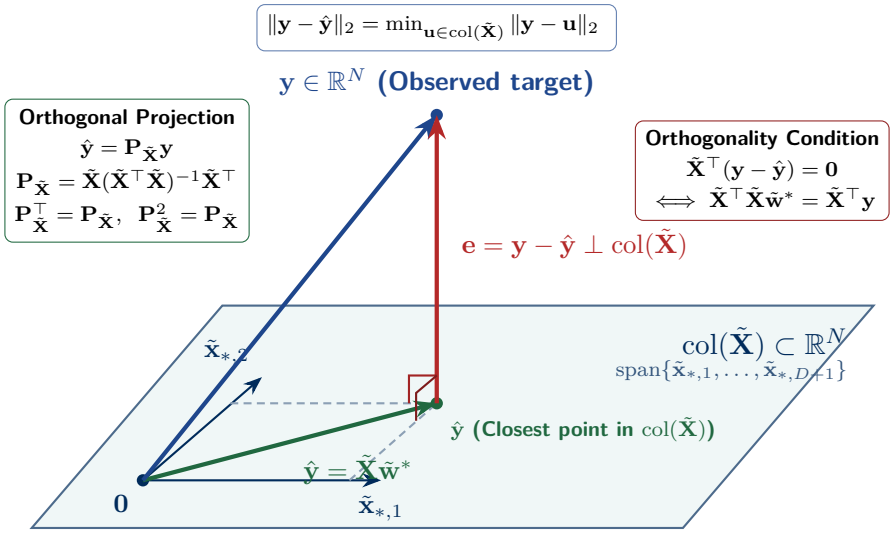


Figure 3.1: Geometric interpretation of Ordinary Least Squares as an orthogonal projection onto the column space $\mathrm{col}(\tilde{\mathbf{X}}) \subset \mathbb{R}^N$. The target observation vector $\mathbf{y}$ lies in general position in the N-dimensional sample space. The linear predictor's feasible output space is restricted to the $(D+1)$-dimensional subspace $\mathrm{col}(\tilde{\mathbf{X}})$ spanned by the columns of the design matrix. The least squares solution $\hat{\mathbf{y}} = \tilde{\mathbf{X}}\tilde{\mathbf{w}}^*$ is the unique vector in $\mathrm{col}(\tilde{\mathbf{X}})$ closest to $\mathbf{y}$ in Euclidean norm, attained precisely when the residual error vector $\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$ is perpendicular to the subspace.

Despite its mathematical closed-form elegance, deploying the Normal Equations in real-world machine learning encounters severe computational and theoretical limitations:

Cubic computational complexity $\mathcal{O}(D^3)$. To compute $\tilde{\mathbf{w}}^* = (\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}})^{-1}\tilde{\mathbf{X}}^\top\mathbf{y}$:

- Multiplying $\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}$ costs $\mathcal{O}(ND^2)$ operations.
- Multiplying $\tilde{\mathbf{X}}^\top\mathbf{y}$ costs $\mathcal{O}(ND)$ operations.
- Inverting the $(D+1)\times(D+1)$ matrix (or solving the linear system via Cholesky factorization $\mathbf{L}\mathbf{L}^\top$) costs $\mathcal{O}(D^3)$ operations.

Total time complexity is $\mathcal{O}(ND^2 + D^3)$, and memory storage is $\mathcal{O}(D^2)$ to store the Gram matrix. In modern applications (such as natural language processing or visual feature representations) where $D \approx 10^5$–10^6, $\mathcal{O}(D^3) \approx 10^{15}$ operations, requiring gigabytes of RAM and days of compute. Closed-form matrix inversion becomes computationally impossible.

Rank deficiency in the high-dimensional regime ($D > N$). When the number of features exceeds the number of observations ($D > N$), the maximum possible rank of the design matrix is bounded by the number of samples:

$$\mathrm{rank}(\tilde{\mathbf{X}}) \leq \min(N, D+1) = N < D+1. \tag{3.19}$$

Consequently, the $(D+1) \times (D+1)$ Gram matrix $\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}$ has rank at most N. It has at least $(D+1-N)$ zero eigenvalues, which implies its determinant is identically zero: $\det(\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}) = 0$.

The Gram matrix is strictly **singular and non-invertible**. The Normal Equations become an under-determined system with infinitely many parameter vectors that achieve zero training error. Standard least squares provides no mechanism to choose among these infinite solutions, and direct inversion fails catastrophically.

Collinearity and numerical instability. Even when $N > D$, if two or more input features are strongly correlated, the columns of $\tilde{\mathbf{X}}$ become nearly linearly dependent. The smallest eigenvalue $\lambda_{\min}$ of $\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}$ approaches zero, causing the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ to

explode. Matrix inversion in floating-point arithmetic amplifies minute measurement noise in $\mathbf{y}$, leading to wild swings and explosive parameter variances.

These computational bottlenecks and structural failure modes reveal that closed-form Ordinary Least Squares cannot suffice for high-dimensional or non-linear prediction. Even when $N > D$ and the Gram matrix is strictly invertible, an affine hyperplane cannot capture curved physical patterns without expanding the input feature representation. In the next subsection, we venture beyond linearity and uncover the delicate interplay between model capacity and generalization.

3.1.2 Non-linear basis expansions

When the true data-generating distribution is non-linear, a simple affine hyperplane $h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ produces severe systematic errors. We can extend the expressive power of linear models by transforming the input vector $\mathbf{x} \in \mathbb{R}^D$ through a fixed non-linear feature mapping:

$$\boldsymbol{\phi}(\mathbf{x}) = [\phi_0(\mathbf{x}), \phi_1(\mathbf{x}), \ldots, \phi_M(\mathbf{x})]^\top \in \mathbb{R}^{M+1}, \tag{3.20}$$

where $\phi_j(\mathbf{x})$ are called **basis functions** (with $\phi_0(\mathbf{x}) \equiv 1$ serving as the bias dummy feature). The predictive hypothesis becomes:

$$h_{\mathbf{w}}(\mathbf{x}) = \sum_{j=0}^{M} w_j \phi_j(\mathbf{x}) = \mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x}). \tag{3.21}$$

The decisive theoretical property of basis function expansion is that while $h_{\mathbf{w}}(\mathbf{x})$ is a **non-linear function of the input $\mathbf{x}$**, it remains strictly a **linear function of the parameters $\mathbf{w}$**.
Because the parameters enter linearly, the loss function $J(\mathbf{w}) = \frac{1}{2N}\|\boldsymbol{\Phi}\mathbf{w} - \mathbf{y}\|_2^2$ remains a strictly quadratic, convex bowl! The Normal Equations still apply identically by replacing the raw design matrix $\tilde{\mathbf{X}}$ with the transformed feature matrix $\boldsymbol{\Phi} \in \mathbb{R}^{N \times (M+1)}$:

$$\mathbf{w}^* = (\boldsymbol{\Phi}^\top \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^\top \mathbf{y}. \tag{3.22}$$

In one-dimensional regression ($x \in \mathbb{R}$), the most classic basis expansion is the **polynomial basis**:

$$\boldsymbol{\phi}(x) = [1, x, x^2, x^3, \ldots, x^M]^\top \implies h_{\mathbf{w}}(x) = \sum_{j=0}^{M} w_j x^j. \tag{3.23}$$

The corresponding feature matrix $\boldsymbol{\Phi}$ is the classical **Vandermonde matrix**:

$$\boldsymbol{\Phi} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^M \\ 1 & x_2 & x_2^2 & \cdots & x_2^M \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \cdots & x_N^M \end{bmatrix} \in \mathbb{R}^{N \times (M+1)}. \tag{3.24}$$

3.1.3 Model capacity and overfitting

The polynomial degree M acts as a direct control dial for the **capacity** of the hypothesis class $\mathcal{H}_M = \{h(x) = \sum_{j=0}^{M} w_j x^j\}$. Capacity measures the richness or flexibility of the functional shapes that the model can represent. Every model class is constrained by its **inductive bias**—the structural assumptions that restrict its search space before observing data. For

polynomial regression, the inductive bias is that the relationship between x and y can be faithfully approximated by a smooth polynomial of order $\leq M$.

To investigate the dynamics of capacity, consider $N = 10$ noisy observations sampled from a ground-truth non-linear function corrupted by Gaussian noise:

$$y_i = \sin(2\pi x_i) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, 0.2^2), \quad x_i \in [0, 1]. \tag{3.25}$$

We fit polynomial models across three representative degrees: $M = 1$, $M = 3$, and $M = 9$ (illustrated in Figure 3.2).

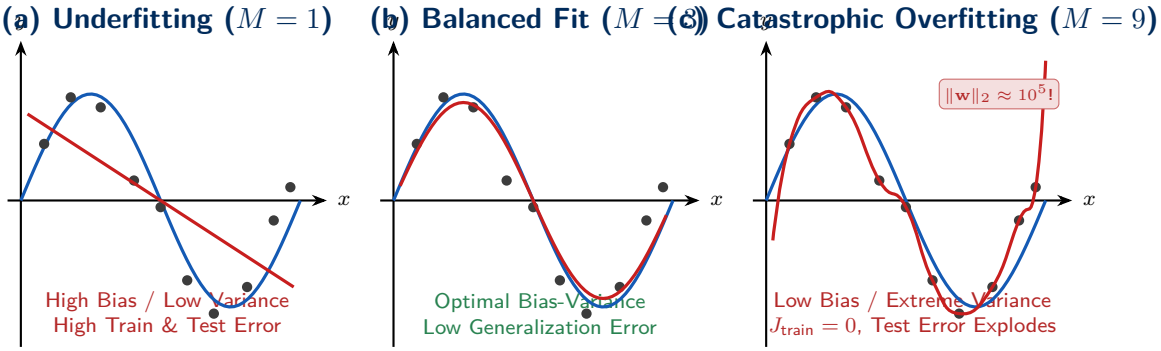


Figure 3.2: The capacity triad in polynomial curve fitting ($N = 10$ noisy sinusoidal samples). (a) Degree $M = 1$ possesses insufficient capacity, underfitting the true non-linear generating curve (high bias). (b) Degree $M = 3$ achieves optimal capacity, balancing expressive power and noise immunity. (c) Degree $M = 9$ possesses 10 degrees of freedom, passing identically through every training sample ($J_{\text{train}} = 0$). Between data points, the curve oscillates wildly with massive parameter magnitudes ($\|\mathbf{w}\|_2 \sim 10^5$), causing catastrophic generalization failure.

1. **Underfitting ($M = 1$)**: A linear hypothesis $w_0 + w_1 x$ possesses insufficient capacity. It cannot flex to follow the sinusoidal curve. Both the training error $\hat{R}_{\text{train}}$ and test error $\hat{R}_{\text{test}}$ are large. The model is crippled by **high bias** (excessively rigid inductive bias).

2. **Optimal fit ($M = 3$)**: The cubic polynomial provides just enough degrees of freedom to capture the inflection points and curvature of $\sin(2\pi x)$ while smoothing over the stochastic noise ϵ_i. Both training error and test error are low and closely matched.

3. **Overfitting ($M = 9$)**: With $M = 9$, the model has $M + 1 = 10$ parameters to fit $N = 10$ points. The system has as many degrees of freedom as data points! The polynomial passes *identically through every single observation*, achieving perfect training error: $J(\mathbf{w}^*) = 0$. However, between adjacent data points, the curve oscillates wildly into unphysical extremes. When evaluated on fresh test samples from $\sin(2\pi x)$, the test error explodes toward infinity. The model suffers from **high variance**: fitting the idiosyncratic noise of the training sample rather than the underlying physical law.

Oral Exam Insight & Pitfall Alert: The Signature of Overfitting: Weight Explosion

How does a degree-9 polynomial manage to oscillate violently while hitting 10 specific coordinates? As Christopher Bishop emphasizes (Bishop, 2024), **overfitting leaves an indelible mathematical signature in the magnitude of the learned parameters**. Table 3.1 displays the estimated parameter values for increasing degrees M.

Table 3.1: Parameter estimates $\mathbf{w}^*$ for polynomial models of varying degree M ($N = 10$). Notice the dramatic explosion of parameter magnitudes for $M = 9$.

Parameter	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0	0.48	0.82	0.31	0.35
w_1	—	−1.27	11.23	232.37
w_2	—	—	−33.84	−5,321.80
w_3	—	—	21.95	48,562.10
w_4	—	—	—	−231,240.50
w_5	—	—	—	612,340.20
w_6	—	—	—	−941,560.80
w_7	—	—	—	832,110.40
w_8	—	—	—	−389,220.10
w_9	—	—	—	74,512.90

For $M = 9$, the weights blow up into the hundreds of thousands, alternating wildly with opposing positive and negative signs! The model achieves zero training error through a precarious, razor-thin numerical balancing act: massive positive powers are cancelled out by massive negative powers at the exact sample coordinates x_i, but a fraction of a millimeter away from x_i, this cancellation collapses, unleashing catastrophic oscillations. This profound empirical discovery establishes the cornerstone of modern regularization theory: **to prevent overfitting, we must constrain the magnitudes of the weight vector**.

Validation protocols, the generalization gap, and k-fold cross-validation. Because empirical training error $\hat{R}_{\text{train}}(\mathbf{w})$ is an overly optimistic, biased estimator of model performance, we must implement rigorous statistical protocols to evaluate **generalization**:

Definition 3.1 (Generalization Gap)**.** Let $\mathcal{R}(h) = \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}[\ell(h(\mathbf{x}), y)]$ denote the true expected risk over the data-generating distribution, and let $\hat{R}_N(h) = \frac{1}{N}\sum_{i=1}^{N} \ell(h(\mathbf{x}_i), y_i)$ denote the empirical training risk. The **generalization gap** is defined as:

$$\Delta_{\text{gen}}(h) \triangleq \mathcal{R}(h) - \hat{R}_N(h). \tag{3.26}$$

As model capacity increases, $\hat{R}_N(h)$ decreases monotonically toward zero, but the generalization gap $\Delta_{\text{gen}}(h)$ widens dramatically, causing true risk $\mathcal{R}(h)$ to follow a characteristic U-shaped curve.

To measure true risk without bias and perform hyperparameter selection (e.g., choosing polynomial degree M or regularization strength λ), the dataset $\mathcal{D}$ must be partitioned into three mutually disjoint subsets:

1. **Training set** (60%–80%): Used exclusively to solve the Normal Equations and estimate parameter weights $\tilde{\mathbf{w}}^*$.
2. **Validation set** (10%–20%): Evaluated across candidate models to tune hyperparameters (selecting optimal M^* or λ^*).
3. **Test set** (10%–20%): Held out in a metaphorical sealed vault. It is accessed *only once* after all modeling choices are frozen, providing an uncontaminated estimate of real-world risk. Evaluating on the test set iteratively to make modeling decisions is a fatal methodological error known as **data leakage** or **data snooping**.

When total sample size N is small, holding out a large static validation set is excessively wasteful. To utilize data efficiently while maintaining low variance, we employ **k-fold cross-validation**:

Algorithm 1 k-Fold Cross-Validation Protocol

Require: Dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, number of folds k, candidate hyperparameter configuration θ.

1: Randomly partition $\mathcal{D}$ into k equal-sized, mutually disjoint folds: $\mathcal{D} = \bigcup_{j=1}^{k} \mathcal{D}_j$, where $|\mathcal{D}_j| \approx N/k$.
2: **for** $j = 1$ **to** k **do**
3: Form validation fold $\mathcal{D}_{\text{val}}^{(j)} = \mathcal{D}_j$ and training fold $\mathcal{D}_{\text{train}}^{(j)} = \mathcal{D} \setminus \mathcal{D}_j$.
4: Train model on $\mathcal{D}_{\text{train}}^{(j)}$ to obtain optimal parameters $\hat{\mathbf{w}}^{(-j)}(\theta)$.
5: Compute validation loss on the held-out fold:
$$\mathcal{L}_j(\theta) = \frac{1}{|\mathcal{D}_j|} \sum_{(\mathbf{x},y)\in\mathcal{D}_j} \ell\left(h_{\hat{\mathbf{w}}^{(-j)}(\theta)}(\mathbf{x}), y\right).$$
6: **end for**
7: Compute the overall cross-validation score:
$$\text{CV}_{(k)}(\theta) = \frac{1}{k} \sum_{j=1}^{k} \mathcal{L}_j(\theta).$$
8: **return** $\text{CV}_{(k)}(\theta)$.

Standard practice sets $k = 5$ or $k = 10$. In the extreme limit where $k = N$, the procedure is called **Leave-One-Out Cross-Validation (LOOCV)**. In each iteration, exactly one sample is held out for validation, and the model is trained on the remaining $N - 1$ samples.

While training N separate models is computationally prohibitive for general algorithms, ordinary least squares admits a celebrated closed-form shortcut for LOOCV known as **Allen's PRESS statistic** (Allen, 1974):

$$\text{LOOCV} = \frac{1}{N} \sum_{i=1}^{N} \left(\frac{y_i - \hat{y}_i}{1 - H_{ii}} \right)^2, \tag{3.27}$$

where $\hat{y}_i$ is the prediction of the model trained on the *full* dataset, and $H_{ii} = [\mathbf{P}_{\tilde{\mathbf{X}}}]_{ii} = [\tilde{\mathbf{X}}(\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}})^{-1}\tilde{\mathbf{X}}^\top]_{ii}$ is the i-th diagonal element of the hat projection matrix (measuring the leverage of observation i). Remarkably, Eq. (3.27) computes the exact LOOCV error in a single closed-form pass without retraining the model N times!

Although basis function expansions enable linear models to approximate intricate non-linear phenomena, unconstrained capacity comes at a severe operational cost: when the basis dimension M is large, the parameter vector experiences explosive weight oscillations ($|w_j| \to \infty$) that fit idiosyncratic sample noise and decimate test generalization. Furthermore, in high-dimensional or collinear regimes, the design matrix $\mathbf{\Phi}$ yields a singular or severely ill-conditioned Gram matrix $\mathbf{\Phi}^\top\mathbf{\Phi}$, causing ordinary least squares to lose its uniqueness and collapse into extreme numerical instability. Rather than crudely truncating candidate feature dimensions, statistical learning demands a principled framework that preserves expressive capacity while systematically penalizing excessive parameter magnitude. This motivates the regularization paradigm explored in Section 3.2.

3.2 Regularization for linear models

In high-dimensional learning, the core challenge is balancing model capacity against generalization error. When a linear predictor possesses more degrees of freedom than can be reliably constrained by available observations, the empirical risk minimizer naturally exploits these excess dimensions to achieve zero training error by driving parameter values to astronomical positive and negative extremes. These runaway weights cancel each other out over the training points, but oscillate wildly across unobserved regions of the input space. Regularization counteracts this failure mode by incorporating a structural inductive bias directly into the objective function, imposing a controlled penalty on parameter complexity.

From an algebraic perspective, regularization addresses the fatal pathology of rank deficiency and ill-conditioning in ordinary least squares. When features are correlated or when the input dimensionality exceeds the dataset size ($D > N$), the sample covariance matrix possesses zero or near-zero eigenvalues, causing standard matrix inversion to fail or explode under floating-point arithmetic. By augmenting the empirical loss with a penalty on parameter norms, regularization reshapes the optimization landscape, dampening the variance of the estimator and restoring mathematical well-posedness to underdetermined systems.

3.2.1 Regularization principles

In Section 3.1.3, we discovered that overfitting in linear basis models manifests as an explosive divergence in parameter magnitudes $\|\mathbf{w}\|_2 \to \infty$. Rather than crudely restricting the hypothesis space by truncating the polynomial degree M (which limits expressive resolution), modern statistical learning adopts the **regularization paradigm**: we allow rich, high-dimensional basis representations, but augment the objective function with a penalty term that discourages complex, runaway weight configurations:

$$\mathcal{L}_{\text{reg}}(\mathbf{w}) \triangleq \mathcal{L}_{\text{data}}(\mathbf{w}) + \lambda\Omega(\mathbf{w}) = \frac{1}{2N}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda\Omega(\mathbf{w}), \tag{3.28}$$

where:

- $\mathcal{L}_{\text{data}}(\mathbf{w})$ is the empirical data misfit loss (e.g., mean squared error).
- $\Omega(\mathbf{w})$ is the **regularizer** (complexity penalty), penalizing undesirable weight properties.
- $\lambda \geq 0$ is the **regularization hyperparameter** governing the trade-off between fitting training observations ($\lambda \to 0$, unconstrained least squares) and forcing parameters toward zero ($\lambda \to \infty$, infinite penalty).

Remark 3.1 (Why the Bias Term is Never Regularized)**.** By universal convention, the intercept (bias) b is **strictly excluded** from the regularization penalty $\Omega(\mathbf{w})$. Regularizing the bias would make the model's predictions dependent on the choice of origin for the target variable y. For example, adding an arbitrary constant offset $+100$ to all targets y_i would force b to absorb $+100$, incurring an artificial regularization penalty. Therefore, the data matrix $\mathbf{X} \in \mathbb{R}^{N\times D}$ is centered ($\sum_{i=1}^{N} x_{ij} = 0$), the targets are centered ($\bar{y} = 0 \implies b = 0$), and $\Omega(\mathbf{w})$ penalizes only the directional weight vector $\mathbf{w} \in \mathbb{R}^D$.

3.2.2 Lasso regularization (L_1)

When dealing with high-dimensional input spaces ($D \gg 1$), many candidate features are redundant, noisy, or completely irrelevant to the underlying physical task. While penalizing weights smoothly can reduce overfitting, interpreting dense parameter vectors remains difficult when thousands of features retain non-zero coefficients. To achieve true parameter sparsity and automated feature selection, Robert Tibshirani (1996) introduced the **Lasso** (*Least Absolute Shrinkage and Selection Operator*), which replaces the Euclidean norm with the L_1 norm penalty:

$$\Omega(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_{j=1}^{D} |w_j|. \tag{3.29}$$

The Lasso objective function is:

$$\mathcal{L}_{\text{Lasso}}(\mathbf{w}) = \frac{1}{2N}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \sum_{j=1}^{D} |w_j|. \tag{3.30}$$

Subgradient calculus and non-smooth optimization. Because the absolute value function $|w|$ possesses a sharp kink (a non-differentiable point) at $w = 0$, standard gradient calculus is inapplicable. Optimization must proceed through **subgradient calculus**:

Definition 3.2 (Subdifferential of the Absolute Value Function)**.** The subdifferential $\partial|w|$ of the scalar absolute value function is the set of all valid subgradients satisfying the subgradient inequality:

$$\partial|w| = \begin{cases} \{+1\} & \text{if } w > 0, \\ \{-1\} & \text{if } w < 0, \\ [-1, +1] & \text{if } w = 0. \end{cases} \tag{3.31}$$

Soft-thresholding operator under decoupled design. To see how the L_1 penalty induces exact sparsity, consider the idealized setting of an **orthonormal design matrix** where $\mathbf{X}^\top \mathbf{X} = \mathbf{I}_D$ (or after feature whitening). Under this decoupling, the loss function decomposes into D completely independent scalar optimization problems:

$$\min_{w_j} \frac{1}{2}(w_j - \hat{w}_j^{\text{OLS}})^2 + \lambda|w_j|, \quad \text{where } \hat{w}_j^{\text{OLS}} = \mathbf{x}_{*,j}^\top \mathbf{y}. \tag{3.32}$$

The first-order optimality condition requires that zero belongs to the subdifferential:

$$0 \in (w_j - \hat{w}_j^{\text{OLS}}) + \lambda \partial|w_j|. \tag{3.33}$$

Analyzing the three branches of the subdifferential:

1. If $w_j^* > 0$: $w_j^* - \hat{w}_j^{\text{OLS}} + \lambda = 0 \implies w_j^* = \hat{w}_j^{\text{OLS}} - \lambda$. This branch requires $\hat{w}_j^{\text{OLS}} > \lambda$.
2. If $w_j^* < 0$: $w_j^* - \hat{w}_j^{\text{OLS}} - \lambda = 0 \implies w_j^* = \hat{w}_j^{\text{OLS}} + \lambda$. This branch requires $\hat{w}_j^{\text{OLS}} < -\lambda$.
3. If $w_j^* = 0$: $0 - \hat{w}_j^{\text{OLS}} + \lambda s = 0$ for some $s \in [-1, +1]$, which implies $|\hat{w}_j^{\text{OLS}}| \leq \lambda$.

Combining these three cases yields the celebrated closed-form **Soft-Thresholding Operator** $\mathcal{S}_\lambda$:

$$w_j^* = \mathcal{S}_\lambda(\hat{w}_j^{\text{OLS}}) \triangleq \text{sign}(\hat{w}_j^{\text{OLS}}) \max\left(0, |\hat{w}_j^{\text{OLS}}| - \lambda\right). \tag{3.34}$$

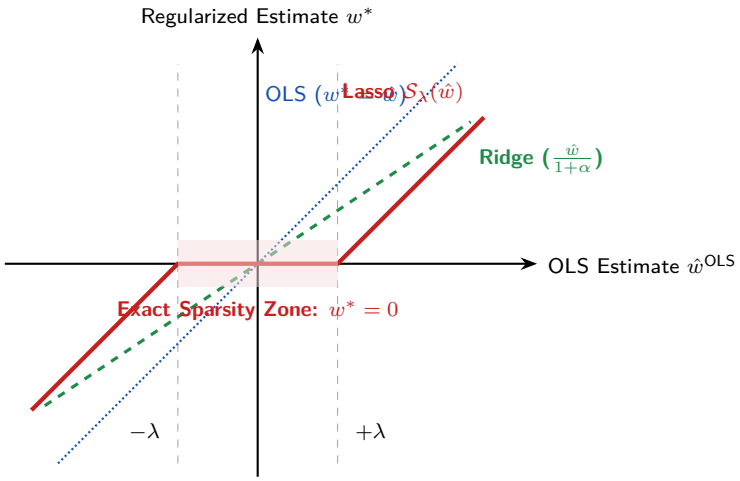


Figure 3.3: Comparison of thresholding operators under orthonormal design. Unconstrained OLS (dotted blue) preserves all values. Ridge regression (dashed green) scales estimates by a constant factor $\frac{1}{1+\alpha}$, shrinking values toward zero but never setting them to zero. Lasso (solid red) applies the soft-thresholding operator $\mathcal{S}_\lambda$: all coefficients with magnitude $|\hat{w}^{\text{OLS}}| \leq \lambda$ are driven **identically to zero**, while coefficients exceeding λ are shifted by constant offset λ.

As visually proved in Figure 3.3:

- If the OLS estimate has magnitude smaller than λ ($|\hat{w}_j^{\text{OLS}}| \leq \lambda$), Lasso drives the parameter **strictly to zero**: $w_j^* = 0$. The feature is completely excised from the model.
- If the magnitude exceeds λ, Lasso translates the coefficient toward zero by constant amount λ.
- Ridge, by contrast, shrinks coefficients proportionally: $w_j^* = \frac{1}{1+\alpha}\hat{w}_j^{\text{OLS}}$, retaining all features in the active set.

Operational trade-offs and limitations of Lasso. Despite its remarkable capacity for exact feature selection, Lasso regression is subject to notable practical and algebraic limitations:

1. **Non-differentiability and computational complexity**: Because the L_1 penalty is non-smooth at the origin, the global optimum cannot be expressed as a linear closed-form matrix inverse. Solving Lasso requires iterative non-smooth convex optimization algorithms (such as coordinate descent or proximal gradient descent).
2. **Feature saturation in high dimensions**: When the feature dimensionality exceeds the sample size ($D > N$), Lasso can select at most N features before saturating, ignoring any remaining relevant variables.
3. **Instability under collinearity**: When multiple features are strongly correlated, Lasso arbitrarily picks one feature from the group and drives the others to zero, displaying erratic sensitivity to small perturbations in training data.
4. **Lack of guaranteed matrix conditioning**: Lasso does not provide guaranteed diagonal eigenvalue loading to stabilize arbitrary ill-conditioned Gram matrices in closed form.

These limitations motivate L_2 regularization, which restores smooth differentiability, provides an unconditional closed-form solution, and guarantees numerical invertibility even when $D > N$.

3.2.3 Ridge regularization (L_2)

When the penalty is the squared Euclidean (L_2) norm of the weight vector, the method is known as **Ridge regression** (or *Tikhonov regularization*):

$$\Omega(\mathbf{w}) = \frac{1}{2}\|\mathbf{w}\|_2^2 = \frac{1}{2}\sum_{j=1}^{D} w_j^2 = \frac{1}{2}\mathbf{w}^\top\mathbf{w}. \tag{3.35}$$

The Ridge regression objective is:

$$\mathcal{L}_{\text{Ridge}}(\mathbf{w}) = \frac{1}{2N}(\mathbf{X}\mathbf{w} - \mathbf{y})^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) + \frac{\lambda}{2}\mathbf{w}^\top\mathbf{w}. \tag{3.36}$$

Closed-form analytical solution. Because the L_2 penalty is quadratic and smooth, we can derive its closed-form solution by differentiating with respect to $\mathbf{w}$:

$$\nabla_{\mathbf{w}}\mathcal{L}_{\text{Ridge}}(\mathbf{w}) = \frac{1}{N}\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda\mathbf{w} = \mathbf{0}. \tag{3.37}$$

Multiplying by N and collecting terms with respect to $\mathbf{w}$:

$$\mathbf{X}^\top\mathbf{X}\mathbf{w} - \mathbf{X}^\top\mathbf{y} + N\lambda\mathbf{w} = \mathbf{0} \iff (\mathbf{X}^\top\mathbf{X} + N\lambda\mathbf{I}_D)\mathbf{w} = \mathbf{X}^\top\mathbf{y}. \tag{3.38}$$

Defining $\alpha \triangleq N\lambda$, the regularized normal equations yield:

$$\mathbf{w}_{\text{Ridge}}^* = (\mathbf{X}^\top\mathbf{X} + \alpha\mathbf{I}_D)^{-1}\mathbf{X}^\top\mathbf{y}. \tag{3.39}$$

Theorem 3.1 (Guaranteed Invertibility and Well-Posedness of Ridge Regression)**.** *For any positive regularization parameter $\alpha > 0$, the regularized matrix $(\mathbf{X}^\top\mathbf{X} + \alpha\mathbf{I}_D)$ is **strictly positive definite and unconditionally invertible**, even when $D > N$ or when the columns of $\mathbf{X}$ are linearly dependent.*

Proof. The unregularized Gram matrix $\mathbf{A} \triangleq \mathbf{X}^\top \mathbf{X} \in \mathbb{R}^{D \times D}$ is symmetric and positive semi-definite. By the Spectral Theorem, it admits an orthonormal eigenvalue decomposition $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$, where the eigenvalues satisfy $\mu_i \geq 0$ for all $i \in \{1, \ldots, D\}$.

Adding the identity matrix scaled by $\alpha > 0$:

$$\mathbf{A} + \alpha \mathbf{I}_D = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top + \alpha \mathbf{Q}\mathbf{Q}^\top = \mathbf{Q}(\mathbf{\Lambda} + \alpha \mathbf{I}_D)\mathbf{Q}^\top. \tag{3.40}$$

The eigenvalues of $(\mathbf{X}^\top \mathbf{X} + \alpha \mathbf{I}_D)$ are simply shifted by α:

$$\lambda_i(\mathbf{X}^\top \mathbf{X} + \alpha \mathbf{I}_D) = \mu_i + \alpha \geq \alpha > 0 \quad \forall i \in \{1, \ldots, D\}. \tag{3.41}$$

Because all eigenvalues are strictly bounded below by $\alpha > 0$, the determinant is strictly positive: $\det(\mathbf{X}^\top \mathbf{X} + \alpha \mathbf{I}_D) = \prod_{i=1}^{D}(\mu_i + \alpha) \geq \alpha^D > 0$. Hence, the matrix is guaranteed to be non-singular and invertible under all conditions. □

Operational trade-offs of Ridge regression. While Ridge regression elegantly resolves singular matrix breakdown and admits a direct, non-iterative closed-form solution via Cholesky decomposition, it incurs a significant interpretability cost: because the L_2 penalty is isotropic and smooth everywhere with vanishing gradient at zero, it shrinks weights continuously but **never drives any parameter identically to zero**. Consequently, all D features are retained in the final model, preserving full dense dimensionality.

While linear regression, Lasso, and Ridge establish the mathematical foundation for predicting continuous targets, real-world machine learning frequently requires classifying observations into discrete qualitative categories—such as diagnosing an ultrasound lesion as benign ($y = 0$) versus malignant ($y = 1$). A tempting initial instinct is to deploy this exact same regularized linear framework on binary indicator targets and apply a hard threshold to continuous predictions. However, as revealed in Section 3.3, ordinary least squares encounters fundamental theoretical and geometrical failure modes when applied to discrete classification, necessitating the development of the probabilistic logistic model.

3.3 Logistic regression

In classical statistical learning, ordinary least squares linear regression provides an elegant, closed-form mapping from continuous input features $\mathbf{x} \in \mathbb{R}^d$ to continuous targets $y \in \mathbb{R}$ through the analytical Normal Equations. When faced with classification problems where targets represent discrete qualitative outcomes—such as clinical diagnoses (benign versus malignant lesions), fraud detection, or document categorizations—a tempting intuition is to reuse this established regression machinery. By treating class labels as binary numerical indicators ($y \in \{0, 1\}$), one might hope to fit a continuous affine hyperplane $h_{\boldsymbol{\theta}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ and obtain discrete predictions by simply thresholding the continuous output at an arbitrary decision cutoff such as 0.5.

However, direct application of squared error regression to discrete classification targets triggers fundamental theoretical and geometric pathologies. The range of an affine predictor is the unbounded real line $(-\infty, +\infty)$, producing predictions that severely violate the Kolmogorov axioms of probability ($P \in [0, 1]$) and leaving extreme outputs uninterpretable as posterior confidences. Even more catastrophically, ordinary least squares penalizes errors quadratically; as a result, correctly classified samples that lie far from the decision boundary incur massive squared residuals, exerting enormous leverage on the solution and rotating the decision boundary away from clearly separable clusters.

3.3.1 Failure of least squares on classification

Consider a real-world clinical diagnostic task: automated tumor diagnosis from medical ultrasound imaging. The input $\mathbf{x} \in \mathbb{R}^d$ represents quantitative acoustic texture features extracted from

an ultrasound scan, and the clinical goal is to output a discrete diagnostic category $y \in \{0, 1\}$:

$$y = \begin{cases} 0 & \text{benign lesion,} \\ 1 & \text{malignant neoplasm.} \end{cases} \tag{3.42}$$

To adapt continuous regression to such discrete tasks, a practitioner might attempt to fit a standard affine hypothesis:

$$h_{\boldsymbol{\theta}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b, \tag{3.43}$$

and then establish a discrete classification rule by thresholding the continuous prediction at a cutoff boundary (conventionally 0.5):

$$\hat{y} = \begin{cases} 1 & \text{if } h_{\boldsymbol{\theta}}(\mathbf{x}) \geq 0.5, \\ 0 & \text{if } h_{\boldsymbol{\theta}}(\mathbf{x}) < 0.5. \end{cases} \tag{3.44}$$

Despite its apparent simplicity, using ordinary least squares with squared error loss for classification suffers from two fatal theoretical and practical failure modes:

Unbounded outputs and probabilistic invalidity. The target $y_i \in \{0, 1\}$ represents the categorical state of an event, which in a probabilistic framework corresponds to the posterior probability $P(y = 1 \mid \mathbf{x})$. However, the range of the linear hypothesis $h_{\boldsymbol{\theta}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ is the entire real line:

$$h_{\boldsymbol{\theta}}(\mathbf{x}) \in (-\infty, +\infty). \tag{3.45}$$

For extreme feature vectors, the linear model inevitably predicts values such as $h_{\boldsymbol{\theta}}(\mathbf{x}) = 2.4$ or $h_{\boldsymbol{\theta}}(\mathbf{x}) = -0.7$. Such values violate the foundational Kolmogorov axioms of probability ($P \in [0, 1]$), rendering the raw outputs uninterpretable as posterior confidences.

Extreme sensitivity to outliers and decision threshold shift. The second flaw is even more devastating. Ordinary least squares minimizes the sum of squared residuals:

$$J(\mathbf{w}, b) = \frac{1}{2N} \sum_{i=1}^{N} \Big((\mathbf{w}^\top \mathbf{x}_i + b) - y_i \Big)^2 . \tag{3.46}$$

Because the squared penalty grows quadratically with error, data points located far from the decision boundary exert enormous leverage on the optimal hyperplane.

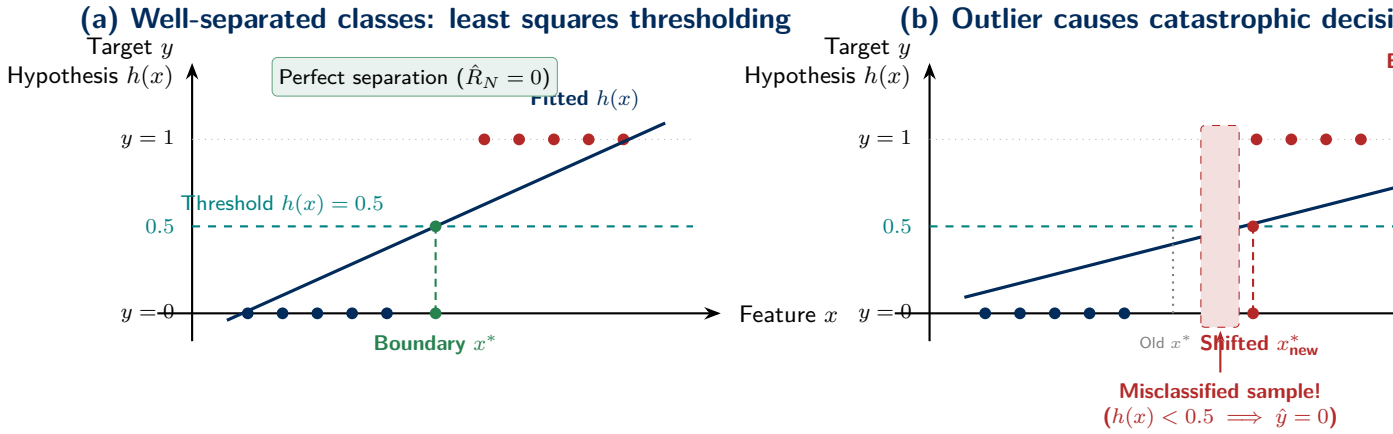


Figure 3.4: The failure of ordinary least squares for binary classification. (a) A linear regression model fitted to well-separated binary classes with decision threshold $h_{\boldsymbol{\theta}}(x) = 0.5$ correctly separating the two groups ($\hat{R}_N = 0$). (b) Introducing a single extreme positive sample far to the right (an outlier with $x \gg 0, y = 1$) exerts quadratic leverage on the squared-loss line, rotating it downward. As the line tilts, the intersection point $h_{\boldsymbol{\theta}}(x) = 0.5$ shifts significantly to the right (from x^* to x^*_{new}), misclassifying several previously correct positive samples.

Consider the scenario depicted in Figure 3.4. In panel (a), we observe a dataset where negative samples ($y = 0$) and positive samples ($y = 1$) are linearly separable along the horizontal axis. Fitting a least squares line yields a threshold intersection $h_{\boldsymbol{\theta}}(x) = 0.5$ that cleanly separates the two classes.

Now, consider what occurs when we introduce a new positive training sample located far to the right along the horizontal axis ($x \gg 0, y = 1$), as shown in panel (b). Intuitively, this data point is indisputably positive and should only reinforce our confidence in the classification boundary. However, for this distant point, the linear prediction might be $h_{\boldsymbol{\theta}}(x) = 4.0$. Because the target label is $y = 1$, the least squares residual for this single point is:

$$(4.0 - 1.0)^2 = 9.0. \tag{3.47}$$

To reduce this large quadratic penalty, the OLS fitting procedure tilts and rotates the regression line downward toward the horizontal axis. As the slope flattens, the decision threshold location where $h_{\boldsymbol{\theta}}(x) = 0.5$ is forced to shift substantially to the right. Consequently, several positive data points near the original boundary now fall below 0.5 and are **erroneously misclassified as negative**!

Thus, least squares penalizes samples for being *too confidently correct*, distorting the geometric decision boundary. This confirms that regression with squared error is structurally inappropriate for classification.

3.3.2 The logistic sigmoid model

To resolve the unbounded range problem, we must apply a non-linear squashing function that maps the real line $\mathbb{R}$ smoothly and monotonically into the open probability interval $(0, 1)$. This function is the **logistic sigmoid**:

$$\sigma(z) \triangleq \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}. \tag{3.48}$$

By composing the linear predictor $z = \mathbf{w}^\top \mathbf{x} + b$ with the sigmoid activation, we define the **Logistic Regression hypothesis**:

$$h_{\boldsymbol{\theta}}(\mathbf{x}) \triangleq \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}. \tag{3.49}$$

Mathematical properties of the sigmoid activation. The sigmoid function exhibits several fundamental mathematical properties:

1. **Bounded Range**: For all $z \in \mathbb{R}$, $0 < \sigma(z) < 1$, with asymptotic limits:
$$\lim_{z \to -\infty} \sigma(z) = 0, \quad \lim_{z \to +\infty} \sigma(z) = 1. \tag{3.50}$$

2. **Point Symmetry**: The function is symmetric around the inflection point $(0, 0.5)$:
$$\sigma(-z) = \frac{1}{1+e^{z}} = \frac{e^{-z}}{e^{-z}+1} = 1 - \frac{1}{1+e^{-z}} = 1 - \sigma(z). \tag{3.51}$$

3. **Compact Analytical Derivative**: The derivative of $\sigma(z)$ with respect to its argument can be expressed purely in terms of the function itself:
$$\begin{aligned} \frac{d\sigma(z)}{dz} &= \frac{d}{dz}(1+e^{-z})^{-1} = -(1+e^{-z})^{-2}(-e^{-z}) = \frac{e^{-z}}{(1+e^{-z})^2} \\ &= \left(\frac{1}{1+e^{-z}}\right)\left(\frac{e^{-z}}{1+e^{-z}}\right) = \left(\frac{1}{1+e^{-z}}\right)\left(1 - \frac{1}{1+e^{-z}}\right) \\ &= \sigma(z)(1-\sigma(z)). \end{aligned} \tag{3.52}$$
This identity greatly simplifies backpropagation and gradient calculus.

Odds ratio and the logit link function. Interpreting the hypothesis output as the posterior probability of the positive class, $p \equiv P(y=1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$, the probability of the negative class is $1 - p = 1 - \sigma(\mathbf{w}^\top \mathbf{x} + b)$.

The **odds** of an event is defined as the ratio of the probability that the event occurs to the probability that it does not occur:
$$\text{Odds} \triangleq \frac{p}{1-p} = \frac{\sigma(z)}{1-\sigma(z)} = \frac{\frac{1}{1+e^{-z}}}{\frac{e^{-z}}{1+e^{-z}}} = \frac{1}{e^{-z}} = e^{z} = e^{\mathbf{w}^\top \mathbf{x} + b}. \tag{3.53}$$

Taking the natural logarithm of the odds defines the **log-odds** (or **logit** function):
$$\text{logit}(p) \triangleq \ln\left(\frac{p}{1-p}\right) = \ln\left(e^{\mathbf{w}^\top \mathbf{x} + b}\right) = \mathbf{w}^\top \mathbf{x} + b. \tag{3.54}$$

Equation (3.54) reveals that logistic regression is a **generalized linear model (GLM)**: while the output probability p is non-linear in $\mathbf{x}$, the log-odds is strictly affine. The decision boundary separating $P(y=1 \mid \mathbf{x}) \geq 0.5$ from $P(y=1 \mid \mathbf{x}) < 0.5$ corresponds precisely to $\text{logit}(p) = 0$, which is the linear hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$.

3.3.3 Failure of squared error on logistic models

Now that the linear predictor has been mapped into valid probabilities via the sigmoid squashing function $\hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b) \in (0, 1)$, a natural follow-up question arises: *can we simply retain our familiar Mean Squared Error (MSE) loss function and optimize the parameters via gradient descent?*

Suppose we construct the empirical risk under squared loss:
$$J_{\text{MSE}}(\mathbf{w}, b) = \frac{1}{2N} \sum_{i=1}^{N} \Big(\sigma(\mathbf{w}^\top \mathbf{x}_i + b) - y_i\Big)^2. \tag{3.55}$$

However, this seemingly straightforward formulation suffers from two severe mathematical flaws:

Vanishing gradient saturation on egregious errors. To observe how gradient descent behaves under Equation (3.55), let us differentiate the loss for a single observation with respect to parameter vector $\mathbf{w}$ using the chain rule:

$$\begin{aligned}\nabla_{\mathbf{w}} \ell_{\text{MSE},i} &= \frac{\partial}{\partial \mathbf{w}} \left[\frac{1}{2} (\sigma(z_i) - y_i)^2 \right] \\ &= (\sigma(z_i) - y_i) \cdot \frac{d\sigma(z_i)}{dz_i} \cdot \frac{\partial z_i}{\partial \mathbf{w}} \\ &= (\hat{y}_i - y_i) \cdot \hat{y}_i(1 - \hat{y}_i) \cdot \mathbf{x}_i. \end{aligned} \tag{3.56}$$

Notice the presence of the multiplicative factor $\hat{y}_i(1 - \hat{y}_i) = \sigma'(z_i)$. This term induces a fatal optimization pathology:

- Suppose the true clinical label is malignant ($y_i = 1$), but the current model parameters output an egregiously wrong prediction ($\hat{y}_i = 0.0001$, so $z_i \ll 0$).
- The raw residual is near its theoretical maximum: $(\hat{y}_i - y_i) = (0.0001 - 1.0) \approx -1.0$.
- However, because the sigmoid is deeply saturated in its negative tail, the derivative factor collapses:
$$\hat{y}_i(1 - \hat{y}_i) = 0.0001 \times (1 - 0.0001) \approx 0.0001 \approx 0. \tag{3.57}$$
- Multiplying by this factor drives the parameter gradient to zero:
$$\lim_{z_i \to -\infty} \nabla_{\mathbf{w}} \ell_{\text{MSE},i} = (-1.0) \times 0 \times \mathbf{x}_i = \mathbf{0}. \tag{3.58}$$

This means that precisely when the classifier is most catastrophically mistaken, the gradient vanishes! The learning signal drops to zero, and gradient descent becomes immobilized on extensive flat saturation plateaus.

Destruction of convexity. In linear regression, $J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$ is a purely quadratic bowl with a strictly positive semi-definite Hessian $\mathbf{X}^\top \mathbf{X} \succeq 0$, guaranteeing strict convexity and a unique global minimum.

However, composing squared loss with the non-linear sigmoid activation fundamentally breaks convexity. Computing the second derivative with respect to scalar w_j yields:

$$\frac{\partial^2 \ell_{\text{MSE},i}}{\partial w_j^2} = x_{ij}^2 \left[(\sigma(z_i) - y_i)\sigma''(z_i) + (\sigma'(z_i))^2 \right]. \tag{3.59}$$

Because the second derivative of the sigmoid $\sigma''(z) = \sigma'(z)(1 - 2\sigma(z))$ changes sign across the domain, the term $(\sigma(z_i) - y_i)\sigma''(z_i)$ can become substantially negative. Whenever this negative term outweighs $(\sigma'(z_i))^2$, the Hessian eigenvalue along that direction becomes negative ($\lambda_k < 0$).

Consequently, the loss surface $J_{\text{MSE}}(\mathbf{w}, b)$ develops complex configurations of local minima, ridges, and deceptive saddle points. An iterative optimizer initialized far from the optimum can easily get trapped in a shallow, non-optimal local basin.

Oral Exam Insight & Pitfall Alert: Oral Exam Question: Why Not MSE with Sigmoid?

A recurrent oral exam question asks: *"Why do we abandon Mean Squared Error when moving to Logistic Regression, instead of simply keeping MSE and applying a sigmoid to the output?"* Candidates must immediately articulate the two complementary justifications:

1. **Loss Surface Geometry (Non-Convexity)**: Composing squared error with the sigmoidal squashing function turns a convex optimization problem into a non-convex landscape riddled with local minima and saddle points, destroying all convergence guarantees.

2. **Gradient Saturation (Vanishing Gradients)**: The factor $\sigma'(z) = \sigma(z)(1-\sigma(z))$ inside the MSE gradient vanishes when the model is confident but completely wrong ($y = 1, \hat{y} \approx 0$). In contrast, binary cross-entropy algebraically cancels this denominator, preserving a strong linear gradient $(\hat{y} - y)\mathbf{x}$ and restoring strict convexity.

Having established that mapping linear outputs through a sigmoidal link rectifies the range pathology and yields well-calibrated posterior probabilities, we discovered that pairing the sigmoid with classical squared error introduces fatal optimization pathologies: non-convex local traps and gradient saturation on egregious errors. To optimize probabilistic classifiers reliably, we require an objective function whose loss landscape is strictly convex, whose gradients penalize catastrophic misclassifications with linear restoration forces, and whose statistical formulation aligns with maximum likelihood principles. This motivates the deep study of loss function geometry and binary cross-entropy in Section 3.4.

3.4 Loss functions and geometry

The choice of loss function is the central design decision that governs both the statistical validity and the geometric solvability of a classification algorithm. In an ideal world, one would directly minimize the discrete 0-1 classification loss, which counts the number of misclassified samples. However, because the 0-1 loss is piecewise constant with zero derivative almost everywhere and discontinuous at the decision boundary, empirical risk minimization under this metric is an NP-hard combinatorial problem, rendering gradient-based learning completely impossible.

To make progress, statistical learning replaces the intractable 0-1 step loss with smooth, continuous surrogate functions defined over the classification margin. Yet, choosing an ill-suited surrogate can have disastrous consequences: as proven in Section 3.3.3, squared error penalizes over-confident correct classifications while destroying convexity and inducing vanishing gradients. In contrast, cross-entropy loss—derived from Maximum Likelihood Estimation on Bernoulli and Multinomial distributions—possesses a remarkably benign geometry: it is strictly convex, penalizes confident mistakes with severe logarithmic penalties, and algebraically cancels the vanishing derivative of sigmoidal and softmax links.

3.4.1 Surrogate loss geometry

In supervised learning, the loss function $\ell(h(\mathbf{x}), y)$ serves as the mathematical objective that steers the parameter vector $\mathbf{w}$ across the parameter space. Optimization algorithms possess no intrinsic semantic understanding of the downstream application—whether diagnosing medical lesions, detecting credit card fraud, or forecasting physical temperatures; they simply follow the local descent trajectory dictated by the gradients of the chosen loss surface. Consequently, defining a mathematically sound and task-aligned loss function is the single most critical modeling decision in machine learning.

Aligning loss functions with downstream task objectives. The loss function must faithfully reflect the statistical nature and operational requirements of the downstream task:

- **Continuous regression tasks**: When predicting continuous physical quantities ($y \in \mathbb{R}$), discrepancies between predictions and targets represent quantitative deviations. Standard L_2 squared error penalizes errors symmetrically and quadratically, estimating the conditional expectation $\mathbb{E}[y \mid \mathbf{x}]$, whereas L_1 absolute error penalizes deviations linearly, estimating the conditional median and conferring robust resistance against extreme anomalies.

- **Discrete classification tasks**: When assigning samples to qualitative categories ($y \in \{0, 1\}$ or $y \in \{1, \ldots, K\}$), the primary objective is correct discrete decision-making. Outputs must be interpreted as well-calibrated posterior probabilities $P(y = k \mid \mathbf{x})$ rather than unconstrained physical magnitudes. The loss function must enforce probabilistic consistency and provide non-vanishing restorative signals when classifications are wrong.

The pathology of Mean Squared Error for classification. To appreciate why loss function design is so critical, recall the catastrophic failure modes uncovered in Section 3.3 when Mean Squared Error (MSE) is misapplied to classification tasks:

1. **Outlier distortion via penalization of correct predictions**: When binary labels are encoded symmetrically as $y \in \{-1, +1\}$, performance is naturally evaluated over the **functional margin** $z \triangleq y \cdot h(\mathbf{x})$. A positive margin ($z > 0$) signifies correct classification, while a negative margin ($z < 0$) indicates an error. Under this formulation, squared error becomes:

$$\ell_{\text{SE}}(z) = (1 - z)^2. \tag{3.60}$$

   Notice the severe flaw: if a test sample is classified correctly with substantial confidence (for instance, $z = +4$), squared error does not assign zero penalty! Instead, it penalizes the point quadratically with $(1 - 4)^2 = 9$. Extremely confident, easily separable observations dominate the empirical risk gradient, forcefully tilting and rotating the decision hyperplane away from clean cluster separation.
2. **Non-convexity and vanishing gradient plateaus**: When squared error is paired with a non-linear link function such as the sigmoid $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ to restrict outputs to $[0, 1]$, the loss landscape becomes non-convex. For egregious prediction errors (e.g., $y = 1$ but $\hat{y} \to 0$), the gradient contains the derivative factor $\sigma'(z) = \hat{y}(1 - \hat{y}) \to 0$. The learning signal vanishes precisely when the prediction is most severely wrong, trapping gradient-based optimizers in high-loss, flat plateaus.

The discrete ideal vs. continuous surrogate losses. The ideal, direct measure of classification error is the discrete **0-1 loss**:

$$\ell_{0\text{-}1}(z) = \mathbb{I}\{z \leq 0\} = \begin{cases} 0 & \text{if } z > 0 \text{ (correct classification)}, \\ 1 & \text{if } z \leq 0 \text{ (misclassification)}. \end{cases} \tag{3.61}$$

Direct empirical risk minimization under $\ell_{0\text{-}1}(z)$ counts the exact number of mistakes. However, $\ell_{0\text{-}1}$ is a non-convex, piecewise-constant step function whose derivative is zero everywhere ($z \neq 0$) and undefined at $z = 0$. Because $\nabla_{\mathbf{w}} \ell_{0\text{-}1} = \mathbf{0}$ almost everywhere, gradient-based learning receives zero directional guidance. In fact, minimizing 0-1 loss over arbitrary training sets is known to be an NP-hard combinatorial problem.

Statistical machine learning overcomes this hurdle by replacing the intractable 0-1 step loss with continuous, smooth, convex **surrogate losses** defined over the functional margin $z = y \cdot h(\mathbf{x})$:

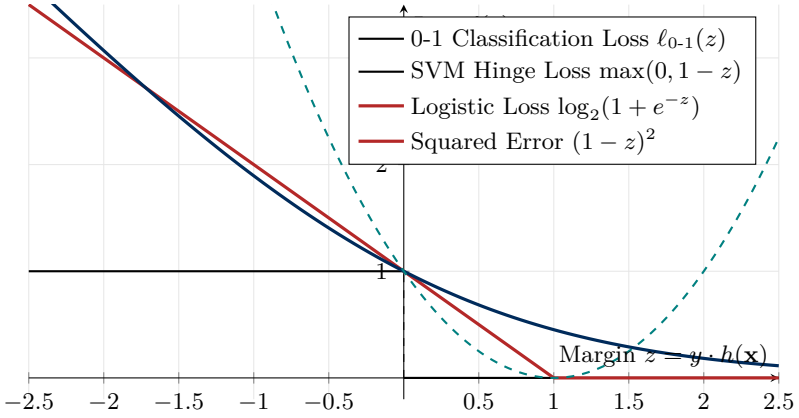


Figure 3.5: Analytical geometry of classification loss functions plotted against the margin $z = y \cdot h(\mathbf{x})$. While the discrete 0-1 step loss has zero derivative almost everywhere, continuous convex surrogates—such as the SVM hinge loss $\max(0, 1-z)$, squared error $(1-z)^2$, and logistic cross-entropy loss $\log_2(1+e^{-z})$—provide informative learning signals. Notice how squared error penalizes points with margin $z > 1$, explaining why it fails on classification.

As visually summarized in Figure 3.5:

1. **0-1 Loss**: $\ell_{0\text{-}1}(z) = \mathbb{I}\{z \leq 0\}$. Directly penalizes misclassifications, but yields zero gradient almost everywhere.
2. **Squared Error Loss**: $\ell_{\text{SE}}(z) = (1-z)^2$. Convex and differentiable, but penalizes confident correct classifications ($z > 1$), corrupting the decision boundary.
3. **SVM Hinge Loss**: $\ell_{\text{hinge}}(z) = \max(0, 1-z)$. Adopted in the primal Support Vector Machine (SVM) formulation:

$$\min_{\mathbf{w},b} \frac{1}{N} \sum_{i=1}^{N} \max\Big(0,\ 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\Big) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2. \tag{3.62}$$

   When a sample is classified correctly with sufficient safety margin ($z \geq 1$), the loss is **identically zero**, completely avoiding outlier distortion. When the margin is violated ($z < 1$), the penalty grows linearly.
4. **Logistic Loss (Cross-Entropy)**: $\ell_{\log}(z) = \ln(1+e^{-z})$. Infinitely smooth, strictly convex, upper-bounds the 0-1 loss, and approaches zero asymptotically as $z \to +\infty$.

3.4.2 Maximum likelihood loss generation

Rather than hand-crafting heuristic loss formulas, statistical machine learning relies on the principled foundation of **Maximum Likelihood Estimation (MLE)**. Under this paradigm, candidate loss functions are derived directly by adopting explicit probabilistic models for the conditional distribution $p(y \mid \mathbf{x}; \boldsymbol{\theta})$ of targets given inputs.

For an independently and identically distributed (i.i.d.) dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, the joint dataset likelihood is the product of individual conditional probabilities:

$$L(\boldsymbol{\theta}) = \prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i; \boldsymbol{\theta}). \tag{3.63}$$

Because maximizing a product of minute probabilities is numerically unstable and induces arithmetic underflow, we take the natural logarithm, converting the product into a sum of log-probabilities:

$$\ell(\boldsymbol{\theta}) \triangleq \ln L(\boldsymbol{\theta}) = \sum_{i=1}^{N} \ln p(y_i \mid \mathbf{x}_i; \boldsymbol{\theta}). \tag{3.64}$$

Because optimization routines universally minimize cost functions, we define the objective as the **Negative Log-Likelihood (NLL)**, normalized by sample size N:

$$J(\boldsymbol{\theta}) \triangleq -\frac{1}{N}\ell(\boldsymbol{\theta}) = -\frac{1}{N}\sum_{i=1}^{N} \ln p(y_i \mid \mathbf{x}_i; \boldsymbol{\theta}). \tag{3.65}$$

Different distributional assumptions for $p(y \mid \mathbf{x}; \boldsymbol{\theta})$ directly generate the most foundational loss functions in machine learning:

1. Gaussian noise assumption $\implies$ Mean Squared Error (L_2 Loss). Assume a continuous regression setting $y \in \mathbb{R}$ where targets are corrupted by zero-mean additive Gaussian noise: $y = h_{\mathbf{w}}(\mathbf{x}) + \epsilon$, with $\epsilon \sim \mathcal{N}(0, \sigma^2)$. The conditional density is:

$$p(y \mid \mathbf{x}; \mathbf{w}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - h_{\mathbf{w}}(\mathbf{x}))^2}{2\sigma^2}\right). \tag{3.66}$$

Substituting Equation (3.66) into the Negative Log-Likelihood:

$$J(\mathbf{w}) = -\frac{1}{N}\sum_{i=1}^{N}\left[-\frac{1}{2\sigma^2}(y_i - h_{\mathbf{w}}(\mathbf{x}_i))^2 - \frac{1}{2}\ln(2\pi\sigma^2)\right] = \frac{1}{2N\sigma^2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \text{const.} \tag{3.67}$$

Minimizing the NLL under Gaussian noise is mathematically equivalent to minimizing **Mean Squared Error (MSE)**. The resulting estimator recovers the conditional expectation $\mathbb{E}[y \mid \mathbf{x}]$.

2. Laplace noise assumption $\implies$ Mean Absolute Error (L_1 Loss). If errors follow a Laplace distribution $p(\epsilon) = \frac{1}{2b}\exp(-|\epsilon|/b)$ with scale parameter b, the negative log-likelihood becomes:

$$J(\mathbf{w}) = \frac{1}{Nb}\sum_{i=1}^{N}|y_i - h_{\mathbf{w}}(\mathbf{x}_i)| + \ln(2b) \propto \frac{1}{N}\sum_{i=1}^{N}|y_i - h_{\mathbf{w}}(\mathbf{x}_i)|. \tag{3.68}$$

This recovers the **Mean Absolute Error (MAE)**. Because the Laplace density has heavier tails than the Gaussian, MAE estimates the conditional median and exhibits robust insensitivity to outliers.

3. Bernoulli likelihood assumption $\implies$ Binary Cross-Entropy (BCE). For binary classification $y \in \{0, 1\}$, given input $\mathbf{x}$, the target follows a Bernoulli distribution parameterized by the posterior probability $p(\mathbf{x}) = \sigma(\mathbf{w}^\top\mathbf{x} + b) = \hat{y}$:

$$P(y \mid \mathbf{x}; \mathbf{w}, b) = \hat{y}^y(1 - \hat{y})^{1-y}. \tag{3.69}$$

Taking the Negative Log-Likelihood normalized by N yields the **Binary Cross-Entropy (BCE)** loss:

$$J(\mathbf{w}, b) = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \ln\hat{y}_i + (1 - y_i)\ln(1 - \hat{y}_i)\right], \quad \text{where } \hat{y}_i = \sigma(\mathbf{w}^\top\mathbf{x}_i + b). \tag{3.70}$$

Notice its pedagogical behavior:

- When ground truth is $y_i = 1$, the loss reduces to $-\ln(\hat{y}_i)$. If $\hat{y}_i = 1$, cost is 0; if $\hat{y}_i \to 0$, cost $-\ln(\hat{y}_i) \to +\infty$ (infinite penalty for confident false negatives).
- When ground truth is $y_i = 0$, the loss reduces to $-\ln(1 - \hat{y}_i)$. If $\hat{y}_i = 0$, cost is 0; if $\hat{y}_i \to 1$, cost $-\ln(1 - \hat{y}_i) \to +\infty$ (infinite penalty for confident false positives).

4. Multinomial/Categorical likelihood $\implies$ Categorical Cross-Entropy. For multiclass classification with K classes, one-hot encoded targets $\mathbf{y}_i \in \{0,1\}^K$, and softmax probabilities $\hat{y}_{ik} = \frac{\exp(z_{ik})}{\sum_{j=1}^{K}\exp(z_{ij})}$, the likelihood is Categorical:

$$P(\mathbf{y}_i \mid \mathbf{x}_i) = \prod_{k=1}^{K} \hat{y}_{ik}^{y_{ik}} \implies \mathcal{L}_{\text{CE}}(\boldsymbol{\theta}) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\ln \hat{y}_{ik}. \tag{3.71}$$

Analytical gradient of Binary Cross-Entropy via the chain rule. To compute parameter updates for logistic regression, we evaluate the gradient of the cross-entropy objective $J(\mathbf{w}, b)$ with respect to $\mathbf{w}$ using the multivariate chain rule across three nested layers:

$$f(g(h(\mathbf{w}))), \quad \text{where} \quad \begin{cases} h(\mathbf{w}) = z_i = \mathbf{w}^\top \mathbf{x}_i + b & \text{(affine pre-activation)}, \\ g(z_i) = \hat{y}_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}} & \text{(sigmoid link)}, \\ f(\hat{y}_i) = -\left[y_i \ln \hat{y}_i + (1-y_i)\ln(1-\hat{y}_i)\right] & \text{(cross-entropy loss)}. \end{cases} \tag{3.72}$$

By the chain rule:

$$\frac{\partial f}{\partial \mathbf{w}} = \frac{\partial f}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial \mathbf{w}}. \tag{3.73}$$

Evaluating the three individual components:

1. **Derivative of affine pre-activation**: $\frac{\partial z_i}{\partial \mathbf{w}} = \mathbf{x}_i$, $\frac{\partial z_i}{\partial b} = 1$.
2. **Derivative of sigmoid link**: $\frac{\partial \hat{y}_i}{\partial z_i} = \sigma(z_i)(1-\sigma(z_i)) = \hat{y}_i(1-\hat{y}_i)$.
3. **Derivative of loss with respect to prediction**:

$$\frac{\partial f}{\partial \hat{y}_i} = -\left[\frac{y_i}{\hat{y}_i} - \frac{1-y_i}{1-\hat{y}_i}\right] = \frac{\hat{y}_i - y_i}{\hat{y}_i(1-\hat{y}_i)}. \tag{3.74}$$

Multiplying these terms together reveals an extraordinary algebraic cancellation:

$$\frac{\partial f}{\partial \mathbf{w}} = \left(\frac{\hat{y}_i - y_i}{\hat{y}_i(1-\hat{y}_i)}\right) \cdot (\hat{y}_i(1-\hat{y}_i)) \cdot \mathbf{x}_i = (\hat{y}_i - y_i)\mathbf{x}_i. \tag{3.75}$$

The term $\hat{y}_i(1-\hat{y}_i)$ in the denominator of the loss derivative perfectly cancels the vanishing derivative of the sigmoid! Averaging over all N training samples yields the full vector gradients:

$$\nabla_{\mathbf{w}} J(\mathbf{w}, b) = \frac{1}{N}\sum_{i=1}^{N}\left(\sigma(\mathbf{w}^\top \mathbf{x}_i + b) - y_i\right)\mathbf{x}_i, \tag{3.76}$$

$$\frac{\partial J(\mathbf{w}, b)}{\partial b} = \frac{1}{N}\sum_{i=1}^{N}\left(\sigma(\mathbf{w}^\top \mathbf{x}_i + b) - y_i\right). \tag{3.77}$$

Notice the striking structural parallel with the gradient of Mean Squared Error:

$$\nabla_{\mathbf{w}} f(\mathbf{w}) = \frac{1}{N}\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) = \frac{1}{N}\sum_{i=1}^{N}(\mathbf{w}^\top \mathbf{x}_i - y_i)\mathbf{x}_i. \tag{3.78}$$

In both foundational estimation paradigms, the gradient is an error-weighted linear combination of input feature vectors $\mathbf{x}_i$, where each sample contributes proportionally to its prediction residual $(\hat{y}_i - y_i)$.

3.4.3 Necessity of iterative optimization

Equipped with the analytical gradients for both linear regression and logistic cross-entropy, a fundamental question arises: *why can we not simply set the gradient to zero ($\nabla_{\mathbf{w}} J(\mathbf{w}) = \mathbf{0}$) and solve for the optimal parameter vector in a single exact closed-form step?*

To answer this question, we examine two contrasting mathematical scenarios: one where an analytical closed form exists but collapses under computational scaling, and one where closed-form solutions are mathematically impossible due to transcendental equations.

Example 1: Linear Regression — When a closed form exists, but matrix inversion fails in practice. In Ordinary Least Squares regression, setting the gradient $\nabla_{\mathbf{w}} f(\mathbf{w}) = \mathbf{0}$ from Equation (3.78) produces:

$$\frac{1}{N}\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{0} \iff \mathbf{X}^\top\mathbf{X}\mathbf{w} = \mathbf{X}^\top\mathbf{y}. \tag{3.79}$$

Because the prediction $\mathbf{X}\mathbf{w}$ is affine in $\mathbf{w}$, the stationary condition is a linear system. Assuming the $d \times d$ Gram matrix $\mathbf{X}^\top\mathbf{X}$ is non-singular, inverting it yields the celebrated closed-form Normal Equations solution:

$$\mathbf{w}^* = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}. \tag{3.80}$$

While algebraically elegant, computing this exact closed-form solution directly via matrix inversion is crippled by severe operational and numerical drawbacks in real-world applications:

1. **Cubic computational complexity** $\mathcal{O}(d^3)$: Forming the Gram matrix $\mathbf{X}^\top\mathbf{X}$ requires $\mathcal{O}(Nd^2)$ operations, and inverting it (via Gaussian elimination, LU decomposition, or Cholesky factorization) requires $\mathcal{O}(d^3)$ floating-point operations. In modern statistical learning and deep learning, parameter dimensionality easily reaches $d \approx 10^5$ to 10^7 (and up to $d \geq 10^{11}$ in large foundation models). For $d = 10^6$, an $\mathcal{O}(d^3)$ operation demands 10^{18} floating-point operations—an intractable compute burden even for supercomputing clusters.
2. **Quadratic memory footprint** $\mathcal{O}(d^2)$: Storing the $d \times d$ Gram matrix requires $\mathcal{O}(d^2)$ contiguous memory. For $d = 10^6$, storing $(\mathbf{X}^\top\mathbf{X})$ in 32-bit single precision demands $10^{12} \times 4$ bytes $= 4$ Terabytes of memory, far exceeding GPU and host RAM capacities.
3. **Singularity, rank deficiency, and numerical instability**: When features are collinear or when the input dimensionality exceeds the dataset size ($d > N$), the Gram matrix satisfies $\text{rank}(\mathbf{X}^\top\mathbf{X}) \leq N < d$. The matrix is strictly singular and non-invertible. Even when $d \leq N$, strong feature correlations cause the condition number $\kappa(\mathbf{X}^\top\mathbf{X}) = \lambda_{\max}/\lambda_{\min}$ to explode, where minute floating-point rounding errors are amplified catastrophically, destroying numerical accuracy.
4. **Incompatibility with streaming, online, and distributed data**: The closed-form formula requires the entire design matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$ to reside in memory at once. It cannot update parameters incrementally as new streaming data arrives, nor can it process massive datasets partitioned across distributed clusters without recomputing the entire matrix product and inversion from scratch.

Example 2: Logistic Regression — When setting the gradient to zero yields transcendental equations. In logistic regression, setting the Binary Cross-Entropy gradient $\nabla_{\mathbf{w}} J(\mathbf{w}, b) = \mathbf{0}$ from Equation (3.76) produces:

$$\frac{1}{N}\sum_{i=1}^{N}\Big(\sigma(\mathbf{w}^\top\mathbf{x}_i + b) - y_i\Big)\,\mathbf{x}_i = \mathbf{0}. \tag{3.81}$$

Expanding the non-linear sigmoid activation function reveals the exact algebraic structure:

$$\sum_{i=1}^{N} \frac{\mathbf{x}_i}{1 + \exp\left(-\sum_{j=1}^{d} w_j x_{ij} - b\right)} = \sum_{i=1}^{N} y_i \mathbf{x}_i. \tag{3.82}$$

Examine Equation (3.82) closely. The unknown parameter vector $\mathbf{w}$ is entangled inside the non-linear denominator of an exponential function across a summation of N heterogeneous terms. This constitutes a coupled system of d non-linear **transcendental equations**.

Unlike polynomial algebraic equations, transcendental equations involving exponential and logarithmic functions cannot be solved, uncoupled, or inverted using a finite sequence of elementary algebraic operations (addition, multiplication, roots, or matrix inversions). There is **no closed-form algebraic formula** that can express the optimal parameter vector $\mathbf{w}^*$.

The causal bridge to numerical optimization. These two contrasting examples reveal a profound theoretical truth:

- In linear models where closed forms exist, matrix inversion is computationally intractable ($\mathcal{O}(d^3)$) and memory-prohibitive ($\mathcal{O}(d^2)$) in high dimensions.
- In probabilistic classification (logistic regression) and all deep neural architectures, closed-form algebraic solutions are mathematically impossible due to non-linear transcendental couplings.

Therefore, modern machine learning must abandon closed-form linear algebra and rely entirely on **Iterative Numerical Optimization**: initializing parameters with a guess $\mathbf{w}_0$ and executing small, localized downhill steps guided by gradient vectors until convergence. This motivates the formal study of first-order numerical optimization, loss landscapes, and gradient descent in Section 3.5.

3.5 Gradient descent

When analytical solutions are mathematically impossible or computationally prohibitive, machine learning algorithms must acquire knowledge through iterative numerical optimization. Rather than leaping to the global optimum via closed-form matrix algebra, the model begins with an initial parameter guess and refines it through a sequence of discrete, localized updates across the loss landscape. Each iteration queries the objective function and its local derivatives to determine an exploratory step that decreases empirical error.

However, navigating high-dimensional loss landscapes is fraught with peril. An optimizer operating without global visibility can be likened to an explorer traversing an unfamiliar mountain shrouded in dense fog: one can only evaluate the local inclination directly beneath one's feet. If the step size is chosen too conservatively, optimization crawls at an agonizingly slow pace, stalling on shallow plateaus; if chosen too aggressively, updates overshoot valleys and oscillate wildly, driving loss values toward numerical overflow. Moreover, the presence of stationary points—where gradients vanish identically—poses constant risks of premature stagnation.

3.5.1 Gradient descent geometry

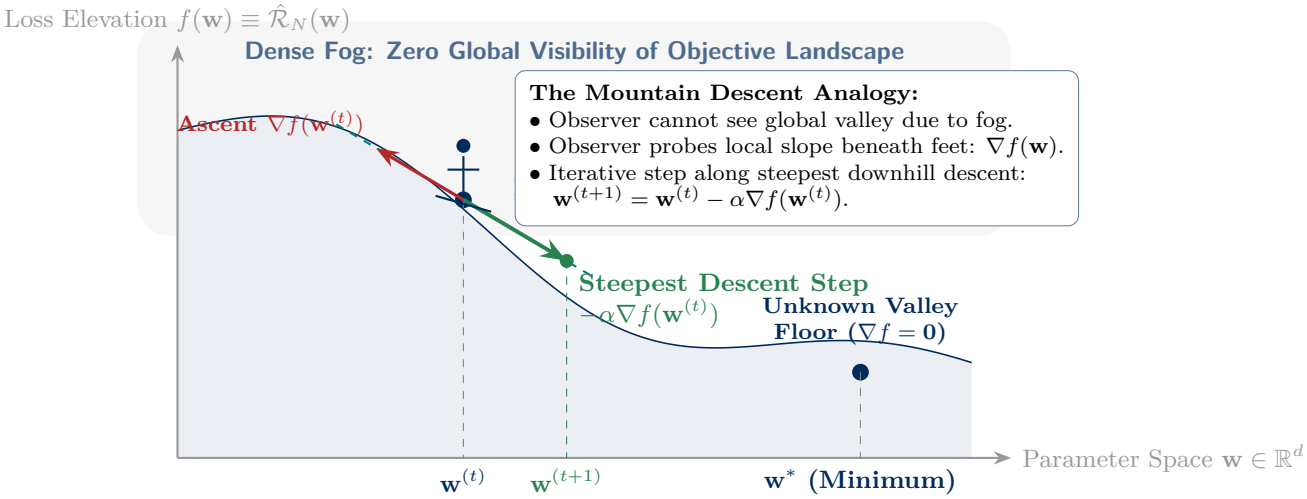


Figure 3.6: The physical intuition of gradient-based optimization: navigating an unfamiliar mountain terrain enveloped in dense fog. Lacking global visibility of the overall topography, the optimizer can only evaluate the local slope immediately beneath its current position to take an iterative step in the direction of steepest downward descent toward a valley minimum.

The optimization landscape and the mountain descent analogy. The physical intuition underlying gradient-based optimization is captured by the classic mountain descent metaphor. Consider standing on a mountain enveloped in dense fog. You cannot perceive the global topography from above; all you can evaluate is the local inclination of the ground beneath your feet. To reach the bottom of the valley as quickly as possible, you assess the local tilt in every direction and step downhill along the path of steepest slope.

In machine learning, the foggy mountain terrain corresponds to the empirical loss landscape $f(\mathbf{w}) \equiv \hat{\mathcal{R}}_N(\mathbf{w})$ over parameter space $\mathbb{R}^d$, and the local slope is quantified analytically by the negative gradient vector $-\nabla f(\mathbf{w})$.

Loss surface topology: 3D surfaces, 2D contours, and vector fields. To visualize multi-dimensional parameter spaces, optimization problems in $\mathbb{R}^2$ are examined through three synchronized representations:

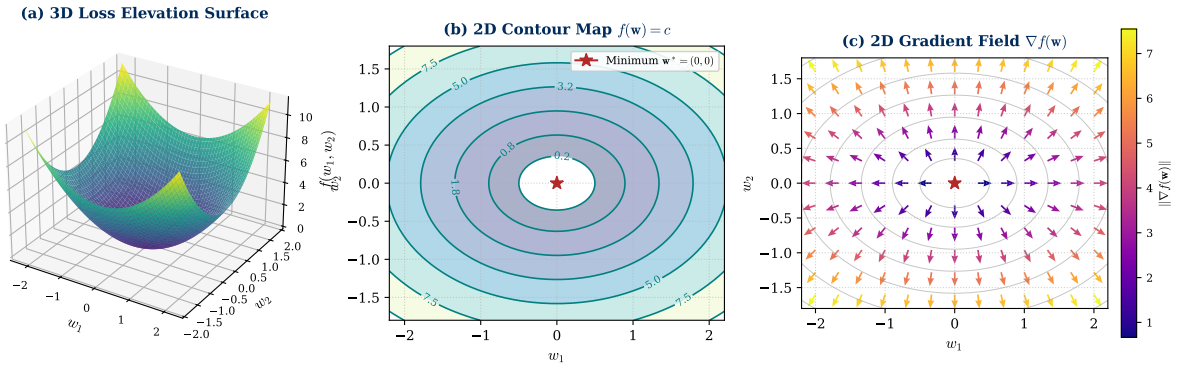


Figure 3.7: Three synchronized representations of an optimization landscape $f(x, y)$: (Left) 3D surface plot displaying elevation peaks, valleys, and saddle regions; (Middle) 2D contour map showing level curves $f(x, y) = c$, where concentric closed loops enclose local minima or maxima; (Right) 2D vector field of gradients, where vector lengths indicate slope magnitude and arrows point in the direction of steepest ascent.

As displayed in Figure 3.7:

1. **3D Surface Plot (Elevation)**: Directly graphs the scalar cost value $z = f(x, y)$ as a function of the two parameters. Elevation reflects loss magnitude; global and local minima correspond to valleys.

2. **2D Contour Map (Level Curves)**: Slices the 3D surface horizontally at constant loss elevations $f(x, y) = c$. Concentric closed isocurves enclose local extrema (minima or maxima). Spacing between contour lines indicates steepness: densely packed contours signify precipitous cliffs, whereas widely spaced contours indicate flat plateaus.

3. **2D Gradient Vector Field**: Maps every coordinate (x, y) to its gradient vector $\nabla f(x, y) = [\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}]^\top$. The arrows point in the direction of maximum instantaneous ascent, and their lengths represent the magnitude of the slope $\|\nabla f\|$.

Orthogonality of the gradient to level curves. A foundational geometric theorem is that the gradient vector is strictly **orthogonal** to contour level curves (or level hypersurfaces in higher dimensions).

Let $\mathcal{C}_c \triangleq \{\mathbf{w} \in \mathbb{R}^d : f(\mathbf{w}) = c\}$ be an isocurve of constant value c. Let $\mathbf{r}(t)$ be an arbitrary smooth path lying entirely within the level set, such that $f(\mathbf{r}(t)) = c$ for all t. Differentiating both sides with respect to scalar parameter t via the multivariate chain rule:

$$\frac{d}{dt} f(\mathbf{r}(t)) = \nabla f(\mathbf{r}(t))^\top \mathbf{r}'(t) = 0. \tag{3.83}$$

Here, $\mathbf{r}'(t) \equiv \mathbf{v}$ is the tangent velocity vector along the contour curve. Equation (3.83) proves that:

$$\langle \nabla f(\mathbf{w}), \mathbf{v} \rangle = 0 \quad \text{for any tangent vector } \mathbf{v} \text{ to the isocurve.} \tag{3.84}$$

Because the inner product vanishes, the directional derivative along the isocurve is zero: moving tangentially to a level set produces zero instantaneous change in loss.

Conversely, the direction perpendicular to the tangent space maximizes the inner product, proving that the gradient vector $\nabla f(\mathbf{w})$ is **perpendicular to the level curves**, pointing toward steepest ascent, while the negative gradient $-\nabla f(\mathbf{w})$ points perpendicular to the level curves toward steepest descent (as illustrated in Figure 3.9).

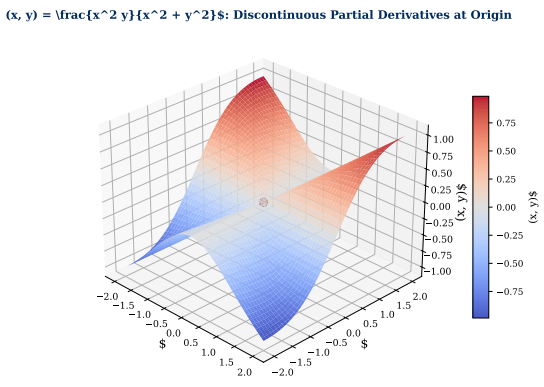


Figure 3.8: 3D surface plot of the counterexample function $f(x, y) = \frac{x^2 y}{x^2+y^2}$ around the origin. Although partial derivatives exist everywhere, they are discontinuous at $(0, 0)$, rendering the surface non-differentiable.

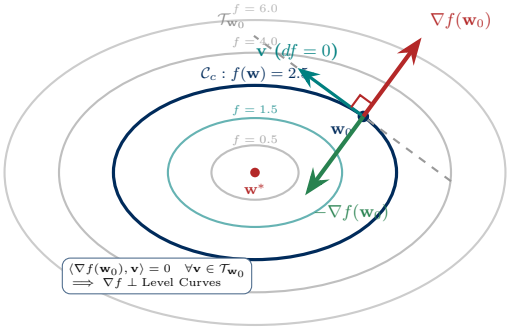


Figure 3.9: Geometric orthogonality of the gradient to contour level curves. Along the isocurve $f(x, y) = 2.5$, the directional derivative along any tangent vector $\mathbf{v}$ is zero ($\langle \nabla f, \mathbf{v} \rangle = 0$). The negative gradient $-\nabla f$ points strictly perpendicular to the level curve in the direction of steepest descent.

Postilla: Existence of all partial derivatives vs. true differentiability. Gradient descent updates require the objective function $f : \mathbb{R}^d \to \mathbb{R}$ to be differentiable. A frequent trap in mathematical analysis and oral examinations is confusing the existence of partial derivatives with true differentiability:

The existence of all partial derivatives (or even all directional derivatives) does NOT imply that a function is differentiable!
A multivariable function $f : \mathbb{R}^d \to \mathbb{R}$ is differentiable at $\mathbf{x}_0$ if and only if there exists a linear map (the gradient $\nabla f(\mathbf{x}_0)$) such that:

$$\lim_{\mathbf{h} \to \mathbf{0}} \frac{f(\mathbf{x}_0 + \mathbf{h}) - f(\mathbf{x}_0) - \nabla f(\mathbf{x}_0)^\top \mathbf{h}}{\|\mathbf{h}\|_2} = 0. \tag{3.85}$$

By the Total Differential Theorem, a sufficient condition for differentiability is that all partial derivatives exist and are **continuous** ($f \in C^1$). If partial derivatives exist but are discontinuous, the function may fail to be differentiable, causing gradient descent to fail.

To understand this distinction concretely, consider the classic counterexample:

$$f(x, y) = \begin{cases} 0 & \text{at } (x, y) = (0, 0), \\ \dfrac{x^2 y}{x^2 + y^2} & \text{elsewhere.} \end{cases} \tag{3.86}$$

Evaluating the partial derivatives of Equation (3.86) at the coordinate origin via the difference quotient:

$$\frac{\partial f}{\partial x}(0, 0) = \lim_{h \to 0} \frac{f(h, 0) - f(0, 0)}{h} = \lim_{h \to 0} \frac{0 - 0}{h} = 0, \tag{3.87}$$

$$\frac{\partial f}{\partial y}(0, 0) = \lim_{h \to 0} \frac{f(0, h) - f(0, 0)}{h} = \lim_{h \to 0} \frac{0 - 0}{h} = 0. \tag{3.88}$$

Both partial derivatives exist and equal zero at the origin. However, computing the partial

derivative with respect to y along the line $y = 0$ for any $x \neq 0$:

$$\frac{\partial f}{\partial y}(x, 0) = \left.\frac{x^2(x^2 + y^2) - x^2 y(2y)}{(x^2 + y^2)^2}\right|_{y=0} = \frac{x^4}{x^4} = 1. \tag{3.89}$$

Meanwhile, along the y-axis ($x = 0$): $\frac{\partial f}{\partial y}(0, y) = 0$. Therefore, $\lim_{(x,y)\to(0,0)} \frac{\partial f}{\partial y}(x, y)$ does not exist! The partial derivative is **discontinuous at the origin**, proving that $f(x, y)$ is not differentiable at $(0, 0)$, despite all directional derivatives being well-defined (Figure 3.8).

Postilla continued: Subgradient calculus for non-smooth functions. What occurs when an objective function or neural network activation is continuous and convex, but contains non-differentiable points where classical gradients cease to exist?

- The **ReLU activation function** $f(z) = \max(0, z)$ has a sharp corner (non-differentiable kink) at $z = 0$.
- The **SVM Hinge loss** $\ell(z) = \max(0, 1 - z)$ has a kink at $z = 1$.
- L_1 **Lasso regularization** $\Omega(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_{j=1}^{d} |w_j|$ is non-differentiable at $w_j = 0$.

To optimize such objectives, classical differential calculus is generalized to convex analysis via the **subgradient** and **subdifferential** [6, 32]:

Definition 3.3 (Subgradient and Subdifferential)**.** Let $f : \mathbb{R}^d \to \mathbb{R}$ be a convex function. A vector $\mathbf{g} \in \mathbb{R}^d$ is defined as a **subgradient** of f at point $\mathbf{w}$ if for all $\mathbf{v} \in \mathbb{R}^d$:

$$f(\mathbf{v}) \geq f(\mathbf{w}) + \mathbf{g}^\top(\mathbf{v} - \mathbf{w}). \tag{3.90}$$

Geometrically, Equation (3.90) asserts that the affine hyperplane $L(\mathbf{v}) = f(\mathbf{w}) + \mathbf{g}^\top(\mathbf{v} - \mathbf{w})$ everywhere globally underestimates or touches the epigraph of f. The set of all valid subgradients at $\mathbf{w}$ is called the **subdifferential**, denoted $\partial f(\mathbf{w}) \subset \mathbb{R}^d$.

If f is continuously differentiable at $\mathbf{w}$, the subdifferential is a singleton containing only the standard gradient: $\partial f(\mathbf{w}) = \{\nabla f(\mathbf{w})\}$. At non-differentiable corners, $\partial f(\mathbf{w})$ forms a closed, convex set.

For the rectified linear unit $f(z) = \max(0, z)$:

$$\partial f(z) = \begin{cases} \{0\} & \text{if } z < 0, \\ [0, 1] & \text{if } z = 0, \\ \{1\} & \text{if } z > 0. \end{cases} \tag{3.91}$$

At the singular corner point $z = 0$, any scalar $g \in [0, 1]$ satisfies the subgradient inequality $f(v) \geq 0 + g \cdot v$. In deep learning frameworks such as PyTorch, the implementation adopts the convention $g = 0$ at $z = 0$.

The subgradient descent algorithm updates parameters by selecting an arbitrary valid element $\mathbf{g}_t \in \partial f(\mathbf{w}_t)$:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \mathbf{g}_t. \tag{3.92}$$

Unlike standard gradient descent on smooth functions, subgradient descent is not guaranteed to decrease the objective at every individual step ($f(\mathbf{w}_{t+1})$ may momentarily exceed $f(\mathbf{w}_t)$). However, if the learning rate sequence satisfies the diminishing harmonic step-size conditions:

$$\sum_{t=1}^{\infty} \eta_t = \infty \quad \text{and} \quad \sum_{t=1}^{\infty} \eta_t^2 < \infty, \tag{3.93}$$

the distance to the optimal set $\|\mathbf{w}_t - \mathbf{w}^*\|_2$ contracts monotonically, guaranteeing asymptotic convergence to the global minimum.

3.5.2 Stationary points and initialization

A point $\mathbf{x}^* \in \mathbb{R}^d$ where the gradient vanishes identically is defined as a **stationary point** (or critical point):

$$\nabla f(\mathbf{x}^*) = \mathbf{0}. \tag{3.94}$$

Evaluating the gradient descent update rule at a stationary point reveals an immediate consequence:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \alpha \nabla f(\mathbf{x}^{(t)}) = \mathbf{x}^{(t)} - \mathbf{0} = \mathbf{x}^{(t)}. \tag{3.95}$$

Standard first-order gradient descent **gets stuck** at any stationary point, terminating all parameter updates.

However, a fundamental distinction must be drawn:

$$\text{Stationary Point } (\nabla f = \mathbf{0}) \quad \not\Longleftrightarrow \quad \text{Local Minimum!} \tag{3.96}$$

A stationary point can be a local minimum, a local maximum, or a saddle point.

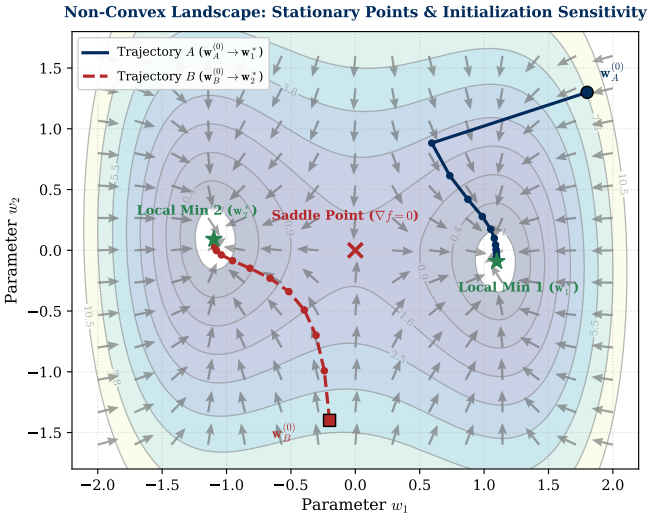


Figure 3.10: Optimization trajectories on a non-convex vector field from different parameter initializations. Starting from the upper-right point, gradient updates follow the negative gradient arrows directly into a local minimum basin. Starting from the lower-left point, updates diverge along a completely different path toward a saddle region or alternative basin.

As visualized in Figure 3.10, which stationary point gradient descent reaches depends entirely on the initial parameter configuration $\mathbf{x}^{(0)}$. In complex non-convex landscapes, different initializations lead to vastly different local basins or saddle geometries.

3.5.3 First-order updates

We parameterize a finite step of length $\eta > 0$ along the negative gradient direction, yielding displacement $\Delta\mathbf{w} = -\eta \nabla f(\mathbf{w})$. The first-order Taylor series approximation around point $\mathbf{w}_t$ is:

$$f(\mathbf{w}_t - \eta \nabla f(\mathbf{w}_t)) \approx f(\mathbf{w}_t) - \eta \|\nabla f(\mathbf{w}_t)\|_2^2 + \mathcal{O}(\eta^2). \tag{3.97}$$

Because $\|\nabla f(\mathbf{w}_t)\|_2^2 > 0$ for any non-stationary point, there exists a sufficiently small step size $\eta > 0$ such that the second-order remainder is dominated by the linear decrease:

$$f(\mathbf{w}_t - \eta \nabla f(\mathbf{w}_t)) < f(\mathbf{w}_t). \tag{3.98}$$

This establishes the classical **Gradient Descent update rule**:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla f(\mathbf{w}_t), \tag{3.99}$$

where $\eta > 0$ is the **learning rate** (step size hyperparameter).

The foundational premise of first-order gradient descent is that the objective function can be effectively approximated by its flat tangent hyperplane. While this linear approximation is strictly valid in an infinitesimal local neighborhood, realistic loss surfaces possess curvature. If an update takes too large a step across a curved landscape, the first-order approximation breaks down, leading to instability. To formally characterize how curvature governs the admissible step size, we must turn to second-order analysis.

3.5.4 Second-order curvature and the Hessian

First-order optimization methods navigate loss landscapes solely on the basis of instantaneous slope, assuming that local descent directions remain reliable over finite distances. However, real-world objective surfaces are curved and non-isotropic. The rate at which the gradient itself varies across parameter space—quantified by second-order partial derivatives—critically determines whether an update step successfully descends toward a minimum or violently overshoots into divergence.

Let $f : \mathbb{R}^d \to \mathbb{R}$ be twice continuously differentiable (C^2). The second-order Taylor series approximation of f around point $\mathbf{w}$ for an arbitrary displacement $\Delta\mathbf{w} \in \mathbb{R}^d$ is:

$$f(\mathbf{w} + \Delta\mathbf{w}) \approx f(\mathbf{w}) + \nabla f(\mathbf{w})^\top \Delta\mathbf{w} + \frac{1}{2}\Delta\mathbf{w}^\top \mathbf{H}(\mathbf{w})\Delta\mathbf{w}, \tag{3.100}$$

where $\mathbf{H}(\mathbf{w}) \in \mathbb{R}^{d\times d}$ is the **Hessian matrix** of second-order partial derivatives:

$$\mathbf{H}_{jk}(\mathbf{w}) = \frac{\partial^2 f(\mathbf{w})}{\partial w_j \partial w_k}, \quad j,k \in \{1,\dots,d\}. \tag{3.101}$$

By Schwarz's theorem on the equality of mixed partials, $\frac{\partial^2 f}{\partial w_j \partial w_k} = \frac{\partial^2 f}{\partial w_k \partial w_j}$, guaranteeing that $\mathbf{H}$ is a real symmetric matrix ($\mathbf{H} = \mathbf{H}^\top$).

By the spectral theorem, the symmetric Hessian admits an orthogonal eigendecomposition:

$$\mathbf{H} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top = \sum_{i=1}^{d} \lambda_i \mathbf{q}_i \mathbf{q}_i^\top, \tag{3.102}$$

where $\mathbf{Q} = [\mathbf{q}_1,\dots,\mathbf{q}_d] \in \mathbb{R}^{d\times d}$ is an orthonormal matrix of eigenvectors ($\mathbf{q}_i^\top \mathbf{q}_j = \delta_{ij}$), and $\mathbf{\Lambda} = \operatorname{diag}(\lambda_1,\dots,\lambda_d)$ contains real eigenvalues ordered such that $\lambda_{\max} \equiv \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \equiv \lambda_{\min}$.

The quadratic form $\frac{1}{2}\Delta\mathbf{w}^\top \mathbf{H}\Delta\mathbf{w}$ characterizes directional curvature:

- If $\Delta\mathbf{w}$ aligns with eigenvector $\mathbf{q}_i$, the second-order change is $\frac{1}{2}\lambda_i \|\Delta\mathbf{w}\|_2^2$.
- A positive eigenvalue ($\lambda_i > 0$) signifies positive curvature (the function curves upward like a bowl).
- A negative eigenvalue ($\lambda_i < 0$) signifies negative curvature (the function curves downward like a dome).
- A zero eigenvalue ($\lambda_i = 0$) signifies flat curvature (the function is locally linear along $\mathbf{q}_i$).

Optimal step size and the stability bound. Substituting the gradient descent step $\Delta\mathbf{w} = -\eta\mathbf{g}$ (where $\mathbf{g} = \nabla f(\mathbf{w})$) into Equation (3.100):

$$f(\mathbf{w} - \eta\mathbf{g}) \approx f(\mathbf{w}) - \eta\mathbf{g}^\top\mathbf{g} + \frac{1}{2}\eta^2 \mathbf{g}^\top \mathbf{H}\mathbf{g}. \tag{3.103}$$

To find the step size η^* that maximizes expected decrease, we differentiate Equation (3.103) with respect to scalar η and set the derivative to zero:

$$\frac{d}{d\eta}\left[f(\mathbf{w}) - \eta\|\mathbf{g}\|_2^2 + \frac{1}{2}\eta^2\mathbf{g}^\top\mathbf{H}\mathbf{g}\right] = -\|\mathbf{g}\|_2^2 + \eta\mathbf{g}^\top\mathbf{H}\mathbf{g} = 0 \implies \eta^* = \frac{\mathbf{g}^\top\mathbf{g}}{\mathbf{g}^\top\mathbf{H}\mathbf{g}}. \tag{3.104}$$

If the gradient vector aligns with the dominant eigenvector of the Hessian ($\mathbf{g} \approx \mathbf{q}_1$), the optimal learning rate contracts to $\eta^* = 1/\lambda_{\max}$. Moreover, convergence strictly requires:

$$0 < \eta < \frac{2}{\lambda_{\max}(\mathbf{H})}. \tag{3.105}$$

If $\eta \geq 2/\lambda_{\max}$, the quadratic term exceeds the linear decrease, and the iterates diverge.

The condition number and ill-conditioned ravines. A fundamental computational bottleneck for first-order gradient descent is the phenomenon of **ill-conditioned ravines** (also known as valleys or canyons).

Definition 3.4 (Hessian condition number)**.** For a convex objective with positive definite Hessian ($\mathbf{H} \succ 0$, so all $\lambda_i > 0$), the **condition number** $\kappa \geq 1$ is defined as the ratio of maximum to minimum eigenvalue:

$$\kappa(\mathbf{H}) = \frac{\lambda_{\max}(\mathbf{H})}{\lambda_{\min}(\mathbf{H})}. \tag{3.106}$$

When $\kappa \approx 1$, the contour lines of the loss surface are isotropic hyperspheres. The gradient vector points directly toward the minimum, and gradient descent converges rapidly.

However, when $\kappa \gg 1$ (e.g., $\kappa = 10^3$ or 10^5), the problem is termed **ill-conditioned**. The contour lines form elongated, highly eccentric ellipsoids.

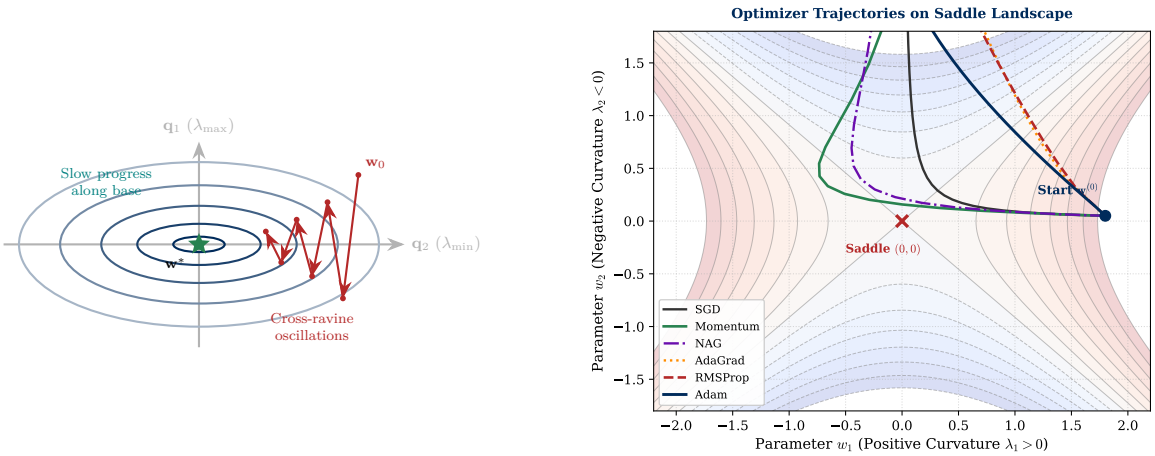


Figure 3.11: Optimization dynamics in ill-conditioned ravines and complex landscapes. Left: Native vector representation of an ill-conditioned quadratic valley ($\kappa = \lambda_{\max}/\lambda_{\min} \gg 1$), where first-order gradient descent oscillates violently across the steep walls along $\mathbf{q}_1$ while making negligible progress along the flat base $\mathbf{q}_2$. Right: Benchmark comparison across optimizers (SGD, Momentum, NAG, AdaGrad, AdaDelta, RMSProp) traversing a saddle topology, demonstrating how momentum and adaptive learning rates overcome anisotropic curvature.

As visualized in Figure 3.11:

- Along the direction of maximum curvature $\mathbf{q}_1$ (across the ravine walls), the loss changes rapidly with eigenvalue $\lambda_{\max}$.

- Along the direction of minimum curvature $\mathbf{q}_2$ (along the ravine base leading to the minimum), the loss changes extremely slowly with eigenvalue $\lambda_{\min}$.
- The stability bound in Equation (3.105) restricts the global learning rate to $\eta < 2/\lambda_{\max}$.
- Because η must be chosen small to prevent explosive divergence across the steep walls, progress along the flat base is scaled by $\eta\lambda_{\min} < 2\frac{\lambda_{\min}}{\lambda_{\max}} = \frac{2}{\kappa} \ll 1$.

Consequently, first-order gradient descent oscillates violently back and forth between the opposing walls of the ravine, making negligible forward progress toward the true minimizer.

3.6 The learning rate

In first-order gradient optimization, the learning rate $\eta > 0$ represents the single most influential hyperparameter governing model training. Acting as the global step-size multiplier, it dictates how far parameters are displaced along the negative gradient vector at every discrete update. Selecting η demands resolving a profound operational dilemma: an excessively small step size guarantees stable local descent but stalls optimization on shallow plateaus, while an excessively large step size risks catastrophic overshooting, bouncing violently across ravines and destabilizing parameter updates toward numerical overflow.

Controlling this behavior requires a comprehensive understanding of step dynamics on contour landscapes, the theoretical stability bounds imposed by local curvature, and the deterministic decay schedules that modulate the step size throughout optimization.

3.6.1 Learning rate dynamics and stability

The magnitude of the learning rate η critically dictates optimization stability:

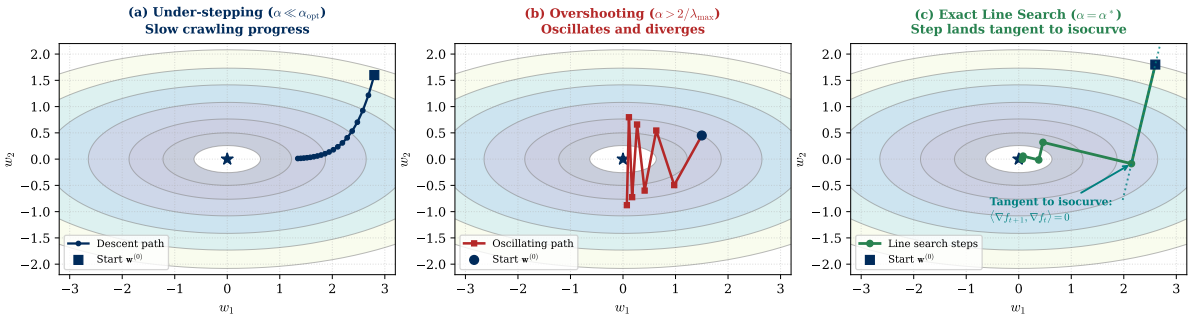


Figure 3.12: Contour dynamics of gradient descent under varying step sizes α: (Left) An excessively small learning rate causes crawling, agonizingly slow progress along the contour gradient; (Middle) An excessively large learning rate overshoots the minimum, oscillating across valley walls with increasing amplitude and ultimately diverging; (Right) An optimal step size determined via line search $\arg\min_\alpha f(\mathbf{x}^{(t)} - \alpha\nabla f(\mathbf{x}^{(t)}))$, where each update lands precisely where the descent direction is tangent to a contour isocurve.

As illustrated across the contour landscapes in Figure 3.12:

- **Effective Step Length**: An essential mathematical nuance is that the geometric length of the parameter update is not simply the scalar learning rate α, but rather the norm-scaled quantity $\alpha\|\nabla f(\mathbf{x}^{(t)})\|$. In regions where the gradient is steep, the parameter displacement is proportionally magnified; conversely, on shallow plateaus where $\|\nabla f\| \approx 0$, progress stalls regardless of α.

- **Under-stepping** ($\alpha \to 0$): When α is excessively conservative, parameter updates $\Delta\mathbf{x}^{(t)}$ are microscopic. Iterations advance imperceptibly, requiring thousands of steps to cross moderate distances and rendering the optimizer vulnerable to premature convergence on deceptive plateaus.
- **Overshooting and Divergence** ($\alpha \gg \alpha_{\mathbf{crit}}$): When α exceeds the critical threshold dictated by the maximum eigenvalue of the Hessian ($\alpha > 2/\lambda_{\max}$, derived in Equation (3.105)), the first-order Taylor approximation breaks down completely. Updates violently oscillate back and forth between opposing valley walls with geometrically increasing amplitude, driving objective values toward infinity: $\lim_{t\to\infty} f(\mathbf{x}^{(t)}) = \infty$.
- **Optimal Step Size via Exact Line Search**: In classical optimization, the optimal step size is determined adaptively by solving a one-dimensional sub-problem along the negative gradient ray:
$$\alpha^* = \arg\min_{\alpha>0} f\left(\mathbf{x}^{(t)} - \alpha\nabla f(\mathbf{x}^{(t)})\right). \tag{3.107}$$
Evaluating the optimality condition by differentiating with respect to α via the multivariate chain rule:
$$\frac{d}{d\alpha} f\left(\mathbf{x}^{(t)} - \alpha\nabla f(\mathbf{x}^{(t)})\right) = -\nabla f\left(\mathbf{x}^{(t+1)}\right)^\top \nabla f\left(\mathbf{x}^{(t)}\right) = 0. \tag{3.108}$$
Equation (3.108) reveals a fundamental geometric theorem: at the exact line-search minimizer, the new gradient $\nabla f(\mathbf{x}^{(t+1)})$ is strictly perpendicular to the previous search direction $-\nabla f(\mathbf{x}^{(t)})$. In terms of contour topology, this means the gradient direction at the landing point is **tangent to the local isocurve** (level set), producing the characteristic orthogonal zig-zag trajectory shown in the right panel of Figure 3.12.

3.6.2 Learning rate decay schedules

Fixing a constant scalar learning rate α forces an undesirable trade-off: a large learning rate enables rapid traversal of distant plateaus early in training but prevents convergence near the minimum due to endless cross-valley oscillations; conversely, a small learning rate ensures stable fine-grained convergence but crawls agonizingly slowly at initialization.

To reconcile these competing objectives, the learning rate can be modulated dynamically over iterations according to a deterministic **decay schedule** parameterized by a decay factor $\rho > 0$:

Linear decay schedule. The linear decay schedule interpolates parameter updates uniformly from an initial step size $\alpha^{(0)}$ down to a designated final floor $\alpha^{(\rho)}$ over a budget of ρ iterations:
$$\alpha^{(t+1)} = \left(1 - \frac{t}{\rho}\right)\alpha^{(0)} + \frac{t}{\rho}\alpha^{(\rho)}, \quad \text{for } 0 \le t \le \rho. \tag{3.109}$$
At iteration $t = 0$, $\alpha^{(1)} = \alpha^{(0)}$. At the terminal horizon $t = \rho$, the learning rate contracts precisely to $\alpha^{(\rho)}$ (typically chosen orders of magnitude smaller than $\alpha^{(0)}$). For iterations $t > \rho$, the rate remains fixed at $\alpha^{(\rho)}$. This schedule provides predictable, constant-rate deceleration.

Inverse decay schedule (Time-based decay). The inverse decay schedule attenuates the step size inversely with the elapsed iteration count:
$$\alpha^{(t+1)} = \frac{\alpha^{(t)}}{1 + \rho t}. \tag{3.110}$$
In its direct non-recursive formulation, $\alpha^{(t)} = \frac{\alpha^{(0)}}{1+\rho t}$. This schedule enforces an asymptotic decay rate of $\mathcal{O}(1/t)$, mirroring the classical Robbins-Monro stochastic convergence rate (detailed in Section 3.9.2) while ensuring rapid initial dampening that smoothly transitions into a long-tailed fine-tuning regime.

Exponential decay schedule. The exponential decay schedule applies continuous geometric dampening scaled by decay rate $\rho > 0$:

$$\alpha^{(t+1)} = \alpha^{(0)} e^{-\rho t}. \tag{3.111}$$

Because the step size contracts by a constant multiplicative factor $e^{-\rho}$ at every iteration, exponential decay rapidly quenches parameter updates. In contemporary deep learning practice, this is frequently discretized into *step decay* (e.g., decaying α by a factor of 0.1 or 0.2 every fixed number of epochs).

Although learning rate decay schedules effectively dampen updates over time to stabilize asymptotic convergence, they apply an identical scalar multiplier uniformly across all spatial coordinates. Consequently, decay schedules are fundamentally incapable of resolving the directional anisotropy of ill-conditioned ravines ($\kappa \gg 1$), where optimization requires aggressive dampening across steep valley walls alongside massive acceleration along gentle valley floors. Furthermore, standard gradient descent operates as a purely reactive, memoryless physical system, resetting its step direction at every iteration solely from instantaneous slope. Overcoming these fundamental geometric bottlenecks requires imbuing the optimizer with physical momentum and developing adaptive coordinate-wise scaling algorithms, as established in Section 3.7.

3.7 Accelerated gradient methods

In high-dimensional non-convex optimization, standard first-order gradient descent behaves like a memoryless physical system: at every iteration, the parameter vector acts as an idealized massless particle that instantly resets its velocity to match the instantaneous negative gradient. While this reactive behavior works satisfactorily on perfectly isotropic quadratic surfaces, it fails catastrophically in the presence of anisotropic curvature and ill-conditioned ravines ($\kappa \gg 1$). In such geometries, the optimizer is buffeted back and forth between opposing valley walls, dissipating nearly all its computational energy in high-frequency cross-valley oscillations while making negligible longitudinal progress toward the minimum.

To overcome this fundamental limitation without incurring the prohibitive $\mathcal{O}(d^3)$ cost of explicit second-order Hessian inversion, statistical learning turns to the principles of classical mechanics. By imbuing the optimizer with **physical momentum**, the parameter vector is modeled not as a massless point, but as a heavy marble rolling down a frictioned potential well. Historical gradient vectors are accumulated into an internal velocity state, allowing persistent directional forces along the valley floor to build up constructive kinetic energy while high-frequency cross-ravine gradient oscillations cancel each other out destructively.

3.7.1 Classical momentum

Learning rate influence on final convergence and the need for adaptive dynamics. In standard first-order optimization, the choice of the scalar learning rate α governs an irreconcilable operational trade-off. As established by the second-order stability bound ($\alpha < 2/\lambda_{\max}$), preventing explosive cross-canyon divergence in an ill-conditioned ravine requires setting α small enough to accommodate the maximum curvature across steep walls. However, this microscopic step size severely penalizes progress along the gentle valley floor, where the longitudinal convergence rate is throttled by $\alpha\lambda_{\min} < 2/\kappa \ll 1$. Conversely, increasing α to accelerate longitudinal progress inevitably causes catastrophic overshooting and divergence across the ravine walls.

Because an isotropic, memoryless scalar step size cannot simultaneously satisfy these competing geometric demands, the effective learning rate must be varied dynamically. The optimizer requires a mechanism that dampens high-frequency transverse bouncing while accelerating progress along persistent descent directions.

Physical mechanism in ill-conditioned valleys. The classical momentum method, formulated by Boris Polyak in 1964 [71], resolves this challenge by drawing direct inspiration from classical mechanics. The optimizer models the parameter vector not as a massless particle whose velocity instantly matches the local negative gradient, but as a heavy marble rolling down a frictioned potential well.

The algorithm introduces an internal state vector $\mathbf{v}^{(t)} \in \mathbb{R}^d$, termed the **velocity**, and updates parameters according to the coupled recurrence:

$$\mathbf{v}^{(t+1)} = \lambda \mathbf{v}^{(t)} - \alpha \nabla f(\mathbf{x}^{(t)}), \tag{3.112}$$

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \mathbf{v}^{(t+1)}, \tag{3.113}$$

where $\alpha > 0$ is the base learning rate, and $\lambda \in [0, 1)$ is the **momentum coefficient** (or friction parameter, typically set to $\lambda = 0.9$ or 0.99).

When $\lambda = 0$, the velocity simplifies to $\mathbf{v}^{(t+1)} = -\alpha \nabla f(\mathbf{x}^{(t)})$, collapsing immediately to standard gradient descent. When $\lambda > 0$, the velocity retains an exponential moving average of historical gradient directions. Decomposing the optimization dynamics into the orthogonal eigenspaces of the local Hessian:

- **Transverse cross-ravine direction ($\mathbf{q}_1$, curvature $\lambda_{\max}$)**: Because the optimizer repeatedly bounces back and forth across opposing valley walls, the signs of consecutive partial derivatives alternate: $\nabla f(\mathbf{x}^{(t)})_{\mathbf{q}_1} \approx -\nabla f(\mathbf{x}^{(t-1)})_{\mathbf{q}_1}$. In the velocity accumulator of Equation (3.112), these opposing vectors cancel out destructively:
$$\mathbf{v}_{\mathbf{q}_1}^{(t+1)} \approx \lambda \mathbf{v}_{\mathbf{q}_1}^{(t)} - \alpha \nabla f(\mathbf{x}^{(t)})_{\mathbf{q}_1} \approx \mathbf{0}. \tag{3.114}$$
This destructive interference aggressively dampens high-frequency cross-ravine oscillations.
- **Longitudinal base direction ($\mathbf{q}_2$, curvature $\lambda_{\min}$)**: Along the gentle descent floor of the valley leading toward the minimizer, consecutive gradients point consistently in the same downhill direction: $\nabla f(\mathbf{x}^{(t)})_{\mathbf{q}_2} \approx \nabla f(\mathbf{x}^{(t-1)})_{\mathbf{q}_2}$. Here, the historical velocity terms add constructively, accumulating directional speed along the base.

Effective terminal step length and aligned gradient acceleration. A central pedagogical result is quantifying the exact acceleration gain when gradients are persistently aligned.

Suppose the optimizer traverses a uniform slope where the gradient remains constant over consecutive iterations: $\nabla f(\mathbf{x}^{(t)}) \approx \mathbf{g}$. Assuming initial velocity $\mathbf{v}^{(0)} = \mathbf{0}$, unrolling the recurrence in Equation (3.112) yields:

$$\begin{aligned}
\mathbf{v}^{(1)} &= -\alpha \mathbf{g}, \\
\mathbf{v}^{(2)} &= -\lambda \alpha \mathbf{g} - \alpha \mathbf{g} = -\alpha(1 + \lambda)\mathbf{g}, \\
\mathbf{v}^{(3)} &= -\lambda \alpha (1 + \lambda)\mathbf{g} - \alpha \mathbf{g} = -\alpha(1 + \lambda + \lambda^2)\mathbf{g}, \\
&\vdots \\
\mathbf{v}^{(t+1)} &= -\alpha \mathbf{g} \sum_{k=0}^{t} \lambda^k.
\end{aligned} \tag{3.115}$$

Because $\lambda \in [0, 1)$, the finite summation in Equation (3.115) is a geometric progression. Taking the limit as $t \to \infty$:

$$\lim_{t \to \infty} \sum_{k=0}^{t} \lambda^k = \frac{1}{1 - \lambda}. \tag{3.116}$$

Consequently, the terminal velocity and effective parameter displacement per step converge to:

$$\lim_{t \to \infty} \|\mathbf{v}^{(t+1)}\| = \frac{1}{1 - \lambda} \alpha \|\mathbf{g}\| = \frac{1}{1 - \lambda} \alpha \|\nabla f\|. \tag{3.117}$$

Equation (3.117) reveals the remarkable speedup factor of momentum:

- For standard gradient descent, the step length along a flat plateau with gradient $\|\nabla f\|$ is strictly $\alpha\|\nabla f\|$.
- For momentum with $\lambda = 0.9$, the effective step length expands to $\frac{1}{1-0.9}\alpha\|\nabla f\| = \mathbf{10}\alpha\|\nabla \mathbf{f}\|$ (10× acceleration).
- For momentum with $\lambda = 0.99$, the effective step length expands to $\frac{1}{1-0.99}\alpha\|\nabla f\| = \mathbf{100}\alpha\|\nabla \mathbf{f}\|$ (100× acceleration).

This dynamic amplification enables momentum to surge rapidly through flat regions and provides the kinetic energy required to carry parameter updates over shallow local barriers into deeper, more generalizing minima basins.

3.7.2 Unrolling optimization dynamics

To understand how momentum transforms optimization trajectories over an entire training run, we analyze the optimization recurrence in unrolled form.

Unrolling standard gradient descent. For standard gradient descent without momentum, parameter updates follow $\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \alpha\nabla f(\mathbf{x}^{(t)})$. Unrolling step by step from initialization $\mathbf{x}^{(0)}$:

$$\begin{aligned}
\mathbf{x}^{(1)} &= \mathbf{x}^{(0)} - \alpha\nabla f(\mathbf{x}^{(0)}),\\
\mathbf{x}^{(2)} &= \mathbf{x}^{(1)} - \alpha\nabla f(\mathbf{x}^{(1)}) = \mathbf{x}^{(0)} - \alpha\nabla f(\mathbf{x}^{(0)}) - \alpha\nabla f(\mathbf{x}^{(1)}),\\
&\vdots\\
\mathbf{x}^{(t+1)} &= \mathbf{x}^{(0)} - \alpha\sum_{i=1}^{t}\nabla f(\mathbf{x}^{(i)}).
\end{aligned} \tag{3.118}$$

Equation (3.118) shows that in standard gradient descent, the net parameter displacement $\mathbf{x}^{(t+1)} - \mathbf{x}^{(0)}$ is simply the unweighted sum of all historical gradients, multiplied by constant step size $-\alpha$. Every historical gradient receives identical weighting, regardless of whether it was computed at the start of training or in the current iteration.

Unrolling gradient descent with momentum. Now consider the momentum update defined by Equations (3.112)–(3.113). Assuming $\mathbf{v}^{(0)} = \mathbf{0}$, the velocity at any iteration $k \geq 1$ is an exponentially decayed sum of past gradients:

$$\mathbf{v}^{(k)} = -\alpha\sum_{j=1}^{k}\lambda^{k-j}\nabla f(\mathbf{x}^{(j-1)}). \tag{3.119}$$

Since $\mathbf{x}^{(t+1)} = \mathbf{x}^{(0)} + \sum_{k=1}^{t+1}\mathbf{v}^{(k)}$, the cumulative displacement after $t+1$ steps is:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(0)} - \alpha\sum_{k=1}^{t+1}\sum_{j=1}^{k}\lambda^{k-j}\nabla f(\mathbf{x}^{(j-1)}). \tag{3.120}$$

To determine the net coefficient multiplying each individual gradient $\nabla f(\mathbf{x}^{(i)})$ (for $i = 1, \ldots, t$), we observe how often $\nabla f(\mathbf{x}^{(i)})$ enters into subsequent velocity terms:

- At iteration $k = i+1$, $\nabla f(\mathbf{x}^{(i)})$ enters with weight $-\alpha\lambda^0 = -\alpha$.
- At iteration $k = i+2$, it persists decayed by $-\alpha\lambda^1$.
- At iteration $k = i+3$, it persists decayed by $-\alpha\lambda^2$.
- ...
- At iteration $k = t+1$, it persists decayed by $-\alpha\lambda^{t-i}$.

Summing these contributions across all subsequent steps gives a finite geometric series of length $t-i+1$:

$$\sum_{m=0}^{t-i} \lambda^m = \frac{1-\lambda^{t-i+1}}{1-\lambda} = \frac{1-\lambda^{t+1-i}}{1-\lambda}. \tag{3.121}$$

Substituting this closed-form coefficient into the unrolled parameter position yields the exact momentum unrolling formula:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(0)} - \alpha \sum_{i=1}^{t} \left(\frac{1-\lambda^{t+1-i}}{1-\lambda} \right) \nabla f(\mathbf{x}^{(i)}). \tag{3.122}$$

For early gradients where $t-i \gg 0$, $\lambda^{t+1-i} \to 0$, so their cumulative weight saturates at $-\frac{\alpha}{1-\lambda}$. For very recent gradients ($i \approx t$), the weight is approximately $-\alpha$. This provides a smooth, recency-weighted memory filter over the optimization trajectory.

3.7.3 Adaptive learning rate optimizers

The unified diagonal scaling matrix framework. The unrolled formulations of Equations (3.118) and (3.122) represent special cases of a far more general algebraic structure. Any first-order optimization method can be expressed in the **unified unrolled form**:

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(0)} - \alpha \sum_{i=1}^{t} \mathbf{\Gamma}_i^t \nabla f(\mathbf{x}^{(i)}), \tag{3.123}$$

where $\mathbf{\Gamma}_i^t \in \mathbb{R}^{d \times d}$ is a time-varying **diagonal scaling matrix**:

$$\mathbf{\Gamma}_i^t = \operatorname{diag}\left(\gamma_{i,1}^t,\, \gamma_{i,2}^t,\, \ldots,\, \gamma_{i,d}^t \right). \tag{3.124}$$

This formulation highlights a profound limitation of classical momentum: it applies an identical scalar coefficient λ to all d coordinate axes simultaneously. However, in deep neural networks, parameters across different layers experience radically disparate gradient magnitudes and curvatures (e.g., infrequent word embeddings vs. dense output classification weights). The diagonal scaling matrix $\mathbf{\Gamma}_i^t$ enables the optimizer to assign coordinate-specific acceleration strengths and adaptive effective step sizes along each individual coordinate direction:

$$\alpha_{t,j}^{\text{eff}} = \alpha \cdot \gamma_{t,j}^t. \tag{3.125}$$

Complete taxonomy of optimizer variations. This unified matrix framework elegantly subsumes the primary optimization algorithms studied in modern deep learning:

1. **Standard Gradient Descent (SGD)**:
   - *General Idea*: Evaluates instantaneous local slopes with zero historical memory and isotropic uniform scaling.
   - *Mathematical Formula*: $\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \alpha \nabla f(\mathbf{x}^{(t)})$.
   - *Scaling Matrix*: $\mathbf{\Gamma}_i^t = \mathbf{I}_d$, assigning identical unit weight across all dimensions.
2. **Classical Polyak Momentum**:
   - *General Idea*: Models physical inertia to cancel cross-ravine oscillations and accelerate along persistent valley base gradients.
   - *Mathematical Formula*: $\mathbf{v}^{(t+1)} = \lambda \mathbf{v}^{(t)} - \alpha \nabla f(\mathbf{x}^{(t)})$, $\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \mathbf{v}^{(t+1)}$.
   - *Scaling Matrix*: $\mathbf{\Gamma}_i^t = \left(\frac{1-\lambda^{t+1-i}}{1-\lambda} \right) \mathbf{I}_d$.
3. **Nesterov Accelerated Gradient (NAG)**:

- *General Idea*: Anticipatory "look-ahead" momentum. Rather than evaluating the gradient at the current parameter position $\mathbf{x}^{(t)}$, NAG evaluates the gradient at the predicted future point $\mathbf{x}^{(t)} + \lambda\mathbf{v}^{(t)}$. This provides an analytical braking mechanism that curbs overshooting when approaching a valley bottom.
- *Mathematical Formula*:

$$\mathbf{v}^{(t+1)} = \lambda\mathbf{v}^{(t)} - \alpha\nabla f\left(\mathbf{x}^{(t)} + \lambda\mathbf{v}^{(t)}\right), \tag{3.126}$$

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} + \mathbf{v}^{(t+1)}. \tag{3.127}$$

- *Scaling Matrix*: $\mathbf{\Gamma}_i^t = \left(\frac{1-\lambda^{t+1-i}}{1-\lambda}\right)\mathbf{I}_d$, applied to the look-ahead Jacobian-adjusted gradient trajectory.

4. **AdaGrad (Adaptive Gradient Algorithm)** [24]:

- *General Idea*: Adapts step sizes per coordinate inversely proportional to the cumulative historical gradient energy. Infrequently occurring features receive larger exploratory steps, while frequently occurring features are dampened.
- *Mathematical Formula*: Accumulate historical squared gradients:

$$G_{t,j} = G_{t-1,j} + \left(\nabla f(\mathbf{x}^{(t)})_j\right)^2 = \sum_{k=1}^{t}\left(\nabla f(\mathbf{x}^{(k)})_j\right)^2. \tag{3.128}$$

Coordinate parameter update:

$$x_j^{(t+1)} = x_j^{(t)} - \frac{\alpha}{\sqrt{G_{t,j}+\epsilon}}\nabla f(\mathbf{x}^{(t)})_j. \tag{3.129}$$

- *Scaling Matrix*: $\mathbf{\Gamma}_t^t = \operatorname{diag}\left(\frac{1}{\sqrt{G_{t,1}+\epsilon}}, \ldots, \frac{1}{\sqrt{G_{t,d}+\epsilon}}\right)$.
- *Critical Drawback*: Because $G_{t,j}$ sums positive squared gradients monotonically, $G_{t,j} \to \infty$ as $t \to \infty$. Consequently, the effective learning rate $\alpha_{t,j}^{\text{eff}} \propto 1/\sqrt{t} \to 0$ collapses monotonically to zero, causing premature training freeze before convergence.

5. **RMSprop (Root Mean Square Propagation)** [32] & **AdaDelta**:

- *General Idea*: Resolves AdaGrad's premature learning rate collapse by replacing the monotonic sum with an Exponential Moving Average (EMA) of squared gradients, introducing a forgetting factor (effective sliding window).
- *Mathematical Formula*: Second moment accumulator ($\beta \approx 0.9$):

$$v_{t,j} = \beta v_{t-1,j} + (1-\beta)\left(\nabla f(\mathbf{x}^{(t)})_j\right)^2. \tag{3.130}$$

Coordinate parameter update:

$$x_j^{(t+1)} = x_j^{(t)} - \frac{\alpha}{\sqrt{v_{t,j}+\epsilon}}\nabla f(\mathbf{x}^{(t)})_j. \tag{3.131}$$

- *Scaling Matrix*: $\mathbf{\Gamma}_t^t = \operatorname{diag}\left(\frac{1}{\sqrt{v_{t,1}+\epsilon}}, \ldots, \frac{1}{\sqrt{v_{t,d}+\epsilon}}\right)$.
- *Advantage*: The effective learning rate $\alpha/\sqrt{v_{t,j}+\epsilon}$ stabilizes at an active, responsive plateau rather than decaying to zero.

6. **Adam (Adaptive Moment Estimation)** [51]:

- *General Idea*: Combines first-moment momentum tracking in the numerator (velocity accumulation) and second-moment uncentered variance tracking in the denominator (RMSprop scaling), with analytical bias correction to fix initialization at zero.

- *Mathematical Formula*: First raw moment ($\beta_1 = 0.9$):

$$m_{t,j} = \beta_1 m_{t-1,j} + (1-\beta_1)\nabla f(\mathbf{x}^{(t)})_j. \tag{3.132}$$

Second uncentered moment ($\beta_2 = 0.999$):

$$v_{t,j} = \beta_2 v_{t-1,j} + (1-\beta_2)\left(\nabla f(\mathbf{x}^{(t)})_j\right)^2. \tag{3.133}$$

Bias-corrected estimates:

$$\hat{m}_{t,j} = \frac{m_{t,j}}{1-\beta_1^t}, \quad \hat{v}_{t,j} = \frac{v_{t,j}}{1-\beta_2^t}. \tag{3.134}$$

Parameter update rule:

$$x_j^{(t+1)} = x_j^{(t)} - \frac{\alpha}{\sqrt{\hat{v}_{t,j}} + \epsilon}\hat{m}_{t,j}. \tag{3.135}$$

- *Scaling Matrix*: $\mathbf{\Gamma}_i^t = \operatorname{diag}\left(\frac{1}{\sqrt{\hat{v}_{t,1}}+\epsilon}, \ldots, \frac{1}{\sqrt{\hat{v}_{t,d}}+\epsilon}\right) \cdot \left(\frac{1-\beta_1^{t+1-i}}{1-\beta_1}\right)$.
- *Signal-to-Noise Ratio Interpretation*: The update scales parameters by $\frac{\hat{m}_{t,j}}{\sqrt{\hat{v}_{t,j}}} \approx \frac{\mathbb{E}[g_j]}{\sqrt{\mathbb{E}[g_j^2]}}$. Along directions with consistent gradient signs, the ratio approaches ± 1, executing confident full steps; along noisy or oscillating directions, the ratio contracts toward zero, providing automatic coordinate regularization.

Learning rate variation across training epochs. To understand how these formulations alter learning rate dynamics over the course of training, Figure 3.13 compares deterministic decay schedules with adaptive coordinate-wise optimizer dynamics across 100 training epochs:

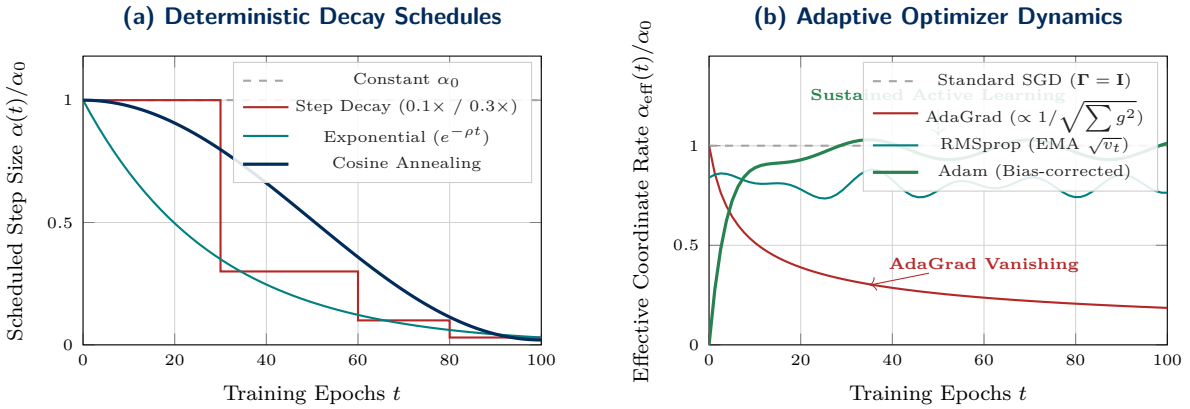


Figure 3.13: Learning rate variation across epochs under deterministic decay schedules versus adaptive coordinate-wise optimizer dynamics. (a) Deterministic schedules attenuate the global step size monotonically to guarantee fine asymptotic settling. (b) Adaptive optimizers rescale step sizes dynamically per coordinate: AdaGrad suffers from catastrophic learning rate decay toward zero ($\propto 1/\sqrt{t}$) due to monotonically growing gradient sums, while RMSprop and Adam stabilize at active, responsive step plateaus that prevent optimization stall.

As visualized across both panels:

- In **Panel (a)**, deterministic decay schedules enforce monotonic global dampening. Step decay executes discrete discrete drops at designated milestone epochs, exponential decay quenches updates continuously, and cosine annealing smoothly transitions from aggressive early exploration to delicate asymptotic fine-tuning.

- In **Panel (b)**, adaptive optimizers dynamically recalibrate effective step sizes $\alpha_{\text{eff}}(t)$ based on gradient history. While standard SGD maintains a static multiplier, AdaGrad exhibits rapid, irreversible decay toward zero ($\approx 1/\sqrt{t}$) because historical squared gradients accumulate indefinitely, freezing parameters prematurely. In sharp contrast, RMSprop and Adam employ exponential moving averages to restrict memory to a recent horizon, allowing effective step sizes to dynamically fluctuate and stabilize around an active, healthy plateau that preserves learning capacity throughout training.

Although momentum and adaptive diagonal scaling provide powerful mechanisms for traversing ill-conditioned valleys and accelerating through flat regions, they were originally conceived within the paradigm of convex or locally unimodal optimization. Deep neural architectures, however, present a radically more complex challenge: the composition of multi-layer nonlinear transformations shatters convexity completely, creating high-dimensional loss landscapes populated by astronomically many critical points—including local minima, local maxima, and complex saddle surfaces. Furthermore, simply locating an empirical minimum does not ensure predictive success, as different minima exhibit wildly divergent generalization properties on unseen data. To optimize deep models effectively, we must therefore investigate the topology of critical points and the delicate interplay between curvature and generalization, as explored in Section 3.8.

3.8 Critical points and generalization

In classical convex optimization, the landscape of the objective function is inherently benign: every stationary point where the gradient vanishes is guaranteed to be a global minimum, ensuring that local search trajectories converge to globally optimal parameters regardless of initialization. In deep neural networks, however, permutation symmetries among hidden units and the cascaded nesting of non-linear activations shatter convexity completely. The resulting parameter space is characterized by an intricate, non-convex terrain punctuated by an astronomical proliferation of critical points, flat plateaus, and treacherous ravines.

For decades, the central fear in non-convex optimization was that gradient-based algorithms would become irrevocably trapped in suboptimal, high-loss local minima. However, modern high-dimensional probability and random matrix theory have revolutionized our understanding of deep loss surfaces: in millions of dimensions, the probability that all eigenvalues of the Hessian are simultaneously positive at a high loss elevation decays exponentially to zero. The primary obstacles that impede training are not isolated local minima, but rather high-index saddle points and vast degenerate plateaus. Crucially, the ultimate objective of machine learning is not to achieve the absolute lowest empirical training loss, but to identify parameter configurations that generalize robustly to unseen test distributions.

3.8.1 Critical point classification

At a critical point $\mathbf{w}^*$, the linear Taylor term in Equation (3.100) vanishes, leaving the local behavior entirely governed by the quadratic form $\frac{1}{2}\Delta\mathbf{w}^\top\mathbf{H}(\mathbf{w}^*)\Delta\mathbf{w}$:

1. **Local Minimum**: The Hessian is positive definite ($\mathbf{H} \succ 0$, all $\lambda_i > 0$). Any displacement strictly increases the loss: $f(\mathbf{w}^* + \Delta\mathbf{w}) > f(\mathbf{w}^*)$.

2. **Local Maximum**: The Hessian is negative definite ($\mathbf{H} \prec 0$, all $\lambda_i < 0$). Any displacement strictly decreases the loss: $f(\mathbf{w}^* + \Delta\mathbf{w}) < f(\mathbf{w}^*)$.

3. **Saddle Point**: The Hessian has mixed eigenvalues ($\exists\lambda_i > 0$ and $\exists\lambda_j < 0$). The function curves upward along eigenvector $\mathbf{q}_i$ and curves downward along eigenvector $\mathbf{q}_j$.

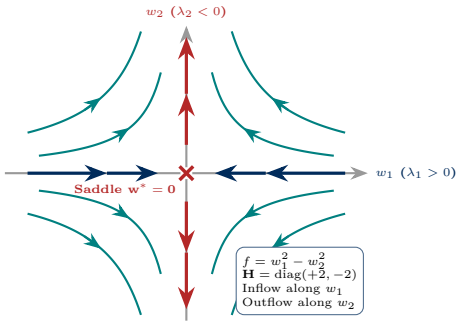


Figure 3.14: Vector field and contour topology around critical points, illustrating how gradients vanish ($\nabla f = \mathbf{0}$) at stationary points and distinguishing a hyperbolic saddle point from a local minimum basin.

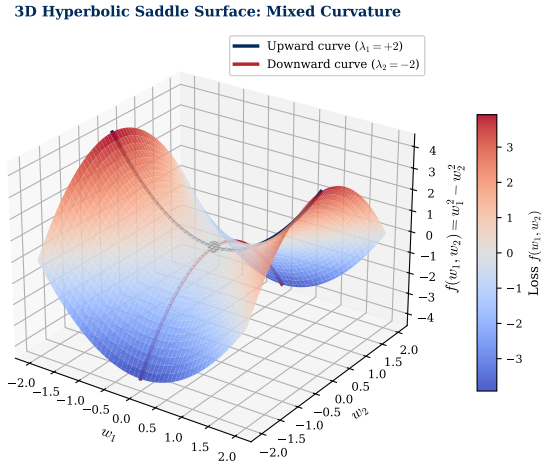


Figure 3.15: High-resolution 3D visualization of the hyperbolic saddle surface, illustrating mixed eigenvalues $\lambda_1 > 0$ and $\lambda_2 < 0$.

Figures 3.14 and 3.15 illustrate a canonical hyperbolic saddle surface $f(w_1, w_2) = w_1^2 - w_2^2$. At the origin $(0, 0)$, the gradient is zero: $\nabla f = [2w_1, -2w_2]^\top = [0, 0]^\top$. The Hessian matrix is constant:

$$\mathbf{H} = \begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix}, \tag{3.136}$$

yielding eigenvalues $\lambda_1 = +2 > 0$ (along the w_1 axis) and $\lambda_2 = -2 < 0$ (along the w_2 axis).

3.8.2 Saddle point proliferation

In popular machine learning discourse, a persistent misconception is that training deep neural networks fails because first-order gradient descent gets trapped in poor local minima.

In a landmark paper presented at NeurIPS 2014, Dauphin, Bengio et al. [17] disproved this hypothesis using random matrix theory and Bray-Dean statistical physics of high-dimensional non-convex glasses. They established that:

1. In a d-dimensional parameter space ($d \sim 10^7$ to 10^{11} in modern deep learning), the probability that all d random eigenvalues of the Hessian share the same positive sign scales asymptotically as:

$$\mathbb{P}(\mathbf{H} \succ 0) \propto \left(\frac{1}{2}\right)^d \approx 0. \tag{3.137}$$

   For $d = 1000$, $(1/2)^{1000} \approx 10^{-301}$, which is physically zero. Therefore, almost all critical points encountered in deep architectures are **saddle points**, not local minima!

2. High-loss critical points are overwhelmingly surrounded by directions of negative curvature ($\lambda_i < 0$), providing open escape paths. True local minima exist almost exclusively near the bottom of the loss landscape, where empirical risk is already low and model generalization is strong.

3. However, because $\|\nabla f(\mathbf{w})\| \approx 0$ in the vicinity of a saddle point, standard first-order gradient descent decelerates dramatically, spending exponential time navigating plateau-like saddle neighborhoods unless escaping noise is injected.

3.8.3 Flat versus sharp minima

In modern statistical learning, a foundational insight overturns classical intuition:

"We are rarely interested in the global optimum!"

In classical convex optimization, the global minimum is the undisputed goal. In deep learning, however, searching for the absolute global minimum of the empirical risk $\hat{\mathcal{R}}_n(\mathbf{w})$ is not merely unnecessary—it is actively hazardous:

1. **Overparameterization and Memorization**: Modern deep networks contain vastly more parameters than training samples ($d \gg n$). A network with millions of parameters possesses sufficient capacity to achieve zero empirical loss ($\hat{\mathcal{R}}_n(\mathbf{w}^*) = 0$) by memorizing every arbitrary training sample, including mislabeled instances and random noise.

2. **Generalization Failure**: Reaching the absolute global minimum of empirical loss often drives parameters into extreme overfitted configurations. Because the training set is merely a finite empirical sample from the underlying data distribution P, perfectly fitting the training sample creates large errors on unseen test data ($\mathcal{R}(h_{\mathbf{w}^*}) \gg 0$).

3. **Sub-optimal Local Basins Generalize Better**: Any reasonable local minimum that achieves low (but non-zero) loss and resides in a broad, well-behaved basin yields vastly superior generalization compared to the degenerate empirical global optimum.

To formalize why certain local minima generalize while others overfit, optimization theory examines the **spectral flatness** of the loss basin (Hochreiter & Schmidhuber [39]; Keskar et al. [50]), characterized by the eigenvalues of the local Hessian matrix.

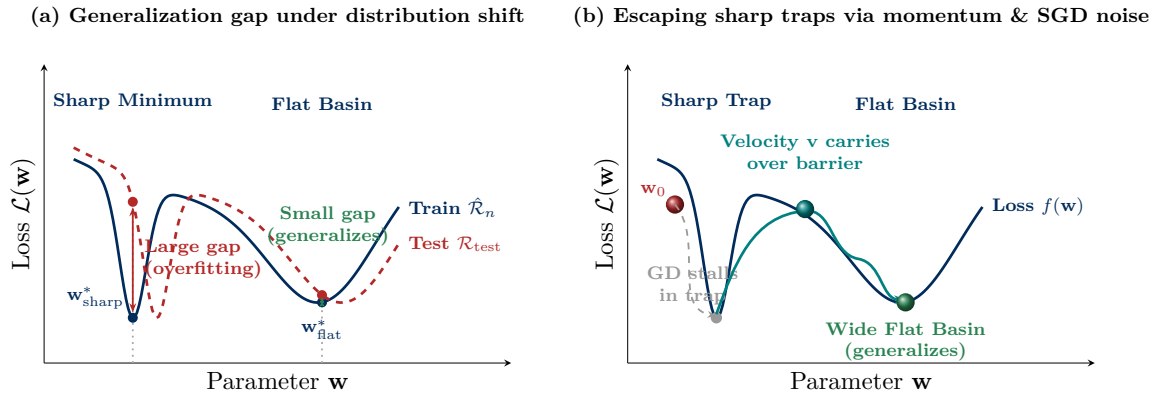


Figure 3.16: The geometry of generalization in non-convex optimization. (a) Under empirical-to-test distribution shift $\Delta\mathbf{w}$, evaluating a sharp minimum $\mathbf{w}^*_{\text{sharp}}$ (large Hessian eigenvalues $\lambda_i \gg 0$) causes test loss to explode ($\Delta\mathcal{R} \gg 0$), resulting in severe overfitting. Conversely, a flat minimum $\mathbf{w}^*_{\text{flat}}$ (small Hessian eigenvalues $\lambda_i \approx 0$) experiences negligible loss increase, ensuring robust generalization. (b) Standard gradient descent readily gets trapped in narrow, high-loss sharp valleys, whereas momentum velocity $\mathbf{v}$ and stochastic mini-batch noise provide the kinetic energy to escape barriers and settle into wide, flat basins.

As demonstrated in Figure 3.16:

- **Sharp Minima ($\lambda_{\max} \gg 0$)**: Characterized by narrow, steep ravines. Because the empirical training distribution $\hat{P}$ differs slightly from the true underlying distribution P, the test loss landscape is shifted horizontally by some small displacement $\Delta\mathbf{w}$. In a sharp well, moving $\mathbf{w}^*_{\text{sharp}} + \Delta\mathbf{w}$ catapults the loss value up the steep canyon wall, causing a catastrophic generalization gap.

- **Flat Minima** ($\lambda_{\max} \approx 0$): Characterized by wide, gentle valleys. Under the exact same parameter displacement $\Delta\mathbf{w}$, the test loss remains virtually unchanged: $\mathcal{R}_{\text{test}}(\mathbf{w}^*_{\text{flat}} + \Delta\mathbf{w}) \approx \hat{\mathcal{R}}_n(\mathbf{w}^*_{\text{flat}})$. Flat minima are inherently robust to distribution shift and parameter quantization.
- **Kinetic Escape**: While standard batch gradient descent easily gets trapped in steep, non-generalizing local basins, momentum velocity $\mathbf{v}$ and stochastic mini-batch noise $\boldsymbol{\xi}_t$ act as dynamic perturbations that kick parameters over low barriers, naturally filtering out sharp wells in favor of expansive flat basins.

Although first-order gradient descent equipped with momentum and adaptive scaling provides a robust framework for navigating non-convex landscapes toward flat minima, its full-batch formulation suffers from a crushing operational bottleneck in modern data-rich environments: computing the exact gradient requires evaluating forward and backward passes across every single training observation in the dataset. When training sets scale to millions or billions of examples, evaluating a single full-batch parameter update becomes prohibitively sluggish, requiring hours or days of compute. Moreover, deterministic full-batch trajectories lack any exploratory stochasticity to jump over low energetic barriers separating sharp, overfitted wells from wide, generalizing valleys. To unlock efficient learning on massive datasets while harnessing beneficial stochastic perturbation, we must transition from deterministic batch optimization to stochastic gradient descent and mini-batch dynamics in Section 3.9.

3.9 Stochastic gradient descent

The empirical risk objective in supervised learning is structured as an arithmetic average of loss terms computed across individual training samples. By linearity of differentiation, computing the exact full-batch gradient requires summing the loss gradients evaluated over the entire dataset. In an era where benchmark datasets encompass millions of high-resolution images, billions of web documents, or continuous streaming logs, executing this exhaustive summation for every parameter update creates a paralyzing computational bottleneck that brings optimization to a virtual standstill.

Stochastic gradient descent (SGD) resolves this computational crisis through a profound statistical principle: an average computed over a small, randomly sampled subset of observations serves as an unbiased estimator of the true population expectation. While this subsampling injects high-frequency variance into parameter updates, this stochastic noise is far from a mere nuisance: it acts as a beneficial thermal agitation—an implicit Langevin regularizer that actively prevents parameters from settling into narrow, sharp overfitted minima, nudging the optimization trajectory toward wide, flat basins with superior generalization capabilities.

3.9.1 Mini-batch gradient descent

Formally, the empirical risk objective $\hat{\mathcal{R}}_n(\boldsymbol{\theta})$ is defined as the arithmetic mean of loss functions evaluated over n training observations:

$$\ell(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \hat{\ell}_i(\boldsymbol{\theta}), \quad \text{where } \hat{\ell}_i(\boldsymbol{\theta}) \equiv \ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i). \tag{3.138}$$

By linearity of differentiation, computing the full batch gradient requires evaluating:

$$\nabla\ell(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \nabla\hat{\ell}_i(\boldsymbol{\theta}). \tag{3.139}$$

Equation (3.139) exposes the **two fundamental bottlenecks** that render full-batch gradient descent impractical for contemporary machine learning:

1. **Bottleneck 1: Number of examples** n. In modern datasets (ImageNet, web corpora), n ranges from 10^6 to 10^{12}. Evaluating all n forward-backward passes to execute a single parameter step takes hours or days, stalling progress completely.
2. **Bottleneck 2: Number of parameters** d. Neural networks contain millions to billions of parameters ($d \sim 10^7$–10^{11}). Storing, communicating, and computing gradients across d dimensions per sample magnifies computational overhead.

To break the dataset size bottleneck n, we approximate the full summation over training set $\mathcal{T}$ ($|\mathcal{T}| = n$) by evaluating the gradient over a small, representative, uniformly drawn subset $\mathcal{B} \subset \mathcal{T}$ of cardinality $m \ll n$ (termed a **mini-batch**):

$$\frac{1}{m}\sum_{i\in\mathcal{B}} \nabla\hat{\ell}_i(\boldsymbol{\theta}) \approx \frac{1}{n}\sum_{i\in\mathcal{T}} \nabla\hat{\ell}_i(\boldsymbol{\theta}). \tag{3.140}$$

A concrete benchmark speedup calculation: MNIST. To appreciate the computational advantage concretely, consider the canonical MNIST handwritten digit benchmark:

- Full training set size: $n = 60{,}000$ images.
- Mini-batch size: $m = 10$ images.
- Speedup factor per gradient update:

$$\text{Speedup} = \frac{n}{m} = \frac{60{,}000}{10} = \mathbf{6{,}000}\times \text{ faster per step!} \tag{3.141}$$

In the time full-batch GD takes to evaluate a single update, mini-batch SGD executes 6,000 parameter updates, making massive progress across the loss terrain.

Algorithmic cycle and epoch progression. The procedural execution of mini-batch stochastic gradient descent proceeds cyclically as formalized in Algorithm 2.

Algorithm 2 Mini-Batch Stochastic Gradient Descent (SGD)

Require: Training dataset $\mathcal{T} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, loss function ℓ, learning rate schedule $\{\alpha_t\}$, mini-batch size m, total epochs E.
Ensure: Optimized parameter vector $\boldsymbol{\theta}^*$.

```
Initialize parameter vector θ^(0) ∈ R^d (e.g., via He or Xavier initialization)
Initialize global step counter t ← 0
for epoch = 1 to E do
    Randomly shuffle training dataset T
    Partition T into K = ⌈n/m⌉ contiguous mini-batches {B_1, B_2, ..., B_K}
    for k = 1 to K do
        Compute mini-batch stochastic gradient estimator:
          g_t ← (1/|B_k|) Σ_{i∈B_k} ∇_θ ℓ(f_{θ^(t)}(x_i), y_i)
        Execute parameter update step:
          θ^(t+1) ← θ^(t) − α_t g_t
        Advance step counter: t ← t + 1
    end for
end for
return θ^(t)
```

Remark 3.2 (Constant Update Cost)**.** A crucial property is that the update cost per iteration is $\mathcal{O}(m \cdot d)$, which is **strictly constant regardless of the total size of dataset** $\mathcal{T}$. This invariance enables deep learning algorithms to scale gracefully to virtually infinite streaming datasets.

3.9.2 Statistical gradient properties

We formalize the statistical properties of the stochastic gradient estimator:

Unbiasedness. Let indices in mini-batch $\mathcal{B}$ be drawn uniformly at random without replacement from $\{1, \ldots, n\}$. The expected value of the estimator is:

$$\mathbb{E}_{\mathcal{B}}\left[\frac{1}{m}\sum_{i\in\mathcal{B}}\nabla\hat{\ell}_i(\boldsymbol{\theta})\right] = \frac{1}{m}\sum_{i\in\mathcal{B}}\mathbb{E}\left[\nabla\hat{\ell}_i(\boldsymbol{\theta})\right] = \frac{1}{m}\cdot m\cdot\left(\frac{1}{n}\sum_{k=1}^{n}\nabla\hat{\ell}_k(\boldsymbol{\theta})\right) = \nabla\ell(\boldsymbol{\theta}). \tag{3.142}$$

The mini-batch gradient is an **unbiased estimator** of the true full-batch gradient.

Variance reduction via batch size m. Let $\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}\|\nabla\hat{\ell}_i(\boldsymbol{\theta}) - \nabla\ell(\boldsymbol{\theta})\|_2^2$ denote sample gradient variance. The covariance of the mini-batch estimator satisfies:

$$\mathbb{E}\left[\left\|\frac{1}{m}\sum_{i\in\mathcal{B}}\nabla\hat{\ell}_i(\boldsymbol{\theta}) - \nabla\ell(\boldsymbol{\theta})\right\|_2^2\right] = \frac{\sigma^2}{m}\left(\frac{n-m}{n-1}\right) \approx \frac{\sigma^2}{m}. \tag{3.143}$$

Variance contracts as $\mathcal{O}(1/m)$, while computational cost scales as $\mathcal{O}(m)$. Due to the square-root law of diminishing returns (StdDev $\propto 1/\sqrt{m}$), increasing batch size from $m = 32$ to 1024 increases compute by $32\times$, but only reduces noise by $\sqrt{32} \approx 5.65\times$. Hence, moderate batch sizes ($32 \leq m \leq 256$) offer the optimal trade-off between parallel throughput and stochastic exploration.

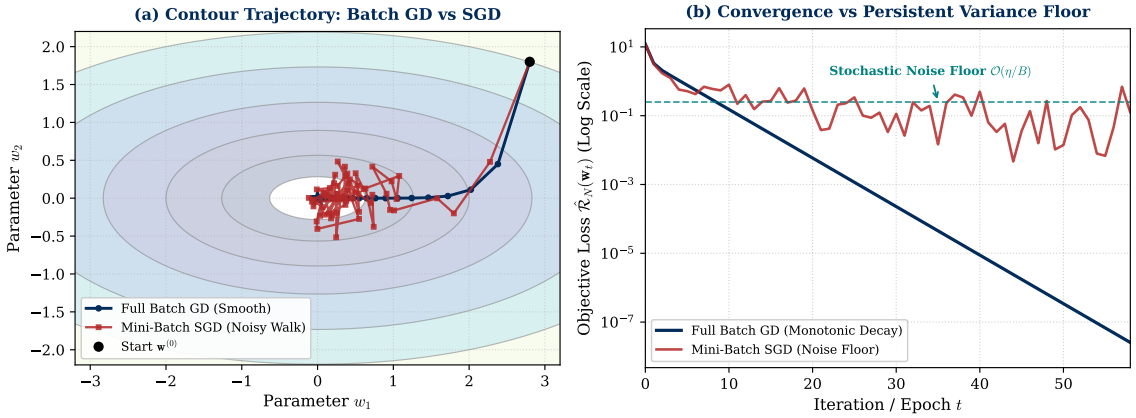


Figure 3.17: Stochastic gradient descent dynamics: (Left) Contour trajectory showing the noisy Brownian-like walk around the loss minimum without exact stationary convergence; (Right) Parameter weight evolution over epochs on a logarithmic scale, illustrating rapid initial progress followed by persistent steady-state stochastic variance fluctuations induced by random data subsampling.

Steady-state fluctuations around the minimum. As illustrated in Figure 3.17, a defining hallmark of SGD is that **it does not halt at the minimum**:

$$\lim_{t\to\infty}\mathbb{E}\left[\|\boldsymbol{\theta}_t - \boldsymbol{\theta}^*\|_2^2\right] \approx \mathcal{O}(\alpha\sigma^2). \tag{3.144}$$

Even when the true gradient vanishes at the minimizer ($\nabla\ell(\boldsymbol{\theta}^*) = \mathbf{0}$), individual sample gradients $\nabla\hat{\ell}_i(\boldsymbol{\theta}^*)$ do not vanish! Consequently, mini-batch gradients retain stochastic noise, causing parameters to wander indefinitely in a steady-state covariance cloud around $\boldsymbol{\theta}^*$.

Asymptotic variance extinction and the Robbins-Monro condition. To extinguish persistent stochastic fluctuations and guarantee almost-sure convergence to a stationary point ($\lim_{t\to\infty} \boldsymbol{\theta}_t = \boldsymbol{\theta}^*$), the learning rate sequence $\{\alpha_t\}_{t=1}^{\infty}$ must satisfy the classical Robbins-Monro conditions [76]:

$$\sum_{t=1}^{\infty} \alpha_t = \infty \quad \text{and} \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty. \tag{3.145}$$

The first condition ensures an infinite exploration horizon so the optimizer never stalls prematurely, while the second guarantees that cumulative variance contracts to zero asymptotically.

3.9.3 Practical stochastic optimization

To deploy SGD effectively in contemporary deep neural networks, five foundational practical principles govern implementation and training stability:

1. **Data Shuffling to Eliminate Ordering Bias**: Mini-batches must be drawn by randomly shuffling the dataset at the start of every epoch. If data exhibits sequential correlation (e.g., all class 1 digits grouped together), sequential mini-batches will violently drag parameters toward class 1, causing catastrophic forgetting of preceding classes and inducing cyclic optimization orbits.

2. **Hardware Parallelization and SIMD Efficiency**: While pure SGD ($m = 1$) is optimal in theory, single-sample operations fail to exploit GPU parallelism. Mini-batch sizes $m \in \{32, 64, 128, 256, 512\}$ are chosen to match GPU Streaming Multiprocessor warp sizes and saturate Tensor Core matrix multiplication units (GEMM).

3. **Two-Phase Optimization Dynamics**: SGD exhibits an asymmetric temporal profile: **remarkably rapid initial progress** in early epochs as parameters drop down steep macro-gradients into low-loss valleys, followed by **slow asymptotic fine-tuning** where parameter wanderings are dominated by gradient variance.

4. **Implicit Regularization of Small Mini-Batches**: Small mini-batches inject high stochastic gradient noise ($\propto \sigma^2/m$). This noise acts as an implicit regularizer, preventing parameters from settling into sharp overfitted minima and nudging trajectories into wide flat basins. However, because variance is high, practitioners must reduce the learning rate α (or apply linear learning rate scaling $\alpha \propto m$) to preserve numerical stability.

5. **The Practitioner's Fundamental Maxim**: This crystallizes the ultimate operational rule of deep learning optimization:

   "SGD can find a low value of the loss quickly enough to be useful, even if it's not a minimum."

   In real-world engineering, achieving an exact stationary point ($\nabla \ell = \mathbf{0}$) is neither required nor desirable. Finding a robust, flat basin with low empirical loss within practical time constraints delivers state-of-the-art generalization.

Chapter 4

Deep feedforward networks and backpropagation

The quest to construct artificial intelligence originated with a profound biological ambition: to emulate the computational architecture of the human brain. While classical machine learning algorithms—such as Ordinary Least Squares, Ridge regression, and linear support vector machines—operate by mapping inputs directly to predictions via fixed, handcrafted feature representations, natural sensory perception defies static linear modeling. Human cognition effortlessly navigates complex perceptual manifolds: recognizing a handwritten digit, segmenting a tumor in an acoustic scan, or parsing visual scenes across dramatic shifts in illumination, scale, and orientation.

To transcend the rigid confines of linear predictors, connectionist theory introduced the **artificial neural network**—a massively parallel computational architecture composed of elementary interconnected processing nodes. This intellectual journey began with Frank Rosenblatt's 1958 Perceptron, an elegant bio-inspired unit that computed affine combinations thresholded by a step function. Yet, as Marvin Minsky and Seymour Papert demonstrated in their devastating 1969 critique, single-layer perceptrons possess a fatal geometric blind spot: bounded strictly by linear separability, they fail on elementary non-linear predicates as simple as the Exclusive-OR (XOR) function, precipitating the First AI Winter.

The modern deep learning revolution was born from overcoming this linear barrier through two foundational mathematical innovations. First, by cascading multiple layers of neurons equipped with continuous, non-linear activation functions into **Multilayer Perceptrons (MLPs)**, networks acquire the capacity for **representation learning**. Rather than relying on human engineers to handcraft feature spaces, deep architectures automatically fold, stretch, and warp input coordinate space, transforming complex non-linear manifolds into representations that are cleanly linearly separable. The celebrated Hornik-Cybenko Universal Approximation Theorem formalizes this expressive power, guaranteeing that feedforward networks can approximate any continuous function on a compact domain to arbitrary precision.

Yet, representational power is useless without an efficient training engine. Because deep compositions of non-linear activations entangle parameters in transcendental algebraic systems, closed-form matrix inversions are mathematically impossible, and numerical finite differences require an intractable $\mathcal{O}(W^2)$ computational complexity. This computational gridlock was broken by formulating deep networks as Directed Acyclic Graphs (DAGs) and applying reverse-mode automatic differentiation: the **Backpropagation algorithm**. By propagating adjoint error sensitivities backward through the network in a single sweep, backpropagation evaluates the exact gradient across millions of parameters in linear $\mathcal{O}(W)$ time, establishing the foundational mathematical engine of contemporary artificial intelligence.

4.1 The perceptron

The quest to construct artificial intelligence originated with a profound biological ambition: to understand and replicate the computational mechanisms of the brain. In the 1940s and 1950s, the emergence of early digital computing was defined by von Neumann architectures designed for serial instruction execution and exact symbolic arithmetic. While these mechanical and electronic calculators could perform millions of numerical operations per second, they proved utterly incapable of solving elementary perceptual tasks that animal brains resolve effortlessly and instantaneously—such as recognizing a visual pattern, distinguishing spoken phonemes, or adapting to sensory variations in an uncertain environment.

This striking divergence highlighted a fundamental insight: biological intelligence does not operate via centralized instruction registers, brittle algorithmic decision trees, or hand-crafted symbolic lookup tables. Instead, cognitive function emerges from the collective dynamics of billions of interconnected, elementary biological cells—neurons—transmitting electrical impulses across plastic synaptic junctions. Despite individual biological neurons operating at millisecond timescales (orders of magnitude slower than electronic switching circuits), their massive parallel connectivity enables robust associative memory, real-time sensory processing, and the ability to learn directly from experience.

Inspired by these neurophysiological observations, early cyberneticians and cognitive scientists—beginning with Warren McCulloch and Walter Pitts in 1943, Donald Hebb in 1949, and culminating in Frank Rosenblatt in 1958—sought to abstract the physical machinery of the biological neuron into an elementary mathematical learning unit. By modeling synaptic plasticity as adjustable numerical weights and dendritic summation as an affine score thresholded by an excitation activation, they aimed to create an artificial computing system capable of discovering patterns and categorizing sensory observations autonomously without explicit human programming.

4.1.1 Architecture and convergence

Biological morphology and Rosenblatt's artificial threshold unit. In biological nervous systems, an individual neuron consists of four primary morphological structures:

- **Dendrites**: Branching nerve fibers that receive incoming chemical neurotransmitter signals from adjacent upstream neurons across synaptic clefts.
- **Soma (Cell Body)**: The metabolic hub containing the nucleus, which integrates and accumulates incoming post-synaptic electrical potentials over space and time.
- **Axon**: A long, slender nerve transmission cable (often insulated by fatty myelin sheaths) that propagates an all-or-nothing action potential (voltage spike) down its length once the soma's membrane potential surpasses an excitation threshold.
- **Synapses**: Chemical junctions connecting axon terminal boutons to recipient dendrites, where biological synaptic plasticity modulates the efficacy of electrical transmission.

In 1958, Frank Rosenblatt [78] formalized this biological mechanism into the **Perceptron**. Given an input vector of sensory features $\mathbf{x} = [x_1, \ldots, x_d]^\top \in \mathbb{R}^d$, the artificial neuron associates each input channel with a tunable real-valued weight $w_j \in \mathbb{R}$, incorporates a scalar bias threshold $b \in \mathbb{R}$, and computes a linear pre-activation score:

$$z \triangleq \mathbf{w}^\top \mathbf{x} + b = \sum_{j=1}^{d} w_j x_j + b. \tag{4.1}$$

The continuous scalar score z is then passed through a discontinuous Heaviside step activation function to produce a binary classification decision:

$$\hat{y} = f(z) = \operatorname{sgn}(z) = \begin{cases} +1 & \text{if } \mathbf{w}^\top \mathbf{x} + b \geq 0, \\ -1 & \text{if } \mathbf{w}^\top \mathbf{x} + b < 0. \end{cases} \tag{4.2}$$

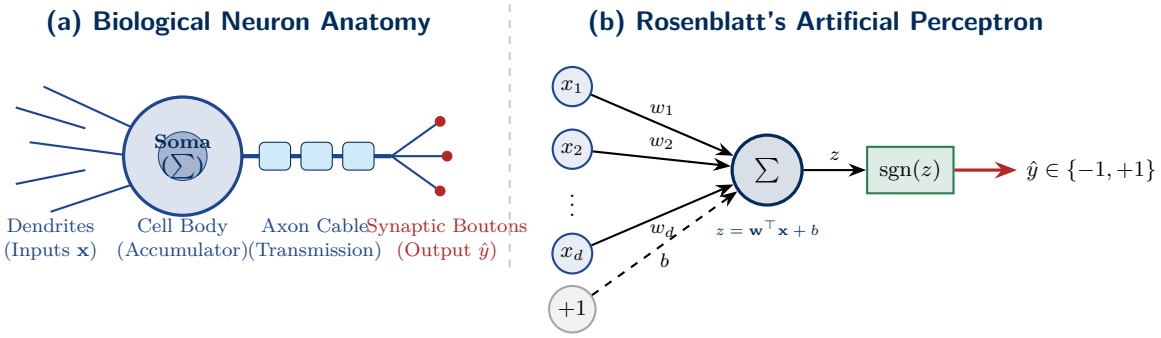


Figure 4.1: Morphological and mathematical correspondence between biological and artificial neural processing. (a) A biological neuron integrates post-synaptic potentials across dendrites into the metabolic soma, firing an action potential down the myelinated axon toward synaptic terminals. (b) Rosenblatt's artificial perceptron models this mechanism: numerical inputs x_j are modulated by synaptic parameters w_j, accumulated by affine summation $z = \mathbf{w}^\top \mathbf{x} + b$, and thresholded by a Heaviside sign function to produce binary classification $\hat{y} \in \{-1, +1\}$.

Decision hyperplane geometry. Geometrically, the equation $\mathbf{w}^\top \mathbf{x} + b = 0$ defines a $(d-1)$-dimensional **decision hyperplane** that partitions the d-dimensional input space $\mathcal{X} \subseteq \mathbb{R}^d$ into two open half-spaces: a positive half-space where $\mathbf{w}^\top \mathbf{x} + b \geq 0$ (yielding $\hat{y} = +1$) and a negative half-space where $\mathbf{w}^\top \mathbf{x} + b < 0$ (yielding $\hat{y} = -1$). The parameter vector $\mathbf{w} \in \mathbb{R}^d$ is the orthogonal normal vector to the hyperplane pointing directly toward the positive decision half-space. The scalar bias $b \in \mathbb{R}$ offsets the hyperplane from the origin, establishing a signed perpendicular Euclidean distance of $\frac{|b|}{\|\mathbf{w}\|_2}$.

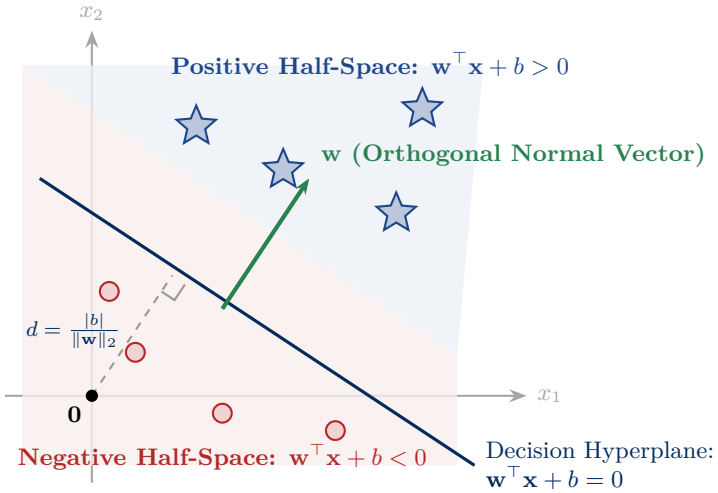


Figure 4.2: The geometry of the linear decision boundary defined by the perceptron. The parameter vector $\mathbf{w} \in \mathbb{R}^d$ is the orthogonal normal vector to the hyperplane pointing directly toward the positive decision half-space. The scalar bias $b \in \mathbb{R}$ offsets the hyperplane from the origin, establishing a signed perpendicular Euclidean distance of $\frac{|b|}{\|\mathbf{w}\|_2}$.

The perceptron learning algorithm. Given a supervised training dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ with binary labels $y_i \in \{-1, +1\}$, the perceptron learning algorithm operates in an online, mistake-driven fashion. Parameters are initialized arbitrarily (typically $\mathbf{w}^{(0)} = \mathbf{0}, b^{(0)} = 0$).

At step t, the algorithm inspects an incoming training instance $(\mathbf{x}_i, y_i)$. If the current hypothesis correctly classifies the instance ($y_i(\mathbf{w}^{(t)\top} \mathbf{x}_i + b^{(t)}) > 0$), no modification is made. If

the instance is misclassified ($y_i(\mathbf{w}^{(t)\top}\mathbf{x}_i + b^{(t)}) \leq 0$), parameters are updated via the additive correction:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \eta y_i \mathbf{x}_i, \tag{4.3}$$

$$b^{(t+1)} = b^{(t)} + \eta y_i R^2, \tag{4.4}$$

where $\eta > 0$ is the learning rate (often set to $\eta = 1$ without loss of generality, as scaling both sides leaves the sign of predictions unchanged), and $R = \max_i \|\mathbf{x}_i\|_2$ is the radius of the smallest sphere enclosing the data. In parameter space, this update executes vector addition; in input space, it rotates the decision hyperplane directly toward the misclassified sample, pulling it into the correct decision region.

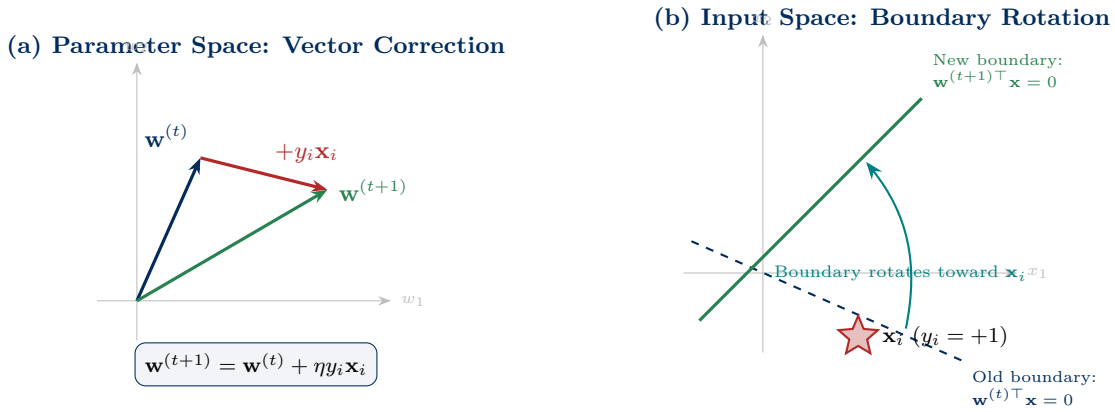


Figure 4.3: Geometric dynamics of the Perceptron Learning Algorithm update. (a) In parameter space, whenever a training sample $(\mathbf{x}_i, y_i)$ is misclassified, the algorithm performs vector addition $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + y_i\mathbf{x}_i$. (b) In input space, adding $y_i\mathbf{x}_i$ rotates the orthogonal decision hyperplane directly toward the misclassified sample, pulling it into the correct decision region.

Novikoff's convergence theorem. The historical significance of Rosenblatt's perceptron stems from the fact that it was the first machine learning model accompanied by a rigorous mathematical convergence proof, formalized by Albert Novikoff in 1962 [70].

Theorem 4.1 (Novikoff's Perceptron Convergence Theorem)**.** *Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ be a training dataset where $\|\mathbf{x}_i\|_2 \leq R$ for all i. Assume the dataset is strictly **linearly separable**: that is, there exists an optimal unit-norm parameter vector $\mathbf{w}^*$ ($\|\mathbf{w}^*\|_2 = 1$) and an optimal margin $\gamma > 0$ such that:*

$$y_i(\mathbf{w}^{*\top}\mathbf{x}_i) \geq \gamma, \quad \forall i \in \{1, \ldots, n\}. \tag{4.5}$$

Then, starting from $\mathbf{w}^{(0)} = \mathbf{0}$, the perceptron learning algorithm makes at most k mistake updates before converging to a perfectly separating hyperplane, where:

$$k \leq \left(\frac{R}{\gamma}\right)^2. \tag{4.6}$$

Proof. Let $\mathbf{w}^{(k)}$ denote the weight vector after k mistaken updates. Assume the k-th update was triggered by misclassified sample $(\mathbf{x}_i, y_i)$, meaning $y_i(\mathbf{w}^{(k-1)\top}\mathbf{x}_i) \leq 0$. The update rule is $\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} + y_i\mathbf{x}_i$.

We bound the inner product $\mathbf{w}^{(k)\top}\mathbf{w}^*$ from below. Expanding recursively:

$$\mathbf{w}^{(k)\top}\mathbf{w}^* = (\mathbf{w}^{(k-1)} + y_i\mathbf{x}_i)^\top\mathbf{w}^* = \mathbf{w}^{(k-1)\top}\mathbf{w}^* + y_i\mathbf{x}_i^\top\mathbf{w}^* \geq \mathbf{w}^{(k-1)\top}\mathbf{w}^* + \gamma. \tag{4.7}$$

Inductively from $\mathbf{w}^{(0)} = \mathbf{0}$, after k mistakes:

$$\mathbf{w}^{(k)\top}\mathbf{w}^* \geq k\gamma. \tag{4.8}$$

Next, we bound the squared Euclidean norm $\|\mathbf{w}^{(k)}\|_2^2$ from above:

$$\begin{aligned}\|\mathbf{w}^{(k)}\|_2^2 &= \|\mathbf{w}^{(k-1)} + y_i\mathbf{x}_i\|_2^2 = \|\mathbf{w}^{(k-1)}\|_2^2 + 2y_i\mathbf{w}^{(k-1)\top}\mathbf{x}_i + \|\mathbf{x}_i\|_2^2 \\ &\leq \|\mathbf{w}^{(k-1)}\|_2^2 + 0 + R^2 = \|\mathbf{w}^{(k-1)}\|_2^2 + R^2,\end{aligned} \tag{4.9}$$

where we used the mistake condition $y_i\mathbf{w}^{(k-1)\top}\mathbf{x}_i \leq 0$ and the radius bound $\|\mathbf{x}_i\|_2 \leq R$. Inductively from $\mathbf{w}^{(0)} = \mathbf{0}$:

$$\|\mathbf{w}^{(k)}\|_2^2 \leq kR^2 \implies \|\mathbf{w}^{(k)}\|_2 \leq \sqrt{k}R. \tag{4.10}$$

By the Cauchy-Schwarz inequality, for any unit vector $\mathbf{w}^*$:

$$\mathbf{w}^{(k)\top}\mathbf{w}^* \leq \|\mathbf{w}^{(k)}\|_2\|\mathbf{w}^*\|_2 = \|\mathbf{w}^{(k)}\|_2. \tag{4.11}$$

Combining Equation (4.8) and Equation (4.10):

$$k\gamma \leq \mathbf{w}^{(k)\top}\mathbf{w}^* \leq \|\mathbf{w}^{(k)}\|_2 \leq \sqrt{k}R \implies k\gamma \leq \sqrt{k}R \implies \sqrt{k} \leq \frac{R}{\gamma} \implies k \leq \left(\frac{R}{\gamma}\right)^2. \tag{4.12}$$

This completes the proof. □

4.1.2 The fatal flaws

Despite the mathematical elegance of Novikoff's theorem, the single-layer perceptron suffers from fundamental theoretical limitations that ultimately triggered the historical First AI Winter.

The non-separability catastrophe: Boundary thrashing and infinite cycling. Novikoff's Perceptron Convergence Theorem (Theorem 4.1) holds strictly under the premise that the training dataset $\mathcal{D}$ is linearly separable, meaning that a strictly positive margin exists: $\gamma > 0$.

However, in realistic real-world datasets, sensory noise, label ambiguity, and overlapping class probability distributions inevitably render data **linearly non-separable** ($\gamma \leq 0$). Under non-separability, the convergence upper bound $k \leq (R/\gamma)^2$ becomes undefined ($\gamma \to 0$ implies $k \to \infty$).

Crucially, the presence of an inseparable outlier or overlapping class instance (such as a negative point located within a cluster of positive samples) triggers a catastrophic failure mode:

- **Boundary thrashing and non-terminating ping-pong oscillation**: Because the outlier violates the margin condition, the algorithm executes the additive update $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + y_i\mathbf{x}_i$, forcibly rotating the decision boundary from $H^{(t)}$ to $H^{(t+1)}$ to satisfy that outlier. However, this corrective rotation inevitably sweeps across previously well-classified instances, instantly converting them into misclassified errors. In subsequent iterations, these newly displaced points trigger counter-updates that swing the boundary back toward $H^{(t)}$. The algorithm thus becomes trapped in an infinite ping-pong loop, thrashing perpetually between conflicting hyperplanes without ever terminating ($k \to \infty$).

To mitigate this catastrophic behavior in practical applications, Stephen Gallant introduced the heuristic **Pocket Algorithm** [27], which runs the perceptron algorithm for a predetermined number of iterations while storing ("in its pocket") the parameter vector that achieved the lowest classification error seen so far. Nevertheless, the fundamental theoretical guarantee of exact convergence is irrevocably lost on non-separable data.

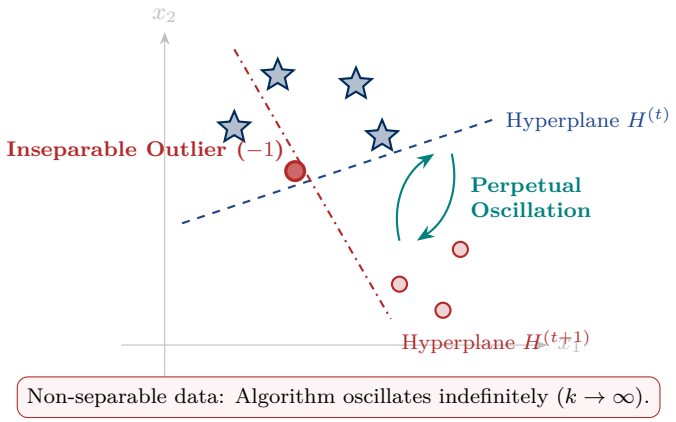


Figure 4.4: Flaw 1 of the Perceptron: Non-separability and perpetual boundary thrashing. When classes overlap or contain an inseparable outlier, no separating hyperplane exists ($\gamma \leq 0$). The perceptron learning algorithm continually rotates to satisfy the misclassified outlier, which promptly causes previously classified points to fail, trapping the algorithm in an infinite ping-pong oscillation between conflicting hyperplanes $H^{(t)}$ and $H^{(t+1)}$.

Inability to compute Exclusive-OR (XOR). The non-separability catastrophe demonstrates that the perceptron fails in the presence of noise and outliers. Yet a far deeper question remains: what if the data contains zero noise, zero outliers, and is completely deterministic? Can a single-layer perceptron learn even the simplest logical functions?

In their landmark 1969 monograph *Perceptrons*, Marvin Minsky and Seymour Papert [64] established that a single-layer perceptron is geometrically incapable of computing basic non-linear Boolean predicates, most famously the **Exclusive-OR (XOR)** function.

Consider the four canonical vertices of the unit square $[0,1]^2$ defining the XOR logic gate:

$$\begin{aligned} \mathbf{x}_1 = [0,0]^\top &\implies y = -1, \qquad \mathbf{x}_3 = [1,0]^\top \implies y = +1, \\ \mathbf{x}_2 = [0,1]^\top &\implies y = +1, \qquad \mathbf{x}_4 = [1,1]^\top \implies y = -1. \end{aligned} \tag{4.13}$$

To classify these four points correctly with zero training error, a perceptron parameterized by weights (w_1, w_2) and bias b must satisfy the four simultaneous strict linear inequalities:

$$\text{For } \mathbf{x}_1 = [0,0]^\top: \quad w_1 \cdot 0 + w_2 \cdot 0 + b < 0 \implies b < 0, \tag{4.14}$$

$$\text{For } \mathbf{x}_2 = [0,1]^\top: \quad w_1 \cdot 0 + w_2 \cdot 1 + b \geq 0 \implies w_2 + b \geq 0, \tag{4.15}$$

$$\text{For } \mathbf{x}_3 = [1,0]^\top: \quad w_1 \cdot 1 + w_2 \cdot 0 + b \geq 0 \implies w_1 + b \geq 0, \tag{4.16}$$

$$\text{For } \mathbf{x}_4 = [1,1]^\top: \quad w_1 \cdot 1 + w_2 \cdot 1 + b < 0 \implies w_1 + w_2 + b < 0. \tag{4.17}$$

We now derive the formal algebraic impossibility proof:

Proof. Summing the two positive-class inequalities (4.15) and (4.16):

$$(w_2 + b) + (w_1 + b) \geq 0 \implies w_1 + w_2 + 2b \geq 0 \implies w_1 + w_2 + b \geq -b. \tag{4.18}$$

Because Equation (4.14) requires $b < 0$, multiplying by -1 yields $-b > 0$. Therefore:

$$w_1 + w_2 + b \geq -b > 0 \implies w_1 + w_2 + b > 0. \tag{4.19}$$

However, Equation (4.17) strictly requires:

$$w_1 + w_2 + b < 0. \tag{4.20}$$

Equations (4.19) and (4.20) are directly contradictory: a real quantity cannot be strictly positive and strictly negative simultaneously ($0 < w_1 + w_2 + b < 0$, a manifest contradiction in $\mathbb{R}$). □

Hence, **no parameter vector $\mathbf{w} \in \mathbb{R}^2$ and bias $b \in \mathbb{R}$ can ever classify the XOR problem**. Geometrically, as illustrated in Figure 4.5, the positive points $(0, 1)$ and $(1, 0)$ and negative points $(0, 0)$ and $(1, 1)$ occupy opposite diagonals of the unit square. Any straight line drawn through the plane can at best isolate three of the four points, always misclassifying at least one vertex.

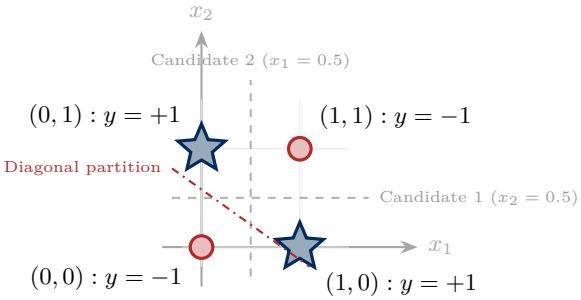


Figure 4.5: Flaw 2 of the Perceptron: The XOR non-linear separability impasse. The Exclusive-OR (XOR) function assigns positive labels ($y = +1$) to coordinates $(0, 1)$ and $(1, 0)$, and negative labels ($y = -1$) to $(0, 0)$ and $(1, 1)$. Because positive and negative pairs occupy alternate diagonals of the unit square $[0, 1]^2$, no single linear decision hyperplane can separate the two classes.

The publication of Minsky and Papert's critique had devastating historical ramifications. Because early AI researchers lacked an algorithm to train multi-layer neural architectures with hidden units, the field broadly concluded that artificial neural networks were fundamentally flawed and unscalable, causing government and academic funding to evaporate and initiating the decade-long First AI Winter.

Zero derivative and optimization incompatibility. The third fatal flaw of Rosenblatt's perceptron lies in its activation function. The discontinuous step activation:

$$f(z) = \operatorname{sgn}(z) = \begin{cases} +1 & \text{if } z \geq 0, \\ -1 & \text{if } z < 0, \end{cases} \tag{4.21}$$

is piecewise constant. Its derivative is zero everywhere that it is defined:

$$f'(z) = \frac{d}{dz} \operatorname{sgn}(z) = 0, \quad \forall z \neq 0, \tag{4.22}$$

and is completely undefined (or infinite) at the discontinuity $z = 0$.

Consequently, the parameter gradient of the pre-activation with respect to weights:

$$\nabla_{\mathbf{w}} f(\mathbf{w}^\top \mathbf{x} + b) = f'(z) \cdot \mathbf{x} = \mathbf{0}, \quad \forall z \neq 0. \tag{4.23}$$

Because the gradient is identically zero almost everywhere, **gradient-based optimization algorithms cannot receive any directional learning signal**. Perturbing the weights by an infinitesimal step $\Delta\mathbf{w}$ either produces zero change in output or an abrupt, unpredictable jump across the step discontinuity.

The perceptron's inability to provide smooth gradient signals, coupled with its linear separability barrier on problems like XOR, established the definitive conclusion of the classical era: single-layer linear architectures cannot serve as universal intelligent learners. To break through this barrier, machine learning had to transition to layered compositions of continuous, differentiable units—leading directly to the Multi-Layer Perceptron in Section 4.2.

4.2 Multilayer perceptrons and approximation

To liberate neural computing from the linear separability impasse, the architecture of the artificial neuron must be fundamentally reconceived. The failure on XOR and the non-differentiability of step functions revealed that a single hyperplane cutting through raw input space cannot capture complex decision geometries. A learner must be capable of transforming the geometry of the input space itself.

However, resolving this challenge is not as straightforward as simply stacking multiple linear perceptrons into a cascading sequence. As linear algebra dictates, any sequence of cascaded affine transformations collapses into a single affine transformation, offering zero representational advantage over a single-layer model. To unlock true hierarchical expressiveness, two simultaneous transformations are mathematically mandatory: (1) interleaving linear matrix projections with continuous, non-linear activation functions that break affine subspace closure, and (2) organizing neurons into intermediate **hidden layers** that execute automated **representation learning**.

4.2.1 Beyond linearity and representation

Why we must go beyond linearity. In classical linear models, decision boundaries are strictly constrained to $(D-1)$-dimensional affine hyperplanes of the form $\mathbf{w}^\top \mathbf{x} + b = 0$. While computationally tractable and admitting closed-form solutions in regression tasks, linear decision boundaries are fundamentally incapable of modeling complex real-world data distributions. Perceptual data—such as natural images, acoustic speech signals, and semantic text—invariably lie on highly curved, non-linear sub-manifolds embedded within high-dimensional ambient space.

When class distributions exhibit topological complexity, non-convex geometry, or inter-class interleaving—as exemplified by the classic Exclusive-OR (XOR) logic function—a linear boundary cannot isolate the classes without incurring catastrophic classification error. To build expressive learning machines, we must endow models with the mathematical capacity to construct non-linear decision boundaries.

The compositional collapse of purely linear networks. A naive intuition might suggest that multi-layer expressiveness could be attained simply by cascading multiple linear layers, feeding the output of one linear perceptron as input to the next. Consider an L-layer network where each intermediate layer computes an affine transformation without activation functions:

$$\begin{aligned}
\mathbf{h}^{(1)} &= \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1, \\
\mathbf{h}^{(2)} &= \mathbf{W}_2 \mathbf{h}^{(1)} + \mathbf{b}_2 = \mathbf{W}_2(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2 \mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2 \mathbf{b}_1 + \mathbf{b}_2), \\
&\vdots \\
\hat{\mathbf{y}} &= \mathbf{W}_L \mathbf{h}^{(L-1)} + \mathbf{b}_L.
\end{aligned} \tag{4.24}$$

Proof. We prove by mathematical induction on the layer depth L that any cascading sequence of affine maps collapses to a single affine transformation.

Base Case ($L = 1$): For a single layer, $\hat{\mathbf{y}} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1$, which is trivially affine with effective weight matrix $\mathbf{W}_{\text{eff}}^{(1)} = \mathbf{W}_1$ and bias $\mathbf{b}_{\text{eff}}^{(1)} = \mathbf{b}_1$.

Inductive Hypothesis: Assume that for any layer $k \geq 1$, the representation $\mathbf{h}^{(k)}$ is an affine function of the input $\mathbf{x}$:

$$\mathbf{h}^{(k)} = \mathbf{W}_{\text{eff}}^{(k)} \mathbf{x} + \mathbf{b}_{\text{eff}}^{(k)}, \quad \text{with } \mathbf{W}_{\text{eff}}^{(k)} \in \mathbb{R}^{d_k \times d_0},\ \mathbf{b}_{\text{eff}}^{(k)} \in \mathbb{R}^{d_k}. \tag{4.25}$$

Inductive Step: Applying the affine transformation of layer $k+1$ yields:

$$\begin{aligned}\mathbf{h}^{(k+1)} &= \mathbf{W}_{k+1}\mathbf{h}^{(k)} + \mathbf{b}_{k+1} \\ &= \mathbf{W}_{k+1}\left(\mathbf{W}_{\text{eff}}^{(k)}\mathbf{x} + \mathbf{b}_{\text{eff}}^{(k)}\right) + \mathbf{b}_{k+1} \\ &= \left(\mathbf{W}_{k+1}\mathbf{W}_{\text{eff}}^{(k)}\right)\mathbf{x} + \left(\mathbf{W}_{k+1}\mathbf{b}_{\text{eff}}^{(k)} + \mathbf{b}_{k+1}\right). \end{aligned} \tag{4.26}$$

Defining the composite parameter matrices:

$$\mathbf{W}_{\text{eff}}^{(k+1)} \triangleq \mathbf{W}_{k+1}\mathbf{W}_{\text{eff}}^{(k)} \in \mathbb{R}^{d_{k+1}\times d_0}, \quad \mathbf{b}_{\text{eff}}^{(k+1)} \triangleq \mathbf{W}_{k+1}\mathbf{b}_{\text{eff}}^{(k)} + \mathbf{b}_{k+1} \in \mathbb{R}^{d_{k+1}}, \tag{4.27}$$

the representation $\mathbf{h}^{(k+1)}$ remains strictly an affine function of $\mathbf{x}$. By induction, an L-layer purely linear network computes:

$$\hat{\mathbf{y}} = \mathbf{W}_{\text{eff}}\mathbf{x} + \mathbf{b}_{\text{eff}}, \quad \text{with } \mathbf{W}_{\text{eff}} = \prod_{l=L}^{1}\mathbf{W}_l = \mathbf{W}_L\mathbf{W}_{L-1}\cdots\mathbf{W}_1, \tag{4.28}$$

and $\mathbf{b}_{\text{eff}} = \mathbf{b}_L + \sum_{l=1}^{L-1}\left(\prod_{j=L}^{l+1}\mathbf{W}_j\right)\mathbf{b}_l$. □

Because matrix multiplication is associative and affine transformations form a mathematically closed set under functional composition, **stacking arbitrary depths of purely linear layers yields zero additional representational power over a single-layer linear model**. No matter how many millions of linear units are assembled, the composition remains incapable of bending a hyperplane.

The imperative for non-linear activation functions. To escape compositional collapse, we must break affine subspace closure. The mathematical remedy is to interleave each linear matrix projection with an element-wise, non-linear activation function $\sigma : \mathbb{R} \to \mathbb{R}$:

$$\mathbf{h}^{(l)} = \sigma\left(\mathbf{W}_l\mathbf{h}^{(l-1)} + \mathbf{b}_l\right), \quad \forall l \in \{1, \ldots, L\}. \tag{4.29}$$

Because $\sigma(\cdot)$ is non-linear, the operator σ cannot commute with matrix multiplication: $\mathbf{W}_2\sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) \neq \tilde{\mathbf{W}}\mathbf{x} + \tilde{\mathbf{b}}$. The non-linearity prevents the successive weight matrices from collapsing into a single matrix product. Geometrically, non-linear activations fold, stretch, and twist the coordinate axes, allowing the network to carve curved, multi-region decision boundaries that solve topologically complex tasks.

Resolving the XOR dilemma via representation learning. To see concretely how non-linear hidden representations shatter the linear separability barrier, consider Minsky and Papert's fatal XOR counterexample. The XOR function assigns positive labels ($y = 1$) to $[1, 0]^\top$ and $[0, 1]^\top$, and negative labels ($y = 0$) to $[0, 0]^\top$ and $[1, 1]^\top$.

We construct a minimal two-layer feedforward network with two input units, two hidden units equipped with the Rectified Linear Unit ($\text{ReLU}(z) \triangleq \max(0, z)$), and one linear output unit:

$$\mathbf{h} = \text{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \quad \text{with } \mathbf{W}_1 = \begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix}, \mathbf{b}_1 = \begin{bmatrix}0\\ -1\end{bmatrix}, \tag{4.30}$$

$$\hat{y} = \mathbf{w}_2^\top\mathbf{h} + b_2, \quad \text{with } \mathbf{w}_2 = \begin{bmatrix}1\\ -2\end{bmatrix}, b_2 = 0. \tag{4.31}$$

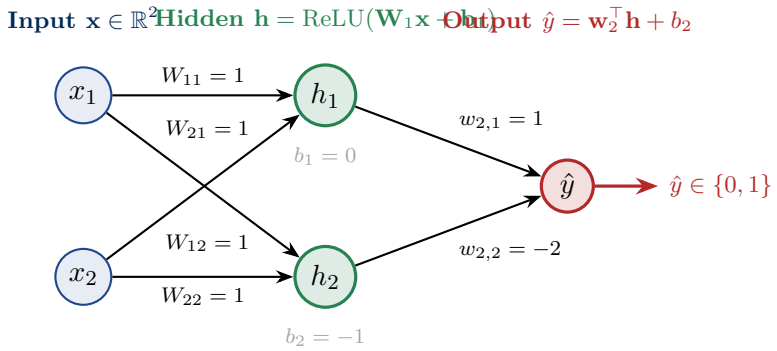


Figure 4.6: The Multi-Layer Perceptron (MLP) architecture resolving XOR. Input coordinates (x_1, x_2) are projected into a 2D hidden layer $\mathbf{h} = \mathrm{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$ that extracts non-linear representations, followed by a linear output unit that classifies the transformed points cleanly.

Evaluating the forward pass step-by-step across all four boolean input combinations:

1. **Input $\mathbf{x} = [0, 0]^\top$ (Class 0)**:

$$\mathbf{W}_1\mathbf{x} + \mathbf{b}_1 = \begin{bmatrix} 0 \\ -1 \end{bmatrix} \implies \mathbf{h} = \mathrm{ReLU}\left(\begin{bmatrix} 0 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies \hat{y} = [1, -2]\begin{bmatrix} 0 \\ 0 \end{bmatrix} = 0. \tag{4.32}$$

2. **Input $\mathbf{x} = [1, 0]^\top$ (Class 1)**:

$$\mathbf{W}_1\mathbf{x} + \mathbf{b}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \implies \mathbf{h} = \mathrm{ReLU}\left(\begin{bmatrix} 1 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \implies \hat{y} = [1, -2]\begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1. \tag{4.33}$$

3. **Input $\mathbf{x} = [0, 1]^\top$ (Class 1)**:

$$\mathbf{W}_1\mathbf{x} + \mathbf{b}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \implies \mathbf{h} = \mathrm{ReLU}\left(\begin{bmatrix} 1 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \implies \hat{y} = [1, -2]\begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1. \tag{4.34}$$

4. **Input $\mathbf{x} = [1, 1]^\top$ (Class 0)**:

$$\mathbf{W}_1\mathbf{x} + \mathbf{b}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \implies \mathbf{h} = \mathrm{ReLU}\left(\begin{bmatrix} 2 \\ 1 \end{bmatrix}\right) = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \implies \hat{y} = [1, -2]\begin{bmatrix} 2 \\ 1 \end{bmatrix} = 2 - 2 = 0. \tag{4.35}$$

Every input point is classified with zero error ($\hat{y} = y$).

The geometric explanation behind this triumph is illustrated in Figure 4.7. In the original input space $\mathcal{X}$, the positive instances $[1, 0]^\top$ and $[0, 1]^\top$ are positioned along the off-diagonal, diagonally interleaved between the negative instances $[0, 0]^\top$ and $[1, 1]^\top$; no straight line can separate them. The non-linear hidden transformation $\mathbf{h} = \mathrm{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$ acts as a coordinate folding operation: it projects the two separate positive points $[1, 0]^\top$ and $[0, 1]^\top$ onto the exact same coordinate $[1, 0]^\top$ in the latent hidden space $\mathcal{H}$, while mapping $[0, 0]^\top \to [0, 0]^\top$ and $[1, 1]^\top \to [2, 1]^\top$. In this newly formed hidden representation space $\mathcal{H}$, the class distributions are strictly linearly separable by the hyperplane $h_1 - 2h_2 = 0.5$.

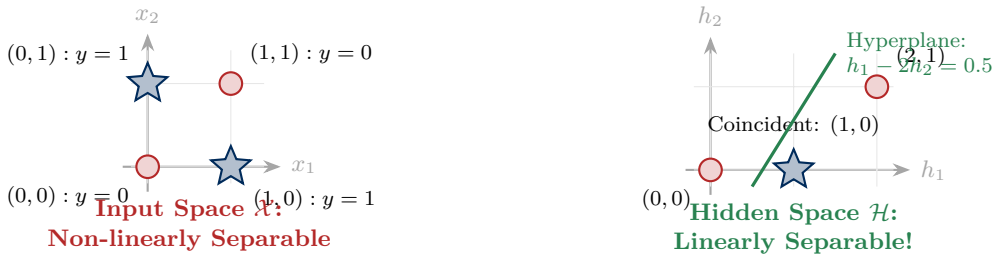


Figure 4.7: The geometric mechanism of representation learning resolving the XOR problem. Left: In the raw input space $\mathcal{X}$, no single line can separate the positive stars from the negative circles. Right: The non-linear hidden layer transformation $\mathbf{h} = \text{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$ collapses both positive points $[0,1]^\top$ and $[1,0]^\top$ onto the single coordinate $[1,0]^\top$, while mapping $[1,1]^\top$ to $[2,1]^\top$. In this hidden space $\mathcal{H}$, the classes are cleanly separated by a single linear hyperplane $h_1 - 2h_2 = 0.5$.

This reveals the foundational paradigm of modern deep learning: **representation learning**. Rather than requiring human engineers to manually design domain-specific features to make inputs linearly separable, the hidden layer of a neural network autonomously transforms raw coordinate space into a latent geometry where linear classification succeeds.

4.2.2 Universal approximation and depth

The Universal Approximation Theorem. How expressive is a feedforward neural network equipped with non-linear activation functions? The foundational theoretical answer is established by the celebrated **Universal Approximation Theorem**, proven by George Cybenko (1989) [15] for sigmoidal activations and generalized by Kurt Hornik (1991) [40] to arbitrary continuous, non-constant activations (and subsequently extended to non-polynomial continuous activations including ReLU).

Theorem 4.2 (Universal Approximation Theorem [15, 40])**.** *Let $\sigma : \mathbb{R} \to \mathbb{R}$ be any continuous, non-constant, bounded (or continuous non-polynomial) activation function. Let $K \subset \mathbb{R}^D$ be a compact (closed and bounded) subset of Euclidean space. Then, for any continuous target function $f \in C(K)$ and any arbitrary error tolerance $\epsilon > 0$, there exists an integer $M \in \mathbb{N}$, input weight vectors $\mathbf{w}_j \in \mathbb{R}^D$, biases $b_j \in \mathbb{R}$, and output weights $\alpha_j \in \mathbb{R}$ such that the single-hidden-layer network:*

$$F(\mathbf{x}) = \sum_{j=1}^{M} \alpha_j \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j), \tag{4.36}$$

satisfies uniform approximation over all $\mathbf{x} \in K$:

$$\sup_{\mathbf{x} \in K} |F(\mathbf{x}) - f(\mathbf{x})| < \epsilon. \tag{4.37}$$

Constructive geometric proof via localized bump functions. To understand why Theorem 4.2 holds, consider the constructive geometric proof intuition formulated by Cybenko and developed in modern treatments by Bishop [6] and Prince [73].

In one dimension ($D = 1$), consider two sigmoid units with identical steep slope $w \gg 0$ but offset biases $b_1 = -wa$ and $b_2 = -wb$ (with $a < b$):

$$\text{bump}(x) \triangleq \sigma(w(x - a)) - \sigma(w(x - b)). \tag{4.38}$$

As $w \to \infty$, the sigmoid $\sigma(w(x - a))$ converges to a Heaviside step jumping from 0 to 1 at $x = a$, while $\sigma(w(x - b))$ jumps from 0 to 1 at $x = b$. Subtracting them produces a localized

rectangular pulse (bump) that evaluates to approximately 1 on the interval $[a, b]$ and decays to 0 everywhere outside.

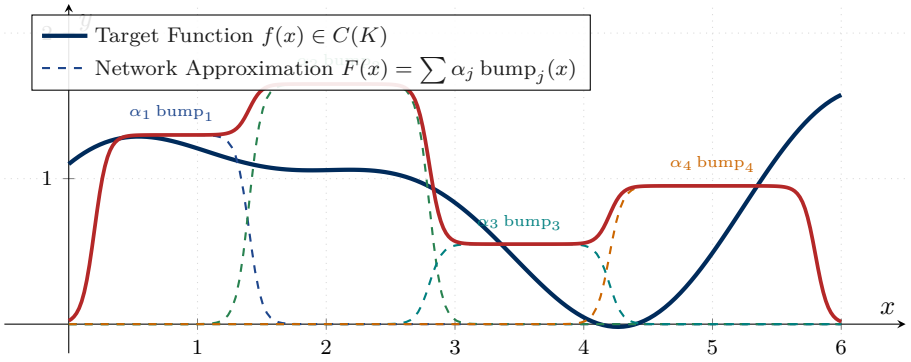


Figure 4.8: The constructive geometric mechanism of the Universal Approximation Theorem. Pairs of steep sigmoidal neurons subtract to form localized rectangular pulse functions (bumps). By tiling the compact input domain with M localized bumps scaled by coefficients $\alpha_j \approx f(x_j)$, a single-hidden-layer network constructs a Riemann-like step approximation that converges uniformly to any continuous function $f(x)$ as width $M \to \infty$.

Any continuous target function $f(x)$ on a compact interval $[x_{\min}, x_{\max}]$ is uniformly continuous. Given tolerance $\epsilon > 0$, we can partition the interval into M sub-intervals of width $\Delta x = \epsilon / L_f$ (where L_f is the Lipschitz constant of f). Associating a bump function with each interval $[x_{j-1}, x_j]$ and weighting it by the target function's midpoint value $\alpha_j = f(x_j)$ yields a Riemann step approximation $F(x) = \sum_{j=1}^{M} \alpha_j \, \text{bump}_j(x)$. As depicted in Figure 4.8, the approximation error satisfies $|F(x) - f(x)| < \epsilon$ everywhere on the domain.

In D-dimensional space $\mathbb{R}^D$, this construction generalizes directly: $2D$ sigmoidal hyperplanes intersect to enclose a localized multidimensional hypercube, forming a localized D-dimensional bump. Summing these localized bumps enables the network to tile any compact domain in $\mathbb{R}^D$. For Rectified Linear Units (ReLU), each hidden neuron introduces an affine bend (spline knot); summing M such units constructs a continuous piecewise linear (CPWL) manifold whose planar facets triangulate any arbitrary smooth function.

From shallow approximations to deep feedforward networks. Although Theorem 4.2 guarantees that a single hidden layer is theoretically sufficient to approximate any continuous function, practical machine learning almost universally deploys **deep feedforward neural networks** (also termed **Fully Connected Networks (FCNs)** or **Multi-Layer Perceptrons (MLPs)**).

Formally, an L-layer Fully Connected Network computes a sequence of coupled affine projections and element-wise non-linear activations:

$$\mathbf{a}^{(0)} \equiv \mathbf{x} \in \mathbb{R}^{d_0}, \tag{4.39}$$

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}, \, \mathbf{b}^{(l)} \in \mathbb{R}^{d_l}, \quad \forall l \in \{1, \ldots, L\}, \tag{4.40}$$

$$\mathbf{a}^{(l)} = \sigma\left(\mathbf{z}^{(l)}\right) \in \mathbb{R}^{d_l}, \tag{4.41}$$

where $\mathbf{z}^{(l)}$ is the layer-l pre-activation vector, $\sigma(\cdot)$ is the non-linear activation function, $\mathbf{a}^{(l)}$ denotes the hidden activations, and the final output is $\hat{\mathbf{y}} \equiv \mathbf{a}^{(L)}$. The total parameter count scales with layer dimensions:

$$N_{\text{dense parameters}} = \sum_{l=1}^{L} \left(d_{l-1} \cdot d_l + d_l\right). \tag{4.42}$$

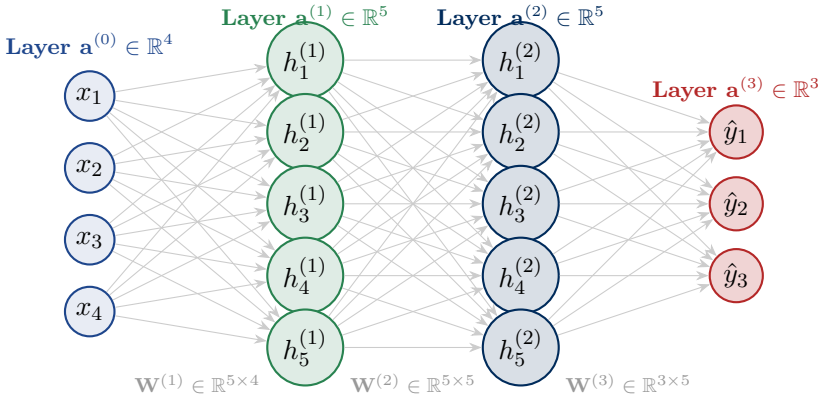


Figure 4.9: The architectural topology of a deep Fully Connected Network (FCN). Each layer l performs an affine transformation followed by an element-wise non-linear activation: $\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$. In this dense topology, the bipartite graph between adjacent layers is complete: every neuron receives input from all neurons of the preceding layer.

Why do we transition from wide, single-hidden-layer networks to deep, multi-layer architectures? The fundamental theoretical justification lies in the **exponential efficiency of depth**:

1. **Exponential Linear Regions**: As rigorously proved by Montufar et al. (2014) [66], a shallow network with W ReLU neurons can partition the input space $\mathbb{R}^D$ into at most $\mathcal{O}(W^D)$ linear regions. In contrast, a deep network with L layers and width W partitions the space into:
$$\mathcal{O}\left(\left(\frac{W}{D}\right)^{(L-1)D} W^D\right) \tag{4.43}$$
linear regions. Depth enables an **exponential explosion in expressive capacity** for the exact same number of parameters, as subsequent layers repeatedly fold and recombine the piecewise linear boundaries created by earlier layers.
2. **Compositional Feature Reuse**: Natural signals (vision, speech, natural language) are governed by hierarchical, compositional physics: pixels combine into edges, edges into motifs and textures, textures into object parts, and parts into complete scenes. Deep architectures embed this hierarchical compositional inductive bias directly into their computational structure: intermediate layers compute reusable features at increasing levels of abstraction, achieving representations with polynomial parameter efficiency $\mathcal{O}(D)$ that would require exponential width $\mathcal{O}(2^D)$ in a shallow network.

Critical drawbacks and limitations of the Universal Approximation Theorem. While the Universal Approximation Theorem provides a comforting mathematical guarantee of expressiveness, its practical utility in real-world machine learning is constrained by four fundamental limitations:

1. **Existential Guarantee vs. Algorithmic Learnability**: The theorem is purely non-constructive and existential. It proves that there *exists* some choice of width M and parameters $(\mathbf{W}, \mathbf{b}, \boldsymbol{\alpha})$ that achieves an approximation error below ϵ. Crucially, it provides **zero guarantee that gradient-based optimization algorithms can ever discover these parameters**. The empirical loss landscape of an ultra-wide shallow network is characterized by pathological saddle points, plateaus, and bad local minima that prevent standard gradient descent from converging to the theoretical solution.
2. **The Curse of Dimensionality and Exponential Width**: The constructive proof partitions each input dimension into $\mathcal{O}(1/\epsilon)$ intervals. To tile a D-dimensional compact

unit hypercube $[0,1]^D$, the required number of localized bump units scales exponentially with dimension:

$$M \sim \mathcal{O}\left(\left(\frac{1}{\epsilon}\right)^D\right). \tag{4.44}$$

For high-dimensional inputs typical of modern tasks (e.g., $D = 3 \times 224 \times 224 \approx 1.5 \times 10^5$ for image classification), M astronomical exceeds the number of elementary particles in the observable universe. Shallow universal approximation is therefore computationally and physically impossible in high dimensions.

3. **Zero Generalization Guarantees and Lookup-Table Memorization**: The theorem guarantees uniform approximation only on the compact training domain K; it offers **no generalization bound** on unseen test distributions. An ultra-wide shallow network with $M \to \infty$ units can act as a brute-force lookup table or memorization spline, dedicating individual localized bumps to interpolate every training point exactly. Such a model achieves zero training error but learns zero underlying structure, leading to catastrophic overfitting and failure to generalize.
4. **Exponential Sample Complexity**: Accurately configuring the weights $\alpha_j \approx f(\mathbf{x}_j)$ across $M \sim \mathcal{O}((1/\epsilon)^D)$ localized bump functions requires observing training samples in virtually every sub-cube of the domain. In high dimensions, the volume of space is vast and finite training sets leave the input space virtually empty (the empty space phenomenon). Without the strong compositional inductive biases enforced by deep hierarchical architectures, a shallow network requires an exponentially large sample size $N \sim \mathcal{O}((1/\epsilon)^D)$ to avoid arbitrary interpolation errors in unobserved regions.

Yet, while deep feedforward networks possess extraordinary theoretical expressive power, an urgent operational crisis emerges: *how do we compute the parameter updates for a deep network containing millions of entangled weights?* In the next section, we discover why classical algebraic methods fail and how the computational graph resolves this crisis.

4.3 Computational graphs and backpropagation

Having established that deep feedforward networks possess the expressive capacity to represent complex non-linear manifolds, we confront the central operational challenge of machine learning: **parameter optimization**. In classical Ordinary Least Squares (Chapter 3), parameters admit an exact, globally optimal analytical solution via the Normal Equations $\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top \mathbf{y}$. In deep neural networks, however, parameters are deeply entangled across multi-layer compositions of non-linear activations. Optimizing these models requires computing the gradient of a scalar loss function with respect to millions or billions of parameters efficiently and accurately.

4.3.1 Motivation for computational graphs

The algebraic impasse and non-convexity. A natural instinct is to ask whether an analytical closed-form solution exists for deep neural networks, analogous to the Normal Equations of linear regression. Consider an L-layer neural network hypothesis evaluating an input $\mathbf{x} \in \mathbb{R}^{d_0}$:

$$f(\mathbf{x};\boldsymbol{\theta}) = \mathbf{W}^{(L)}\sigma\left(\mathbf{W}^{(L-1)}\sigma\left(\dots\sigma\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right)\dots\right) + \mathbf{b}^{(L-1)}\right) + \mathbf{b}^{(L)}, \tag{4.45}$$

evaluated across training dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ under the empirical risk objective:

$$\hat{\mathcal{R}}_N(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}\left(f(\mathbf{x}_i;\boldsymbol{\theta}),\, y_i\right). \tag{4.46}$$

To obtain an exact analytical minimizer, one must equate the gradient with respect to all parameter matrices to zero: $\nabla_{\boldsymbol{\theta}}\hat{\mathcal{R}}_N(\boldsymbol{\theta}) = \mathbf{0}$. This stationary condition defines a massive system

of coupled, transcendental non-linear algebraic equations. Because early parameters $\mathbf{W}^{(1)}$ are deeply nested within cascading non-linear activation functions $\sigma(\cdot)$, they cannot be decoupled, factorized, or isolated through matrix inversion or linear algebraic operations. Furthermore, as established in Chapter 3, the loss surface of deep networks is non-convex, characterized by uncountably many saddle points, local minima, and permutation symmetries. Analytical closed-form solutions are mathematically impossible, necessitating iterative numerical optimization via gradient descent.

The failure of numerical differentiation. To execute gradient descent, we must compute the exact gradient vector $\nabla_{\boldsymbol{\theta}}\mathcal{L}$ for networks containing millions to billions of parameters ($W \sim 10^7$–10^{11}). The simplest numerical approach for computing derivatives is **finite differences** (numerical perturbation):

$$\frac{\partial \mathcal{L}}{\partial \theta_j} \approx \frac{\mathcal{L}(\boldsymbol{\theta} + \epsilon \mathbf{e}_j) - \mathcal{L}(\boldsymbol{\theta})}{\epsilon}, \tag{4.47}$$

where $\mathbf{e}_j$ is the j-th standard basis vector and $\epsilon \approx 10^{-5}$ is an infinitesimal perturbation. Evaluating Equation (4.47) across all W parameters requires evaluating $W+1$ complete forward passes through the network. In a standard deep network where $W = 10^8$ (such as ResNet or AlexNet), computing a single parameter update step would require 100,000,001 forward network evaluations. If a single forward pass takes 10 milliseconds, evaluating one gradient step would require over **11 days of continuous GPU computation**, yielding an intractable computational complexity of $\mathcal{O}(W^2)$. Finite-difference numerical differentiation is therefore completely unusable for deep learning optimization.

4.3.2 DAGs and reverse-mode differentiation

The general principle of computational DAGs. Modern deep learning frameworks (such as PyTorch, JAX, and TensorFlow) resolve the differentiation impasse by reformulating network evaluation as a **Directed Acyclic Graph (DAG)**. In a computational DAG:

- **Nodes** represent mathematical variables (tensors: scalars, vectors, matrices, or higher-order arrays).
- **Directed Edges** represent primitive, differentiable elementary operations (such as addition $+$, scalar/matrix multiplication $\times$, element-wise activation σ, and logarithms $\ln$).

Because the graph is acyclic, its nodes admit a **topological sorting** that defines a valid execution order. Evaluating the network on the DAG naturally decomposes into two phases: (1) a **Forward Pass**, where inputs flow along directed edges according to topological sort order, computing and caching intermediate activations at each node; and (2) a **Backward Pass**, where the chain rule of differential calculus propagates derivative signals in reverse topological order, modulating incoming error signals by local Jacobian matrices.

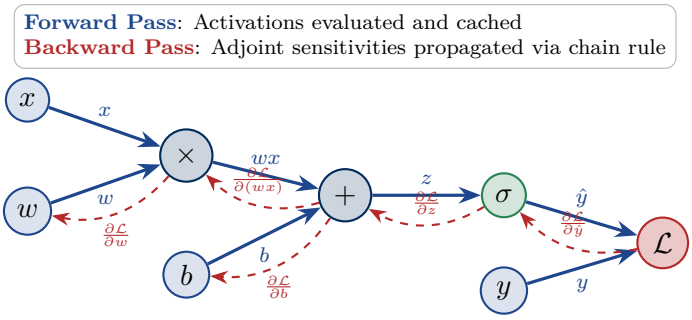


Figure 4.10: A Directed Acyclic Graph (DAG) representation of a computational graph for an artificial neuron with scalar loss $\mathcal{L}(\sigma(wx+b), y)$. Forward evaluation (solid blue lines) evaluates and caches intermediate variables. Reverse-mode automatic differentiation (dashed red lines) propagates adjoint sensitivities backwards from the scalar loss.

Reverse-mode automatic differentiation. Let $f : \mathbb{R}^n \to \mathbb{R}^m$ be an arbitrary composite differentiable function executed on a DAG. Automatic differentiation computes exact derivatives at machine precision by decomposing f into elementary operations and systematically applying the chain rule. It operates in one of two fundamental orientations:

1. **Forward Tangent Mode**: Tracks directional derivatives (tangents) alongside intermediate variables during the forward pass: $\dot{v}_i \triangleq \sum_{j=1}^{n} \frac{\partial v_i}{\partial x_j} \dot{x}_j$. To compute the full Jacobian $\mathbf{J} \in \mathbb{R}^{m \times n}$, forward-mode autodiff requires seeding the input tangents with standard basis vectors $\dot{\mathbf{x}} = \mathbf{e}_k$ across n independent forward sweeps. Forward mode is computationally optimal when the number of inputs is much smaller than the number of outputs ($n \ll m$).
2. **Reverse Adjoint Mode**: Executes in two sweeps. First, the forward sweep evaluates and caches all intermediate node values v_i. Second, beginning at the terminal output nodes, it propagates adjoint sensitivity variables backward:

$$\bar{v}_i \triangleq \frac{\partial \mathcal{L}}{\partial v_i} = \sum_{j \in \text{children}(i)} \bar{v}_j \frac{\partial v_j}{\partial v_i}. \tag{4.48}$$

   To evaluate the complete Jacobian, reverse mode seeds the output adjoints $\bar{\mathbf{y}} = \mathbf{e}_k$ and sweeps backward m times.

In deep learning, the training objective is a **scalar loss** ($\mathcal{L} \in \mathbb{R}$, so output dimension $m = 1$), while the parameter space is massive ($n = W \sim 10^8$). Because $m = 1$, reverse-mode automatic differentiation evaluates the **entire parameter gradient vector $\nabla_{\boldsymbol{\theta}} \mathcal{L} \in \mathbb{R}^W$ in exactly ONE single backward pass**! It reduces computational complexity from $\mathcal{O}(W^2)$ to $\mathcal{O}(W)$, slashing gradient evaluation time by eight orders of magnitude.

Table 4.1: Comparison of differentiation mechanisms across algorithmic paradigms.

Method	Exactness	Complexity	Memory	Suitability for Deep Learning
Numerical (Finite Diff.)	Approximate ($\mathcal{O}(\epsilon)$)	$\mathcal{O}(W^2)$	$\mathcal{O}(1)$	Unusable: 10^8 forward passes per step.
Symbolic (Algebraic)	Exact	Exponential	Memory blowup	Unusable: expression swell on deep DAGs.
Forward-Mode Autodiff	Exact	$\mathcal{O}(n \cdot W)$	$\mathcal{O}(1)$	Sub-optimal: requires W passes for scalar loss.
Reverse-Mode Autodiff	Exact	$\mathcal{O}(m \cdot W)$	$\mathcal{O}(W)$	**Ideal**: exactly **ONE backward pass** for scalar loss ($m = 1$).

4.3.3 The backpropagation algorithm

The general idea behind backpropagation. The Backpropagation algorithm is the concrete, layer-wise realization of reverse-mode automatic differentiation applied to neural network architectures. Formulated in optimal control by Paul Werbos (1974) [97] and popularized in connectionist machine learning by David Rumelhart, Geoffrey Hinton, and Ronald Williams (1986) [79], backpropagation represents the foundational algorithmic engine of modern deep learning.

The central intuition of backpropagation is solving the **credit assignment problem**: determining how much each individual weight and bias in an arbitrary layer contributed to the final prediction error. Instead of computing derivatives independently from scratch for every parameter, backpropagation transforms credit assignment into a localized, recursive message-passing scheme. Information flows in two directions: forward signals compute intermediate representations, while backward signals (error adjoints) propagate from the loss back toward the inputs. Each layer receives an error message from the layer above, uses its cached forward activations to compute its own parameter gradients, and multiplies the error by its transposed weight matrix to pass an error message to the layer below.

The algorithmic cycle of backpropagation. The execution of backpropagation across an L-layer network proceeds through five sequential steps:

1. **Forward Propagation**: Feed input $\mathbf{x} \equiv \mathbf{a}^{(0)}$ through all layers $l = 1, \ldots, L$. At each layer, compute and cache pre-activations $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$ and post-activations $\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$.
2. **Loss and Output Error Evaluation**: Evaluate the scalar loss $\mathcal{L}(\mathbf{a}^{(L)}, \mathbf{y})$ and compute the output layer error adjoint $\boldsymbol{\delta}^{(L)} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(L)}}$.
3. **Backward Error Propagation**: Recursively backpropagate the error adjoints $\boldsymbol{\delta}^{(l)}$ from layer $l + 1$ back to layer l for $l = L - 1, L - 2, \ldots, 1$ via transposed matrix multiplication and element-wise activation differentiation.

4. **Parameter Gradient Accumulation**: Compute partial derivatives of the loss with respect to all parameter matrices $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$ and bias vectors $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}}$ using the error adjoints $\boldsymbol{\delta}^{(l)}$ and stored activations $\mathbf{a}^{(l-1)}$.
5. **Parameter Update**: Update all network parameters via gradient descent: $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$ and $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}}$.

Step-by-step mathematical derivation of the four fundamental equations. We define the **layer error adjoint vector** $\boldsymbol{\delta}^{(l)}$ as the partial derivative of the scalar loss with respect to the pre-activation vector $\mathbf{z}^{(l)}$:

$$\boldsymbol{\delta}^{(l)} \triangleq \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} \in \mathbb{R}^{d_l}. \tag{4.49}$$

The vector $\boldsymbol{\delta}^{(l)}$ quantifies the sensitivity of the final loss to perturbations in layer l's pre-activations. The algorithm is governed by four fundamental equations:

1. **Equation BP1: Output Layer Error Adjoint $\boldsymbol{\delta}^{(L)}$.** Applying the multivariate chain rule to the scalar loss $\mathcal{L}$ with respect to coordinate $z_k^{(L)}$:

$$\delta_k^{(L)} = \frac{\partial \mathcal{L}}{\partial z_k^{(L)}} = \sum_{j=1}^{d_L} \frac{\partial \mathcal{L}}{\partial a_j^{(L)}} \frac{\partial a_j^{(L)}}{\partial z_k^{(L)}}. \tag{4.50}$$

   Because the activation function $\sigma(\cdot)$ is applied element-wise, $\frac{\partial a_j^{(L)}}{\partial z_k^{(L)}} = 0$ for all $j \neq k$. The summation collapses to a single term:

$$\delta_k^{(L)} = \frac{\partial \mathcal{L}}{\partial a_k^{(L)}} \sigma'\left(z_k^{(L)}\right) \implies \boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}} \mathcal{L} \odot \sigma'\left(\mathbf{z}^{(L)}\right), \tag{4.51}$$

   where $\odot$ denotes the Hadamard (element-wise) product. For standard Cross-Entropy loss paired with Softmax or Sigmoid activations, this simplifies directly to the residual prediction error: $\boldsymbol{\delta}^{(L)} = \hat{\mathbf{y}} - \mathbf{y}$.
2. **Equation BP2: Hidden Layer Error Recurrence $\boldsymbol{\delta}^{(l)}$.** Suppose the error adjoint vector $\boldsymbol{\delta}^{(l+1)} \in \mathbb{R}^{d_{l+1}}$ at layer $l+1$ has already been computed. By the multivariate chain rule:

$$\delta_j^{(l)} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} = \sum_{k=1}^{d_{l+1}} \frac{\partial \mathcal{L}}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = \sum_{k=1}^{d_{l+1}} \delta_k^{(l+1)} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}}. \tag{4.52}$$

   Since $z_k^{(l+1)} = \sum_{m=1}^{d_l} W_{km}^{(l+1)} \sigma(z_m^{(l)}) + b_k^{(l+1)}$, differentiating with respect to $z_j^{(l)}$ yields $\frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = W_{kj}^{(l+1)} \sigma'(z_j^{(l)})$. Substituting into Equation (4.52):

$$\delta_j^{(l)} = \sum_{k=1}^{d_{l+1}} \delta_k^{(l+1)} W_{kj}^{(l+1)} \sigma'\left(z_j^{(l)}\right) = \left(\sum_{k=1}^{d_{l+1}} W_{kj}^{(l+1)} \delta_k^{(l+1)}\right) \sigma'\left(z_j^{(l)}\right). \tag{4.53}$$

   In vector notation, this produces the core backpropagation recurrence:

$$\boldsymbol{\delta}^{(l)} = \left((\mathbf{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)}\right) \odot \sigma'\left(\mathbf{z}^{(l)}\right). \tag{4.54}$$

   The error adjoint propagates backward by multiplying with the transposed weight matrix $(\mathbf{W}^{(l+1)})^\top$, followed by element-wise modulation by the local activation derivative $\sigma'(\mathbf{z}^{(l)})$.
3. **Equation BP3: Weight Matrix Parameter Gradients.** The gradient of the loss with respect to individual weight entries $W_{jk}^{(l)}$ is:

$$\frac{\partial \mathcal{L}}{\partial W_{jk}^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial W_{jk}^{(l)}} = \delta_j^{(l)} a_k^{(l-1)}. \tag{4.55}$$

In outer-product matrix notation:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^\top \in \mathbb{R}^{d_l \times d_{l-1}}. \tag{4.56}$$

4. **Equation BP4: Bias Vector Parameter Gradients.** Similarly, for the scalar bias parameter $b_j^{(l)}$:

$$\frac{\partial \mathcal{L}}{\partial b_j^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial b_j^{(l)}} = \delta_j^{(l)} \cdot 1 \implies \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)} \in \mathbb{R}^{d_l}. \tag{4.57}$$

In practical machine learning, computations are vectorized over mini-batches of size B. Stacking B training observations as rows in matrix $\mathbf{X} \in \mathbb{R}^{B \times d_0}$, the batched forward pass computes:

$$\mathbf{Z}^{(l)} = \mathbf{A}^{(l-1)} (\mathbf{W}^{(l)})^\top + \mathbf{1}_B (\mathbf{b}^{(l)})^\top, \quad \mathbf{A}^{(l)} = \sigma\left(\mathbf{Z}^{(l)}\right), \tag{4.58}$$

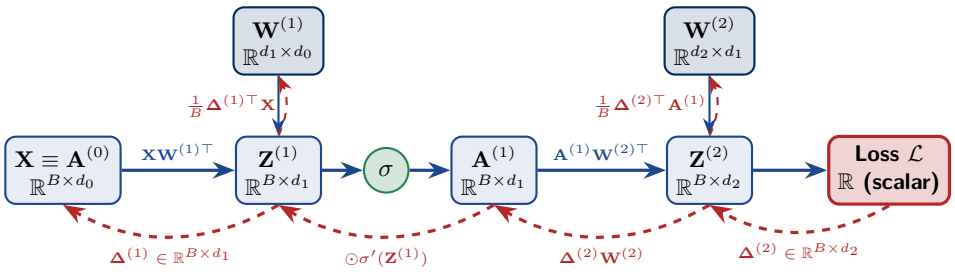
where $\mathbf{1}_B = [1, \ldots, 1]^\top \in \mathbb{R}^B$ broadcasts the bias row vector across all samples. Defining the batch error adjoint matrix $\boldsymbol{\Delta}^{(l)} \triangleq \nabla_{\mathbf{Z}^{(l)}} \mathcal{L} \in \mathbb{R}^{B \times d_l}$, the vectorized batched backpropagation equations evaluate to:

$$\boldsymbol{\Delta}^{(L)} = \nabla_{\mathbf{A}^{(L)}} \mathcal{L} \odot \sigma'\left(\mathbf{Z}^{(L)}\right) \in \mathbb{R}^{B \times d_L}, \tag{4.59}$$

$$\boldsymbol{\Delta}^{(l)} = \left(\boldsymbol{\Delta}^{(l+1)} \mathbf{W}^{(l+1)}\right) \odot \sigma'\left(\mathbf{Z}^{(l)}\right) \in \mathbb{R}^{B \times d_l}, \tag{4.60}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \frac{1}{B} \left(\boldsymbol{\Delta}^{(l)}\right)^\top \mathbf{A}^{(l-1)} \in \mathbb{R}^{d_l \times d_{l-1}}, \tag{4.61}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \frac{1}{B} \left(\boldsymbol{\Delta}^{(l)}\right)^\top \mathbf{1}_B \in \mathbb{R}^{d_l}. \tag{4.62}$$



Forward Pass: Tensors evaluated and activations cached in GPU VRAM
Backward Pass: Adjoint error matrices $\boldsymbol{\Delta}^{(l)}$ and parameter gradients propagated via GEMM contractions

Figure 4.11: Batched tensor computational graph for a multi-layer feedforward network. Forward execution (solid blue) propagates activation matrices $\mathbf{A}^{(l)} \in \mathbb{R}^{B \times d_l}$ through weight matrices. Backward execution (dashed red) propagates adjoint error matrices $\boldsymbol{\Delta}^{(l)} \in \mathbb{R}^{B \times d_l}$, computing parameter gradient matrices via outer-product matrix contractions.

In Equation (4.61), the dimensions contract as $(d_l \times B) \times (B \times d_{l-1}) = (d_l \times d_{l-1})$. The inner batch dimension B contracts completely, accumulating the sample-averaged outer product across all B observations in a single high-performance BLAS GEMM matrix multiplication!

Concrete end-to-end numerical walkthrough. To make the algorithm completely transparent and provide a concrete numerical benchmark, consider a minimal two-layer network evaluated on a single observation:

- Input: $\mathbf{x} = [x_1, x_2]^\top = [0.5, -0.2]^\top \in \mathbb{R}^2$.

- Hidden layer: 2 neurons with logistic sigmoid activation $\sigma(z) = \frac{1}{1+e^{-z}}$, weights $\mathbf{W}^{(1)} = \begin{bmatrix} 0.4 & -0.5 \\ 0.2 & 0.8 \end{bmatrix}$, and bias $\mathbf{b}^{(1)} = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}$.
- Output layer: 1 linear neuron $\hat{y} = z^{(2)} = \mathbf{w}^{(2)\top}\mathbf{a}^{(1)} + b^{(2)}$ with $\mathbf{w}^{(2)} = \begin{bmatrix} 0.7 \\ -0.3 \end{bmatrix}$, $b^{(2)} = 0.2$, target $y = 1.0$, and squared error loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$.

1. Forward Pass Evaluation: The hidden pre-activations evaluate to:

$$\begin{aligned} z_1^{(1)} &= (0.4)(0.5) + (-0.5)(-0.2) + 0.1 = 0.20 + 0.10 + 0.1 = 0.40, \\ z_2^{(1)} &= (0.2)(0.5) + (0.8)(-0.2) - 0.1 = 0.10 - 0.16 - 0.1 = -0.16. \end{aligned} \tag{4.63}$$

Applying the logistic sigmoid activation:

$$\begin{aligned} a_1^{(1)} &= \sigma(0.40) = \frac{1}{1 + e^{-0.40}} \approx 0.5987, \\ a_2^{(1)} &= \sigma(-0.16) = \frac{1}{1 + e^{0.16}} \approx 0.4601. \end{aligned} \tag{4.64}$$

Evaluating the output pre-activation and prediction:

$$\hat{y} = z^{(2)} = (0.7)(0.5987) + (-0.3)(0.4601) + 0.2 \approx 0.4191 - 0.1380 + 0.2 = 0.4811. \tag{4.65}$$

The forward loss is:

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(0.4811 - 1.0)^2 = \frac{1}{2}(-0.5189)^2 \approx 0.1346. \tag{4.66}$$

2. Backward Pass Evaluation: Step 1: Compute the output layer error adjoint $\delta^{(2)}$ via Equation BP1:

$$\delta^{(2)} \triangleq \frac{\partial \mathcal{L}}{\partial z^{(2)}} = \hat{y} - y = 0.4811 - 1.0 = -0.5189. \tag{4.67}$$

The output parameter gradients follow immediately via Equations BP3 and BP4:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_1^{(2)}} &= \delta^{(2)} a_1^{(1)} = (-0.5189)(0.5987) \approx -0.3107, \\ \frac{\partial \mathcal{L}}{\partial w_2^{(2)}} &= \delta^{(2)} a_2^{(1)} = (-0.5189)(0.4601) \approx -0.2387, \\ \frac{\partial \mathcal{L}}{\partial b^{(2)}} &= \delta^{(2)} = -0.5189. \end{aligned} \tag{4.68}$$

Step 2: Backpropagate error adjoints to the hidden layer pre-activations via Equation BP2:

$$\boldsymbol{\delta}^{(1)} = \left((\mathbf{w}^{(2)})^\top \delta^{(2)}\right) \odot \sigma'(\mathbf{z}^{(1)}). \tag{4.69}$$

Evaluating the sigmoid derivatives $\sigma'(z) = \sigma(z)(1 - \sigma(z))$:

$$\begin{aligned} \sigma'(z_1^{(1)}) &= (0.5987)(1 - 0.5987) \approx 0.2403, \\ \sigma'(z_2^{(1)}) &= (0.4601)(1 - 0.4601) \approx 0.2484. \end{aligned} \tag{4.70}$$

Multiplying the backpropagated error by the activation derivatives:

$$\begin{aligned} \delta_1^{(1)} &= (w_1^{(2)} \delta^{(2)}) \sigma'(z_1^{(1)}) = (0.7)(-0.5189)(0.2403) \approx -0.0873, \\ \delta_2^{(1)} &= (w_2^{(2)} \delta^{(2)}) \sigma'(z_2^{(1)}) = (-0.3)(-0.5189)(0.2484) \approx +0.0387. \end{aligned} \tag{4.71}$$

Step 3: Compute gradients for hidden weights and biases via Equations BP3 and BP4:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \boldsymbol{\delta}^{(1)} \mathbf{x}^\top = \begin{bmatrix} -0.0873 \\ +0.0387 \end{bmatrix} \begin{bmatrix} 0.5 & -0.2 \end{bmatrix} = \begin{bmatrix} -0.0437 & +0.0175 \\ +0.0194 & -0.0077 \end{bmatrix},$$
$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = \boldsymbol{\delta}^{(1)} = \begin{bmatrix} -0.0873 \\ +0.0387 \end{bmatrix}. \tag{4.72}$$

In a single backward sweep, all 9 parameter gradients are computed exactly at machine precision.

Computational and memory drawbacks of backpropagation and the mini-batch solution. Despite its foundational role, the backpropagation algorithm incurs two significant computational and physical bottlenecks:

1. **The Computational Drawback of Full-Batch Evaluation**: In standard batch gradient descent, evaluating the true empirical risk gradient requires backpropagating through the entire training dataset:
$$\nabla_{\boldsymbol{\theta}} \hat{\mathcal{R}}_N(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \nabla_{\boldsymbol{\theta}} \mathcal{L}_i(\boldsymbol{\theta}). \tag{4.73}$$
For large datasets (e.g., ImageNet with $N = 1.3 \times 10^6$ images, or modern web-scale text corpora with $N \sim 10^9$–10^{12} tokens), executing a single parameter update requires an intractable computational cost of $\mathcal{O}(N \cdot W)$ operations. Much of this computation is severely redundant, as gradients evaluated on similar examples within a massive dataset largely cancel or repeat directional information.
How Mini-Batching Resolves the Computational Drawback: Mini-batch optimization resolves this computational barrier by approximating the full gradient using a small, randomly sampled subset $\mathcal{B} \subset \{1, \dots, N\}$ of size $B \ll N$ (typically $B \in \{32, 64, 128, 256, 512\}$):
$$\mathbf{g}_{\mathcal{B}}(\boldsymbol{\theta}) = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\boldsymbol{\theta}} \mathcal{L}_i(\boldsymbol{\theta}). \tag{4.74}$$
Because observations are sampled uniformly at random, the mini-batch gradient is an **unbiased estimator** of the true full gradient: $\mathbb{E}_{\mathcal{B}}[\mathbf{g}_{\mathcal{B}}(\boldsymbol{\theta})] = \nabla_{\boldsymbol{\theta}} \hat{\mathcal{R}}_N(\boldsymbol{\theta})$. This reduces the computational cost per update from $\mathcal{O}(N \cdot W)$ down to $\mathcal{O}(B \cdot W)$—an acceleration of several orders of magnitude.
Furthermore, mini-batching unlocks massive hardware efficiency: by stacking B observations into contiguous memory buffers, vector operations are transformed into dense General Matrix Multiplications (GEMM) that saturate the high-throughput Tensor Cores of modern GPUs and TPUs. Performing N/B parameter updates per epoch drastically accelerates convergence relative to full-batch optimization.
2. **The Memory Drawback: Activation Caching and GPU VRAM Blowup**: As revealed by Equation BP3 ($\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)} (\mathbf{a}^{(l-1)})^\top$), computing the weight gradient at layer l strictly requires the forward activation matrix $\mathbf{A}^{(l-1)}$. Consequently, all intermediate activations computed during the forward pass **must be retained in high-bandwidth memory (GPU VRAM)** until the backward pass traverses back to layer l.
This introduces a critical memory asymmetry between inference and training:
   - **Inference Memory**: Intermediate activations can be discarded immediately once the next layer is computed. Peak memory is determined only by the largest single layer: $\mathcal{O}(B \cdot \max_l d_l)$.
   - **Training Memory**: All intermediate activations across **all** L **layers** must be stored concurrently:
$$M_{\text{activations}} = B \sum_{l=1}^{L} d_l \approx \mathcal{O}(L \cdot B \cdot d). \tag{4.75}$$

In ultra-deep networks (such as 152-layer ResNets or deep Transformers), activation memory vastly exceeds parameter memory, forming the primary physical bottleneck on training batch size and model depth. To mitigate this VRAM wall, modern systems implement **Activation Checkpointing** [12], which discards activations during the forward pass and selectively recomputes them during the backward sweep, trading a 30% increase in compute FLOPs for an $\mathcal{O}(\sqrt{L})$ reduction in memory footprint.

Chapter 5

Convolutional neural networks

In Chapter 4, we established that Multi-Layer Perceptrons equipped with non-linear activation functions are universal function approximators: given sufficient neurons, they can represent any continuous manifold. Yet, in empirical computer vision, deploying dense MLPs directly onto pixel arrays yields an immediate, catastrophic failure. Fully connected networks treat each input feature independently; they possess zero awareness of geometry, spatial adjacency, or physical symmetries.

To construct visual recognition systems that generalize efficiently, machine learning must abandon unconstrained fully connected architectures and embed the physical invariances of the natural visual world directly into the network topology. This breakthrough is realized through **Convolutional Neural Networks (CNNs)**.

From Yoshua Bengio's Bayesian perspective, a convolutional network is not merely an engineering heuristic: it is a deep neural network endowed with an **infinitely strong spatial prior**. By enforcing sparse local interactions, parameter sharing across space, and translation equivariance, CNNs slash parameter counts by orders of magnitude, preserve 2D grid topology, and mirror the hierarchical visual processing discovered in the biological mammalian visual cortex by Hubel and Wiesel.

This chapter provides an exhaustive theoretical and mathematical treatment of convolutional architectures: from continuous convolutions and discrete cross-correlations, through receptive field algebra, 1×1 bottlenecks, and global average pooling, to landmark architectural evolutions (AlexNet, VGG, ResNet) and the complete taxonomy of visual tasks (classification, detection, and semantic segmentation).

5.1 Failure of dense networks on vision

In biological vision and modern artificial intelligence, visual sensory inputs are fundamentally distinct from tabular database entries or isolated acoustic waveforms: an image is a continuous, high-dimensional projection of a structured three-dimensional physical environment onto a two-dimensional sensor plane. In Chapter 4, we proved the Universal Approximation Theorem, establishing that Multi-Layer Perceptrons (MLPs) equipped with non-linear activation functions can approximate any continuous compact mapping to arbitrary precision. However, when machine learning practitioners attempt to deploy unconstrained fully connected architectures directly onto raw visual sensory grids, the models suffer an immediate, catastrophic empirical failure.

5.1.1 Image patterns and visual regularities

To understand why dense networks fail and how we can construct architectures tailored for vision, we must first examine the nature of visual data itself. Why do photographic images

possess an inherent internal structure, and why do virtually all natural images share identical low-level statistical patterns?

Natural images versus white noise. Consider an arbitrary 256×256 RGB image with 8-bit color channels. The space of all possible pixel configurations contains:

$$(256)^{256\times256\times3} = 256^{196,608} \approx 10^{473,000} \text{ configurations.} \tag{5.1}$$

If pixel intensities were drawn independently from a uniform or Gaussian distribution, the resulting image would be pure white noise: an unstructured, static television hiss containing zero discernible semantic information. In contrast, the set of all possible **natural images** occupies an infinitesimal, low-dimensional manifold embedded within this unimaginably vast space.

Natural images are non-random because they are optical projections of a **structured 3D physical universe**:

- **Continuous Physical Matter**: Solid physical objects consist of contiguous materials with uniform surface reflectance, mass, and texture.
- **Smooth Optical Illumination**: Environmental light sources cast smooth illumination gradients across physical surfaces rather than fluctuating arbitrarily from millimeter to millimeter.
- **Localized Discontinuities**: Significant variations in pixel values occur predominantly at physical object boundaries, material transitions, and occlusion silhouettes, creating sharp, localized step edges.
- **Perspective Projection**: The 3D geometry of space induces regular perspective transformations, straight lines, vanishing points, and depth planes onto the 2D sensor grid.

Inherent self-similarity and the inpainting proof. Because images project continuous physical scenes, they exhibit profound **statistical self-similarity**: local visual motifs, textures, and color distributions repeat continuously across the canvas. For example, a single photograph contains repeated rectangular patterns on the spokes of a wheel, rows of nearly identical windows across an architectural facade, or consistent skin tones and textural shading between a person's forehead, cheeks, and adjacent visual regions.

A decisive qualitative demonstration of this universal self-similarity is the mechanism of **patch-based inpainting** (as utilized in digital image editing and patch synthesis algorithms). If an object is excised from a scene—such as removing an eagle soaring against a sky and mountain backdrop—the resulting empty hole can be seamlessly reconstructed without relying on external databases. An algorithm can fill the void by simply sampling, copying, and blending coherent texture patches already present in other regions of the exact same image. The fact that an occluded region can be faithfully synthesized from distant patches of the same canvas proves that natural scenes possess immense internal redundancy and spatial self-similarity.

Universal statistical regularities of natural scenes. Empirical investigations into natural image statistics reveal four universal mathematical properties shared across all natural scenes:

1. **High Local Autocorrelation**: The statistical correlation between two pixels $\mathbf{I}(\mathbf{x})$ and $\mathbf{I}(\mathbf{x} + \Delta\mathbf{x})$ approaches unity as the spatial displacement $\|\Delta\mathbf{x}\|_2 \to 0$. Adjacent pixels almost always share identical color and luminance; meaningful variation is concentrated at localized boundaries.

2. **Scale Invariance and the $1/f^2$ Power Spectrum**: Across diverse natural landscapes, the Fourier power spectral density $S(f)$ universally follows a power law:

$$S(f) \propto \frac{1}{f^{2-\eta}}, \quad \text{with } \eta \approx 0.2. \tag{5.2}$$

   This scale invariance indicates that the visual world exhibits identical statistical structure across zoom levels: low spatial frequencies (smooth surfaces) dominate the overall energy, while high spatial frequencies (edges and fine textures) form a sparse, highly informative component.

3. **Universality of Low-Level Edge Primitives**: Across entirely unrelated visual domains—whether inspecting facial portraits, satellite topography, medical histopathology, or urban streetscapes—images are universally constructed from the same elementary visual primitives: **oriented edges, localized color transitions, and corner junctions**.

4. **Hierarchical Compositionality and Semantic Translation Tolerance**: Natural visual semantics are formed compositionally: local pixels combine into oriented edges, edges aggregate into textures and geometric motifs, motifs assemble into object parts (such as eyes or wheels), and parts assemble into complete semantic objects. Furthermore, the semantic identity of an object is translation-tolerant: a cat remains a cat regardless of whether it appears in the upper-left corner, the center, or the lower-right corner of the canvas.

Neurobiological evidence: Hubel and Wiesel. This hierarchical organization is not merely an engineering convenience; it mirrors biological vision. In 1959, neurophysiologists David Hubel and Torsten Wiesel [45, 46] discovered that the mammalian primary visual cortex (area V1) is organized into a hierarchy of specialized cells:

- **Simple Cells**: Possess small, localized receptive fields that activate selectively to oriented light bars and edges at specific angles and spatial positions.
- **Complex Cells**: Pool the responses of multiple simple cells across a local neighborhood, maintaining orientation selectivity while achieving invariance to small spatial shifts.

This biological discovery inspired Kunihiko Fukushima's **Neocognitron** (1980) [26] and Yann LeCun's **LeNet** (1989) [56], establishing the foundation for hierarchical representation learning in artificial vision.

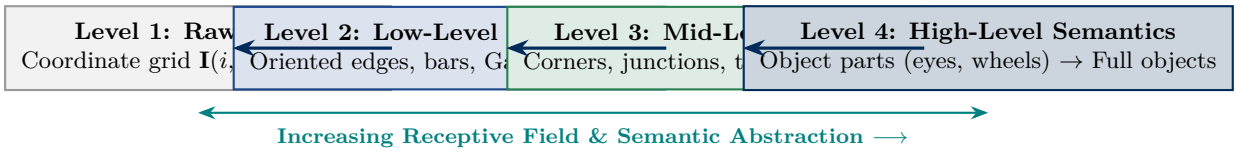


Figure 5.1: The hierarchical compositionality of visual representations. Natural images are formed compositionally: local pixels combine into oriented edges, edges aggregate into textures and geometric motifs, which assemble into object parts, and ultimately into semantic categories. Convolutional networks emulate this visual hierarchy across cascading layers.

5.1.2 Vectorization and parameter sharing failure

Despite the universal regularities governing natural images, standard Multi-Layer Perceptrons fail catastrophically when applied to raw visual grids. This breakdown is driven by two foundational structural limitations: the mandatory vectorization of multidimensional inputs and the complete absence of parameter sharing.

The vectorization requirement and destruction of 2D spatial topology. To process sensory data in an MLP, each artificial neuron computes an affine inner product followed by an element-wise activation:

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{k=1}^{d} w_k x_k + b. \tag{5.3}$$

Because the canonical Euclidean inner product and matrix multiplication $\mathbf{Wx}$ are defined exclusively over one-dimensional coordinate vectors $\mathbf{x} \in \mathbb{R}^d$, a dense layer **cannot natively ingest multidimensional array grids**.

Consequently, any multi-channel image tensor $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ must undergo mandatory **vectorization (flattening)** into an unrolled 1D column vector:

$$\mathbf{x} = \mathrm{vec}(\mathbf{X}) \in \mathbb{R}^d, \quad \text{where } d = H \cdot W \cdot C, \tag{5.4}$$

under which each spatial coordinate (i, j) in channel c is mapped to a serialized linear index $k(i, j, c) = c \cdot (HW) + i \cdot W + j$.

This flattening operation irrevocably dismantles the spatial geometry of the image:

- **Metric Space Disruption**: In the native 2D continuous image plane, distance is measured by standard Euclidean distance between coordinate pairs $\mathbf{p}_1 = (i_1, j_1)$ and $\mathbf{p}_2 = (i_2, j_2)$:

  $$d_{\text{spatial}}(\mathbf{p}_1, \mathbf{p}_2) = \|\mathbf{p}_1 - \mathbf{p}_2\|_2 = \sqrt{(i_1 - i_2)^2 + (j_1 - j_2)^2}. \tag{5.5}$$

  Pixels that are immediate vertical neighbors, such as (i, j) and $(i + 1, j)$, have a physical spatial distance of 1. However, in the flattened vector $\mathbf{x}$, their serialized indices are separated by an entire row of W elements: $|k_1 - k_2| = W$. Conversely, pixels at the end of one row (i, W) and the beginning of the next row $(i + 1, 1)$ appear adjacent in 1D index space ($|k_1 - k_2| = 1$), despite residing on opposite physical edges of the visual canvas!

- **Loss of Locality and Spatial Adjacency**: Natural visual semantics emerge entirely from local spatial coherence: adjacent pixels share illumination and material properties. By discarding the 2D grid, an MLP eliminates the concept of a spatial neighborhood $\mathcal{N}(i, j)$, treating a pixel's relationship with its adjacent neighbor identically to its relationship with a distant pixel hundreds of scanlines away.

- **The Permutation Invariance Flaw**: Because a dense weight matrix $\mathbf{W} \in \mathbb{R}^{M \times d}$ assigns an unconstrained independent parameter $W_{m,k}$ to every coordinate k, an MLP possesses **zero spatial inductive bias**. To demonstrate this formally, let $\mathbf{P}_\pi \in \{0, 1\}^{d \times d}$ be an arbitrary fixed permutation matrix that randomly shuffles the d pixel indices, transforming every image into humanly unrecognizable white noise static. If we simultaneously permute the columns of the weight matrix by $\mathbf{W}' = \mathbf{W}\mathbf{P}_\pi^\top$, then for any scrambled input $\mathbf{x}' = \mathbf{P}_\pi \mathbf{x}$:

  $$\mathbf{W}'\mathbf{x}' = (\mathbf{W}\mathbf{P}_\pi^\top)(\mathbf{P}_\pi \mathbf{x}) = \mathbf{W}(\mathbf{P}_\pi^\top \mathbf{P}_\pi)\mathbf{x} = \mathbf{W}\mathbf{x}. \tag{5.6}$$

  The MLP produces the *exact same forward pre-activations, the exact same backpropagated gradients, and the exact same loss trajectory* on scrambled white noise as it does on structured natural images! An unconstrained dense network is mathematically incapable of distinguishing a photographic scene from scrambled tabular numbers.

Zero parameter sharing and lack of translation equivariance. In an unconstrained dense layer, every hidden neuron m connects to every input coordinate $k = (i, j, c)$ via a dedicated weight parameter $W_{m,(i,j,c)}$:

$$z_m = \sum_{c=1}^{C} \sum_{i=1}^{H} \sum_{j=1}^{W} W_{m,(i,j,c)} X_{i,j,c} + b_m. \tag{5.7}$$

This formulation introduces two fatal limitations:

- **Coordinate-Specific Parameter Binding (Zero Weight Sharing)**: In Equation (5.7), the weight $W_{m,(i,j,c)}$ is bound to the absolute spatial coordinate (i, j) on the canvas. There is **zero parameter sharing**: the weight that examines coordinate $(10, 10)$ shares no algebraic relationship with the weight examining coordinate $(150, 150)$. To share parameters across spatial locations, the matrix $\mathbf{W}$ would have to be constrained to have a doubly block-circulant (or Toeplitz) structure—which unconstrained MLPs lack entirely.

- **Breakdown of Translation Equivariance**: Physical objects satisfy translational symmetry: an object retains its identity regardless of where it appears on the canvas. Mathematically, a representation mapping $f : \mathcal{X} \to \mathcal{H}$ is **translation-equivariant** with respect to spatial shift $T_{\mathbf{v}}$ if:

$$f(T_{\mathbf{v}}(\mathbf{X})) = T_{\mathbf{v}}(f(\mathbf{X})), \quad \forall \mathbf{v} \in \mathbb{Z}^2. \tag{5.8}$$

  In an MLP, because weights are rigidly tied to absolute pixel coordinates, $f(T_{\mathbf{v}}(\mathbf{x})) \neq T_{\mathbf{v}}(f(\mathbf{x}))$. If a hidden neuron learns a set of weights that detects an eye at coordinates $(10, 10)$, and in a new test image the face shifts so the eye appears at $(150, 150)$, the neuron at $(10, 10)$ fails to activate, and the neuron at $(150, 150)$ has never observed an eye during training! The network possesses zero ability to transfer learned patterns across spatial coordinates.

- **Exponential Sample Complexity**: Because an MLP cannot share parameters across space, it cannot generalize across translations. To learn that an object can appear anywhere on the canvas, the MLP must observe that object at **every possible spatial coordinate** (i, j) **during training**. This multiplies the required training dataset size by a factor proportional to the spatial resolution $H \times W$, making sample-efficient visual learning impossible.

Catastrophic parameter explosion and generalization failure. The combination of all-to-all dense connectivity and zero parameter sharing inevitably triggers a catastrophic parameter explosion:

$$N_{\text{weights}} = d_{\text{in}} \times d_{\text{out}} = (H \cdot W \cdot C) \times M, \tag{5.9}$$

where M is the number of neurons in the first hidden layer.

- **Small Resolution (CIFAR-10)**: For a tiny $32 \times 32 \times 3$ thumbnail ($d = 3,072$), connecting to a hidden layer of $M = 1,024$ neurons requires $3,072 \times 1,024 \approx 3.15 \times 10^6$ parameters.
- **Medium Resolution ($256 \times 256 \times 3$)**: Here, $d = 196,608$. Connecting to a modest hidden layer of $M = 2,048$ neurons requires:

$$196,608 \times 2,048 = 402,653,184 \text{ parameters (over 402 million weights!)} \tag{5.10}$$

  for the **first layer alone**! Storing this single weight matrix in 32-bit floating point precision consumes over 1.61 GB of VRAM.
- **High Resolution (Full HD $1080 \times 1920 \times 3$)**: $d \approx 6.22 \times 10^6$. A single hidden layer of $2,048$ neurons would require over 1.27×10^{10} parameters (12.7 billion weights, ~ 50 GB of memory).

By Vapnik-Chervonenkis (VC) generalization theory (Chapter 2.5.5), the sample complexity required to prevent overfitting scales linearly with the number of free parameters ($N \propto \mathcal{O}(|\boldsymbol{\theta}|)$). Training an unconstrained model with hundreds of millions of parameters on typical dataset sizes ($N \sim 10^4$–10^6) guarantees catastrophic overfitting and failure to generalize.

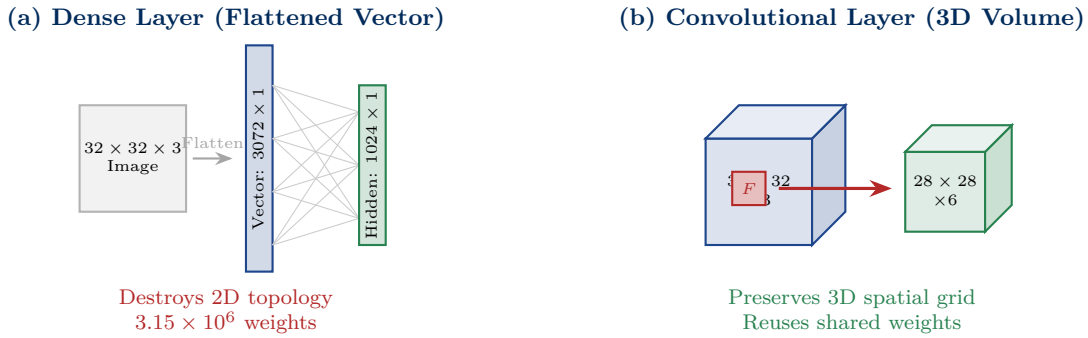


Figure 5.2: The structural contrast between dense and convolutional representations. (a) A dense fully connected layer flattens an image into an unrolled 1D vector, destroying all 2D spatial adjacency and requiring an independent weight for every pixel-neuron pair. (b) A convolutional layer preserves the native 3D tensor volume, sliding localized spatial filters across coordinates with shared parameters.

The catastrophic failure of dense MLPs on images demonstrates an inescapable structural truth: when unconstrained models ignore 2D spatial geometry, parameter explosions and lack of translation equivariance render sample-efficient generalization impossible. This failure motivates the mathematical construction of operators that inherently preserve 2D grid topology while drastically constraining free parameters via local receptive fields and weight tying: **discrete 2D convolution and cross-correlation**.

5.2 Convolution mechanics and geometry

To overcome the structural collapse of dense Multi-Layer Perceptrons on visual data, machine learning must operationalize the spatial physics of natural photographic scenes. The foundational mathematical engine for this task is the convolution operator. Tracing its lineage from functional analysis and continuous differential systems to discrete tensor mechanics, convolution transforms spatial filtering into a localized, sliding affine transformation that respects the spatial metric of continuous physical space.

Unlike unconstrained matrix multiplication—which binds an independent parameter to every absolute coordinate on the canvas—a convolutional filter slides uniformly across the pixel lattice, evaluating inner products over compact receptive fields while sharing an identical kernel tensor across every spatial position. By decoupling filter parameters from image resolution, convolution guarantees translation equivariance while slashing model complexity. Furthermore, the geometric interaction between kernel size, stride, boundary zero-padding modes (valid versus same), and dilation factors dictates the spatial downsampling trajectory and governs boundary information loss across cascading layers.

5.2.1 Continuous and discrete convolution

To establish the theoretical foundations of convolutional neural networks, we examine the convolution operation from a physical and mathematical perspective, tracking how its formulation adapts across dimensions and transitions from continuous functional analysis to discrete computation [6, 32].

The general concept of convolution. At its core, convolution is a mathematical operator that expresses the degree of overlap between two functions as one slides across the other, computing a localized, weighted moving average. To understand its physical intuition, consider tracking the position of a spacecraft using an onboard rangefinder [32]. At continuous time t,

the sensor emits a noisy measurement $x(t)$. To obtain an accurate trajectory estimate $s(t)$, we take a running weighted average of historical readings. Because recent measurements are more reliable than older ones, we apply a weighting kernel $w(a)$, where $a \geq 0$ represents the age of the measurement. Since a measurement acquired at time $\tau \leq t$ has age $a = t - \tau$, weighting each reading $x(\tau)$ by $w(t - \tau)$ and integrating over all history yields the smoothed trajectory $s(t) = \int x(\tau)w(t - \tau)\, d\tau$.

Continuous formulations across dimensions. The continuous convolution operator generalizes across spatial dimensions by extending the domain of integration:

- **One-Dimensional (1D) Convolution**: For continuous time-series or 1D signals $x, w : \mathbb{R} \to \mathbb{R}$:

$$s(t) = (x * w)(t) \triangleq \int_{-\infty}^{\infty} x(\tau)w(t - \tau)\, d\tau. \tag{5.11}$$

  Mechanically, this operation consists of four steps: reflecting the kernel $w(-\tau)$, translating it by offset t to obtain $w(t - \tau)$, computing the pointwise product $x(\tau)w(t - \tau)$, and integrating across the real line.

- **Two-Dimensional (2D) Convolution**: For continuous 2D planar signals (such as continuous optical wavefields or continuous image planes) $I, K : \mathbb{R}^2 \to \mathbb{R}$, integration extends across both spatial axes:

$$S(x, y) = (I * K)(x, y) \triangleq \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} I(\tau_x, \tau_y)K(x - \tau_x,\, y - \tau_y)\, d\tau_x\, d\tau_y. \tag{5.12}$$

- **Three-Dimensional (3D) Convolution**: For continuous volumetric signals $I, K : \mathbb{R}^3 \to \mathbb{R}$, integration extends across three continuous coordinates:

$$S(x, y, z) = (I * K)(x, y, z) \triangleq \iiint I(\tau_x, \tau_y, \tau_z)K(x - \tau_x,\, y - \tau_y,\, z - \tau_z)\, d\tau_x\, d\tau_y\, d\tau_z. \tag{5.13}$$

  This formulation governs volumetric physical domains, such as 3D medical tomography (MRI, CT scans), volumetric point-cloud grids, or spatio-temporal video volumes where the third axis represents time.

The discrete formulation and deep learning cross-correlation. When processing digital signals on computers, continuous functions are sampled onto discrete coordinate lattices, and continuous integrals are replaced by discrete summations:

- **Discrete 1D Convolution**: For sampled 1D arrays $x, w : \mathbb{Z} \to \mathbb{R}$:

$$s[n] = (x * w)[n] \triangleq \sum_{\tau=-\infty}^{\infty} x[\tau]w[n - \tau]. \tag{5.14}$$

- **Discrete 2D Convolution**: For sampled 2D matrices $\mathbf{I} \in \mathbb{R}^{H \times W}$ and discrete filter kernels $\mathbf{K} \in \mathbb{R}^{F \times F}$:

$$S(i, j) = (\mathbf{I} * \mathbf{K})(i, j) \triangleq \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} \mathbf{I}(m, n)\mathbf{K}(i - m,\, j - n). \tag{5.15}$$

- **Discrete 2D Cross-Correlation in Deep Learning**: While formal mathematical convolution requires reflecting the kernel indices $(i-m, j-n)$, modern deep learning libraries (PyTorch, TensorFlow, JAX) omit the reflection step and compute **discrete cross-correlation**:

$$S(i, j) = (\mathbf{I} \star \mathbf{K})(i, j) \triangleq \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} \mathbf{I}(i + m,\, j + n)\mathbf{K}(m, n). \tag{5.16}$$

In classical signal processing with fixed handcrafted filters, kernel reflection is necessary to satisfy commutativity. In deep learning, however, the kernel weights $\mathbf{K}(m, n)$ are free learnable parameters initialized randomly and optimized via backpropagation. If a task requires a reflected filter, gradient descent simply learns the reflected parameter values directly:

$$\min_{\mathbf{K}} \mathcal{L}(\mathbf{I} * \mathbf{K}) \equiv \min_{\mathbf{K}'} \mathcal{L}(\mathbf{I} \star \mathbf{K}'), \quad \text{where } \mathbf{K}'(m, n) = \mathbf{K}(-m, -n). \tag{5.17}$$

Omitting kernel reflection eliminates unnecessary array index flips on GPUs with zero impact on expressive capacity. Throughout machine learning, Equation (5.16) is referred to interchangeably as convolution.

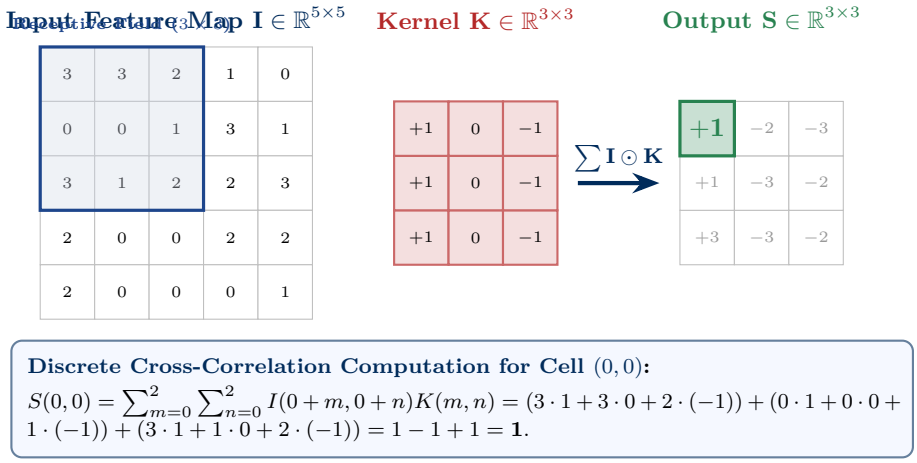


Figure 5.3: Mechanics of 2D discrete cross-correlation in deep learning. A 3×3 sliding kernel $\mathbf{K}$ traverses the input feature map $\mathbf{I} \in \mathbb{R}^{5\times5}$ with unit stride ($S = 1$) and valid padding ($P = 0$). At each spatial location, the element-wise Hadamard product between the kernel weights and the local receptive field is summed to produce the corresponding scalar activation in output feature map $\mathbf{S} \in \mathbb{R}^{3\times3}$.

5.2.2 Stride, padding, and layer geometry

We now formalize the full multi-channel convolutional layer as deployed in modern deep networks.

Input and filter tensor volumes. In deep neural networks, activations are represented as multi-dimensional tensor volumes:

- The input is a 3D tensor volume $\mathbf{X} \in \mathbb{R}^{H_{\text{in}} \times W_{\text{in}} \times C_{\text{in}}}$, where H_{in} and W_{in} denote spatial height and width, and C_{in} denotes the channel depth (e.g., $C_{\text{in}} = 3$ for RGB color images).
- The layer contains a bank of C_{out} distinct convolutional filters.
- **Fundamental Depth Rule**: **Each filter kernel must span the ENTIRE depth C_{in} of the input volume!** While a filter has a local spatial receptive field of size $F \times F$, its volumetric shape is:

$$\mathbf{K}^{(k)} \in \mathbb{R}^{F \times F \times C_{\text{in}}}, \quad \forall k \in \{1, \ldots, C_{\text{out}}\}. \tag{5.18}$$

Each filter $\mathbf{K}^{(k)}$ computes an element-wise dot product across all C_{in} channels simultaneously over a local spatial window, adds a scalar bias $b_k \in \mathbb{R}$, and emits a single 2D spatial activation slice $\mathbf{Y}_{:,:,k} \in \mathbb{R}^{H_{\text{out}} \times W_{\text{out}}}$:

$$\mathbf{Y}_{i,j,k} = \sum_{c=1}^{C_{\text{in}}} \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} \mathbf{X}_{i+m,\, j+n,\, c} \, \mathbf{K}^{(k)}_{m,n,c} + b_k. \tag{5.19}$$

Stacking the resulting C_{out} feature slices along the depth axis yields the complete output tensor volume $\mathbf{Y} \in \mathbb{R}^{H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}}$.

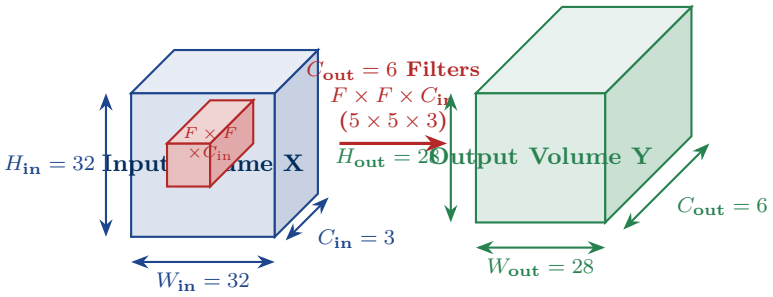


Figure 5.4: The volumetric geometry of multi-channel convolution. An input tensor volume $\mathbf{X} \in \mathbb{R}^{H_{\text{in}} \times W_{\text{in}} \times C_{\text{in}}}$ $(32 \times 32 \times 3)$ is convolved by a bank of $C_{\text{out}} = 6$ distinct filters. Each individual filter spans the complete depth of the input volume with shape $F \times F \times C_{\text{in}}$ $(5 \times 5 \times 3)$, emitting a single 2D feature slice. Stacking the C_{out} activation slices yields the output volume $\mathbf{Y} \in \mathbb{R}^{H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}}$ $(28 \times 28 \times 6)$.

Hyperparameters: Filter size, stride, and padding. The spatial dimensions H_{out} and W_{out} of the output volume are governed by three primary geometric hyperparameters:

- **Filter Size (F)**: The spatial extent (height and width) of the local kernel window $F \times F$ (typically odd integers: $F \in \{3, 5, 7\}$).
- **Stride (S)**: The step displacement in pixels by which the kernel advances across the input lattice. Unit stride $S = 1$ evaluates all adjacent positions; stride $S = 2$ shifts by two pixels, skipping every second position and halving spatial resolution.
- **Zero-Padding (P)**: The width of the symmetrical border of zero-valued pixels appended around the perimeter of the input feature map, controlling spatial downsampling and preventing boundary information loss.

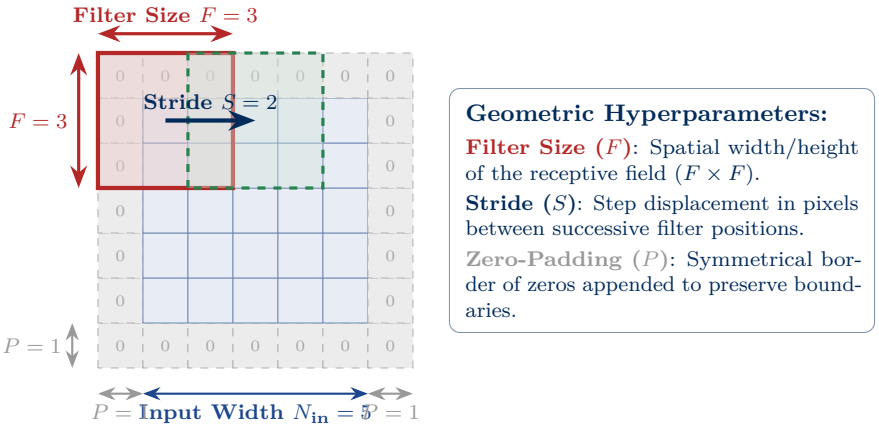


Figure 5.5: Visual illustration of convolutional spatial hyperparameters: filter size F, stride S, and zero-padding P. The kernel of spatial size $F \times F$ (red solid box) traverses the zero-padded input grid (dashed border of width P), advancing by step displacement S (green dashed box) at each convolution step.

Theorem 5.1 (Universal spatial dimension formula)**.** *Given an input spatial dimension N_{in} (height or width), filter spatial size F, zero-padding P, and stride S, the output spatial dimension*

N_{out} is given by:

$$N_{out} = \left\lfloor \frac{N_{in} - F + 2P}{S} \right\rfloor + 1. \tag{5.20}$$

Example 1 (Unit stride with valid padding): Consider an input feature map of size $N_{\text{in}} = 7$, convolved by a filter of size $F = 3$ with unit stride $S = 1$ and no padding ($P = 0$). Evaluating Equation (5.20) gives:

$$N_{\text{out}} = \frac{7 - 3 + 2(0)}{1} + 1 = \frac{4}{1} + 1 = 5 \implies \text{Output is } 5 \times 5. \tag{5.21}$$

Example 2 (Spatial downsampling via strided convolution): Setting stride $S = 2$ on the same input ($N_{\text{in}} = 7, F = 3, P = 0$) shifts the kernel by two pixels at each step, skipping alternate coordinates:

$$N_{\text{out}} = \frac{7 - 3 + 2(0)}{2} + 1 = \frac{4}{2} + 1 = 3 \implies \text{Output is } 3 \times 3. \tag{5.22}$$

Example 3 (Hyperparameter stride compatibility condition): If we select stride $S = 3$ with $N_{\text{in}} = 7, F = 3, P = 0$:

$$\frac{7 - 3 + 0}{3} + 1 = \frac{4}{3} + 1 \approx 2.33 \implies \text{Fractional! The filter does NOT tile the grid uniformly without clipping!} \tag{5.23}$$

In well-formed convolutional designs, hyperparameters must strictly satisfy the integer divisibility condition $(N_{\text{in}} - F + 2P) \pmod S = 0$.

Example 4 (Preserving spatial dimensions via same padding): Without zero-padding ($P = 0$, valid padding), each convolutional layer shrinks the spatial grid by $F - 1$ pixels, causing rapid spatial collapse in deep architectures. To maintain identical input and output dimensions ($N_{\text{out}} = N_{\text{in}}$) under unit stride $S = 1$, we enforce:

$$N_{\text{in}} = \frac{N_{\text{in}} - F + 2P}{1} + 1 \implies 2P = F - 1 \implies P = \frac{F - 1}{2}. \tag{5.24}$$

For standard 3×3 filters ($F = 3$), setting $P = (3-1)/2 = 1$ perfectly preserves spatial resolution ("Same" padding). For 5×5 filters ($F = 5$), setting $P = 2$ achieves the same dimensional preservation.

Parameter and FLOP counting. Let a convolutional layer process input $\mathbf{X} \in \mathbb{R}^{H_{\text{in}} \times W_{\text{in}} \times C_{\text{in}}}$ with C_{out} filters of size $F \times F \times C_{\text{in}}$, stride S, and padding P:

- **Total Learnable Parameters**: Each filter contains $F \times F \times C_{\text{in}}$ kernel weights plus 1 scalar bias. For C_{out} filters:

  $$N_{\text{params}} = (F \times F \times C_{\text{in}} + 1) \times C_{\text{out}}. \tag{5.25}$$

  Notice that N_{params} is completely independent of the input spatial resolution $H_{\text{in}}, W_{\text{in}}$.

- **Total Floating-Point Multiply-Accumulate Operations (FLOPs)**: Computing each scalar output activation requires $(F \times F \times C_{\text{in}})$ multiplications and additions. Emitting the full output volume $H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}}$ requires:

  $$N_{\text{FLOPs}} = 2 \times H_{\text{out}} \times W_{\text{out}} \times (F \times F \times C_{\text{in}}) \times C_{\text{out}}. \tag{5.26}$$

Example 5 (Parameter and FLOP calculation): Consider an input volume of size $32 \times 32 \times 3$ convolved by $C_{\text{out}} = 10$ filters of size 5×5, with stride $S = 1$ and padding $P = 2$:

- Spatial Output: $H_{\text{out}} = W_{\text{out}} = \frac{32-5+2(2)}{1} + 1 = 32 \implies$ Output volume is $32 \times 32 \times 10$.
- Parameters: $(5 \times 5 \times 3 + 1) \times 10 = (75 + 1) \times 10 = 760$ parameters.

An MLP performing the identical mapping from $3,072$ inputs to $32 \times 32 \times 10 = 10,240$ outputs would require $3,072 \times 10,240 \approx 31.4$ million parameters! Convolution achieves a $41,000\times$ **reduction in parameters** while maintaining exact translation equivariance.

While the mechanics and geometry of convolution explain how multi-channel tensors are transformed with spatial parameter sharing, a fundamental theoretical question remains: why does this specific architectural constraint succeed so profoundly in empirical vision, and how can we understand it through Bayesian and biological principles? This question motivates examining convolutional neural networks as an infinitely strong spatial prior rooted in the functional architecture of the mammalian visual cortex.

5.3 CNN priors and structural design

In the mathematical analysis of deep learning, a pervasive misconception treats convolutional networks as merely an empirical engineering heuristic—a clever computational shortcut designed to reduce parameter counts on GPUs. As Yoshua Bengio, Ian Goodfellow, and Aaron Courville established in their foundational work, convolutional networks are deeply grounded in Bayesian probability theory: a CNN is an unconstrained fully connected network upon which an *infinitely strong spatial prior* has been imposed. This prior assigns absolute zero probability mass to any linear transformation that violates spatial locality or translation equivariance, permanently restricting parameter search to a low-dimensional manifold of shared local weights.

Understanding convolutional networks through this Bayesian lens clarifies both their immense empirical power and their theoretical sample complexity. In the absence of an inductive prior, an unconstrained Multi-Layer Perceptron must observe every visual object at every possible spatial coordinate on the canvas to learn translation invariance, scaling sample complexity with image resolution $\Omega(H \cdot W \cdot N_0)$. By hardcoding spatial locality and shift equivariance into the network topology, the convolutional prior slashes sample complexity to $\mathcal{O}(N_0)$.

5.3.1 Sparsity, parameter sharing, and equivariance

In their foundational treatise *Deep Learning*, Ian Goodfellow, Yoshua Bengio, and Aaron Courville [32] establish that convolutional networks improve upon traditional dense feedforward neural architectures by operationalizing three fundamental mathematical ideas: sparse interactions (sparse connectivity), parameter sharing (tied weights), and equivariant representations.

Sparse interactions. Traditional feedforward neural network layers compute representations using unconstrained matrix multiplication:

$$\mathbf{y} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}), \tag{5.27}$$

where every individual output unit interacts with every single input unit via an independent weight parameter $W_{i,j}$. If the input has m units and the output has n units, the transformation requires computing and storing an $n \times m$ matrix, with computational runtime scaling as $\mathcal{O}(m \times n)$.

In stark contrast, Convolutional Neural Networks guarantee sparse interactions by restricting the filter kernel size F to be orders of magnitude smaller than the input spatial dimensions ($F \ll H, W$). Rather than connecting an output unit to all $H \times W$ input pixels, each output unit connects exclusively to a local contiguous receptive field of $F \times F$ pixels (or F taps in 1D), strictly restricting non-zero linear interactions to a localized band of width F.

Crucially, although direct connections at each individual convolutional layer are strictly sparse, units in deeper layers of a multi-layer CNN interact indirectly with a much larger portion of the input visual grid. If each layer uses a 1D filter of width F, a unit in layer l observes an effective receptive field:

$$\mathrm{ERF}_l = l(F-1) + 1 \tag{5.28}$$

input units. By stacking multiple sparsely connected convolutional layers, a deep network allows high-level neurons to synthesize complex, non-local semantic relationships across the entire image canvas while keeping every individual layer's computation strictly localized and memory-efficient.

To demonstrate the profound computational advantage of sparse interactions, Goodfellow, Bengio, and Courville [32] provide a canonical quantitative benchmark: detecting edges in a digital image of size 320×280 pixels, totaling $D = 320 \times 280 = 89,600$ pixels, to produce an output feature map of equivalent dimensions ($89,600$ units).

- *Dense Fully Connected Layer*: An unconstrained dense weight matrix $\mathbf{W} \in \mathbb{R}^{89,600 \times 89,600}$ requires storing:

$$|\mathbf{W}| = (320 \times 280) \times (320 \times 280) \approx 8.028 \times 10^9 \text{ parameters } (> 8 \text{ billion entries}). \tag{5.29}$$

  In standard 32-bit floating point precision (4 bytes per float), storing $\mathbf{W}$ requires:

$$8.028 \times 10^9 \times 4 \text{ bytes} \approx 32.11 \times 10^9 \text{ bytes} \approx 32.1 \text{ GB of RAM}, \tag{5.30}$$

  just to store the parameters of a single layer, far exceeding the memory bandwidth of high-performance GPUs. Evaluating the matrix-vector multiplication requires:

$$2 \times (89,600)^2 \approx 1.606 \times 10^{10} \text{ FLOPs } (\approx 16 \text{ billion operations}). \tag{5.31}$$

- *Convolutional Kernel Layer*: Performing edge detection with a compact 2-element difference kernel $\mathbf{k} = [-1, 1]^\top$ requires storing strictly:

$$|\mathbf{k}| = 2 \text{ parameters } (8 \text{ bytes}). \tag{5.32}$$

  Evaluating the convolution across all $(320 - 1) \times 280 = 89,320$ valid output positions requires:

$$89,320 \times 2 \approx 1.786 \times 10^5 \text{ FLOPs } (\approx 2.68 \times 10^5 \text{ FLOPs including bias additions}). \tag{5.33}$$

Comparing the two implementations reveals that the convolutional approach is over $60,000$ times faster computationally and requires over 4 billion times less parameter storage.

Parameter sharing. The second foundational principle articulated by Bengio is parameter sharing (also known as tied weights): using the exact same parameter value across multiple spatial positions in a network. In a conventional dense MLP, each element $W_{i,j}$ of the weight matrix is used exactly once when multiplying the j-th input unit to compute the i-th output unit. In a CNN, each element of the filter kernel $\mathbf{K} \in \mathbb{R}^{F \times F}$ is applied across every single receptive field throughout the entire input grid.

While parameter sharing does not change the runtime scaling of forward propagation ($\mathcal{O}(k \times n)$ operations are still performed across the spatial grid), it dramatically reduces the number of unique parameters that must be stored in memory to $\mathcal{O}(F^2 C_{\text{in}} C_{\text{out}})$, which is completely independent of the input image dimensions $H \times W$.

Crucially, parameter sharing provides an enormous boost in statistical efficiency. In a dense MLP, a weight connecting to pixel $(10, 10)$ only learns from patterns presented at pixel $(10, 10)$ during training. If a visual feature appears at coordinate $(200, 200)$, the weights at $(10, 10)$ receive identically zero gradient feedback. In a CNN, because the filter $\mathbf{K}$ is shared across all spatial coordinates, every pixel patch across every training image contributes a gradient update to the exact same parameter tensor. If an input image contains $100 \times 100 = 10,000$ receptive fields, a single image provides $10,000$ distinct gradient signals to update $\mathbf{K}$, dramatically slashing the number of training samples required to learn robust feature detectors.

Equivariant representations. The third foundational motivation of CNNs is the emergence of equivariant representations. Formally, a mathematical mapping f is equivariant to a transformation g if:

$$f(g(x)) = g(f(x)). \tag{5.34}$$

Applying the transformation g to the input x and then evaluating f produces the identical result as evaluating f first and then applying g to the representation.

Visual perception imposes two distinct transformation symmetries that must not be conflated:

1. *Translation Invariance*: A mapping $f : \mathcal{X} \to \mathcal{Y}$ is translation-invariant under spatial shift $\mathcal{T}_{\boldsymbol{\delta}}$ if:
$$f\left(\mathcal{T}_{\boldsymbol{\delta}}\mathbf{X}\right) = f(\mathbf{X}), \quad \forall \boldsymbol{\delta} = (\Delta x, \Delta y) \in \mathbb{Z}^2. \tag{5.35}$$
Translation invariance is the mathematically required property for whole-image categorical classification: whether an object sits in the top-left or bottom-right corner of the canvas, its semantic category label (*"Cat"*) must remain unchanged.

2. *Translation Equivariance (Covariance)*: A mapping $f : \mathcal{X} \to \mathcal{Z}$ is translation-equivariant under spatial shift $\mathcal{T}_{\boldsymbol{\delta}}$ if:
$$f\left(\mathcal{T}_{\boldsymbol{\delta}}\mathbf{X}\right) = \mathcal{T}_{\boldsymbol{\delta}}\left[f(\mathbf{X})\right], \quad \forall \boldsymbol{\delta} \in \mathbb{Z}^2. \tag{5.36}$$
Translation equivariance is indispensable for internal feature representations, dense localization, and semantic segmentation: if an object moves three pixels to the right in the visual field, all extracted feature activations in the convolutional feature map must shift by exactly three pixels to preserve physical spatial geometry.

To understand why parameter sharing and translation equivariance cannot simply be learned by dense MLPs, consider an MLP processing a flattened image vector $\mathbf{x} = \mathrm{vec}(\mathbf{X}) \in \mathbb{R}^D$ with $D = H \cdot W$. A fully connected layer computes hidden activations via an unconstrained dense weight matrix $\mathbf{W} \in \mathbb{R}^{D_{\text{out}} \times D}$:

$$h_p = \sigma\left(\sum_{i=1}^{H}\sum_{j=1}^{W} W_{p,(i,j)} X_{i,j} + b_p\right). \tag{5.37}$$

Notice that every input coordinate (i, j) is governed by a completely independent, decoupled parameter $W_{p,(i,j)}$.

Suppose a visual pattern $\mathcal{P}$ (such as an eye or a sharp vertical edge) is presented at spatial coordinate (i_1, j_1) during training. Backpropagation calculates the gradient of the objective $\mathcal{L}$ with respect to the weights connected to that specific coordinate:

$$\frac{\partial \mathcal{L}}{\partial W_{p,(i_1,j_1)}} = \delta_p X_{i_1,j_1} \neq 0. \tag{5.38}$$

However, for any other spatial coordinate $(i_2, j_2) \neq (i_1, j_1)$ where the pattern was not presented, the local input is zero or background, yielding:

$$\frac{\partial \mathcal{L}}{\partial W_{p,(i_2,j_2)}} = 0. \tag{5.39}$$

The weights connected to coordinate (i_2, j_2) receive identically zero gradient updates. Consequently, if the object appears at position (i_2, j_2) at test time, the MLP possesses zero ability to recognize it.

To learn translation equivariance empirically, the MLP must observe every visual concept $\mathcal{P}$ instantiated at every coordinate in the spatial translation group $\mathcal{T} = \{(\Delta x, \Delta y) \mid 0 \leq \Delta x <$

$W,\ 0 \le \Delta y < H\}$. Because the cardinality of this group is $|\mathcal{T}| = H \times W$, the required training sample complexity scales as:

$$N_{\text{MLP}} = \Omega\left(|\mathcal{T}| \cdot N_0\right) = \Omega\left(H \cdot W \cdot N_0\right), \tag{5.40}$$

where N_0 is the sample complexity required to learn the pattern at a single fixed location. For a standard 256×256 input image, $|\mathcal{T}| = 65,536$: an MLP requires up to a $65,000\times$ larger dataset purely to brute-force learn translation invariance through exhaustive spatial data augmentation.

In contrast, a Convolutional Neural Network resolves this dilemma by hardcoding translation equivariance directly into its algebraic structure through parameter sharing. Convolving input $\mathbf{X}$ with shared kernel $\mathbf{K} \in \mathbb{R}^{F \times F}$ guarantees commutativity with translation by algebraic identity:

$$\begin{aligned}
\left[(\mathcal{T}_{\boldsymbol{\delta}}\mathbf{X}) * \mathbf{K}\right](i,j) &= \sum_{m=0}^{F-1}\sum_{n=0}^{F-1} (\mathcal{T}_{\boldsymbol{\delta}}\mathbf{X})(i+m,\, j+n)\, K_{m,n} \\
&= \sum_{m=0}^{F-1}\sum_{n=0}^{F-1} \mathbf{X}(i-\Delta x+m,\, j-\Delta y+n)\, K_{m,n} \\
&= (\mathbf{X} * \mathbf{K})(i-\Delta x,\, j-\Delta y) = \mathcal{T}_{\boldsymbol{\delta}}\left[(\mathbf{X} * \mathbf{K})\right](i,j).
\end{aligned} \tag{5.41}$$

Because Equation (5.41) holds for any kernel values $\mathbf{K}$, translation equivariance is strictly hardcoded prior to observing any training data. Observing a visual feature at a single location (i_1, j_1) produces a gradient that updates the shared kernel $\mathbf{K}$; this single parameter update immediately generalizes detection across all $H \times W$ locations simultaneously:

$$N_{\text{CNN}} = \mathcal{O}(N_0). \tag{5.42}$$

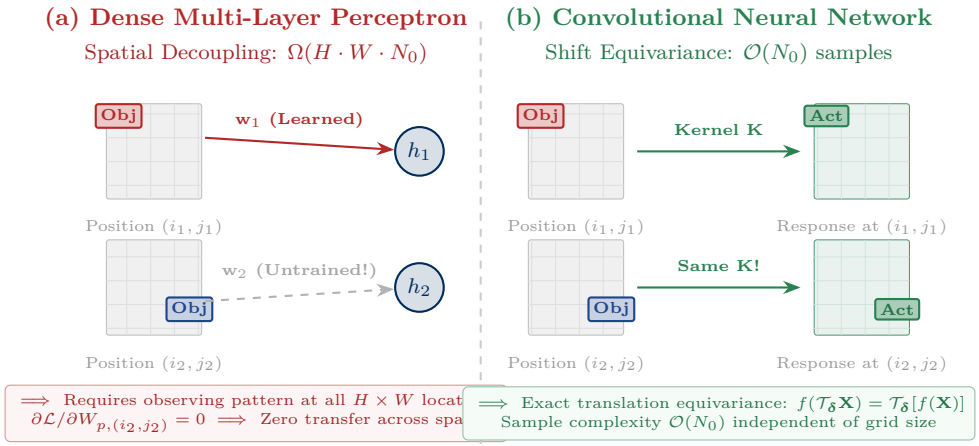


Figure 5.6: Sample complexity contrast between dense MLPs and Convolutional Networks under 2D spatial translation. (a) In an MLP, disjoint weights connect to each spatial pixel; learning a visual concept at position (i_1, j_1) fails to generalize to position (i_2, j_2). (b) In a CNN, parameter sharing ties all spatial positions to identical kernel weights $\mathbf{K}$, guaranteeing exact translation equivariance and sample complexity scaling independent of grid resolution.

Bengio's caveat and locally connected layers. Goodfellow, Bengio, and Courville [32] highlight a critical architectural caveat: parameter sharing across space is beneficial if and only if the underlying data-generating distribution is spatially stationary—that is, when features useful at one location are equally useful at all other locations. While this assumption holds universally for unconstrained natural images, it is fundamentally suboptimal when images possess a fixed, canonical spatial composition.

- *The Canonical Example of Aligned Faces*: Consider a biometric dataset of tightly cropped, pre-aligned human faces (such as LFW or CelebA). In these images, eyes are consistently positioned in the upper quadrant, noses reside in the center, and mouths are located in the lower quadrant. There is no reason to search for an eye in the bottom corner or a mouth at the top of the canvas.

- *The Unshared Locally Connected Solution*: Forced weight sharing across the entire grid forces the network to compromise between detecting eyes and detecting mouths using the same shared filters. In such canonically framed domains, unshared locally connected layers (also known as tiled convolutions without weight sharing) are theoretically and empirically superior: connections remain spatially localized to $F \times F$ patches (preserving sparse interactions), but different spatial positions (i, j) learn completely separate, decoupled weight matrices $\mathbf{K}_{(i,j)}$.

5.3.2 Convolution as an infinitely strong prior

To attain the deepest theoretical understanding of why Convolutional Neural Networks succeed—and precisely when they fail—we must examine their architecture through the lens of Bayesian probability theory and statistical learning theory. In Chapter 9.4 of *Deep Learning*, Ian Goodfellow, Yoshua Bengio, and Aaron Courville [32] introduce a unifying perspective: convolution and pooling can be understood as an infinitely strong prior on the weights of an unconstrained fully connected layer.

Prior probability distributions in Bayesian learning. In Bayesian parameter estimation, a prior probability distribution $p(\boldsymbol{\theta})$ over the model parameters $\boldsymbol{\theta}$ encodes domain assumptions about parameter values before observing training data $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ [6, 32]. Through Bayes' rule, the posterior distribution over parameters is:

$$p(\boldsymbol{\theta} \mid \mathcal{D}) \propto p(\mathcal{D} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta}) = \left[\prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i, \boldsymbol{\theta})\right] p(\boldsymbol{\theta}). \tag{5.43}$$

In Maximum A Posteriori (MAP) estimation, the optimal parameter vector maximizes the log-posterior:

$$\boldsymbol{\theta}_{\text{MAP}} = \arg\max_{\boldsymbol{\theta}} \left[\sum_{i=1}^{N} \log p(y_i \mid \mathbf{x}_i, \boldsymbol{\theta}) + \log p(\boldsymbol{\theta})\right]. \tag{5.44}$$

Priors exist along a continuum of strength determined by their probability entropy:

- *Weak Prior*: High variance and high entropy (such as a Gaussian prior $\mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$ with large σ^2), exerting minimal regularization and allowing empirical data to drive parameter values freely.

- *Strong Prior*: Low variance and low entropy (such as a Gaussian prior with small σ^2), heavily penalizing parameters that deviate from the prior mean.

- *Infinitely Strong Prior*: Places zero probability mass ($p(\boldsymbol{\theta}) = 0$) on certain parameter configurations, permanently forbidding them regardless of empirical evidence. It restricts the parameter space strictly to a lower-dimensional submanifold $\mathcal{M} \subset \mathbb{R}^P$:

$$p(\boldsymbol{\theta}) = \begin{cases} \phi(\boldsymbol{\theta}) & \text{if } \boldsymbol{\theta} \in \mathcal{M}, \\ 0 & \text{if } \boldsymbol{\theta} \notin \mathcal{M}. \end{cases} \tag{5.45}$$

  If $\boldsymbol{\theta} \notin \mathcal{M}$, its posterior probability $p(\boldsymbol{\theta} \mid \mathcal{D})$ remains identically zero for any dataset, even as $N \to \infty$.

Convolution and pooling as infinitely strong priors. Under Bengio's formulation, a convolutional layer can be conceptualized as a standard unconstrained fully connected layer $\mathbf{y} = \mathbf{W}\mathbf{x}$ upon which an infinitely strong spatial prior is imposed [32]. This prior asserts two structural constraints with absolute mathematical certainty:

1. *Hard Receptive Field Sparsity*: A unit at spatial coordinate (i, j) in layer l can only interact with units in layer $l-1$ within its local spatial neighborhood $\Omega(i, j)$. For all connections outside this receptive field, the prior assigns a Dirac delta distribution centered at zero:

$$p(W_{(i,j),(i',j')}) = \delta(W_{(i,j),(i',j')}), \quad \forall (i', j') \notin \Omega(i, j), \tag{5.46}$$

   forcing $W_{(i,j),(i',j')} \equiv 0$ strictly and eliminating all direct long-range connections.

2. *Strict Spatial Parameter Sharing*: The weights connecting an output unit to its receptive field must be strictly invariant to spatial translation. Defining coordinates $\mathbf{u}_1 = (i_1, j_1)$, $\mathbf{u}_2 = (i_2, j_2)$ and displacement $\boldsymbol{\Delta} = (\Delta x, \Delta y)$, weights at identical relative offsets must be identical:

$$p(W_{\mathbf{u}_1, \mathbf{u}_1+\boldsymbol{\Delta}} - W_{\mathbf{u}_2, \mathbf{u}_2+\boldsymbol{\Delta}}) = \delta(W_{\mathbf{u}_1, \mathbf{u}_1+\boldsymbol{\Delta}} - W_{\mathbf{u}_2, \mathbf{u}_2+\boldsymbol{\Delta}}). \tag{5.47}$$

Together, Equations (5.46) and (5.47) collapse the hypothesis space from an unconstrained $D_{\text{out}} \times D_{\text{in}}$ matrix manifold down to a banded subspace of dimension $F \times F \times C_{\text{in}} \times C_{\text{out}}$.

Similarly, spatial pooling (such as max or average pooling over a window of radius R) acts as an infinitely strong prior asserting that the representation must be strictly invariant to small local translations:

$$f(\mathcal{T}_{\boldsymbol{\epsilon}}\mathbf{X}) = f(\mathbf{X}), \quad \forall \|\boldsymbol{\epsilon}\| \leq R. \tag{5.48}$$

This prior encodes the domain belief that the exact sub-window spatial coordinate where a feature occurs is irrelevant, prioritizing feature presence over exact coordinate localization.

Underfitting risk and bias-variance trade-offs. An infinitely strong prior is fundamentally a double-edged sword:

- *When the Prior Holds (Stationary Natural Images)*: If the data adheres to the prior assumptions—as natural images do, with high local spatial correlations and continuous rigid translation symmetries—the prior drastically reduces estimator variance, eliminates the $\mathcal{O}(HW)$ sample complexity overhead, and prevents overfitting without introducing harmful bias.

- *When the Prior Fails (Underfitting and Bias)*: If the true data distribution violates the prior assumptions, an infinitely strong prior results in severe, permanent underfitting due to uncorrectable inductive bias. Because the prior assigns zero probability to functions outside $\mathcal{M}$, the true data-generating function cannot be recovered even with infinite training data ($N \to \infty$).

Concrete failure modes. Several practical vision domains expose the limitations of the convolutional and pooling prior:

1. *Exact Coordinate Regression*: Tasks requiring precise pixel coordinate prediction (such as facial landmark keypoint localization, bounding-box corner regression, or robotic visual servoing) are hindered by spatial pooling, which deliberately discards fine-grained spatial offsets.

2. *Non-Local Long-Range Interactions*: Convolution restricts direct connections to local $F \times F$ windows. Capturing correlations across distant corners of an image requires stacking dozens of layers to build receptive field depth, progressively diluting high-frequency details.

3. *Vision Transformers (ViTs)*: By replacing localized convolutional priors with unconstrained multi-head self-attention, ViTs impose much weaker inductive biases. While requiring substantially more training data to converge from scratch, ViTs learn flexible global dependencies that surpass the asymptotic performance ceiling of rigid local priors on web-scale datasets.

4. *Architectural Mitigations*: Convolutional architectures address these rigidities via dilated convolutions (which expand receptive fields exponentially without adding parameters or downsampling) and multi-branch modules (such as Inception, which compute multiple kernel scales in parallel).

5.4 Architectural design principles

The realization that convolutional networks embody an infinitely strong spatial prior explains their data efficiency, but it leaves an essential engineering challenge unsolved: how should one configure individual convolutional layers into an expressive, deep visual architecture? In the early eras of deep learning, visual networks relied on wide, heterogeneous spatial filters—such as the 11×11, 7×7, and 5×5 convolutions deployed in early CNNs—to rapidly capture extended spatial context. However, large spatial kernels suffer from quadratic computational and parameter complexity ($\mathcal{O}(k^2)$), consuming massive memory and floating-point budgets while severely constraining the total depth that can be trained under practical hardware constraints.

To break past these computational barriers, modern computer vision underwent a structural paradigm shift toward disciplined micro-architectural design principles. Rather than relying on single monolithic convolutions, contemporary deep backbones decompose spatial transformations into cascaded sequences of localized 3×3 filters. This structural decomposition achieves identical or superior effective receptive fields while dramatically slashing weight counts and introducing multiple non-linear activations that geometrically fold and partition the representation space. Combined with spatial pooling operations for progressive spatial downsampling and dense classification heads for global relational synthesis, these principles establish the canonical blueprint of modern vision architectures.

5.4.1 Receptive fields and depth

Having formalized the output geometry of individual convolutional layers, we analyze a decisive architectural paradigm shift in deep learning: *should deep networks employ large convolutional kernels, or should they cascade multiple small* 3×3 *filters?* [6, 87]

Receptive field arithmetic and hyperparameter selection. The receptive field R_l of a neuron in layer l is defined as the spatial extent (diameter along one axis) in the input image $\mathbf{X}^{(0)}$ that contributes to the activation of that neuron [6, 73]. Let k_l denote the kernel size at layer l, and let s_l denote the layer stride. With $R_0 = 1$ (the input pixel itself), the cumulative stride (jump) preceding layer l is $J_{l-1} = \prod_{i=1}^{l-1} s_i$ (with $J_0 = 1$). The effective receptive field progression follows the universal linear recurrence:

$$R_l = R_{l-1} + (k_l - 1) \cdot J_{l-1}. \tag{5.49}$$

In standard convolutional blocks prior to pooling, the stride is unit ($s_l = 1, \forall l \implies J_{l-1} = 1$). Equation (5.49) simplifies to:

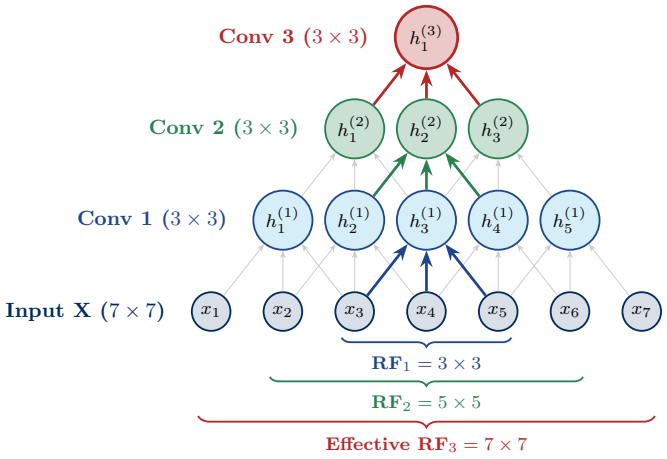
$$R_l = R_{l-1} + (k_l - 1). \tag{5.50}$$

For a sequence of L cascaded convolutional layers with identical kernel size k:

$$R_L = 1 + \sum_{l=1}^{L} (k - 1) = 1 + L(k - 1). \tag{5.51}$$

Example: Specializing Equation (5.51) to standard 3×3 kernels ($k = 3 \implies k - 1 = 2$):

- *Layer 1 ($L = 1$)*: $R_1 = 1 + 1(2) = 3$ (3×3 effective receptive field).
- *Layer 2 ($L = 2$)*: $R_2 = 3 + 2 = 5$ (5×5 effective receptive field). A hidden unit $h_j^{(2)}$ at layer 2 computes a linear combination of three layer-1 units: $\{h_{j-1}^{(1)}, h_j^{(1)}, h_{j+1}^{(1)}\}$. In turn, the leftmost unit $h_{j-1}^{(1)}$ accesses input pixels $\{x_{j-2}, x_{j-1}, x_j\}$, while the rightmost unit $h_{j+1}^{(1)}$ accesses $\{x_j, x_{j+1}, x_{j+2}\}$. The total spatial span on the input is $[j-2, j+2]$, spanning $(j+2)-(j-2)+1 = 5$ pixels along each axis. Stacking two 3×3 layers with C channels requires $2 \times (3^2C^2) = 18C^2$ parameters compared to a single 5×5 layer requiring $1 \times (5^2C^2) = 25C^2$, achieving an exact parameter reduction of $\frac{25-18}{25} = 28\%$.
- *Layer 3 ($L = 3$)*: $R_3 = 5 + 2 = 7$ (7×7 effective receptive field). The three layer-2 units span $[j-3, j+3]$ on the input grid, encompassing 7 pixels along each dimension. Stacking three 3×3 layers requires $3 \times (3^2C^2) = 27C^2$ weights and $3 \times (2 \times 9C^2HW) = 54C^2HW$ FLOPs, compared to a single 7×7 layer requiring $1 \times (7^2C^2) = 49C^2$ weights and $98C^2HW$ FLOPs. This structural decomposition achieves an exact savings of $\frac{49-27}{49} \approx 45\%$ in both parameters and arithmetic operations.



The VGG Stacking Principle

Parameter Savings:

- Two 3×3 vs. one 5×5: $2 \cdot 9C^2 = 18C^2$ vs. $25C^2$ (28% cut)
- Three 3×3 vs. one 7×7: $3 \cdot 9C^2 = 27C^2$ vs. $49C^2$ (45% cut)

Representational Power:
Stacking incorporates 3 non-linear activations (e.g. ReLUs) instead of 1, forming more discriminative features.

Figure 5.7: Effective receptive field expansion of stacked 3×3 convolutional kernels. Cascading two 3×3 filters achieves a 5×5 receptive field with 28% parameter reduction, while cascading three 3×3 filters spans a 7×7 receptive field with 45% parameter savings and three successive non-linearities.

This receptive field arithmetic provides the foundational guideline for selecting the architectural hyperparameters of a CNN. For a deep network to recognize semantic entities or make holistic categorical decisions, the final convolutional neurons must possess an effective receptive field that matches or exceeds the physical scale of the target objects in the input image. If an architect selects hyperparameters that produce an undersized receptive field (for instance, stacking only two convolutional layers on high-resolution 224×224 images), deep neurons remain local feature detectors blind to global spatial context. Conversely, attempting to broaden the receptive field by increasing kernel size F incurs quadratic computational and storage penalties $\mathcal{O}(F^2)$. The principled hyperparameter strategy, codified by the VGG architecture [87], is to fix the spatial kernel size to the minimal non-trivial odd dimension ($F = 3$) and expand the receptive field by increasing architectural depth L alongside periodic spatial downsampling.

Non-linear vs. linear depth: Mathematical proof of collapse. Beyond parameter and FLOP efficiency, cascading convolutions introduces a vital question regarding representational

depth: what happens if intermediate non-linear activations are omitted?

Consider a cascade of L consecutive 2D convolutional layers operating without activation functions (or with identity mappings $\sigma(z) = z$):

$$\mathbf{Y} = \mathbf{W}_L * (\mathbf{W}_{L-1} * (\cdots * (\mathbf{W}_1 * \mathbf{X}) \dots)). \tag{5.52}$$

Recall that 2D continuous and discrete convolution operators are strictly associative and commutative:

$$\mathbf{A} * (\mathbf{B} * \mathbf{C}) = (\mathbf{A} * \mathbf{B}) * \mathbf{C}. \tag{5.53}$$

For any two successive linear convolutional layers parameterized by kernels $\mathbf{W}_1 \in \mathbb{R}^{k_1 \times k_1}$ and $\mathbf{W}_2 \in \mathbb{R}^{k_2 \times k_2}$, the associative property yields:

$$\mathbf{W}_2 * (\mathbf{W}_1 * \mathbf{X}) = (\mathbf{W}_2 * \mathbf{W}_1) * \mathbf{X} = \mathbf{W}_{\text{eff}}^{(2)} * \mathbf{X}, \tag{5.54}$$

where the composite kernel $\mathbf{W}_{\text{eff}}^{(2)} = \mathbf{W}_2 * \mathbf{W}_1$ is obtained by convolving the two kernel tensors directly, spanning a spatial support of $(k_1 + k_2 - 1) \times (k_1 + k_2 - 1)$.

By induction over L successive layers, the entire multi-layer cascade algebraically collapses into a single linear convolution:

$$\mathbf{W}_{\text{eff}} = \mathbf{W}_L * \mathbf{W}_{L-1} * \cdots * \mathbf{W}_1 \in \mathbb{R}^{\left(1+\sum_{l=1}^{L}(k_l-1)\right) \times \left(1+\sum_{l=1}^{L}(k_l-1)\right)}, \tag{5.55}$$

yielding the purely linear mapping:

$$\mathbf{Y} = \mathbf{W}_{\text{eff}} * \mathbf{X} + \mathbf{b}_{\text{eff}}. \tag{5.56}$$

Equation (5.56) proves that linear depth is an illusion: a deep stack of linear convolutions possesses no greater representational capacity than a single convolutional layer with a larger kernel size. It cannot partition input space into non-linear decision regions and merely parameterizes a low-rank, constrained factorization of a single linear filter.

To unlock the true power of architectural depth, each convolutional layer must be coupled with an element-wise non-linear activation function $\sigma(\cdot)$ (such as ReLU):

$$\mathbf{H}_l = \sigma\left(\mathbf{W}_l * \mathbf{H}_{l-1} + \mathbf{b}_l\right). \tag{5.57}$$

Because non-linear activation functions do not commute with convolution ($\sigma(\mathbf{W}*\mathbf{X}) \neq \mathbf{W}*\sigma(\mathbf{X})$), each successive layer geometrically bends, folds, and partitions the representation space into an exponentially large number of linear decision regions $\mathcal{O}(2^L)$ [66, 91]. Non-linear depth allows the network to construct a true hierarchical abstraction: early layers extract elementary oriented edges, intermediate layers combine them into corners and textural primitives, and deep layers assemble complex parts and holistic object geometries.

5.4.2 Pooling operations

Interspersed between convolutional layers are spatial pooling (subsampling) operations. A pooling layer operates independently on each feature channel slice, replacing local spatial neighborhoods with an aggregated summary statistic [32].

The two standard pooling operations are:

- *Max Pooling*: Computes the maximum activation value within a local sliding window of spatial size $F_p \times F_p$ with stride S_p:

$$Y_{i,j,c} = \max_{m,n \in \{0,\dots,F_p-1\}} X_{i \cdot S_p + m,\, j \cdot S_p + n,\, c}. \tag{5.58}$$

  Max pooling acts as a localized feature presence detector (a continuous logical OR): it tests whether a specific visual pattern (e.g., an edge, corner, or texture) is active anywhere within the receptive patch, regardless of its exact sub-window coordinate.

- *Average Pooling*: Computes the arithmetic mean of activations across the window:

$$Y_{i,j,c} = \frac{1}{F_p^2} \sum_{m=0}^{F_p-1} \sum_{n=0}^{F_p-1} X_{i \cdot S_p+m,\, j \cdot S_p+n,\, c}, \tag{5.59}$$

  smoothing spatial responses and capturing background feature energy.

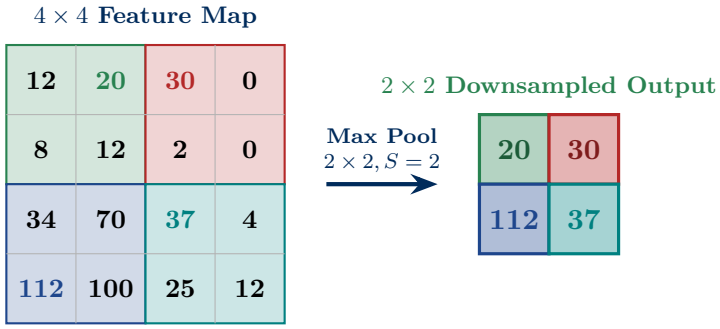


Figure 5.8: Max pooling operation with a 2×2 filter and stride $S = 2$. The input feature map is partitioned into non-overlapping 2×2 colored quadrants. The operation routes the maximum value within each quadrant to the downsampled grid, halving height and width (75% spatial area reduction) with zero learnable parameters.

In the framework of CNN hyperparameter selection, pooling layers serve as critical structural regulators:

- *Kernel and Stride Configuration*: The canonical hyperparameter choice is an unpadded ($P = 0$) non-overlapping window with $F_p = 2$ and $S_p = 2$. Setting $S_p = F_p$ ensures that every input activation belongs to exactly one pooling quadrant, preventing redundant multi-counting.
- *Parameter-Free Computation*: Unlike convolutional layers, pooling involves fixed aggregation arithmetic and contains zero learnable parameters ($N_{\text{params}} = 0$).
- *Spatial Downsampling and FLOP Reduction*: Halving spatial dimensions ($H_{\text{out}} = H_{\text{in}}/2$, $W_{\text{out}} = W_{\text{in}}/2$) reduces spatial activation volume by a factor of 4 (75% area reduction). Because the computational FLOPs of subsequent convolutional layers scale as $\mathcal{O}(H \cdot W \cdot C_{\text{in}} C_{\text{out}} F^2)$, each pooling stage slashes subsequent computational and memory burdens by 75%.
- *Receptive Field Doubling*: Pooling multiplies the cumulative stride of all downstream layers: $J_{\text{post}} = J_{\text{pre}} \cdot S_p$. Consequently, a subsequent 3×3 convolutional kernel jumps across twice as many input pixels, doubling its rate of receptive field expansion without increasing parameter counts.
- *Local Translation Invariance*: If the input image shifts by one pixel, the maximum activation within a 2×2 quadrant often remains unchanged, conferring robust local shift invariance.
- *Hyperparameter Pitfalls*: Choosing excessively large pooling kernels ($F_p > 3$) or inserting pooling stages too aggressively causes premature and irreversible loss of fine-grained spatial localization and high-frequency details.

5.4.3 Building a convolutional network

Designing a convolutional neural network requires synthesizing local convolutions, non-linear activations, and spatial downsampling into a coherent processing hierarchy. Rather than arranging layers arbitrarily, standard CNN architectures follow a principled modular framework structured into sequential computational stages.

Stage-based modular organization. A convolutional backbone is organized into cascading stages. Within each stage $s \in \{1, \dots, S\}$, the spatial resolution (H_s, W_s) is held fixed while multiple convolutional layers extract increasingly expressive features. A complete CNN pipeline comprises:

1. *Input Layer*: Ingests the raw image tensor $\mathbf{X} \in \mathbb{R}^{H \times W \times C_{\text{in}}}$ (e.g., $224 \times 224 \times 3$ for standard RGB images).
2. *Convolutional Stages*: A sequence of stages where each stage contains one or more convolutional layers with non-linear activations.
3. *Transition Downsampling*: Inter-stage transitions that systematically halve the spatial dimensions and expand channel capacity.
4. *Terminal Head*: A final decision module that maps the high-level spatial feature map to downstream task outputs (such as classification probabilities).

Intra-stage layer design. Within each individual stage, the architecture adheres to the small-kernel stacking principle established by VGG:

- *Filter Size ($F = 3$)*: All convolutional kernels are standardized to small 3×3 filters ($F = 3$), representing the minimal footprint capable of capturing directional spatial patterns (up/down, left/right, and diagonals).
- *Unit Stride and Same Padding ($S = 1, P = 1$)*: Setting stride $S = 1$ and zero-padding $P = (F - 1)/2 = 1$ ensures that spatial height and width remain strictly invariant across convolutions:
$$H_{\text{out}} = \frac{H_{\text{in}} - 3 + 2(1)}{1} + 1 = H_{\text{in}}. \tag{5.60}$$
This decouples feature extraction from spatial resolution reduction, allowing the network to grow deep without prematurely extinguishing spatial information.
- *Non-Linear Activations*: Every convolutional layer is immediately followed by a non-linear activation (typically ReLU). As demonstrated in Section 5.4.1, stacking k non-linear 3×3 convolutions expands the receptive field to $2k + 1$ while embedding k independent non-linearities, providing strictly greater representational expressiveness than a single linear filter of equivalent receptive field.

Inter-stage transitions: Spatial halving and channel doubling. Between consecutive stages, the network executes a coordinated spatial downsampling and channel expansion:

- *Spatial Halving*: Spatial dimensions are halved ($H_{s+1} = H_s/2$ and $W_{s+1} = W_s/2$) using either max pooling ($2 \times 2, S = 2$) or a strided convolution ($S = 2$). This reduces the spatial area by 75% (1/4 of the original grid cells), slashes computational FLOPs for all subsequent layers, and doubles the cumulative stride rate ($J_{s+1} = 2J_s$), causing downstream kernels to expand their effective receptive field at twice the rate.
- *Channel Doubling*: Concurrently with spatial halving, the number of feature channels is doubled ($C_{s+1} = 2C_s$, e.g., $64 \to 128 \to 256 \to 512$). Because halving spatial dimensions contracts the canvas by a factor of 4, doubling the channels preserves total representational capacity while halving total activation memory:
$$\left(\frac{H_s}{2}\right) \times \left(\frac{W_s}{2}\right) \times (2C_s) = \frac{1}{2}(H_s \times W_s \times C_s). \tag{5.61}$$
As spatial coordinates become coarser, the network compensates by expanding its dictionary of semantic feature detectors, allocating greater expressive capacity to represent diverse, complex patterns.

Hierarchical feature abstraction. By combining small-kernel stacking with progressive downsampling and channel doubling, the network forms an inductive representational hierarchy:

- *Shallow Layers (Stages 1–2)*: Small receptive fields and high spatial resolution capture low-level, class-agnostic geometric primitives (oriented edges, color contrasts, local gradients, simple textures).
- *Mid-Level Layers (Stage 3)*: Moderate receptive fields aggregate primitives into structured motifs, contours, corners, and object parts (wheels, handles, organic textures).
- *Deep Layers (Stages 4–5)*: Expansive receptive fields covering most or all of the input canvas synthesize global semantic concepts and holistic category geometries (dog faces, vehicle profiles, architectural structures).

This multi-stage pyramid of feature extraction produces a compact, semantically rich tensor $\mathbf{F} \in \mathbb{R}^{H' \times W' \times C'}$ (e.g., $7 \times 7 \times 512$), perfectly positioned for global decision synthesis.

5.4.4 Dense classification heads and parameter distribution

A complete convolutional architecture combines convolutional backbones, non-linear activations, and spatial pooling into an end-to-end processing pipeline, concluding with Fully Connected (dense) layers at the output. This raises a fundamental architectural question: *if Fully Connected Networks fail catastrophically when applied directly to images (as established in Section 5.1.2), why is it a sound architectural design to place a Fully Connected Network at the bottom of a CNN?*

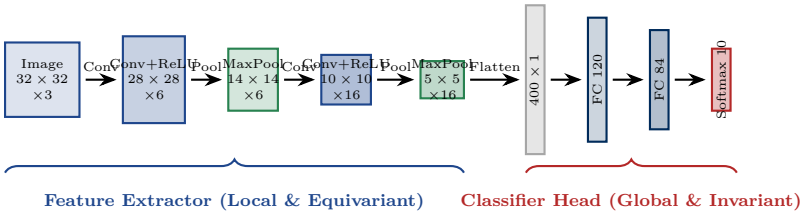


Figure 5.9: The end-to-end architectural pipeline of a Convolutional Neural Network (LeNet-5 archetype). The convolutional backbone extracts translation-equivariant local visual features across cascading spatial scales, culminating in dense fully connected layers that synthesize global spatial evidence to emit categorical class probabilities.

As shown in Figure 5.9, placing a Fully Connected Network at the output of a convolutional backbone realizes a principled division of labor between two distinct computational regimes:

1. *The Convolutional Backbone as a Geometric Feature Extractor*: The primary role of the CNN backbone is to resolve the spatial physics of natural imagery. Operating on local 2D neighborhoods via parameter sharing, it guarantees translation equivariance and builds an increasingly abstracted, multi-scale representation while progressively downsampling spatial dimensions. Each unit at coordinate (i, j) in the final convolutional tensor $\mathbf{F} \in \mathbb{R}^{H' \times W' \times C'}$ answers a localized question: *"Does semantic feature c occur at spatial coordinate (i, j)?"*
2. *The Fully Connected Head as a Global Decision Synthesizer*: Whole-image classification is fundamentally a non-spatial, translation-invariant decision task: mapping input evidence to a probability distribution $\hat{\mathbf{y}} \in \Delta^{K-1}$ over K categories. Recognizing a holistic category requires reasoning over the global spatial co-occurrence and relative geometric arrangement of multiple independent parts (e.g., verifying that two eyes appear above a nose, which in turn appears above a mouth). Because convolutional kernels are spatially constrained, they

cannot establish unconstrained all-to-all cross-spatial correlations between distant quadrants of the canvas without an impractical number of extra layers. By flattening the feature tensor $\mathbf{v} = \text{vec}(\mathbf{F}) \in \mathbb{R}^{H'W'C'}$, the dense head computes:

$$z_m = \sum_{c=1}^{C'} \sum_{i=1}^{H'} \sum_{j=1}^{W'} W_{m,(i,j,c)} F_{i,j,c} + b_m. \tag{5.62}$$

Every neuron in the dense head connects to every spatial coordinate and feature channel simultaneously, enabling unconstrained relational synthesis across the entire visual field to emit categorical class logits.

Parameter distribution across the architecture. To understand the trade-offs of this architectural composition, we must mathematically analyze where parameters are concentrated in a CNN.

While the convolutional backbone accounts for the overwhelming majority of computational FLOPs (due to sliding dot products across $H \times W$ locations), the fully connected head accounts for the vast majority of parameters. Consider the canonical benchmark of VGG-16 operating on 224×224 images:

- *Convolutional Backbone Parameters*: The 13 convolutional layers (utilizing small 3×3 filters across channel depths from 64 to 512) contain a cumulative total of:

$$N_{\text{conv}} \approx 14.71 \times 10^6 \text{ parameters (14.71M)}. \tag{5.63}$$

- *Fully Connected Head Parameters*: The final convolutional volume of size $7 \times 7 \times 512$ is flattened into a 1D vector of dimension $d_{\text{feat}} = 7 \times 7 \times 512 = 25,088$. Connecting this vector to the first dense hidden layer of $4,096$ units requires:

$$|\mathbf{W}_{\text{fc1}}| = 25,088 \times 4,096 = 102,760,448 \text{ parameters } (\approx 102.76\text{M}). \tag{5.64}$$

The second dense layer ($4,096 \to 4,096$) requires $4,096 \times 4,096 \approx 16.78$M weights, and the final classification projection ($4,096 \to 1,000$) requires $4,096 \times 1,000 \approx 4.10$M weights. The dense head alone totals:

$$N_{\text{fc}} = 102.76\text{M} + 16.78\text{M} + 4.10\text{M} \approx 123.64 \times 10^6 \text{ parameters (123.64M)}. \tag{5.65}$$

Out of the total 138.36M parameters in VGG-16, the fully connected layers account for over **89.4**% of all model weights, with the single transition layer (FC_1) alone consuming nearly 75% of the network's entire parameter budget! Similarly, in AlexNet, the dense layers account for 58.6M out of 61.0M parameters ($\approx 96.1\%$).

This mathematical derivation demonstrates that in classical CNN architectures, parameters are overwhelmingly concentrated at the dense classification head. Consequently, while the convolutional backbone provides parameter-efficient feature extraction, the dense head forms the primary bottleneck for memory storage and represents the predominant source of overfitting, necessitating heavy regularization techniques such as Dropout.

5.5 Computer vision tasks

The physical visual world does not present itself as neatly isolated, canonically centered objects floating against uniform backgrounds. In real-world environments, visual scenes are continuous, highly cluttered, and composed of multiple overlapping entities distributed across complex spatial topographies. While whole-image categorical classification provides a vital mathematical benchmark for representation learning, reducing an entire high-dimensional image tensor

$\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ to a single discrete label fundamentally discards spatial coordinates, geometric dimensions, and physical relationships among visual entities. To address diverse real-world perceptual demands, machine learning formulates visual tasks along a structured spectrum of spatial granularity.

Image classification. In whole-image categorical classification, the model maps an input image $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ to a single categorical probability distribution $\hat{\mathbf{y}} \in \Delta^{K-1}$ over K mutually exclusive classes. It answers the holistic question: *"What object category is present in this image?"* Because the objective concerns only the presence of a class regardless of where it appears in the frame, image classification enforces strict **translation invariance** ($f(\mathcal{T}_{\boldsymbol{\delta}}\mathbf{X}) = f(\mathbf{X})$). While intermediate convolutional layers maintain spatial geometry through translation equivariance, the terminal classification head (via Global Average Pooling or flattening into dense layers) intentionally collapses spatial coordinates to emit invariant class predictions.

Object detection. Object detection extends visual recognition by simultaneously answering *"What is present?"* and *"Where is it located?"* for an arbitrary, unknown number of object instances $M \geq 0$ scattered across the canvas. Rather than predicting a single label, the model predicts a set of detections, each parameterized by a discrete categorical class $c \in \{1, \ldots, K\}$ and 4D continuous bounding box coordinates $\mathbf{b} = [x_c, y_c, w, h]^\top \in \mathbb{R}^4$ (specifying the object's spatial centroid, width, and height). Training optimizes a composite multi-task objective combining categorical cross-entropy with continuous bounding box regression (such as Smooth L_1 loss). Detection performance is quantitatively evaluated using the **Intersection over Union (IoU)** metric (the ratio between the area of overlap and the area of union of predicted and ground-truth boxes), scoring detections as true positives when IoU ≥ 0.5. Object detection reconciles translation-equivariant feature extraction across the backbone with spatial coordinate regression in the detection heads.

Semantic segmentation. Semantic segmentation formulates perception as a dense, pixel-level classification problem, assigning a categorical class label to every individual coordinate (i, j) in the spatial grid: $\hat{\mathbf{Y}} \in \{1, \ldots, K\}^{H \times W}$. It answers: *"Which semantic class does each individual pixel belong to?"* Unlike classification, semantic segmentation strictly demands **translation equivariance**: shifting an input image by $\boldsymbol{\delta}$ must shift the corresponding segmentation mask by the identical displacement. Deep segmentation architectures resolve the inherent trade-off between downsampling to capture broad semantic context ("what") and preserving high-resolution spatial details for boundary localization ("where") by deploying encoder-decoder architectures (such as U-Net) equipped with skip connections that route pristine, early-stage edge features directly to the decoding layers. Importantly, semantic segmentation does not distinguish between distinct instances of the same category; all pixels belonging to the same semantic class share an identical label.

Instance segmentation. Instance segmentation synthesizes the complementary strengths of object detection and semantic segmentation. It simultaneously identifies individual object instances and predicts a detailed, pixel-level binary mask for each detected instance. While semantic segmentation labels all contiguous pixels of a given category identically, instance segmentation separates and delineates multiple overlapping or adjacent instances of the same class (e.g., distinguishing five individual sheep grazing together in a field), providing both instance-level localization and precise boundary contours.

Table 5.1: Comparative taxonomy of canonical computer vision tasks across objective, output domain, and spatial symmetry requirements.

Task	Objective	Output Domain	Spatial Symmetry
Image Classification	Holistic categorical recognition	Class distribution $\hat{\mathbf{y}} \in \Delta^{K-1}$	Translation Invariance
Object Detection	Instance classification and 2D bounding box localization	$\{1, \ldots, K\} \times \mathbb{R}^4$ per detected instance	Equivariant backbone, localized heads
Semantic Segmentation	Dense per-pixel categorical classification	Label canvas $\{1, \ldots, K\}^{H \times W}$	Translation Equivariance
Instance Segmentation	Per-instance detection with pixel-level silhouette masks	$\{1, \ldots, K\} \times \{0, 1\}^{H \times W}$ per instance	Equivariant per detected instance

Chapter 6

Deep network training and regularization

In Chapters 4 and 5, we developed the architectural foundations of deep learning: Multilayer Perceptrons capable of universal function approximation and Convolutional Neural Networks endowed with an infinitely strong spatial prior. In theory, once a loss function $\mathcal{L}$ and a computational graph are defined, the Backpropagation algorithm evaluates exact parameter gradients $\nabla_{\boldsymbol{\theta}}\mathcal{L}$, and first-order optimizers (such as Stochastic Gradient Descent or Adam, derived in Chapter 3) iteratively minimize empirical risk.

In practice, however, when researchers in the early 2000s attempted to train feedforward or convolutional networks with more than a few hidden layers, **naive training catastrophically broke down**:

1. **Gradient vanishing and explosion**: Propagating signals across dozens of cascading layers caused gradients to decay exponentially toward zero or explode toward numerical overflow, halting learning in early layers.
2. **Activation pathologies and the dying neuron trap**: Classical activations saturated, while ReLUs suffered from catastrophic deactivation, permanently extinguishing large percentages of the network.
3. **Initialization sensitivity**: Initializing weights with values that were slightly too small caused activation variances to collapse to zero; initializing weights slightly too large drove activations into saturated regimes.
4. **Internal covariate shift**: Continuous parameter updates in early layers altered the distribution of inputs feeding downstream layers, forcing subsequent layers to constantly chase a moving target distribution.
5. **Overfitting in overparameterized regimes**: Deep architectures possessing millions of parameters easily memorized noise and training idiosyncrasies, leading to massive generalization gaps.

To transform deep neural networks from unstable mathematical curiosities into robust, scalable engineering systems, machine learning had to conquer the **numerical physics of deep training**. This chapter investigates the theoretical breakthroughs that unlocked deep network optimization: from activation function dynamics and data preprocessing, through Xavier and He weight initialization theory, to Batch Normalization and modern regularization paradigms.

6.1 Activation dynamics and pathologies

Before analyzing the specific failure modes of deep neural network optimization, we must establish why **activation functions** constitute the foundational engine of representation learning, and why their numerical behavior fundamentally dictates whether a deep architecture can be trained.

The Linear Collapse Theorem: Consider a feedforward network with L hidden layers, weight matrices $\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$, and bias vectors $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$. Suppose we omit the nonlinear activation functions, computing layer activations purely as linear transformations:

$$\mathbf{a}^{(l)} = \mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \text{for } l = 1, \dots, L, \quad \text{with } \mathbf{a}^{(0)} = \mathbf{x}. \tag{6.1}$$

Unrolling this composition from the output layer $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$ back to the input $\mathbf{x}$ yields:

$$\begin{aligned} \hat{\mathbf{y}} &= \mathbf{W}^{(L)} \left(\mathbf{W}^{(L-1)} \left(\cdots \left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)} \right) \cdots \right) + \mathbf{b}^{(L-1)} \right) + \mathbf{b}^{(L)} \\ &= \left(\prod_{l=1}^{L} \mathbf{W}^{(l)} \right) \mathbf{x} + \sum_{k=1}^{L-1} \left(\prod_{m=k+1}^{L} \mathbf{W}^{(m)} \right) \mathbf{b}^{(k)} + \mathbf{b}^{(L)}. \end{aligned} \tag{6.2}$$

Because matrix multiplication is associative and the space of linear transformations is closed under composition, the product of L weight matrices collapses into a single effective matrix $\mathbf{W}_{\text{eff}} \triangleq \prod_{l=1}^{L} \mathbf{W}^{(l)} \in \mathbb{R}^{d_L \times d_0}$, while the accumulated affine shifts collapse into a single effective bias vector $\mathbf{b}_{\text{eff}} \in \mathbb{R}^{d_L}$. Therefore:

$$\hat{\mathbf{y}} = \mathbf{W}_{\text{eff}}\mathbf{x} + \mathbf{b}_{\text{eff}}. \tag{6.3}$$

Equation (6.3) establishes an inescapable theoretical result: **any purely linear neural network, regardless of its depth, parameter count, or number of layers, is mathematically identical to a single-layer linear model**. It cannot compute the simple XOR function, cannot represent non-convex decision boundaries, and possesses zero hierarchical feature abstraction capacity.

Universal Approximation and the Non-Linear Engine: By the Universal Approximation Theorem [15, 41], introducing a continuous, non-polynomial activation function $\sigma : \mathbb{R} \to \mathbb{R}$ after each affine transformation:

$$\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}) = \sigma\left(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \right) \tag{6.4}$$

endows the network with the mathematical expressivity to approximate any continuous function on compact subsets of $\mathbb{R}^d$ to arbitrary precision. The activation function is the sole architectural mechanism responsible for bending, folding, and untangling the geometric manifold of the data.

The Numerical Dilemma of Non-Linearity: However, the activation function is a double-edged sword. While it introduces the non-linearity required for expressive capacity on the forward pass, its **local first derivative** $\sigma'(z)$ acts as a scalar multiplicative gate during the backward pass via the chain rule of calculus. If $\sigma'(z)$ is poorly conditioned, training a deep network encounters severe numerical instabilities:

1. **Vanishing Gradients**: If the derivative satisfies $|\sigma'(z)| < 1$, multiplying derivatives across L layers causes the gradient to decay exponentially toward zero, starving early layers of learning signals.
2. **Exploding Gradients**: If derivatives and weight spectral norms combine such that the effective transition multiplier exceeds 1, gradients grow exponentially, causing numerical overflow and parameter divergence.
3. **Optimization Zigzagging**: If $\sigma(z)$ is strictly non-zero centered (e.g., $\sigma(z) > 0$), all weight gradients entering a neuron share identical signs, restricting parameter updates to two opposite orthants and forcing the optimizer into slow zigzagging trajectories.
4. **Dead Neurons**: If $\sigma(z)$ contains a flat, inactive region with zero derivative ($\sigma'(z) = 0$), large gradient steps can push neurons into permanent comas where gradients vanish across the entire dataset.

5. **Computational Inefficiency**: Activation functions involving transcendental functions ($\exp, \log$) incur significant computational latency on hardware accelerators compared to simple thresholding comparators.

Understanding these five interconnected phenomena is essential to mastering modern deep network optimization.

6.1.1 Vanishing and exploding gradients

To understand why deep networks fail to train out of the box, we analyze the analytical structure of backpropagation across an L-layer network from first principles.

Derivation of the Layer Error Recurrence (Formula 6.1): Consider an L-layer feedforward network defined by the forward recurrence:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \qquad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}), \tag{6.5}$$

where $\mathbf{W}^{(l)} \in \mathbb{R}^{d_l \times d_{l-1}}$, $\mathbf{b}^{(l)} \in \mathbb{R}^{d_l}$, and $\sigma(\cdot)$ is applied element-wise. Let $\mathcal{L}$ denote the scalar training loss. We define the error adjoint vector at layer l as the gradient of the loss with respect to pre-activation vector $\mathbf{z}^{(l)}$:

$$\boldsymbol{\delta}^{(l)} \triangleq \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} = \begin{bmatrix} \frac{\partial \mathcal{L}}{\partial z_1^{(l)}} & \frac{\partial \mathcal{L}}{\partial z_2^{(l)}} & \cdots & \frac{\partial \mathcal{L}}{\partial z_{d_l}^{(l)}} \end{bmatrix}^\top \in \mathbb{R}^{d_l}. \tag{6.6}$$

To evaluate the i-th component $\delta_i^{(l)} = \frac{\partial \mathcal{L}}{\partial z_i^{(l)}}$, we apply the multivariate chain rule across all d_{l+1} units of the subsequent layer $l+1$:

$$\delta_i^{(l)} = \frac{\partial \mathcal{L}}{\partial z_i^{(l)}} = \sum_{j=1}^{d_{l+1}} \frac{\partial \mathcal{L}}{\partial z_j^{(l+1)}} \frac{\partial z_j^{(l+1)}}{\partial a_i^{(l)}} \frac{\partial a_i^{(l)}}{\partial z_i^{(l)}}. \tag{6.7}$$

Evaluating each partial derivative component individually:

- By definition, $\frac{\partial \mathcal{L}}{\partial z_j^{(l+1)}} = \delta_j^{(l+1)}$.
- Because $z_j^{(l+1)} = \sum_{k=1}^{d_l} W_{jk}^{(l+1)} a_k^{(l)} + b_j^{(l+1)}$, the partial derivative with respect to activation $a_i^{(l)}$ is:
$$\frac{\partial z_j^{(l+1)}}{\partial a_i^{(l)}} = W_{ji}^{(l+1)}. \tag{6.8}$$
- Because $a_i^{(l)} = \sigma(z_i^{(l)})$, the derivative of the element-wise activation with respect to its pre-activation is:
$$\frac{\partial a_i^{(l)}}{\partial z_i^{(l)}} = \sigma'(z_i^{(l)}). \tag{6.9}$$

Substituting these three terms back into Equation (6.7):

$$\delta_i^{(l)} = \sum_{j=1}^{d_{l+1}} \delta_j^{(l+1)} W_{ji}^{(l+1)} \sigma'(z_i^{(l)}) = \left(\sum_{j=1}^{d_{l+1}} W_{ji}^{(l+1)} \delta_j^{(l+1)} \right) \sigma'(z_i^{(l)}). \tag{6.10}$$

Recognizing that the summation $\sum_{j=1}^{d_{l+1}} W_{ji}^{(l+1)} \delta_j^{(l+1)}$ represents the i-th element of the matrix-vector product $(\mathbf{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)}$, we express this in compact vector notation:

$$\boldsymbol{\delta}^{(l)} = \left((\mathbf{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)} \right) \odot \sigma'(\mathbf{z}^{(l)}), \tag{6.11}$$

where $\odot$ denotes the Hadamard (element-wise) product, and $\sigma'(\mathbf{z}^{(l)}) \in \mathbb{R}^{d_l}$ is the vector of element-wise activation derivatives. This establishes Formula 6.1.

Derivation of the Unrolled Gradient Chain (Formula 6.2): To analyze how this error propagates backwards across many layers, we convert the element-wise Hadamard product into matrix multiplication. Notice that for any vector $\mathbf{u} \in \mathbb{R}^d$, $\mathbf{u} \odot \sigma'(\mathbf{z}^{(l)}) = \operatorname{diag}\left(\sigma'(\mathbf{z}^{(l)})\right)\mathbf{u}$, where $\operatorname{diag}\left(\sigma'(\mathbf{z}^{(l)})\right) \in \mathbb{R}^{d_l \times d_l}$ is a diagonal matrix containing the local activation derivatives along its main diagonal:

$$\operatorname{diag}\left(\sigma'(\mathbf{z}^{(l)})\right) = \begin{bmatrix} \sigma'(z_1^{(l)}) & 0 & \cdots & 0 \\ 0 & \sigma'(z_2^{(l)}) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma'(z_{d_l}^{(l)}) \end{bmatrix}. \tag{6.12}$$

Equation (6.11) can therefore be written strictly as linear algebraic transformations:

$$\boldsymbol{\delta}^{(l)} = \operatorname{diag}\left(\sigma'(\mathbf{z}^{(l)})\right)(\mathbf{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)}. \tag{6.13}$$

Defining the layer transition Jacobian matrix $\mathbf{J}_k \in \mathbb{R}^{d_k \times d_{k+1}}$ from layer $k+1$ to layer k:

$$\mathbf{J}_k \triangleq \operatorname{diag}\left(\sigma'(\mathbf{z}^{(k)})\right)(\mathbf{W}^{(k+1)})^\top \quad \text{or equivalently} \quad (\mathbf{W}^{(k+1)})^\top \operatorname{diag}\left(\sigma'(\mathbf{z}^{(k+1)})\right), \tag{6.14}$$

we unroll the recurrence recursively from output layer L down to early layer l:

$$\begin{aligned} \boldsymbol{\delta}^{(L-1)} &= \mathbf{J}_{L-1}\boldsymbol{\delta}^{(L)}, \\ \boldsymbol{\delta}^{(L-2)} &= \mathbf{J}_{L-2}\boldsymbol{\delta}^{(L-1)} = \mathbf{J}_{L-2}\mathbf{J}_{L-1}\boldsymbol{\delta}^{(L)}, \\ \boldsymbol{\delta}^{(l)} &= \mathbf{J}_l\mathbf{J}_{l+1}\cdots\mathbf{J}_{L-1}\boldsymbol{\delta}^{(L)} = \left[\prod_{k=l}^{L-1}\mathbf{J}_k\right]\boldsymbol{\delta}^{(L)}. \end{aligned} \tag{6.15}$$

Expanding each Jacobian $\mathbf{J}_k$ explicitly yields Formula 6.2:

$$\boldsymbol{\delta}^{(l)} = \left[\prod_{k=l}^{L-1}(\mathbf{W}^{(k+1)})^\top \operatorname{diag}\left(\sigma'(\mathbf{z}^{(k)})\right)\right]\boldsymbol{\delta}^{(L)}. \tag{6.16}$$

Equation (6.16) proves that the error signal $\boldsymbol{\delta}^{(l)}$ reaching any early layer l is a **repeated product of $L-l$ transition matrices**.

Derivation of the Spectral Bound (Formula 6.3): We now bound the magnitude of $\boldsymbol{\delta}^{(l)}$ using the sub-multiplicative property of induced matrix operator norms. Recall that for any matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ and vector $\mathbf{x} \in \mathbb{R}^n$, the induced Euclidean operator norm (spectral norm, equal to the largest singular value $\sigma_{\max}(\mathbf{A})$) satisfies:

$$\|\mathbf{A}\mathbf{x}\|_2 \le \|\mathbf{A}\|_2\|\mathbf{x}\|_2, \qquad \|\mathbf{A}\mathbf{B}\|_2 \le \|\mathbf{A}\|_2\|\mathbf{B}\|_2. \tag{6.17}$$

Taking the Euclidean norm of both sides of Equation (6.16):

$$\|\boldsymbol{\delta}^{(l)}\|_2 = \left\|\left(\prod_{k=l}^{L-1}\mathbf{J}_k\right)\boldsymbol{\delta}^{(L)}\right\|_2 \le \left(\prod_{k=l}^{L-1}\|\mathbf{J}_k\|_2\right)\|\boldsymbol{\delta}^{(L)}\|_2. \tag{6.18}$$

Furthermore, by the sub-multiplicative norm property, the norm of each transition Jacobian decomposes into the product of the weight matrix norm and the derivative matrix norm:

$$\|\mathbf{J}_k\|_2 \le \|(\mathbf{W}^{(k+1)})^\top\|_2 \|\operatorname{diag}\left(\sigma'(\mathbf{z}^{(k)})\right)\|_2 = \|\mathbf{W}^{(k+1)}\|_2 \max_{i\in\{1,\dots,d_k\}}\left|\sigma'(z_i^{(k)})\right|. \tag{6.19}$$

Let $\gamma \triangleq \max_{k\in\{l,\dots,L-1\}}\|\mathbf{J}_k\|_2$ denote the upper bound on the layer spectral norm. Applying this upper bound across all $L-l$ transitions yields Formula 6.3:

$$\|\boldsymbol{\delta}^{(l)}\|_2 \le \left(\prod_{k=l}^{L-1}\|\mathbf{J}_k\|_2\right)\|\boldsymbol{\delta}^{(L)}\|_2 \le \gamma^{L-l}\|\boldsymbol{\delta}^{(L)}\|_2. \tag{6.20}$$

The Asymptotic Dichotomy: Vanishing vs. Exploding Gradients: Equation (6.20) provides the fundamental mathematical explanation for the fragility of deep networks:

- **Vanishing Gradients ($\gamma < 1$):** If the effective spectral norm $\gamma < 1$, the gradient magnitude decays exponentially with depth: $\|\boldsymbol{\delta}^{(l)}\|_2 \leq \gamma^{L-l}\|\boldsymbol{\delta}^{(L)}\|_2 \to 0$. In a network with $L = 50$ layers, if $\gamma \approx 0.8$, the gradient at the first layer is attenuated by $0.8^{50} \approx 1.4 \times 10^{-5}$; if $\gamma \approx 0.25$ (the maximum derivative of Sigmoid), the attenuation is $(0.25)^{50} \approx 7.8 \times 10^{-31}$! Early layers receive virtually zero gradient updates, leaving their weights frozen at their initial random values. Because early layers extract essential low-level primitives (such as edges and textures), freezing them prevents all downstream layers from forming meaningful representations.
- **Exploding Gradients ($\gamma > 1$):** If $\gamma > 1$, the product grows exponentially: $\|\boldsymbol{\delta}^{(l)}\|_2 \sim \mathcal{O}(\gamma^{L-l}) \to \infty$. A gradient step $\Delta\mathbf{W} = -\eta\nabla_{\mathbf{W}}\mathcal{L}$ takes massive, erratic jumps across the loss surface, catapulting parameters into unstable regions and producing floating-point overflow (`NaN` or `Inf`).

6.1.2 Pathologies of Sigmoid and Tanh

For decades, the default non-linearities in neural computation were the smooth sigmoidal functions: the logistic sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$ and hyperbolic tangent $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$. When deployed in deep architectures, both functions induce severe mathematical pathologies that prevent gradient-based learning from scaling beyond a few layers.

1. Saturation and Gradient Vanishing in Sigmoid. [30] The derivative of the logistic sigmoid is:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)). \tag{6.21}$$

Because $\sigma(z) \in (0, 1)$, we can find the maximum value of $\sigma'(z)$ by differentiating with respect to z and setting to zero: $\sigma''(z) = \sigma'(z)(1 - 2\sigma(z)) = 0$, yielding $\sigma(z^*) = 0.5 \implies z^* = 0$. At this optimum:

$$\max_{z\in\mathbb{R}} \sigma'(z) = \sigma'(0) = 0.5 \times (1 - 0.5) = 0.25. \tag{6.22}$$

This property is devastating when evaluated in light of the spectral bound derived in Equation (6.19):

$$\|\mathbf{J}_k\|_2 \leq \|\mathbf{W}^{(k+1)}\|_2 \max_i |\sigma'(z_i^{(k)})| \leq 0.25\|\mathbf{W}^{(k+1)}\|_2. \tag{6.23}$$

Even under the most optimistic theoretical assumption where all weight matrices $\mathbf{W}^{(l)}$ preserve signal scale ($\|\mathbf{W}\|_2 = 1$), the scalar derivative factor alone shrinks the error gradient by **at least a factor of 4 at every single layer**:

$$\|\boldsymbol{\delta}^{(l)}\|_2 \leq (0.25)^{L-l}\|\boldsymbol{\delta}^{(L)}\|_2. \tag{6.24}$$

In a modest 10-layer network, $(0.25)^{10} \approx 9.5 \times 10^{-7}$. Across 20 layers, $(0.25)^{20} \approx 9.1 \times 10^{-13}$.

Furthermore, when $|z| \geq 4$, $\sigma(z)$ enters its flat, horizontal asymptotic saturation zones ($\sigma(z) \approx 0$ for $z \leq -4$ and $\sigma(z) \approx 1$ for $z \geq 4$). In these saturating regimes, the local derivative drops to $\sigma'(z) \approx 0$. When a neuron saturates, its diagonal entry in $\text{diag}(\sigma'(\mathbf{z}))$ becomes zero, completely annihilating the error gradient flowing through it. If neurons across a layer saturate, the backward gradient is instantly severed, trapping the entire preceding network in stagnation.

2. The Non-Zero-Centered Gradient Catastrophe. The second flaw of the sigmoid activation is equally pernicious: **its outputs are strictly positive**:

$$\sigma(z) \in (0, 1) \implies a_i^{(l)} > 0 \quad \forall i. \tag{6.25}$$

Consider a neuron j in layer $l+1$ computing pre-activation $z_j^{(l+1)} = \sum_i W_{ji}^{(l+1)} a_i^{(l)} + b_j^{(l+1)}$. The gradient of the loss $\mathcal{L}$ with respect to weight $W_{ji}^{(l+1)}$ evaluates to:

$$\frac{\partial \mathcal{L}}{\partial W_{ji}^{(l+1)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l+1)}} \frac{\partial z_j^{(l+1)}}{\partial W_{ji}^{(l+1)}} = \delta_j^{(l+1)} a_i^{(l)}. \tag{6.26}$$

Because the incoming activations $a_i^{(l)}$ are strictly positive ($a_i^{(l)} > 0$ for all i), the sign of the weight gradient $\frac{\partial \mathcal{L}}{\partial W_{ji}^{(l+1)}}$ is **governed entirely by the sign of the scalar error** $\delta_j^{(l+1)}$:

$$\text{sign}\left(\frac{\partial \mathcal{L}}{\partial W_{ji}^{(l+1)}}\right) = \text{sign}\left(\delta_j^{(l+1)}\right) \quad \forall i \in \{1, \ldots, d_l\}. \tag{6.27}$$

Equation (6.27) establishes a severe geometric constraint: **all weights feeding into a neuron must update in the exact same direction**. If $\delta_j^{(l+1)} > 0$, every weight gradient component is positive, forcing all weights to decrease; if $\delta_j^{(l+1)} < 0$, every component is negative, forcing all weights to increase.
The parameter update vector $\nabla_{\mathbf{w}_j} \mathcal{L}$ is restricted exclusively to two opposing orthants (the all-positive and all-negative orthants). If the true optimal parameter update requires increasing W_{j1} while decreasing W_{j2}, gradient descent cannot move directly along that vector. Instead, the optimizer is forced into a high-frequency, orthogonal **zig-zagging trajectory**, crawling along coordinate axes with agonizingly slow convergence.
Moreover, non-zero centered activations introduce a positive forward mean shift:

$$\mathbb{E}[z_j^{(l+1)}] = \sum_i W_{ji}^{(l+1)} \mathbb{E}[a_i^{(l)}] + b_j^{(l+1)}. \tag{6.28}$$

Because $\mathbb{E}[a_i^{(l)}] \approx 0.5 > 0$, this positive expectation cascades through successive layers, systematically shifting pre-activations into large positive regimes where sigmoid saturates ($z \gg 0 \implies \sigma'(z) \to 0$), thereby directly triggering catastrophic gradient vanishing!

3. Hyperbolic Tangent (Tanh): Partial Remedy. The hyperbolic tangent function $\tanh(z)$ is related to the sigmoid by an affine scaling:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1 \in (-1, 1). \tag{6.29}$$

Its derivative evaluates to:

$$\tanh'(z) = 1 - \tanh^2(z). \tag{6.30}$$

Because $\tanh(z)$ is an odd function ($\tanh(-z) = -\tanh(z)$), its outputs are symmetric around zero: $\mathbb{E}[a_i^{(l)}] \approx 0$. This **zero-centered property** eliminates the uniform-sign constraint, allowing weights to update independently in positive and negative directions without zigzagging. Furthermore, its maximum derivative at the origin is $\tanh'(0) = 1.0$, which is $4\times$ larger than that of Sigmoid.

However, tanh remains a **saturating function**: for $|z| \geq 2.5$, $\tanh(z) \to \pm 1$ and $\tanh'(z) \to 0$. In deep networks, as layer pre-activations naturally spread out in variance, activations drift into the saturating asymptotic tails, causing vanishing gradients to persist.

6.1.3 ReLU and the dying ReLU problem

The breakthrough that unlocked modern deep learning was the introduction of the **Rectified Linear Unit (ReLU)** [31, 67]:

$$\mathrm{ReLU}(z) \triangleq \max(0, z). \tag{6.31}$$

The derivative of ReLU is a step function:

$$\mathrm{ReLU}'(z) = \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{if } z < 0, \\ \text{undefined (subgradient } [0,1]) & \text{if } z = 0. \end{cases} \tag{6.32}$$

Why ReLU Revolutionized Deep Learning:

1. **Non-saturating gradient flow**: For any positive input $z > 0$, the derivative is *identically equal to 1* ($\sigma'(z) = 1$). Unlike sigmoid or tanh, ReLU does not compress large signals into flat asymptotes. Gradients flow through active neurons without any attenuation factor, vanquishing the vanishing gradient problem.
2. **Computational efficiency**: Evaluating $\max(0, z)$ and its derivative requires a single hardware comparator instruction, completely eliminating expensive transcendental exponential computations ($\exp(z)$) and accelerating forward and backward passes by a factor of 6 on GPUs (Krizhevsky et al., 2012).
3. **Induced representation sparsity**: For any negative pre-activation ($z \leq 0$), the neuron outputs an exact zero. At any given forward pass, roughly 50% of neurons in the network output zero, creating a sparse, highly decoupled representation that mimics biological cortical activation.

The Dying ReLU Trap (Dying Neuron Pathology): Despite its transformative advantages, standard ReLU possesses a fatal weakness: **it is completely inactive in the negative half-plane**.

Suppose a neuron with pre-activation $z = \mathbf{w}^\top \mathbf{x} + b$ receives a large gradient update during training (for example, caused by an excessively high learning rate η or a large stochastic gradient outlier). The update modifies weights and bias:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}, \quad b \leftarrow b - \eta \nabla_b \mathcal{L}. \tag{6.33}$$

If this update pushes the bias b to a large negative value such that:

$$z_i = \mathbf{w}^\top \mathbf{x}_i + b < 0 \quad \text{for } \mathbf{all} \text{ samples } \mathbf{x}_i \text{ in the training set}, \tag{6.34}$$

the neuron enters an irreversible coma:

1. On the forward pass, $\mathrm{ReLU}(z_i) = 0$ for every single sample. The neuron outputs zero constantly, contributing nothing to subsequent layers.
2. On the backward pass, $\mathrm{ReLU}'(z_i) = 0$ for every single sample. The error gradient $\boldsymbol{\delta}$ flowing into this neuron is multiplied by 0, yielding parameter gradients:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \sum_{i=1}^{N} \delta_i \mathbf{x}_i = \mathbf{0}, \qquad \frac{\partial \mathcal{L}}{\partial b} = \sum_{i=1}^{N} \delta_i = 0. \tag{6.35}$$

Because its gradients are identically zero across the entire dataset, **its parameters can never change again**. The neuron is permanently dead. In poorly tuned networks, 10%–40% of ReLUs can die during the very first epoch, permanently crippling the capacity of the model.

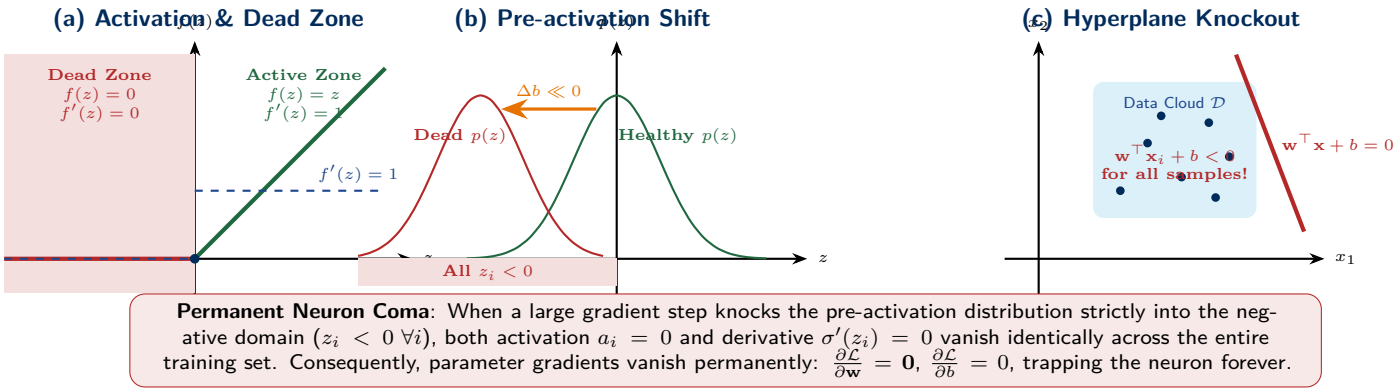


Figure 6.1: The anatomical and geometric genesis of the Dying ReLU trap. (a) The ReLU function and its derivative: inputs in the negative half-plane produce zero output and zero gradient. (b) An excessively large negative gradient update ($\Delta b \ll 0$) knocks the pre-activation distribution $p(z)$ entirely into the negative region. (c) In input space, the decision boundary $\mathbf{w}^\top \mathbf{x} + b = 0$ is knocked completely outside the training data cloud. Because every training sample produces zero derivative, parameter gradients vanish identically, rendering the neuron permanently un-trainable.

6.1.4 Modern non-linear activations

To resolve the dying ReLU trap while preserving non-saturating gradient flow, modern deep learning introduced several principled variants. To systematically analyze their mathematical properties and geometrical behaviors, Figure 6.2 plots each fundamental activation function individually within its own Cartesian coordinate frame, highlighting their saturation limits, zero crossings, and derivative dynamics.

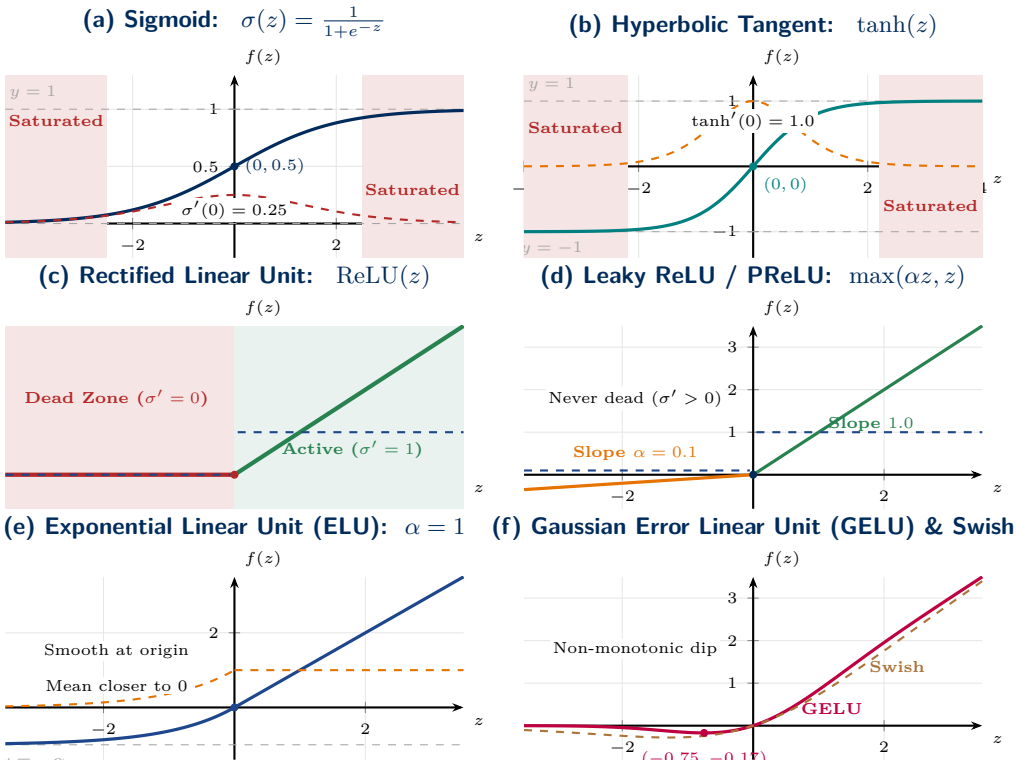


Figure 6.2: Individual Cartesian plane characterizations for classic and modern activation functions. Solid curves represent the activation function $f(z)$; dashed curves depict their first derivatives $f'(z)$. (a) Sigmoid saturates at 0 and 1 with maximum derivative 0.25. (b) Tanh is zero-centered in $(-1, 1)$ with maximum derivative 1.0. (c) Standard ReLU provides non-saturating unit gradient for $z > 0$ but suffers from the dying trap ($\sigma' = 0$) for $z \leq 0$. (d) Leaky ReLU and PReLU maintain a positive slope α in the negative domain, guaranteeing recovery. (e) ELU provides smooth exponential saturation toward $-\alpha = -1$, driving mean activations closer to zero. (f) GELU and Swish introduce smooth, non-monotonic probabilistic gating dips in the negative realm.

1. **Leaky ReLU** [62]: Replaces the flat negative zero regime with a small linear slope α (typically $\alpha = 0.01$):

$$f(z) = \max(\alpha z, z) = \begin{cases} z & \text{if } z > 0, \\ \alpha z & \text{if } z \leq 0. \end{cases} \tag{6.36}$$

   Because its derivative in the negative regime is $f'(z) = \alpha = 0.01 \neq 0$, error signals never completely vanish. A neuron knocked into the negative half-plane continues to receive updates, allowing it to recover and return to the active set.

2. **Parametric ReLU (PReLU)** [35]: Treats the negative slope α as a learnable parameter initialized to 0.25 and updated via gradient descent:

$$\frac{\partial \mathcal{L}}{\partial \alpha} = \sum_i \delta_i \min(0, z_i). \tag{6.37}$$

   PReLU adds negligible parameter overhead (one scalar per channel) while allowing the model to adaptively select between identity mapping ($\alpha = 1$), rectified sparsity ($\alpha = 0$), or leaky behavior.

3. **Exponential Linear Unit (ELU)** [13]: Replaces the linear negative branch with a smooth

exponential curve:

$$f(z) = \begin{cases} z & \text{if } z > 0, \\ \alpha(e^z - 1) & \text{if } z \leq 0, \end{cases} \tag{6.38}$$

where $\alpha > 0$ (typically $\alpha = 1.0$). For $z \leq 0$, $f(z)$ saturates smoothly toward $-\alpha$, providing noise immunity against extreme negative inputs while driving the mean activation $\mathbb{E}[a]$ closer to zero, which accelerates gradient descent.

4. **Gaussian Error Linear Unit (GELU)** [37]: Blends deterministic activation with stochastic regularizing gating. It scales the input by the cumulative distribution function (CDF) of the standard normal distribution $\Phi(z) = P(X \leq z)$ where $X \sim \mathcal{N}(0, 1)$:

$$\text{GELU}(z) \triangleq z\Phi(z) = z \int_{-\infty}^{z} \frac{1}{\sqrt{2\pi}} e^{-\frac{t^2}{2}}\, dt \approx 0.5z \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}(z + 0.044715z^3)\right)\right). \tag{6.39}$$

GELU is smooth, differentiable everywhere, and **non-monotonic**: it dips to a minimum of ≈ -0.17 at $z \approx -0.75$ before approaching zero. GELU has become the universal standard activation in modern Transformer foundation models (BERT, GPT-2/3/4, LLaMA, Vision Transformers).

5. **Swish / SiLU** [75]: Discovered via automated neural architecture search, defined as:

$$\text{Swish}(z) \triangleq z \cdot \sigma(\beta z) = \frac{z}{1 + e^{-\beta z}}, \tag{6.40}$$

where β is a learnable or fixed scaling factor (typically $\beta = 1$). Like GELU, Swish is smooth, non-monotonic, and bounded below while unbounded above, frequently matching or slightly exceeding ReLU on deep visual benchmarks.

Directly below these variants, Figure 6.3 presents the empirical benchmark comparing these 8 activation functions across three deep convolutional architectures (ResNet, Wide ResNet, and DenseNet) on CIFAR-10 (reproduced from Ramachandran et al. [75]).

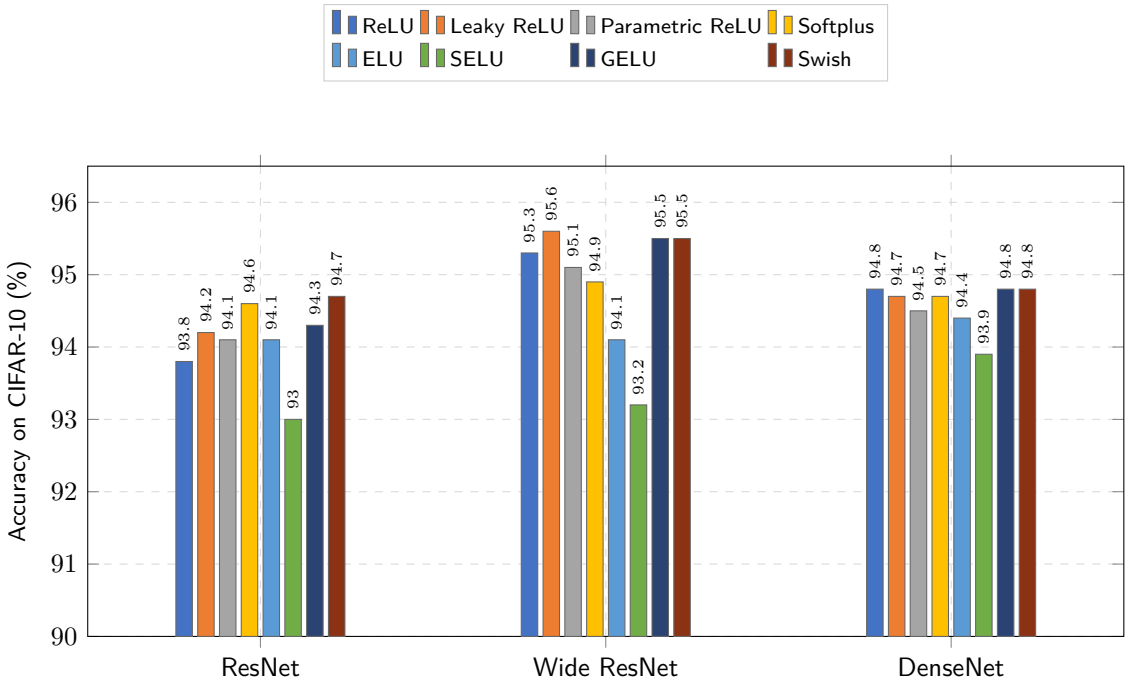


Figure 6.3: Empirical accuracy comparison of 8 modern activation functions across ResNet, Wide ResNet, and DenseNet on CIFAR-10 (originating from Ramachandran et al. [75]). The benchmark highlights two foundational empirical realities: (1) Deprecated activations (Sigmoid and Tanh) are omitted entirely because they suffer from catastrophic gradient vanishing and fail to train; (2) The performance variance across all modern ReLU variants is remarkably narrow ($\leq 1.0\%$), proving that activation selection is almost never the primary source of error in an underperforming deep model.

6.1.5 Diagnostic principles for activations

Table 6.1 provides a comprehensive master comparative taxonomy of all discussed activation functions, summarizing their mathematical formulations, output dynamic ranges, key computational properties, and practical engineering verdicts.

Table 6.1: Comparative theoretical taxonomy of neural activation functions.

Activation	**Formula $f(z)$**	**Range**	**Key Properties**	**Diagnostic Role & Practical Recommendation**
Sigmoid	$\frac{1}{1+e^{-z}}$	$(0, 1)$	Non-zero centered, saturates	**Deprecated for hidden layers**. Severe vanishing gradients ($\sigma' \leq 0.25$); binary output heads only.
Tanh	$\frac{e^z-e^{-z}}{e^z+e^{-z}}$	$(-1, 1)$	Zero-centered, saturates	**Deprecated for deep nets**. Solves centering, but saturation kills deep gradients ($\tanh' \to 0$).
ReLU	$\max(0, z)$	$[0, \infty)$	Non-saturating ($z > 0$), dying risk	**Universal Industry Standard**. 6× GPU speedup, constant unit gradient; default starting baseline.
Leaky ReLU	$\max(\alpha z, z)$	$(-\infty, \infty)$	Non-saturating, slope $\alpha = 0.01$	**Robust Alternative**. Completely eliminates dying ReLU trap with trivial compute overhead.
PReLU	$\max(\alpha z, z)$	$(-\infty, \infty)$	Learnable negative slope α	**Adaptive Slope**. Backpropagates into α at $+1$ parameter per feature channel.
ELU	$\begin{cases} z & z > 0 \\ \alpha(e^z - 1) & z \leq 0 \end{cases}$	$(-\alpha, \infty)$	Smooth, mean near 0, saturates left	**Noise Robust**. Negative saturation provides noise immunity; requires exponential.
GELU	$z\Phi(z)$	$[-0.17, \infty)$	Smooth, non-monotonic dip	**Transformer Standard**. Probabilistic gating; default in BERT, GPT, and Vision Transformers.
Swish	$z \cdot \sigma(\beta z)$	$[-0.28, \infty)$	Smooth, non-monotonic dip	**NAS Discovery**. Matches or slightly exceeds ReLU on deep ResNets ($\beta = 1$).

In empirical deep learning engineering, diagnostic reasoning regarding activation functions involves contrasting two fundamentally distinct performance scenarios:

1. The Catastrophic Failure of Deprecated Activations (Sigmoid and Tanh): If an engineer selects Sigmoid or Tanh for the hidden layers of a modern deep architecture (such as a 50-layer ResNet), **the performance will drop catastrophically**. In most cases, the model will

fail to learn entirely, halting at random guessing accuracy. As derived in Section 6.1.2, because $|\sigma'(z)| \leq 0.25$, gradients decay exponentially across cascading layers as $\sim (0.25)^{50} \approx 7.8 \times 10^{-31}$. Early layers receive strictly zero update, freezing the network at its random initialization. This is why Sigmoid and Tanh are omitted completely from competitive benchmarks such as Figure 6.3: they cannot converge on deep feedforward topologies without specialized layer-wise pre-training.

2. The Narrow Margin of Rectifiers: Activation Choice is Not the Source of Error: In stark contrast, when comparing modern non-saturating rectifiers (ReLU, Leaky ReLU, PReLU, ELU, GELU, Swish), the empirical results in Figure 6.3 reveal that **all variants cluster within a remarkably narrow** $\pm 0.5\%$–1.0% **accuracy margin**:

- On ResNet: ReLU (93.8%) vs. Leaky ReLU (94.2%) vs. Swish (94.7%).
- On Wide ResNet: ReLU (95.3%) vs. Leaky ReLU (95.6%) vs. GELU (95.5%).
- On DenseNet: ReLU (94.8%) vs. Leaky ReLU (94.7%) vs. GELU (94.8%).

This empirical reality leads to a foundational diagnostic principle in machine learning:

> **If a deep neural network is achieving poor performance, suffering from severe underfitting, or failing to converge when using standard ReLU, the activation function is almost certainly NOT the source of error.**

Switching from standard ReLU to Leaky ReLU, ELU, or GELU will never rescue a broken model. The validity of this assessment is directly demonstrated by Figure 6.3: the performance differences among all 8 modern activation functions across three distinct network families are minimal. A network suffering from flawed convergence cannot be repaired by altering the activation function.

When a deep architecture fails or underperforms, the engineer must look past the activation function and systematically diagnose the **true foundational pillars of deep training**:

1. **Learning Rate Misconfiguration**: Is the learning rate η excessively high, causing numerical divergence or knocking neurons into the dying ReLU trap? Or is it excessively low, causing the optimizer to stall in flat regions?

2. **Improper Weight Initialization**: Were weights initialized with unscaled Gaussian noise rather than He/Kaiming variance-preserving initialization (derived in Section 6.3), causing activation variances to vanish or explode during the very first forward pass?

3. **Missing or Misplaced Normalization Layers**: Is the network lacking Batch Normalization or Layer Normalization (Section 6.4), allowing internal covariate shift and gradient scale imbalances to destabilize deeper representations?

4. **Input Data Misconditioning**: Were input images fed as unnormalized $[0, 255]$ integer values rather than being zero-centered and scaled by channel-wise standard deviations (Section 6.2)?

5. **Optimization Scheduling and Regularization**: Is the optimizer lacking learning rate warmup, momentum, or cosine annealing (Section 6.6), or is the generalization gap driven by insufficient data augmentation or mismatched weight decay?

The Practical Baseline Rule: In industrial and research practice, the golden rule is: **Default to standard ReLU**. ReLU is computationally trivial (a single hardware branch comparator), memory-efficient, and universally accelerated across edge devices, GPUs, and TPUs. Only when the entire deep learning system is verified to be fully functional, converging stably, and properly regularized should an engineer experiment with Leaky ReLU, ELU, GELU, or Swish to harvest the final marginal fractions of a percent on benchmark competitions.

6.2 Input preprocessing and conditioning

Before any architectural forward pass or backpropagation update can occur, the input data feeding the network must undergo rigorous mathematical conditioning. In real-world applications, raw data rarely arrives in a form amenable to smooth gradient-based optimization. In computer vision, raw digital images arrive as unsigned 8-bit integers in the discrete range $\{0, 1, \dots, 255\}$; in tabular, speech, and multimodal datasets, features frequently span wildly disparate dynamic ranges, arbitrary non-zero offsets, and dense cross-feature correlations.

Training a deep neural network directly on raw, unconditioned input data triggers severe pathologies across the entire optimization trajectory:

1. **Directional Gradient Locking and Zigzagging**: When raw features are strictly positive (such as pixel intensities $x_i \in [0, 255]$), all incoming weights connected to a first-layer neuron receive gradient updates of identical sign. The optimizer is prevented from navigating into mixed-sign parameter quadrants, forcing it into slow, high-frequency oscillations across the loss surface.

2. **Extreme Loss Surface Eccentricity**: Disparities in raw feature magnitudes distort the local curvature of the objective function. The Hessian matrix develops an extreme condition number $\kappa(\mathbf{H}) = \lambda_{\max}/\lambda_{\min} \gg 1$, creating steep, narrow ravines where first-order gradient descent oscillates violently across the walls while making negligible progress along the base.

3. **Premature Activation Saturation**: Feeding unscaled, large raw inputs into the initial layers pushes pre-activations to extreme values ($|z| \gg 1$), shoving classical saturating activations into their zero-derivative asymptotes and destabilizing rectifier activations with large forward scale shifts.

4. **Arbitrary Sensitivity to Measurement Units**: Unnormalized models lack scale invariance; scaling a single feature from meters to millimeters multiplies its gradient magnitude by 10^3, completely overturning the learning rate calibration across dimensions.

To overcome these obstacles, machine learning practitioners employ a systematic taxonomy of preprocessing operations on raw data matrices $\mathbf{X} \in \mathbb{R}^{N \times D}$:

- **Zero-Centering (Mean Subtraction)**: Translates the empirical centroid of the dataset to the coordinate origin ($\mathbb{E}[\mathbf{x}] = \mathbf{0}$), eliminating sign coupling.
- **Variance Rescaling (Standardization / Z-Score)**: Scales each coordinate dimension to unit empirical variance ($\mathrm{Var}(x_d) = 1$), equalizing diagonal curvature.
- **Min-Max Normalization (Feature Rescaling)**: Linearly maps bounded features into a normalized compact interval, such as $[0, 1]$ or $[-1, 1]$.
- **Principal Component Analysis (PCA Decorrelation)**: Rotates the data onto the orthogonal eigenvector basis of its covariance matrix, annihilating all off-diagonal linear cross-correlations.
- **Whitening (Sphering Transformation)**: Scales decorrelated principal components by the inverse square root of their eigenvalues, yielding an isotropic identity covariance matrix ($\mathrm{Cov}(\mathbf{x}_{\text{white}}) = \mathbf{I}$).
- **Local Contrast / Patch Normalization**: Normalizes spatial neighborhoods by local standard deviations, suppressing illumination variations in vision.

6.2.1 Centering and variance scaling

The two primary operations in feature conditioning are zero-centering and variance scaling. To understand their necessity, we examine their optimization consequences from both qualitative and quantitative perspectives.

1. Motivation of Zero-Centering: Eliminating Sign Coupling and Hessian Spread.

- **Qualitative Perspective**: If input features are uncentered and strictly positive (as is universal for raw image pixels $x \in [0, 255]$), every weight connected to an initial neuron is forced to change in the same direction. The optimizer cannot increase one weight while decreasing another in a single update step. If the loss minimum requires opposing sign adjustments, gradient descent is forced into an inefficient zigzagging pattern, bouncing back and forth across parameter quadrants.

- **Quantitative Derivation (Gradient Sign Coupling)**: Consider a neuron computing pre-activation $a = \mathbf{w}^\top \mathbf{x} + b$. By the multivariate chain rule, the gradient of the scalar loss $\mathcal{L}$ with respect to the weight vector $\mathbf{w} \in \mathbb{R}^D$ is:

$$\nabla_{\mathbf{w}} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \mathbf{w}} = \delta \mathbf{x}, \tag{6.41}$$

  where $\delta \equiv \frac{\partial \mathcal{L}}{\partial a} \in \mathbb{R}$ is the scalar error adjoint. The sign of each individual partial derivative is:

$$\operatorname{sign}\left(\frac{\partial \mathcal{L}}{\partial w_i}\right) = \operatorname{sign}(\delta) \cdot \operatorname{sign}(x_i). \tag{6.42}$$

  When $x_i > 0$ for all $i \in \{1, \ldots, D\}$, every component of $\nabla_{\mathbf{w}} \mathcal{L}$ shares the exact sign of δ:

$$\operatorname{sign}\left(\frac{\partial \mathcal{L}}{\partial w_i}\right) = \operatorname{sign}(\delta) \quad \forall i \in \{1, \ldots, D\}. \tag{6.43}$$

  The gradient vector $\nabla_{\mathbf{w}} \mathcal{L}$ is strictly confined to the positive cone $\mathcal{C}_+ = \{\mathbf{v} : v_i \geq 0\}$ or the negative cone $\mathcal{C}_- = \{\mathbf{v} : v_i \leq 0\}$. As illustrated in Figure 6.4, if the optimum $\mathbf{w}^*$ requires coordinate updates of opposite signs ($w_1 \uparrow, w_2 \downarrow$), gradient descent cannot point toward $\mathbf{w}^*$ and is forced to oscillate diagonally.

- **Quantitative Derivation (Hessian Condition Number Expansion)**: In a local quadratic approximation of the loss $\mathcal{L}(\mathbf{w}) \approx \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$, the Hessian matrix of second derivatives is:

$$\mathbf{H} = \frac{1}{N} \mathbf{X}^\top \mathbf{X} = \frac{1}{N} \sum_{n=1}^{N} \mathbf{x}_n \mathbf{x}_n^\top. \tag{6.44}$$

  If the dataset has an empirical non-zero mean vector $\boldsymbol{\mu} = \frac{1}{N} \sum_{n=1}^{N} \mathbf{x}_n \neq \mathbf{0}$, the second-moment matrix expands around the centered sample covariance $\boldsymbol{\Sigma} = \frac{1}{N} \sum_n (\mathbf{x}_n - \boldsymbol{\mu})(\mathbf{x}_n - \boldsymbol{\mu})^\top$:

$$\mathbf{H} = \boldsymbol{\Sigma} + \boldsymbol{\mu}\boldsymbol{\mu}^\top. \tag{6.45}$$

  The rank-one outer product $\boldsymbol{\mu}\boldsymbol{\mu}^\top$ adds an eigenvalue of magnitude $\|\boldsymbol{\mu}\|_2^2$ along the direction of $\boldsymbol{\mu}$. When raw features have large non-zero offsets (for example, raw pixels where $\mu_i \approx 128 \implies \|\boldsymbol{\mu}\|_2^2 \approx D \times 128^2$), the maximum eigenvalue of the Hessian inflates dramatically:

$$\lambda_{\max}(\mathbf{H}) \approx \lambda_{\max}(\boldsymbol{\Sigma}) + \|\boldsymbol{\mu}\|_2^2, \qquad \lambda_{\min}(\mathbf{H}) \approx \lambda_{\min}(\boldsymbol{\Sigma}). \tag{6.46}$$

  The resulting condition number of the Hessian explodes:

$$\kappa(\mathbf{H}) = \frac{\lambda_{\max}(\mathbf{H})}{\lambda_{\min}(\mathbf{H})} \approx \frac{\lambda_{\max}(\boldsymbol{\Sigma}) + \|\boldsymbol{\mu}\|_2^2}{\lambda_{\min}(\boldsymbol{\Sigma})} \gg 1. \tag{6.47}$$

  Since the convergence rate of first-order gradient descent is bounded by $\left(\frac{\kappa-1}{\kappa+1}\right)^2$, an uncentered non-zero mean $\boldsymbol{\mu}$ directly causes catastrophic optimization slowdown. Zero-centering sets $\boldsymbol{\mu} = \mathbf{0}$, annihilating the $\boldsymbol{\mu}\boldsymbol{\mu}^\top$ term and restoring $\mathbf{H} = \boldsymbol{\Sigma}$.

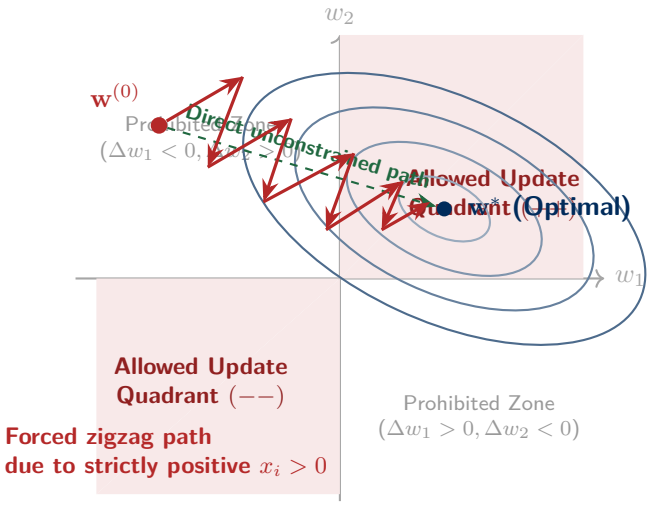


Figure 6.4: The geometric pathology of uncentered data. When input features are strictly positive ($x_i > 0$), parameter updates $\Delta \mathbf{w} = -\eta \frac{\partial \mathcal{L}}{\partial a} \mathbf{x}$ are restricted strictly to quadrants $(++)$ and $(--)$ (shaded regions). Reaching an optimum $\mathbf{w}^*$ situated in a mixed quadrant forces gradient descent into an inefficient, high-frequency zigzag trajectory. Zero-centering ($\mathbf{x} \leftarrow \mathbf{x} - \boldsymbol{\mu}$) eliminates this quadrant constraint, permitting direct gradient descent along the green dashed trajectory.

2. Motivation of Variance Rescaling: Equalizing Axis Curvature.

- **Qualitative Perspective**: If one input feature varies over thousands of units while another varies over fractional decimals, the loss surface forms an extremely steep, narrow canyon. Gradient updates with a uniform learning rate η bounce uncontrollably between the steep walls (along the high-variance feature) while crawling with imperceptible slowness along the flat canyon floor (along the low-variance feature).

- **Quantitative Derivation (Curvature Equalization)**: The diagonal elements of the Hessian matrix correspond directly to the second moments of the individual feature dimensions:

$$H_{ii} = \frac{\partial^2 \mathcal{L}}{\partial w_i^2} = \frac{1}{N} \sum_{n=1}^{N} x_{ni}^2 \approx \sigma_i^2 \quad \text{(for zero-centered features).} \tag{6.48}$$

  If feature 1 has variance σ_1^2 and feature 2 has variance σ_2^2 with $\sigma_1 \gg \sigma_2$, the condition number satisfies $\kappa(\mathbf{H}) \geq \sigma_1^2/\sigma_2^2$. The maximum stable learning rate is upper-bounded by the steepest curvature:

$$\eta < \frac{2}{\lambda_{\max}(\mathbf{H})} \approx \frac{2}{\sigma_1^2}. \tag{6.49}$$

  Under this small learning rate η, the parameter progress along the flat dimension w_2 per iteration is:

$$\Delta w_2 \approx -\eta \sigma_2^2 w_2 < -\frac{2\sigma_2^2}{\sigma_1^2} w_2, \tag{6.50}$$

  requiring $\mathcal{O}\left(\frac{\sigma_1^2}{\sigma_2^2}\right)$ update steps to converge. Dividing each feature by its empirical standard deviation σ_i scales the diagonal entries of the Hessian to $H_{ii} \approx 1$, equalizing axis curvature and enabling rapid, balanced convergence.

3. Evolution of Visual Normalization Conventions. In deep convolutional architectures, input preprocessing standards have evolved across model generations, transitioning from full 3D spatial images toward compact channel-wise scaling:

Architecture	Normalization Protocol	Statistics Shape	Practical Implementation
AlexNet (2012)	Full Mean Image Subtraction	$[H \times W \times C]$	$\mathbf{x} \leftarrow \mathbf{x} - \bar{\mathbf{X}}_{\text{pixel}}$
VGGNet (2014)	Per-Channel Mean Subtraction	[3] (RGB scalars)	$\mathbf{x}_c \leftarrow \mathbf{x}_c - \mu_c$
ResNet (2015)	Per-Channel Mean and Std Scaling	[3] means, [3] stds	$\mathbf{x}_c \leftarrow (\mathbf{x}_c - \mu_c)/\sigma_c$

Table 6.2: Comparison of input image preprocessing standards across landmark deep visual architectures. While early networks subtracted full 3D mean images, modern standard practice subtracts per-channel RGB means and scales by per-channel standard deviations.

For visual image recognition, the empirical golden rule is: **center only, or standardize per-channel**. Never attempt full dataset variance normalization across arbitrary dimensions, and never apply PCA whitening.

Mathematical Definition of Centering and Variance Scaling: Formally, let $\mathbf{x} \in \mathbb{R}^D$ denote an input feature vector. The standard preprocessing pipeline applies two consecutive linear transformations:

$$\mathbf{x}_{\text{centered}} = \mathbf{x} - \boldsymbol{\mu}, \qquad \mathbf{x}_{\text{norm}} = \frac{\mathbf{x} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}, \tag{6.51}$$

where $\boldsymbol{\mu} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{x}_i$ is the empirical dataset mean vector, and $\boldsymbol{\sigma} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\mathbf{x}_i - \boldsymbol{\mu})^2}$ is the empirical standard deviation vector (computed element-wise).

Figure 6.5 visually demonstrates the geometric progression of these two transformations on a two-dimensional feature distribution.

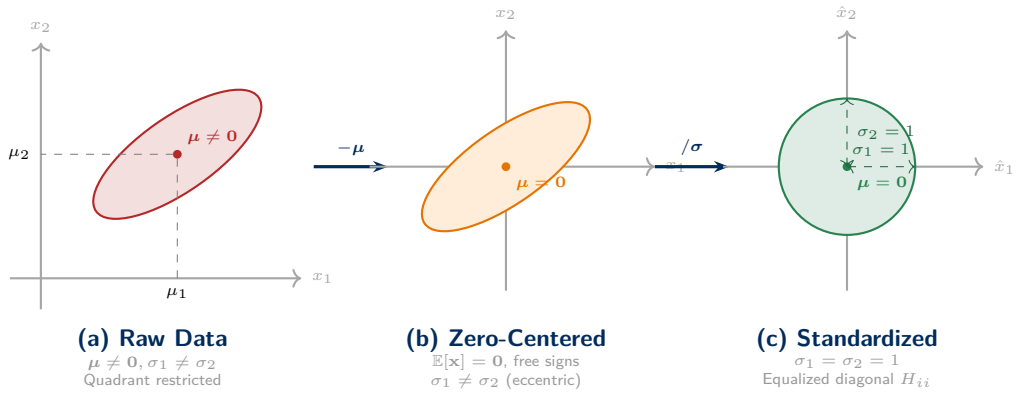


Figure 6.5: The geometric progression of feature conditioning. (a) Raw input features are situated in the positive quadrant with non-zero mean $\boldsymbol{\mu} \neq \mathbf{0}$ and unequal variances $\sigma_1 \neq \sigma_2$. (b) Zero-centering subtracts $\boldsymbol{\mu}$, shifting the empirical centroid to the origin $(0, 0)$ and restoring full 360-degree gradient directional freedom. (c) Variance scaling divides by $\boldsymbol{\sigma}$, normalizing variances to 1 along all coordinate axes and equalizing the diagonal curvature of the loss surface.

6.2.2 PCA and whitening

While zero-centering and coordinate-wise variance scaling normalize each dimension independently, they operate strictly along the canonical coordinate axes. In real-world data, features frequently exhibit strong cross-correlations. To tackle this, we examine PCA and Whitening from qualitative and quantitative perspectives.

1. Motivation of PCA and Whitening: Oblique Correlations and Complete Sphering.

- **Qualitative Perspective**: Standard variance standardization scales the bounding box of the data along the x_1 and x_2 axes. However, if x_1 and x_2 are strongly correlated (such as adjacent pixels in an image or correlated physical measurements), the data cloud forms an oblique, tilted ellipse. In this case, the Hessian matrix contains large off-diagonal entries, causing gradient descent to oscillate across the tilted ridge. PCA decorrelation rotates the coordinate frame to align with the ellipse's principal axes, and Whitening stretches every axis to unit variance, turning the elliptical ravine into a perfectly round spherical bowl where gradient descent points straight to the minimum.

- **Quantitative Derivation (Eigendecomposition and Decorrelation)**: Given a zero-centered data matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$, the empirical sample covariance matrix is:

$$\mathbf{\Sigma} = \frac{1}{N}\mathbf{X}^\top \mathbf{X} \in \mathbb{R}^{D \times D}. \tag{6.52}$$

  Because $\mathbf{\Sigma}$ is real, symmetric, and positive semi-definite, the spectral theorem guarantees an orthonormal eigendecomposition:

$$\mathbf{\Sigma} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top, \tag{6.53}$$

  where $\mathbf{U} \in \mathbb{R}^{D \times D}$ is an orthogonal matrix whose columns $\mathbf{u}_1, \ldots, \mathbf{u}_D$ are the orthonormal eigenvectors ($\mathbf{U}^\top \mathbf{U} = \mathbf{I}$), and $\mathbf{\Lambda} = \text{diag}(\lambda_1, \ldots, \lambda_D)$ contains the non-negative eigenvalues representing the variance along each principal axis ($\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D \geq 0$).

- **Quantitative Derivation (Whitening and Isotropic Hessian)**: Projecting the zero-centered data onto the eigenvector basis defines the PCA-rotated representation:

$$\mathbf{x}_{\text{rot}} = \mathbf{U}^\top \mathbf{x}. \tag{6.54}$$

  The covariance of $\mathbf{x}_{\text{rot}}$ is completely diagonalized:

$$\text{Cov}(\mathbf{x}_{\text{rot}}) = \frac{1}{N}\mathbf{U}^\top \mathbf{X}^\top \mathbf{X}\mathbf{U} = \mathbf{U}^\top(\mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top)\mathbf{U} = \mathbf{\Lambda}. \tag{6.55}$$

  All off-diagonal linear correlations are completely eliminated: $\text{Cov}(x_{\text{rot},i}, x_{\text{rot},j}) = 0$ for all $i \neq j$.

  The whitening transformation subsequently scales each decorrelated principal coordinate by the inverse square root of its eigenvalue:

$$\mathbf{x}_{\text{white}} = \mathbf{\Lambda}^{-1/2}\mathbf{x}_{\text{rot}} = \mathbf{\Lambda}^{-1/2}\mathbf{U}^\top \mathbf{x}. \tag{6.56}$$

  The covariance matrix of the whitened representation evaluates to the exact identity matrix:

$$\text{Cov}(\mathbf{x}_{\text{white}}) = \mathbf{\Lambda}^{-1/2}\,\text{Cov}(\mathbf{x}_{\text{rot}})\mathbf{\Lambda}^{-1/2} = \mathbf{\Lambda}^{-1/2}\mathbf{\Lambda}\mathbf{\Lambda}^{-1/2} = \mathbf{I}_D. \tag{6.57}$$

  Consequently, the empirical Hessian matrix of a quadratic objective computed on whitened features is perfectly isotropic:

$$\mathbf{H}_{\text{white}} = \frac{1}{N}\mathbf{X}_{\text{white}}^\top \mathbf{X}_{\text{white}} = \mathbf{I}_D \implies \kappa(\mathbf{H}_{\text{white}}) = \frac{\lambda_{\max}}{\lambda_{\min}} = 1. \tag{6.58}$$

  With a condition number of exactly 1, the loss surface consists of concentric hyperspherical contours. The negative gradient vector points directly toward the global minimum $\mathbf{w}^*$, enabling gradient descent with learning rate $\eta = 1$ to converge in a single step!

Figure 6.6 illustrates the mathematical progression from correlated zero-centered data to decorrelated PCA coordinates and finally to the isotropic whitened distribution.

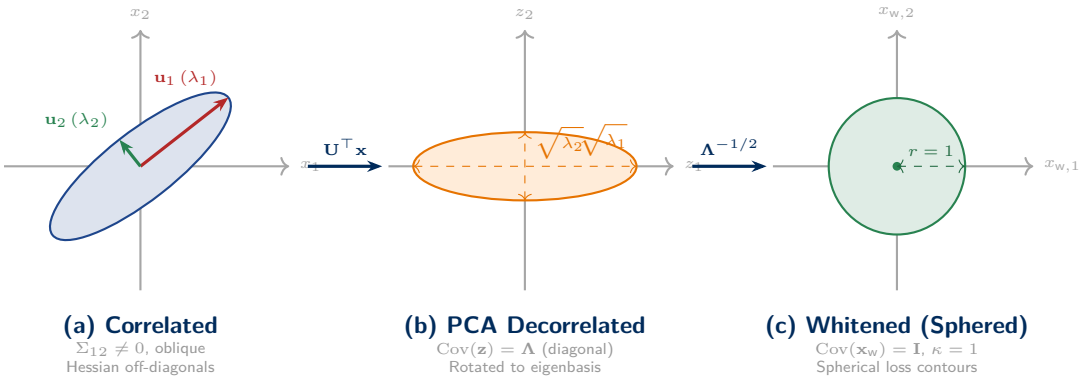


Figure 6.6: The geometric mechanism of PCA decorrelation and whitening. (a) Correlated zero-centered data forming an oblique ellipse with eigenvectors $\mathbf{u}_1, \mathbf{u}_2$ and eigenvalues λ_1, λ_2. (b) PCA projection ($\mathbf{U}^\top\mathbf{x}$) rotates the coordinate system into the principal eigenbasis, eliminating cross-covariance ($\mathrm{Cov}(\mathbf{z}) = \mathbf{\Lambda}$). (c) Whitening ($\mathbf{\Lambda}^{-1/2}\mathbf{U}^\top\mathbf{x}$) scales each principal component by $1/\sqrt{\lambda_i}$, yielding an isotropic unit hypersphere where the Hessian condition number is $\kappa = 1$.

While PCA and whitening are mathematically appealing for tabular models and linear classifiers, **they are strictly avoided in Convolutional Neural Networks**. Why? As proved in Chapter 5, CNNs rely fundamentally on the **spatial stationarity and 2D topology** of natural images: a 3×3 kernel assumes that adjacent pixels in a row (i, j) and $(i, j+1)$ share identical statistical properties regardless of where they appear on the grid.
Projecting an image through the global eigenvector matrix $\mathbf{U}^\top$ completely scrambles the spatial arrangement of pixels: pixels on opposite sides of the image are linearly blended into abstract global components. Whitening destroys spatial stationarity, spatial locality, and translation equivariance, shattering the fundamental inductive bias that gives CNNs their visual power. Therefore, CNNs utilize exclusively per-channel zero-centering and scaling.

6.3 Weight initialization

Having preprocessed the input data, how should the network parameters $\mathbf{W}$ and $\mathbf{b}$ be initialized at iteration $t = 0$?

In convex optimization (such as linear regression or logistic regression, derived in Chapter 3), the loss landscape consists of a single global basin; regardless of where the initial parameters $\mathbf{w}^{(0)}$ are placed, gradient descent is mathematically guaranteed to slide down to the unique global minimum. In that setting, initialization affects only the number of convergence steps, not the identity of the final solution.

In deep neural networks, the situation is fundamentally different. The loss landscape is an extraordinarily complex, high-dimensional non-convex manifold in $\mathbb{R}^P$ ($P \sim 10^6$–10^{11} parameters), characterized by:

1. **Proliferation of Basins of Attraction**: The parameter space is partitioned into countless distinct local minima of vastly different generalization quality. Wide, flat minima generalize well to unseen test distributions, whereas sharp, steep ravines severely overfit to training noise. The initial parameter configuration $\mathbf{W}^{(0)}$ acts as the deterministic seed that decides which basin of attraction the optimizer enters.

2. **Saddle Points and Flat Plateaus**: High-dimensional loss surfaces are dominated by saddle points (where gradients vanish in several directions) and expansive flat plateaus. A flawed initialization can place the network directly into a flat regime where gradients are numerical zeros, trapping the optimizer indefinitely before meaningful learning begins.

3. **Dynamical Signal Survival Across Depth**: In deep feedforward cascades, the forward signal must traverse dozens or hundreds of linear transformations and non-linearities without collapsing to zero or exploding to infinity. Likewise, the backward gradient adjoints must survive backpropagation across the entire depth to deliver informative learning signals to the earliest feature extractors.

4. **Architectural Symmetry**: Hidden layers comprise collections of identical parallel units. If parameters are initialized symmetrically, these units receive identical gradient updates and compute identical features forever, reducing the representational power of a 1,000-unit layer to that of a single neuron.

Therefore, weight initialization is not a cosmetic hyperparameter; it is the fundamental boundary condition that governs gradient survival, breaks symmetry, and steers optimization into generalizable basins of attraction.

6.3.1 Zero and small random initialization

To appreciate the theoretical principles underlying modern initialization, we inspect the failure modes of naive initialization strategies solution-by-solution:

1. The Zero-Initialization Disaster: Symmetry Trap. A naive student might suggest setting all weights and biases to zero: $\mathbf{W}^{(l)} = \mathbf{0}, \mathbf{b}^{(l)} = \mathbf{0}$.

Consider what happens in a fully connected layer: on the forward pass, pre-activations evaluate to $\mathbf{z}^{(1)} = \mathbf{0} \cdot \mathbf{x} + \mathbf{0} = \mathbf{0}$. For any activation function (e.g., ReLU or Tanh), all hidden neurons in the layer output the exact same constant: $a_1^{(1)} = a_2^{(1)} = \cdots = a_{d_1}^{(1)}$.

On the backward pass, by Equation (4.56), the gradient with respect to weight $W_{ji}^{(1)}$ is $\frac{\partial \mathcal{L}}{\partial W_{ji}^{(1)}} = \delta_j^{(1)} x_i$. Because all neurons computed identical values, all error adjoints are identical: $\delta_1^{(1)} = \delta_2^{(1)} = \cdots = \delta_{d_1}^{(1)}$. Consequently, all weights connected to input x_i receive the exact same gradient update:

$$W_{1i}^{(1)} \leftarrow W_{1i}^{(1)} - \eta g_i, \quad W_{2i}^{(1)} \leftarrow W_{2i}^{(1)} - \eta g_i \implies W_{1i}^{(1)} = W_{2i}^{(1)} \quad \forall t. \tag{6.59}$$

The neurons compute identical features forever; **symmetry is never broken**. Stacking 1,000 neurons per layer behaves as if there were only a single neuron. Therefore, **symmetry breaking via stochastic randomization is mathematically mandatory**.

2. Small Random Initialization ($\mathbf{W} \sim \mathcal{N}(0, 0.01^2)$): Signal Collapse. To break symmetry, an intuitive heuristic is to sample weights from a zero-mean Gaussian with small variance: $W_{ij} \sim \mathcal{N}(0, 0.01^2)$.

Consider a 10-layer network with 500 neurons per layer using tanh activations. In the first layer, the pre-activation of neuron j is $z_j = \sum_{i=1}^{500} W_{ji} x_i$. Assuming inputs have unit variance ($\mathrm{Var}(x_i) = 1$):

$$\mathrm{Var}(z_j) = \sum_{i=1}^{500} \mathrm{Var}(W_{ji}) \, \mathrm{Var}(x_i) = 500 \times 0.01^2 \times 1 = 500 \times 0.0001 = 0.05. \tag{6.60}$$

The standard deviation of activations is $\sqrt{0.05} \approx 0.22$. In the next layer, this tiny variance is multiplied again by 0.05:

$$\mathrm{Var}(a^{(l)}) \approx (0.05)^l \, \mathrm{Var}(a^{(0)}). \tag{6.61}$$

By layer 6, $\mathrm{Var}(a^{(6)}) \approx (0.05)^6 \approx 1.5 \times 10^{-8}$. All activations collapse to zero! On the backward pass, because gradients are proportional to activation magnitudes ($\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \boldsymbol{\delta}\mathbf{a}^\top$), parameter gradients vanish to zero. Learning completely halts.

3. Large Random Initialization ($\mathbf{W} \sim \mathcal{N}(0, 1.0^2)$): Saturation Trap. Conversely, if weights are initialized too large ($W_{ij} \sim \mathcal{N}(0, 1^2)$), $\mathrm{Var}(z_j) = 500 \times 1 \times 1 = 500$. The pre-activations have standard deviation $\sqrt{500} \approx 22.3$. All neurons are shoved deep into the saturated asymptotes of tanh ($|z| \gg 2.5 \implies \tanh(z) \approx \pm 1$). Derivatives vanish to zero ($\tanh'(z) \approx 0$), killing all backward gradient flow.

Figure 6.7 provides an intuitive visual synthesis of these three naive initialization pathologies.

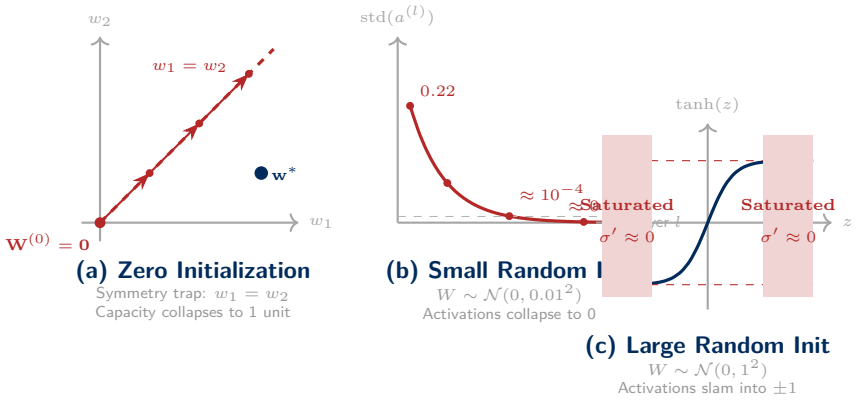


Figure 6.7: Visual breakdown of naive parameter initialization pathologies. (a) **Zero initialization**: weights are locked to the diagonal line $w_1 = w_2$; symmetry is never broken, collapsing multi-neuron capacity to a single unit. (b) **Small random initialization**: activation variance shrinks exponentially across depth ($\mathrm{Var}(a^{(l)}) \to 0$), driving gradients to zero. (c) **Large random initialization**: pre-activations explode ($|z| \gg 2.5$), pushing neurons into the saturated asymptotes of tanh where local derivatives vanish ($\sigma' \approx 0$).

6.3.2 Xavier / Glorot initialization

To eliminate both variance collapse and saturation, Xavier Glorot and Yoshua Bengio [30] formulated the **variance preservation principle**: **the variance of activations on the forward pass must remain constant across all layers, and the variance of error gradients on the backward pass must remain constant across all layers**:

$$\mathrm{Var}(a^{(l)}) = \mathrm{Var}(a^{(l-1)}) \quad \forall l, \qquad \mathrm{Var}(\delta^{(l)}) = \mathrm{Var}(\delta^{(l+1)}) \quad \forall l. \tag{6.62}$$

Theorem 6.1 (Xavier / Glorot Initialization Formula)**.** *For a linear or symmetric activation function (such as* tanh *in its linear regime around zero), satisfying the forward and backward variance preservation criteria dictates that weights must be sampled from a distribution with variance:*

$$\mathrm{Var}(W) = \frac{2}{n_{in} + n_{out}}, \tag{6.63}$$

where n_{in} denotes the fan-in (number of input connections to the layer) and n_{out} denotes the fan-out (number of output connections).

Proof. Consider a feedforward layer computing pre-activation $z_j = \sum_{i=1}^{n_{\text{in}}} W_{ji} a_i$. We assume:

1. Weights W_{ji} are independent and identically distributed with zero mean: $\mathbb{E}[W_{ji}] = 0$.
2. Activations a_i are independent and identically distributed with zero mean: $\mathbb{E}[a_i] = 0$.
3. Weights and activations are mutually independent: $\mathbb{E}[W_{ji}a_i] = \mathbb{E}[W_{ji}]\mathbb{E}[a_i] = 0$.

By the variance of the product of independent zero-mean random variables:

$$\mathrm{Var}(W_{ji}a_i) = \mathbb{E}[(W_{ji}a_i)^2] - (\mathbb{E}[W_{ji}a_i])^2 = \mathbb{E}[W_{ji}^2]\mathbb{E}[a_i^2] = \mathrm{Var}(W)\,\mathrm{Var}(a). \tag{6.64}$$

Summing across all n_{in} independent input connections:

$$\mathrm{Var}(z_j) = \sum_{i=1}^{n_{\text{in}}} \mathrm{Var}(W_{ji}a_i) = n_{\text{in}}\,\mathrm{Var}(W)\,\mathrm{Var}(a). \tag{6.65}$$

Assuming the activation operates in its linear regime near the origin ($\sigma'(0) \approx 1 \implies a_j \approx z_j$), we have $\mathrm{Var}(a_j) \approx \mathrm{Var}(z_j)$. Thus:

$$\mathrm{Var}(a^{(l)}) = n_{\text{in}}\,\mathrm{Var}(W^{(l)})\,\mathrm{Var}(a^{(l-1)}). \tag{6.66}$$

To ensure the activation variance does not collapse or explode ($\mathrm{Var}(a^{(l)}) = \mathrm{Var}(a^{(l-1)})$), we require:

$$n_{\text{in}}\,\mathrm{Var}(W) = 1 \implies \mathrm{Var}(W) = \frac{1}{n_{\text{in}}}. \tag{6.67}$$

Now consider the backward pass. By Equation (6.11), the error signal backpropagates as $\delta_i^{(l)} = \sum_{j=1}^{n_{\text{out}}} W_{ji}^{(l+1)}\delta_j^{(l+1)}\sigma'(z_i^{(l)})$. Assuming $\sigma'(z) \approx 1$ and applying identical independence arguments:

$$\mathrm{Var}(\delta^{(l)}) = n_{\text{out}}\,\mathrm{Var}(W^{(l+1)})\,\mathrm{Var}(\delta^{(l+1)}). \tag{6.68}$$

Preserving backward gradient variance ($\mathrm{Var}(\delta^{(l)}) = \mathrm{Var}(\delta^{(l+1)})$) requires:

$$n_{\text{out}}\,\mathrm{Var}(W) = 1 \implies \mathrm{Var}(W) = \frac{1}{n_{\text{out}}}. \tag{6.69}$$

Unless $n_{\text{in}} = n_{\text{out}}$, we cannot simultaneously satisfy both conditions. Glorot and Bengio proposed taking the **harmonic mean** of the two conditions:

$$\mathrm{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}}. \tag{6.70}$$

□

Practical Implementation Distributions:

- **Xavier Gaussian**: Sample from $\mathcal{N}\left(0,\, \sigma^2 = \frac{2}{n_{\text{in}}+n_{\text{out}}}\right)$.
- **Xavier Uniform**: For a uniform distribution $U[-a, a]$, variance is $\frac{(2a)^2}{12} = \frac{a^2}{3}$. Equating $\frac{a^2}{3} = \frac{2}{n_{\text{in}}+n_{\text{out}}}$ yields:

$$W \sim U\left[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}},\, +\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right]. \tag{6.71}$$

6.3.3 He / Kaiming initialization

While Xavier initialization solved training for Sigmoid and Tanh, it **catastrophically failed when applied to ReLU networks**. Why?

Glorot's derivation assumed that the activation is linear and preserves variance ($\mathbb{E}[\sigma(z)^2] \approx \mathbb{E}[z^2]$). But ReLU sets the entire negative half-space to zero! Assuming symmetric zero-mean pre-activations $z \sim \mathcal{N}(0, \sigma_z^2)$:

$$\mathbb{E}[\mathrm{ReLU}(z)^2] = \int_{-\infty}^{\infty} \max(0, z)^2 p(z)\, dz = \int_0^{\infty} z^2 p(z)\, dz = \frac{1}{2}\int_{-\infty}^{\infty} z^2 p(z)\, dz = \frac{1}{2}\,\mathrm{Var}(z). \tag{6.72}$$

ReLU cuts the variance of activations exactly in half at every layer!

Under Xavier initialization ($n_{\text{in}} \operatorname{Var}(W) = 1$), the activation variance evolves as:

$$\operatorname{Var}(a^{(l)}) = \frac{1}{2} n_{\text{in}} \operatorname{Var}(W) \operatorname{Var}(a^{(l-1)}) = \frac{1}{2} \operatorname{Var}(a^{(l-1)}). \tag{6.73}$$

In a 20-layer ReLU network, Xavier initialization causes activations to decay by a factor of $\left(\frac{1}{2}\right)^{20} \approx 9.5 \times 10^{-7}$, extinguishing all signals.

To fix this, Kaiming He et al. [35] introduced **He (Kaiming) Initialization**: to compensate for the factor of $\frac{1}{2}$, **the weight variance must be doubled**:

$$\frac{1}{2} n_{\text{in}} \operatorname{Var}(W) = 1 \implies \operatorname{Var}(W) = \frac{2}{n_{\text{in}}}. \tag{6.74}$$

Practical Implementation Distributions:

- **He Gaussian**: Sample weights from $\mathcal{N}\left(0, \sigma^2 = \frac{2}{n_{\text{in}}}\right)$.
- **He Uniform**: Sample weights from $U\left[-\sqrt{\frac{6}{n_{\text{in}}}}, +\sqrt{\frac{6}{n_{\text{in}}}}\right]$.
- For convolutional layers: $n_{\text{in}} = K_h \times K_w \times C_{\text{in}}$ (the filter spatial area times input channels).

He initialization was the mathematical key that allowed Kaiming He to train the revolutionary 152-layer ResNet, winning the ImageNet 2015 challenge.

6.4 Batch Normalization

Even with proper He initialization and input data conditioning, training deep networks remains notoriously delicate. In a deep architecture, each intermediate layer receives as its input the activations produced by the preceding layer. As gradient descent simultaneously updates the parameters of all layers at every training iteration, a critical optimization dilemma emerges: **the distribution of inputs to each layer changes continuously throughout training**.

Sergey Ioffe and Christian Szegedy [47] termed this phenomenon **Internal Covariate Shift**:

$$\text{Distribution of intermediate representations } \mathbf{x}^{(l)} \text{ drifts across training iterations } t. \tag{6.75}$$

From a qualitative perspective, internal covariate shift represents a severe optimization bottleneck for several key reasons:

1. **Chasing a Moving Target**: Each layer is tasked with learning a stable representation of high-level features. However, because the underlying coordinate system and statistical distribution of its inputs are constantly drifting, the layer must continually re-adapt its weights to an ever-changing baseline. An update that was optimal at iteration t becomes suboptimal at iteration $t+1$ simply because earlier layers altered their outputs.

2. **Compounding Distributional Drift Across Depth**: Subtle parameter adjustments in the initial layers compound exponentially through cascading non-linearities. For deeper layers ($l \gg 1$), input statistics can fluctuate wildly between mini-batches, destabilizing representation learning.

3. **Activation Saturation and Vanishing Gradients**: Uncontrolled distribution drift frequently pushes activations into the saturated asymptotes of activation functions (for Sigmoid and Tanh, into flat tails where derivatives vanish; for ReLU, into the negative dead zone), causing gradient flow to freeze in deeper subgraphs.

4. **Constrained Optimization Schedules**: To prevent internal distributions from swinging out of control, practitioners are forced to use painfully small learning rates, drag out training across weeks, and rely on hyper-sensitive initialization schemes.

6.4.1 Batch Normalization mechanics

To eliminate internal covariate shift, Ioffe and Szegedy [47] proposed a profound architectural solution: **forcefully normalize intermediate representations directly within the network architecture**.

Figure 6.8 illustrates the structural insertion of Batch Normalization into a neural network layer. In standard architectures, the linear transformation $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ connects directly to the activation function $\sigma(\mathbf{z})$. When Batch Normalization is inserted, the layer is decomposed into a linear operation without bias ($\mathbf{z} = \mathbf{W}\mathbf{x}$), followed immediately by the Batch Normalization operator, and finally passed into the non-linear activation $\sigma(\mathbf{y})$.

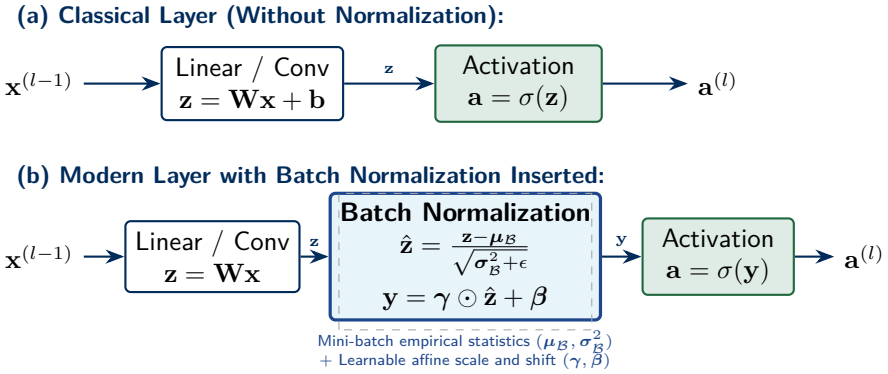


Figure 6.8: Architectural insertion of Batch Normalization into a deep network layer. (a) In classical layers, pre-activations $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ are fed directly into non-linear activations $\sigma(\mathbf{z})$, leaving intermediate distributions unconstrained. (b) Batch Normalization is inserted directly between the linear operation and the non-linearity; the linear bias $\mathbf{b}$ is eliminated (as it is subtracted out by centering), and the normalized activations are scaled by $\boldsymbol{\gamma}$ and shifted by $\boldsymbol{\beta}$ before non-linear activation.

Mathematically, given a mini-batch of activations $\mathcal{B} = \{\mathbf{x}_1, \ldots, \mathbf{x}_m\}$ where each $\mathbf{x}_i \in \mathbb{R}^d$, the Batch Normalization operator applies four sequential operations:

Algorithm 3 Batch Normalization Forward Pass (Ioffe & Szegedy, 2015)

Require: Mini-batch activations $\mathcal{B} = \{\mathbf{x}_1, \ldots, \mathbf{x}_m\}$ where $\mathbf{x}_i \in \mathbb{R}^d$; learnable parameters $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$; numerical stability constant $\epsilon > 0$.

1: Compute mini-batch empirical mean vector:
$\boldsymbol{\mu}_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} \mathbf{x}_i \in \mathbb{R}^d$.
2: Compute mini-batch empirical variance vector:
$\boldsymbol{\sigma}^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} (\mathbf{x}_i - \boldsymbol{\mu}_{\mathcal{B}})^2 \in \mathbb{R}^d$.
3: Normalize activations to zero mean and unit variance:
$\hat{\mathbf{x}}_i \leftarrow \frac{\mathbf{x}_i - \boldsymbol{\mu}_{\mathcal{B}}}{\sqrt{\boldsymbol{\sigma}^2_{\mathcal{B}}+\epsilon}} \in \mathbb{R}^d$.
4: Apply learnable affine scale and shift:
$\mathbf{y}_i \leftarrow \boldsymbol{\gamma} \odot \hat{\mathbf{x}}_i + \boldsymbol{\beta} \equiv \mathrm{BN}_{\gamma,\beta}(\mathbf{x}_i)$.
5: **return** $\{\mathbf{y}_1, \ldots, \mathbf{y}_m\}$.

Why Learnable Scale $\boldsymbol{\gamma}$ and Shift $\boldsymbol{\beta}$ are Indispensable: A critical conceptual question arises: *if the goal is zero-mean unit-variance activations, why do we multiply by $\boldsymbol{\gamma}$ and add $\boldsymbol{\beta}$ in Step 4?*

If we restricted the layer strictly to pure normalization $\hat{\mathbf{x}}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, we would permanently strip the network of expressive power! For instance, if a sigmoid activation receives strictly

$\mathcal{N}(0,1)$ inputs, all values fall within $[-2,2]$, confining the sigmoid to its linear regime around the origin ($\sigma(z) \approx 0.5 + 0.25z$). The network would lose the ability to utilize the non-linear curvature of the activation function.

The parameters $\boldsymbol{\gamma} \in \mathbb{R}^d$ (scale) and $\boldsymbol{\beta} \in \mathbb{R}^d$ (shift) are **learnable weights optimized end-to-end via backpropagation**. They grant the network complete representational freedom:

$$\text{If the optimizer learns } \boldsymbol{\gamma} = \sqrt{\boldsymbol{\sigma}_{\mathcal{B}}^2 + \epsilon} \quad \text{and} \quad \boldsymbol{\beta} = \boldsymbol{\mu}_{\mathcal{B}}, \tag{6.76}$$

then $\mathbf{y}_i = \boldsymbol{\gamma}\hat{\mathbf{x}}_i + \boldsymbol{\beta} = \mathbf{x}_i$ **identically recovers the original unnormalized input**! Batch Normalization does not permanently constrain the layer; it provides an identity bypass, allowing the network to decide precisely how much normalization each individual feature dimension requires.

Analytical Backpropagation through Batch Normalization: Because Batch Normalization is an internal network layer, error gradients must backpropagate through all normalization steps. Applying the multivariate chain rule:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\gamma}} = \sum_{i=1}^{m} \frac{\partial \mathcal{L}}{\partial \mathbf{y}_i} \odot \hat{\mathbf{x}}_i, \tag{6.77}$$

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\beta}} = \sum_{i=1}^{m} \frac{\partial \mathcal{L}}{\partial \mathbf{y}_i}. \tag{6.78}$$

Differentiating with respect to the normalized input $\hat{\mathbf{x}}_i$:

$$\frac{\partial \mathcal{L}}{\partial \hat{\mathbf{x}}_i} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}_i} \odot \boldsymbol{\gamma}. \tag{6.79}$$

Differentiating through the variance $\boldsymbol{\sigma}_{\mathcal{B}}^2$ and mean $\boldsymbol{\mu}_{\mathcal{B}}$ yields the exact closed-form gradient with respect to input $\mathbf{x}_i$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_i} = \frac{\boldsymbol{\gamma}}{m\sqrt{\boldsymbol{\sigma}_{\mathcal{B}}^2 + \epsilon}} \left[m\frac{\partial \mathcal{L}}{\partial \hat{\mathbf{x}}_i} - \sum_{j=1}^{m} \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{x}}_j} - \hat{\mathbf{x}}_i \odot \sum_{j=1}^{m} \left(\frac{\partial \mathcal{L}}{\partial \hat{\mathbf{x}}_j} \odot \hat{\mathbf{x}}_j \right) \right]. \tag{6.80}$$

This formulation guarantees that gradient signals propagate smoothly through the normalization operators during training without numerical instability.

Train-Time vs. Test-Time Mechanics: Running Statistics and Layer Folding: A critical implementation bug in deep learning arises from confusing the behavior of Batch Normalization at **training time** versus **test (inference) time**:

At test time, the model is frequently deployed to classify a **single sample** ($\mathbf{x} \in \mathbb{R}^d$, batch size $m = 1$).

- If we attempted to compute empirical batch mean and variance on a single sample, we would obtain $\boldsymbol{\mu} = \mathbf{x}$ and $\boldsymbol{\sigma}^2 = \mathbf{0}$, producing $\hat{\mathbf{x}} = \frac{\mathbf{x}-\mathbf{x}}{0} \implies$ total numerical collapse!
- Furthermore, a model's prediction on sample $\mathbf{x}_A$ should depend *only* on $\mathbf{x}_A$, not on which other test samples happen to be in the same mini-batch.

During training, the layer continuously maintains running estimates of the global population mean $\boldsymbol{\mu}_{\text{run}}$ and variance $\boldsymbol{\sigma}^2_{\text{run}}$ via exponential moving averages (momentum $\rho \approx 0.1$ or 0.01):

$$\boldsymbol{\mu}_{\text{run}} \leftarrow (1-\rho)\boldsymbol{\mu}_{\text{run}} + \rho\boldsymbol{\mu}_{\mathcal{B}}, \tag{6.81}$$

$$\boldsymbol{\sigma}^2_{\text{run}} \leftarrow (1-\rho)\boldsymbol{\sigma}^2_{\text{run}} + \rho\boldsymbol{\sigma}^2_{\mathcal{B}}. \tag{6.82}$$

At test time, $\boldsymbol{\mu}_{\text{run}}$ and $\boldsymbol{\sigma}^2_{\text{run}}$ are **frozen**. Inference replaces mini-batch statistics with these fixed global constants:

$$\mathbf{y}_{\text{test}} = \frac{\boldsymbol{\gamma}}{\sqrt{\boldsymbol{\sigma}^2_{\text{run}} + \epsilon}} \odot \mathbf{x} + \left(\boldsymbol{\beta} - \frac{\boldsymbol{\gamma} \odot \boldsymbol{\mu}_{\text{run}}}{\sqrt{\boldsymbol{\sigma}^2_{\text{run}} + \epsilon}} \right). \tag{6.83}$$

Notice that Equation (6.83) is a pure affine transformation: $\mathbf{y} = \mathbf{w}_{\text{eff}} \odot \mathbf{x} + \mathbf{b}_{\text{eff}}$. In production deployment (TensorRT, ONNX), the Batch Normalization layer is mathematically **folded (absorbed)** directly into the preceding linear or convolutional weight matrix:

$$\mathbf{W}_{\text{fused}} = \frac{\boldsymbol{\gamma}}{\sqrt{\boldsymbol{\sigma}^2_{\text{run}} + \epsilon}}\mathbf{W}, \quad \mathbf{b}_{\text{fused}} = \frac{\boldsymbol{\gamma}}{\sqrt{\boldsymbol{\sigma}^2_{\text{run}} + \epsilon}}(\mathbf{b} - \boldsymbol{\mu}_{\text{run}}) + \boldsymbol{\beta}. \tag{6.84}$$

At test time, Batch Normalization incurs **zero computational latency and zero memory overhead**!

6.4.2 The normalization family

Spatial Batch Normalization for Convolutional Networks: In fully connected networks, activations have shape (N, D), and normalization is computed across the batch dimension N independently for each of the D feature coordinates.

In convolutional networks, intermediate feature maps form 4D tensors of shape (N, C, H, W). How should batch normalization operate on visual grids?

Recall from Section 5.3.1 that a convolutional filter is applied uniformly across all spatial coordinates (h, w) to enforce translation equivariance. If we computed separate normalization statistics for each spatial location (h, w), adjacent pixels would be normalized by different scales, destroying spatial translation equivariance!

To preserve the convolutional inductive bias, **Spatial Batch Normalization** computes statistics across the batch dimension N **AND** all spatial locations $H \times W$ simultaneously:

$$\text{Total effective samples per channel: } m = N \times H \times W. \tag{6.85}$$

For each channel $c \in \{1, \ldots, C\}$:

$$\mu_c = \frac{1}{NHW} \sum_{n=1}^{N} \sum_{h=1}^{H} \sum_{w=1}^{W} X_{n,c,h,w}, \quad \sigma_c^2 = \frac{1}{NHW} \sum_{n=1}^{N} \sum_{h=1}^{H} \sum_{w=1}^{W} (X_{n,c,h,w} - \mu_c)^2. \tag{6.86}$$

Thus, a convolutional layer with C channels has exactly C scale parameters γ_c and C shift parameters β_c.

Alternative Normalization Paradigms: Decoupling from Batch Size: While Batch Normalization revolutionized deep CNNs, it suffers from a major weakness: **dependence on batch size** N. When training massive models (e.g., high-resolution medical imaging, 3D vision, or object detection) where GPU memory limits the batch size to $N \leq 4$, empirical mini-batch estimates $\boldsymbol{\mu}_{\mathcal{B}}, \boldsymbol{\sigma}^2_{\mathcal{B}}$ become noisy and unstable, causing training to fail.

To decouple normalization from batch size, alternative architectures alter the dimensions over which statistics are computed (illustrated in Figure 6.9):

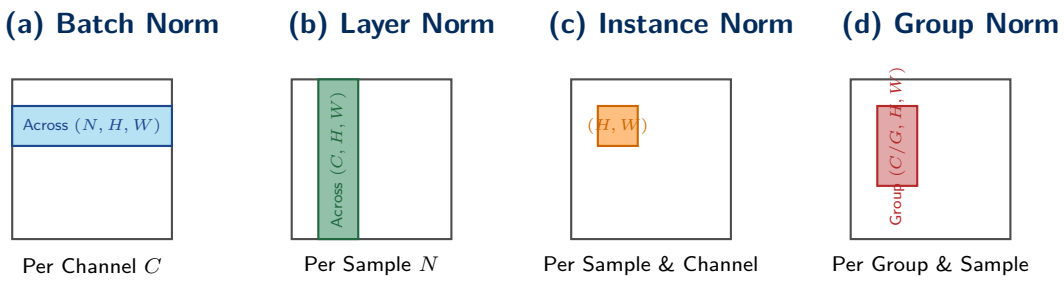


Figure 6.9: Taxonomy of normalization methods across the 4D feature tensor (N, C, H, W). (a) Batch Normalization normalizes across batch and spatial dimensions for each channel. (b) Layer Normalization normalizes across all channels and spatial dimensions for each independent sample (batch-size independent; standard in Transformers). (c) Instance Normalization normalizes across spatial dimensions independently for each channel and sample (standard in style transfer). (d) Group Normalization divides channels into groups G and normalizes across group channels and space (batch-size independent; robust for small mini-batches in CNNs).

1. **Layer Normalization (LN)** [2]: Computes statistics across all channels C and spatial dimensions for a **single observation** i:

$$\mu_i = \frac{1}{C}\sum_{c=1}^{C} x_{i,c}, \quad \sigma_i^2 = \frac{1}{C}\sum_{c=1}^{C}(x_{i,c} - \mu_i)^2. \tag{6.87}$$

   LN is completely independent of batch size N. It behaves identically at training time and test time, making it the universal standard for Recurrent Neural Networks, Transformers, and Large Language Models.

2. **Instance Normalization (IN)** [94]: Normalizes across spatial dimensions (H, W) independently for each individual sample n and channel c. By stripping contrast and lighting variations while preserving structural geometry, IN is standard in image style transfer and GANs.

3. **Group Normalization (GN)** [98]: Divides the C channels into G groups (typically $G = 32$) and normalizes across $(C/G, H, W)$ for each sample. GN delivers performance parity with Batch Normalization in CNNs while remaining completely invariant to batch size, excelling in memory-constrained detection and segmentation pipelines.

6.5 Deep regularization

Once deep networks can be stably initialized and trained via He initialization and Batch Normalization, they confront their central theoretical challenge: **generalization in overparameterized regimes**.

In classical statistics, models are intentionally underparameterized ($P < N$): the number of parameters is kept strictly smaller than the number of training samples to constrain model capacity and prevent memorization. In modern deep learning, models are massively overparameterized ($P \gg N$), often containing tens of millions to hundreds of billions of parameters trained on datasets orders of magnitude smaller than the parameter count.

From a qualitative perspective, why must we deal with overparameterization, and what are its consequences?

1. **The Capacity to Memorize Arbitrary Noise**: As proved empirically by Chiyuan Zhang et al. [103] in *"Understanding deep learning requires rethinking generalization"*, deep architectures possess sufficient expressive capacity to memorize completely random training labels or pure

Gaussian noise with 100% training accuracy. Without constraints, a network will simply memorize training instances rather than discovering semantic rules.

2. **Infinite Zero-Error Solutions**: In overparameterized regimes, the empirical loss surface features an infinite manifold of parameter configurations that achieve zero training error ($\mathcal{L}_{\text{train}} = 0$). However, these configurations are not equally generalizable: many correspond to sharp, brittle minima with extreme curvature where small input perturbations cause wild output fluctuations.

3. **The Imperative for Regularization**: Because training loss alone cannot distinguish between a smooth function that generalizes well and a convoluted function that memorizes noise, we must introduce **regularization**—both explicit inductive penalties and implicit optimization biases—to guide the model toward flat, robust solutions.

6.5.1 L_1 and L_2 regularization

The classical foundation of explicit regularization is adding a norm penalty on the network weights to the empirical training objective:

$$\mathcal{L}_{\text{reg}}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \lambda\Omega(\mathbf{w}). \tag{6.88}$$

L_1 Regularization and Exact Sparsity: Under L_1 regularization (Lasso), the penalty corresponds to the Manhattan norm of the weight vector:

$$\Omega(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_{j=1}^{D} |w_j|. \tag{6.89}$$

The gradient (subgradient) of the L_1 penalty with respect to weight w_j is given by:

$$\frac{\partial \|\mathbf{w}\|_1}{\partial w_j} = \text{sign}(w_j) = \begin{cases} +1 & \text{if } w_j > 0, \\ -1 & \text{if } w_j < 0, \\ [-1, 1] & \text{if } w_j = 0. \end{cases} \tag{6.90}$$

Under gradient descent with learning rate η, the parameter update step becomes:

$$w_{t+1} = w_t - \eta\nabla\mathcal{L}(w_t) - \eta\lambda\,\text{sign}(w_t). \tag{6.91}$$

Notice the regularizing term $-\eta\lambda\,\text{sign}(w_t)$: at each step, regardless of whether a weight is large ($w = 100$) or microscopic ($w = 0.001$), the weight magnitude is reduced by a **constant amount** $\eta\lambda$. If $|w_t| \leq \eta\lambda$, the update clamps the weight to **exactly zero**. Consequently, L_1 regularization drives a large fraction of the weights to exact numerical zero, performing automated **feature selection and parameter sparsity**.

Why L_1 Regularization Fails in Deep Networks and Motivates L_2 Regularization: While exact sparsity is appealing for linear tabular models, L_1 regularization is rarely used as the primary regularizer in deep networks due to severe practical limitations:

- **Subgradient Discontinuity and Optimization Chatter**: The derivative of $|w|$ is discontinuous at $w = 0$. When weights hover near zero, stochastic gradient descent chatters and oscillates across the non-differentiable singularity, destabilizing backpropagation across deep layers.
- **Arbitrary Knockout of Correlated Features**: In deep representations, features in parallel channels or neurons are inherently correlated. When faced with collinear features, L_1 arbitrarily selects one feature and sets all others to zero. This destroys the distributed, redundant feature representation that gives deep networks their fault tolerance and robustness.

- **Linear Penalty Insensitivity**: L_1 penalizes huge weights at the exact same constant rate as tiny weights. It provides weak resistance against exploding weights that cause activation saturation.

These structural limitations directly motivate L_2 **Regularization (Weight Decay)**:

$$\mathcal{L}_{\text{reg}}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2 = \mathcal{L}(\mathbf{w}) + \frac{\lambda}{2}\sum_{j=1}^{D} w_j^2. \tag{6.92}$$

Under gradient descent:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta\left(\nabla\mathcal{L}(\mathbf{w}_t) + \lambda\mathbf{w}_t\right) = (1-\eta\lambda)\mathbf{w}_t - \eta\nabla\mathcal{L}(\mathbf{w}_t). \tag{6.93}$$

Notice the factor $(1-\eta\lambda)$: at every single optimization step, before moving in the direction of the empirical data gradient $-\eta\nabla\mathcal{L}$, the parameter vector is multiplied by a shrinkage factor strictly less than one:

$$(1-\eta\lambda) < 1 \quad (\text{since } \eta > 0, \lambda > 0). \tag{6.94}$$

Unlike L_1's constant subtraction, L_2 exerts a shrinkage force **strictly proportional to the weight magnitude**: large weights are shrunk aggressively, while tiny weights experience negligible decay. This smoothly compresses all weights toward the origin without forcing hard zeros, preserving distributed representations.

To understand how L_2 weight decay affects the geometry of the learned representation, we expand the empirical loss $\mathcal{L}(\mathbf{w})$ around a local minimum $\mathbf{w}^*$ using a second-order Taylor expansion (Bishop, 2024):

$$\mathcal{L}(\mathbf{w}) \approx \mathcal{L}(\mathbf{w}^*) + \frac{1}{2}(\mathbf{w}-\mathbf{w}^*)^\top\mathbf{H}(\mathbf{w}-\mathbf{w}^*), \tag{6.95}$$

where $\mathbf{H} = \nabla^2\mathcal{L}(\mathbf{w}^*)$ is the Hessian matrix. Differentiating the regularized objective and setting to zero:

$$\mathbf{H}(\mathbf{w}-\mathbf{w}^*) + \lambda\mathbf{w} = \mathbf{0} \implies (\mathbf{H}+\lambda\mathbf{I})\mathbf{w} = \mathbf{H}\mathbf{w}^* \implies \mathbf{w} = (\mathbf{H}+\lambda\mathbf{I})^{-1}\mathbf{H}\mathbf{w}^*. \tag{6.96}$$

Expressing this in the orthonormal eigenvector coordinates of the Hessian $\mathbf{H} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ (where $\mathbf{\Lambda} = \text{diag}(\lambda_1, \ldots, \lambda_D)$):

$$w_j = \left(\frac{\lambda_j}{\lambda_j+\lambda}\right) w_j^*. \tag{6.97}$$

Geometric Insight: Along directions of **large curvature** (large Hessian eigenvalues $\lambda_j \gg \lambda$, where moving away from $\mathbf{w}^*$ sharply increases loss), $\frac{\lambda_j}{\lambda_j+\lambda} \approx 1$: the parameter is preserved. Along directions of **small curvature** (small Hessian eigenvalues $\lambda_j \ll \lambda$, flat directions susceptible to fitting noise), $\frac{\lambda_j}{\lambda_j+\lambda} \to 0$: the weights are shrunken aggressively to zero! Weight decay selectively suppresses low-curvature parameter combinations that overfit noise.

6.5.2 Early stopping

While L_2 regularization explicitly modifies the objective function by penalizing large weight vectors, **Early Stopping** achieves an identical regularizing effect without adding any penalty terms to the loss function.

In early stopping, the dataset is partitioned into training, validation, and test subsets. As iterative gradient optimization progresses, the training loss decreases monotonically. The validation error initially decreases as the network learns genuine underlying patterns, but eventually begins to climb as the network memorizes idiosyncratic training noise (the classical U-shaped generalization curve). Early stopping terminates training at iteration τ^*, preserving the parameter snapshot that minimized validation risk.

Mathematical Derivation of the Equivalence (Bishop 1995): Christopher M. Bishop [5] proved that for a quadratic error surface, **early stopping after τ iterations with learning rate η is mathematically equivalent to L_2 weight decay with penalty parameter $\lambda \approx \frac{1}{\tau\eta}$.**

Consider expanding the unregularized empirical loss $\mathcal{L}(\mathbf{w})$ as a second-order Taylor series around the unconstrained global minimum $\mathbf{w}^*$:

$$\mathcal{L}(\mathbf{w}) \approx \mathcal{L}(\mathbf{w}^*) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^*)^\top \mathbf{H}(\mathbf{w} - \mathbf{w}^*), \tag{6.98}$$

where $\mathbf{H} = \nabla^2\mathcal{L}(\mathbf{w}^*)$ is the positive semi-definite Hessian matrix. The gradient evaluates to $\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}) = \mathbf{H}(\mathbf{w} - \mathbf{w}^*)$.

Under standard gradient descent initialized at the origin ($\mathbf{w}^{(0)} = \mathbf{0}$) with constant learning rate η:

$$\begin{aligned} \mathbf{w}^{(\tau)} &= \mathbf{w}^{(\tau-1)} - \eta\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}^{(\tau-1)}) \\ &= \mathbf{w}^{(\tau-1)} - \eta\mathbf{H}(\mathbf{w}^{(\tau-1)} - \mathbf{w}^*). \end{aligned} \tag{6.99}$$

Subtracting $\mathbf{w}^*$ from both sides yields the linear recurrence:

$$\mathbf{w}^{(\tau)} - \mathbf{w}^* = (\mathbf{I} - \eta\mathbf{H})(\mathbf{w}^{(\tau-1)} - \mathbf{w}^*) = (\mathbf{I} - \eta\mathbf{H})^\tau(-\mathbf{w}^*). \tag{6.100}$$

We diagonalize the Hessian via its orthonormal eigendecomposition $\mathbf{H} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$, where $\mathbf{Q}$ is an orthogonal matrix of eigenvectors and $\mathbf{\Lambda} = \text{diag}(\sigma_1, \ldots, \sigma_D)$ contains the non-negative eigenvalues. Projecting the parameter trajectory into the eigenvector coordinate frame:

$$\mathbf{Q}^\top(\mathbf{w}^{(\tau)} - \mathbf{w}^*) = (\mathbf{I} - \eta\mathbf{\Lambda})^\tau(-\mathbf{Q}^\top\mathbf{w}^*), \tag{6.101}$$

which rearranges to:

$$\mathbf{Q}^\top\mathbf{w}^{(\tau)} = \left[\mathbf{I} - (\mathbf{I} - \eta\mathbf{\Lambda})^\tau\right]\mathbf{Q}^\top\mathbf{w}^*. \tag{6.102}$$

Now recall the exact analytical solution for explicit L_2 regularization (weight decay) derived in Equation (6.97):

$$\tilde{\mathbf{w}} = (\mathbf{H} + \lambda\mathbf{I})^{-1}\mathbf{H}\mathbf{w}^* \implies \mathbf{Q}^\top\tilde{\mathbf{w}} = (\mathbf{\Lambda} + \lambda\mathbf{I})^{-1}\mathbf{\Lambda}\mathbf{Q}^\top\mathbf{w}^*. \tag{6.103}$$

Using the matrix identity $(\mathbf{\Lambda} + \lambda\mathbf{I})^{-1}\mathbf{\Lambda} = \mathbf{I} - (\mathbf{\Lambda} + \lambda\mathbf{I})^{-1}\lambda\mathbf{I}$, we write:

$$\mathbf{Q}^\top\tilde{\mathbf{w}} = \left[\mathbf{I} - (\mathbf{\Lambda} + \lambda\mathbf{I})^{-1}\lambda\mathbf{I}\right]\mathbf{Q}^\top\mathbf{w}^*. \tag{6.104}$$

Comparing the bracketed shrinkage operators in Equation (6.102) and Equation (6.104), the early stopping parameter vector $\mathbf{w}^{(\tau)}$ coincides exactly with the L_2-regularized vector $\tilde{\mathbf{w}}$ if and only if:

$$(\mathbf{I} - \eta\mathbf{\Lambda})^\tau = (\mathbf{\Lambda} + \lambda\mathbf{I})^{-1}\lambda\mathbf{I}. \tag{6.105}$$

For each individual eigenvalue σ_i, this scalar equality requires:

$$(1 - \eta\sigma_i)^\tau = \frac{\lambda}{\sigma_i + \lambda} = \frac{1}{1 + \frac{\sigma_i}{\lambda}}. \tag{6.106}$$

Taking the natural logarithm of both sides:

$$\tau \log(1 - \eta\sigma_i) = -\log\left(1 + \frac{\sigma_i}{\lambda}\right). \tag{6.107}$$

Assuming standard non-divergent gradient step sizes ($\eta\sigma_i \ll 1$) and small eigenvalues relative to the penalty ($\sigma_i/\lambda \ll 1$), we expand both sides via first-order Taylor series $\log(1 + x) \approx x$:

$$-\tau\eta\sigma_i \approx -\frac{\sigma_i}{\lambda} \implies \tau\eta \approx \frac{1}{\lambda} \iff \lambda \approx \frac{1}{\tau\eta}. \tag{6.108}$$

Theorem 6.2 (Bishop's Early Stopping Equivalence Theorem)**.** *Under a quadratic approximation of the loss surface, early stopping after τ iterations with learning rate η is mathematically equivalent to L_2 weight decay with penalty parameter:*

$$\lambda_{\mathit{eff}} = \frac{1}{\tau\eta}. \tag{6.109}$$

The total cumulative step capacity $\tau\eta$ acts as the reciprocal of the weight decay regularization parameter.

The fundamental insight of Bishop (1995) [5] and Goodfellow et al. (2016) [32] is that early stopping restricts the parameter trajectory to a small, compact volume in parameter space in the immediate neighborhood of initialization $\mathbf{w}^{(0)}$. Because the gradient is bounded, limiting the product of iterations τ and step size η physically prevents the parameter vector from traveling into distant, high-curvature regions where it can contort to memorize noise.
Furthermore, Equation (6.102) reveals an intrinsic directional selectivity: parameter coordinates aligned with **large curvature** (large eigenvalues σ_i) satisfy $(1 - \eta\sigma_i)^\tau \to 0$ rapidly, allowing the network to converge quickly along dominant structural directions. In contrast, coordinates aligned with **flat directions** (small eigenvalues $\sigma_i \approx 0$) evolve slowly, remaining tightly anchored near zero. Early stopping automatically determines the optimal regularization strength along each eigenvector direction without requiring expensive grid searches over λ.

6.5.3 The double descent phenomenon

For nearly a century, classical statistical learning theory dictated that model complexity must be strictly balanced against dataset size: increasing model parameters beyond the number of training samples ($P > N$) inevitably causes catastrophic overfitting, as depicted by the traditional U-shaped risk curve.

Modern deep neural networks defy this classical doctrine: architectures containing hundreds of millions of parameters achieve superior test accuracy when trained on only a few tens of thousands of samples, achieving 100% training accuracy while generalizing remarkably well. This apparent contradiction was formally resolved by Mikhail Belkin et al. [3] through the discovery of the **Double Descent** curve (Figure 6.10).

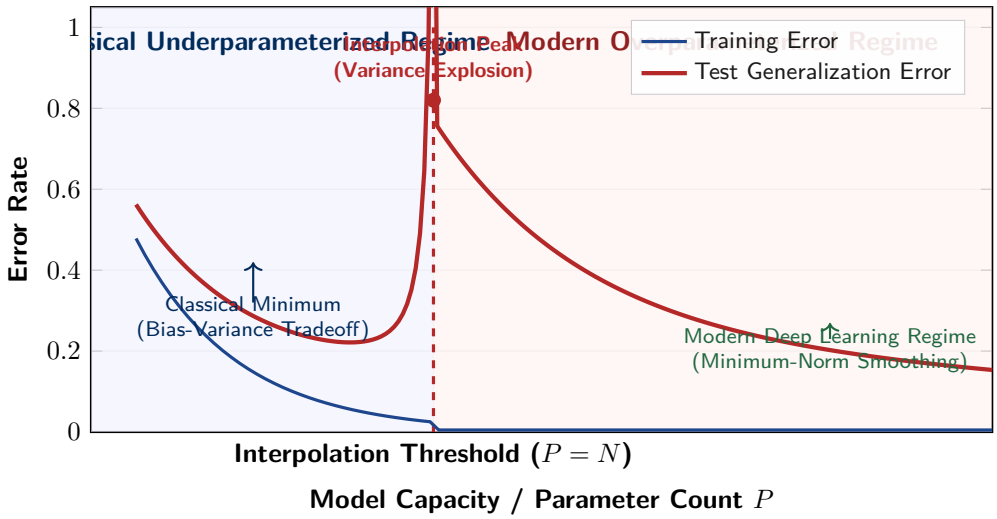


Figure 6.10: The Double Descent generalization curve reconciling classical statistics with modern deep learning (Belkin et al., 2019; Bishop, 2024). In the underparameterized regime ($P < N$), test error follows the classical U-shaped bias-variance tradeoff. At the interpolation threshold ($P = N$), the model has just enough capacity to memorize all training points, causing function derivatives to oscillate violently and generalization variance to explode into a sharp peak. In the overparameterized regime ($P \gg N$), stochastic gradient descent selects the minimum-norm interpolant among infinite zero-training-error solutions, monotonically reducing test error below the classical minimum.

The double descent curve establishes three distinct operational regimes:

1. **Underparameterized Classical Regime ($P < N$)**: The number of learnable parameters is strictly smaller than the number of training samples. Here, the network lacks sufficient expressive capacity to fit the training set perfectly. As capacity P increases, bias decreases monotonically, variance increases, and the test error exhibits the classical U-shaped minimum.

2. **The Interpolation Threshold ($P \approx N$)**: The capacity of the network matches the effective degrees of freedom required to achieve zero empirical training error ($\mathcal{L}_{\text{train}} = 0$). In this critical regime, there exists **exactly one unique interpolating function** that passes through all training points (including label noise and outliers). Because the model has zero surplus capacity to choose a smooth solution, it must contort violently between adjacent data points. Its spatial derivatives explode, and test error spikes catastrophically into an **interpolation peak**.

3. **Overparameterized Modern Regime ($P \gg N$)**: As capacity expands far beyond the interpolation threshold, the network enters an under-determined space with an infinite number of parameter configurations that achieve zero training error ($\mathcal{L}_{\text{train}} = 0$). When optimized via Stochastic Gradient Descent from near-zero initialization, the algorithm possesses an **implicit inductive bias toward the minimum Euclidean norm solution**:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \|\mathbf{w}\|_2 \quad \text{subject to} \quad f(\mathbf{x}_i; \mathbf{w}) = y_i \quad \forall i \in \{1, \dots, N\}. \tag{6.110}$$

Because excess parameters provide extra geometric dimensions through which the function can smoothly interpolate the data points without erratic oscillations, the interpolating surface becomes progressively flatter. Generalization variance drops, and test error decreases monotonically, frequently outperforming the classical underparameterized optimum!

Forms of Double Descent in Modern Practice (Nakkiran et al., 2020): Preetum Nakkiran et al. [68] generalized double descent beyond model width to two additional fundamental training axes:

- **Epoch Double Descent**: Training an overparameterized model for an extended number of epochs exhibits a temporary peak in test error when the effective capacity of the network passes through the interpolation threshold, before descending to optimal generalization at late epochs.
- **Sample Double Descent**: Counter-intuitively, collecting a slightly larger training dataset can temporarily **worsen test performance** if the additional data shifts the interpolation threshold directly onto the model's fixed parameter count ($N \to P$).

6.5.4 Dropout regularization

Introduced by Nitish Srivastava et al. [88], **Dropout** is a foundational stochastic regularization technique designed to prevent complex co-adaptation in deep representations.

In standard neural network training, individual neurons can become codependent on other specific units to correct their errors. When units co-adapt, the network behaves like an interconnected brittle system where no neuron extracts autonomous, robust features. Dropout solves this problem by injecting stochastic noise directly into the network architecture during each training iteration: it randomly deactivates (drops out) a subset of neurons with probability $1 - p$, severing all their incoming and outgoing connections. Because any neuron might vanish at any step, each individual neuron is forced to learn features that are useful and reliable across diverse, random combinations of companion units.

Forward Pass with Inverted Dropout: At each training iteration, for each intermediate hidden activation vector $\mathbf{h} \in \mathbb{R}^d$, we sample an independent vector of binary Bernoulli random variables:

$$\mathbf{r} \sim \text{Bernoulli}(p)^d, \quad \text{where } P(r_j = 1) = p, \quad P(r_j = 0) = 1 - p, \tag{6.111}$$

where $p \in (0, 1]$ is the **keep probability** (typically $p = 0.5$ for dense layers, $p = 0.8$–0.9 for convolutional layers).

In classical dropout, the activation was $\tilde{h}_j = r_j h_j$, which reduced the expected activation to $\mathbb{E}[\tilde{h}_j] = p h_j$. At test time, one had to remember to multiply all weights by p ($\mathbf{W}_{\text{test}} = p\mathbf{W}$).

Modern frameworks employ **Inverted Dropout**: activations are scaled by $\frac{1}{p}$ **during training**:

$$\tilde{h}_j = \frac{r_j}{p} h_j. \tag{6.112}$$

Evaluating the expectation during training:

$$\mathbb{E}[\tilde{h}_j] = \frac{\mathbb{E}[r_j]}{p} h_j = \frac{p}{p} h_j = h_j. \tag{6.113}$$

Because the expected activation during training matches the unregularized activation exactly, **test-time inference requires zero modification**:

$$\mathbf{h}_{\text{test}} = \mathbf{h} \quad \text{(all neurons active, deterministic computation)}. \tag{6.114}$$

Figure 6.11 illustrates the operational difference between the stochastic training pass and deterministic test-time inference.

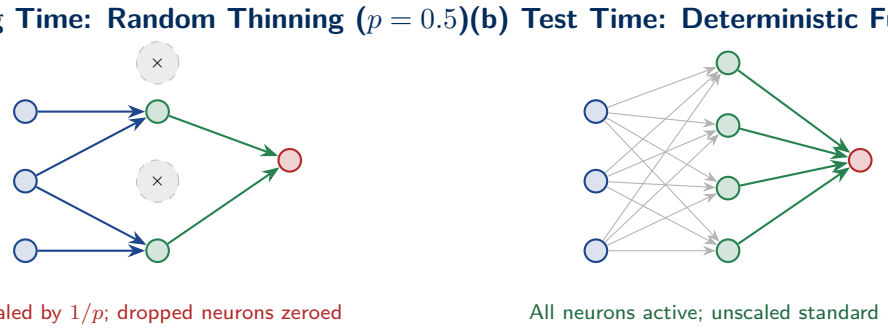


Figure 6.11: Inverted Dropout mechanics. (a) During training, hidden units are randomly zeroed with probability $1 - p$ (dropped neurons marked with $\times$), and remaining units are scaled by $1/p$. (b) At test time, dropout is disabled: all neurons and connections remain active, and the network executes standard deterministic inference without any scaling modifications.

Theoretical Interpretations: Co-Adaptation and Bagging:

1. **Preventing Complex Co-Adaptation**: When dropout randomly extinguishes companion neurons, each unit is forced to learn robust, self-reliant features that are useful in conjunction with many random subsets of other units.

2. **Extreme Ensemble Bagging (Model Averaging)**: An M-neuron layer trained with dropout possesses 2^M possible binary mask configurations. In a standard dense layer with 4,096 units (such as `fc6` in VGG-16), there are:

$$2^{4096} \approx 10^{1233} \text{ possible binary subnetworks!} \tag{6.115}$$

   Sampling a different random mask $\mathbf{r}$ at each optimization step effectively trains an ensemble of astronomical scale with extensive parameter sharing. At test time, executing a deterministic forward pass over the unmasked network acts as an approximate geometric mean over the entire ensemble's predictions, slashing prediction variance.

Architectural Obsolescence in Modern Convolutional Networks: A critical architectural insight emphasized across modern computer vision concerns where Dropout is applied across network generations:

- In early deep networks such as **AlexNet (2012)** and **VGG-16 (2014)**, over 85%–90% of all learnable parameters were concentrated in the final fully connected layers (`fc6` alone accounted for 102M weights in VGG-16). Dropout with $p = 0.5$ was indispensable to prevent these dense matrices from memorizing training labels.
- Modern architectures—beginning with **GoogLeNet (2014)** and **ResNet (2015)**—completely eliminated dense fully connected layers, replacing them with **Global Average Pooling (GAP)**. In ResNet-50, a $7 \times 7 \times 2048$ feature volume is collapsed into a 2,048-dimensional vector, followed by a single linear projection matrix of only $2048 \times 1000 \approx 2$M parameters. Because convolutional layers have parameter sharing and significantly fewer degrees of freedom, and because Batch Normalization acts as a strong regularizer, **modern convolutional backbones do not use Dropout at all**.

Residual Connections as Implicit Landscape Regularizers: Beyond skip connections serving as backpropagation highways (derived in Chapter 5), Hao Li et al. [58] demonstrated that residual connections act as powerful **implicit regularizers**. In standard deep feedforward networks without skip connections (e.g. ResNet-56 without shortcuts), the high-dimensional loss surface is chaotic, non-convex, and fractured by millions of sharp, ill-conditioned local minima.

Residual connections create a multi-scale ensemble of shallow and deep paths, smoothing the non-convex landscape into a wide, spherical, convex-like basin. This loss surface smoothing prevents optimization trajectories from becoming trapped in sharp, high-generalization-error crevices.

6.5.5 Data augmentation and modern regularizers

The single most effective regularization strategy in machine learning is **expanding the training dataset**. In supervised learning, error bounds scale inversely with dataset size N. However, gathering, curating, and manually annotating massive real-world datasets is expensive, time-consuming, and frequently infeasible.

Data Augmentation solves this fundamental data-scarcity bottleneck by synthetically generating new, plausible training instances directly from existing samples. The core mathematical mechanism relies on applying **label-preserving stochastic transformations** $\mathcal{T} : \mathcal{X} \to \mathcal{X}$ to the input data such that the semantic ground-truth label remains invariant:

$$P(y \mid \mathcal{T}(\mathbf{x})) = P(y \mid \mathbf{x}). \tag{6.116}$$

By training the network on transformed pairs $(\mathcal{T}(\mathbf{x}), y)$, data augmentation effectively forces the optimization algorithm to discover parameter configurations that are **invariant to the transformations** $\mathcal{T}$. Geometrically, data augmentation flattens the empirical loss landscape around each training example: rather than fitting isolated discrete points, the model is compelled to fit continuous orbits in input space generated by the transformation group.

Figure 6.12 provides a structured visual taxonomy of data augmentation techniques categorized into geometric and photometric domains.

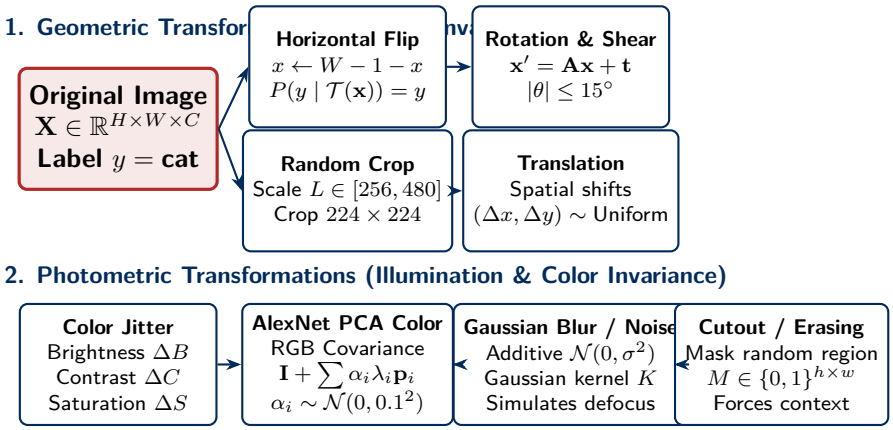


Figure 6.12: Visual taxonomy of data augmentation techniques across geometric and photometric domains. (1) Geometric transformations perturb spatial grid coordinates to induce invariance to viewpoint, translation, scale, and reflection. (2) Photometric transformations perturb pixel values in color and frequency space to induce invariance to illumination spectrum, exposure, contrast, and sensor noise.

Geometric Transformations in Computer Vision:

- **Horizontal Reflection (Flips)**: Flipping an image horizontally ($x \leftarrow W-1-x$) preserves semantics (a cat flipped horizontally is still a cat). Note: vertical flips are generally invalid for natural images (cats do not walk upside down).
- **Random Cropping and Rescaling (ResNet Protocol)**: During training, sample a random scale $L \in [256, 480]$, resize the image such that its shorter side equals L, and crop a random 224×224 patch. At test time, standard practice evaluates a fixed ensemble of

10 crops across 5 scales: $\{224, 256, 384, 480, 640\}$, taking the 4 corners, center crop, and their horizontal flips.

- **Task-Dependent Transformation Constraints**: Transformations must be strictly aligned with the target problem. For instance, a 180° rotation or vertical flip is perfectly valid for aerial satellite imagery (where overhead perspective possesses rotational symmetry), but catastrophically destroys label semantics in handwritten digit recognition (rotating a 6 produces a 9).
- **The Label Transformation Synchronization Rule**: Whenever a data augmentation transformation alters spatial coordinates (such as translation, rotation, shearing, or cropping), **ground-truth spatial labels (bounding boxes, segmentation masks, keypoints) must undergo the exact same transformation synchronously**.

Photometric Transformations and Color Perturbations:

- **Color Jitter**: Adding random perturbations to brightness, contrast, saturation, and hue, simulating changes in illumination conditions.
- **AlexNet PCA Color Augmentation**: Performing PCA on the RGB pixel covariance matrix of ImageNet and adding multiples of the principal components: $\mathbf{I}_{xy} \leftarrow \mathbf{I}_{xy} + [\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3][\alpha_1\lambda_1, \alpha_2\lambda_2, \alpha_3\lambda_3]^\top$, where $\alpha_i \sim \mathcal{N}(0, 0.1^2)$. This captures the natural variation in illumination spectrum while preserving object identity.

The Unified Noise Injection Taxonomy: A unifying pattern in modern deep regularization is: **inject randomness during training, and average out/marginalize the randomness at test time**:

Regularizer	Training Noise Mechanism	Test-Time Deterministic Behavior
Dropout	Randomly zero activations with mask $\mathbf{r}$	Scale activations (or unscaled Inverted Dropout)
DropConnect	Randomly zero weights $W_{ji} \leftarrow 0$	Marginalize Gaussian mixture of weights
BatchNorm	Normalize with random mini-batch stats	Normalize with fixed global running $(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$
Stochastic Depth	Randomly skip residual blocks with prob p_l	Scale block contributions by survival rate
Fractional Pooling	Randomize pooling grid spatial stride	Average predictions over multiple random grids
Cutout / Erasing	Zero out random rectangular patches	Full unmasked deterministic image
Mixup	Convex blend of input pairs & labels	Standard single-image unmixed inference

Table 6.3: The unified taxonomy of stochastic noise injection regularizers. Each method injects controlled stochasticity during training to prevent co-adaptation, and integrates or marginalizes out the randomness at inference time.

6.6 Training diagnostics and learning rates

Having engineered the model architecture, input preprocessing, weight initialization, normalization layers, and regularizers, the practitioner confronts the dynamic optimization phase: **babysitting the learning process**.

In classical convex optimization, training is a predictable, closed-form, or monotonically convergent numerical procedure: once the objective and solver are specified, the algorithm is guaranteed to converge to the unique global minimum without human intervention. In deep learning, the reality is fundamentally different. Deep neural network optimization is a high-dimensional, non-convex, stochastic dynamical process where millions or billions of parameters interact through non-linear cascades across non-stationary mini-batches.

A deep learning training run is never a "set-it-and-forget-it" endeavor. Without continuous, real-time monitoring and empirical diagnostics:

1. **Silent Numerical Failures**: A network can train stably for hours and then suddenly encounter an unstable gradient shock, causing weights to overflow and producing fatal NaN loss values that destroy the checkpoint.

2. **Gradient Starvation and Latent Deadlock**: Activation functions can quietly saturate or die (such as dying ReLU units or saturated Sigmoids), reducing the effective learning capacity of layers to zero while the loss plateaus deceptively.

3. **Early Implementation Bugs**: Subtle software defects—such as unnormalized input channels, misaligned data augmentation labels, detached regularization gradients, or swapped train/eval flags—can completely sabotage learning without throwing explicit code exceptions.

4. **Resource Conservation**: Training modern visual and multimodal models consumes days or weeks of expensive GPU cluster compute. Babysitting allows practitioners to diagnose fatal pathologies within the first hundred iterations, killing doomed experiments immediately and redirecting compute toward promising configurations.

To turn this empirical babysitting into a disciplined engineering science rather than haphazard guesswork, we follow a systematic workflow.

6.6.1 Systematic training protocol

Justin Johnson outlines a rigorous, disciplined 7-step procedure for initializing, debugging, and verifying a deep learning experiment before committing substantial compute resources:

1. **Step 1: Sanity Check the Initial Loss**: Before applying weight decay or data augmentation, execute a single forward pass with random initialization. For a classification problem with C balanced classes using cross-entropy loss, the expected initial loss is:

$$\mathcal{L}_{\text{init}} = -\log\left(\frac{1}{C}\right) = \log(C). \tag{6.117}$$

   For CIFAR-10 ($C = 10$), $\mathcal{L}_{\text{init}} \approx -\log(0.1) \approx 2.302$. For ImageNet ($C = 1000$), $\mathcal{L}_{\text{init}} \approx -\log(0.001) \approx 6.908$. If the observed initial loss is substantially higher, weights are improperly scaled; if it is lower, the network has trivial inductive leakage.

2. **Step 2: Verify Regularization Penalty**: Turn on weight decay ($\lambda > 0$). Confirm that the total loss strictly increases relative to Step 1. If the loss does not increase, the regularization gradient is detached from the computational graph.

3. **Step 3: Overfit a Tiny Sample (Sanity Check)**: Isolate a tiny subset of the dataset (e.g., 20 samples), disable all regularization ($\lambda = 0, p_{\text{drop}} = 1.0$), and train for 20–50 iterations. **The model must achieve 100% training accuracy and drive loss to near zero ($\mathcal{L} < 10^{-4}$).** If a high-capacity deep network cannot memorize 20 data points, there is an egregious bug in the code: gradients are zeroed, labels are misaligned, or activations are dead.

4. **Step 4: Find a Learning Rate That Makes Loss Drop**: Using the full training set and a small weight decay ($\lambda \approx 10^{-4}$), execute 100 iterations across a logarithmic grid of learning rates: $\alpha \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$. Identify the learning rate that causes training loss to decrease significantly within the first 100 steps.

5. **Step 5: Coarse Grid Search (1–5 Epochs)**: Sample a coarse grid of learning rates and weight decay values around the promising region from Step 4. Train a small ensemble of models for 1–5 epochs. Good standard baselines: $\alpha \in [10^{-4}, 10^{-1}]$ and $\lambda \in [0, 10^{-4}]$.

6. **Step 6: Refine Grid and Train Longer**: Select the top performing hyperparameter configurations from Step 5 and train them for 10–20 epochs without learning rate decay.

7. **Step 7: Inspect Diagnostic Curves**: Plot the empirical loss and accuracy trajectories to diagnose optimization health.

Drawback of Static Checkpoints and Transition to Dynamic Curves: While this 7-step checklist guarantees that the code is free of fundamental implementation bugs at initialization, it provides only static, discrete checkpoints. Once long-horizon training begins on the full dataset, static checks cannot reveal how the model's representations are evolving across epochs or why training slows down. To understand and steer the ongoing training process, practitioners must continuously analyze **dynamic learning curves**.

6.6.2 Diagnostic curves

Analyzing the trajectory and geometry of learning curves provides immediate actionable insight into optimization health (Figure 6.13):

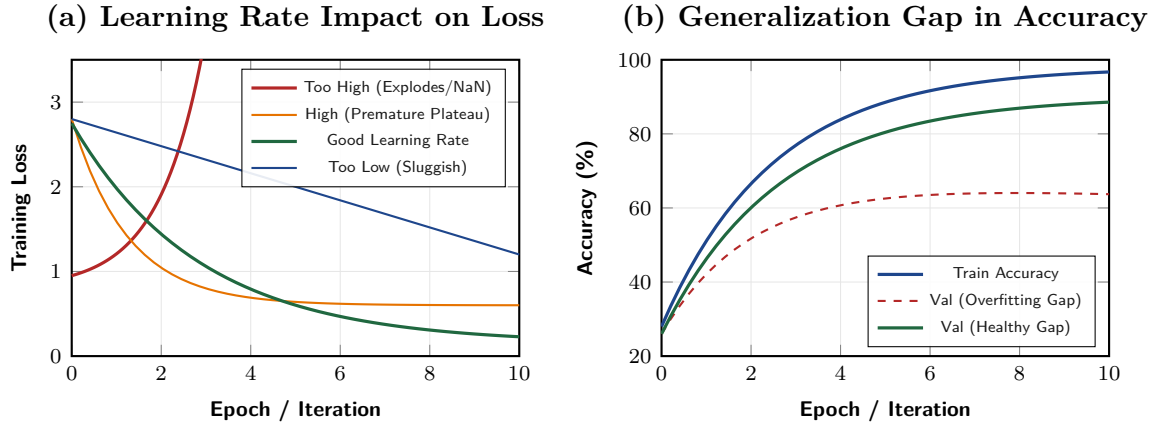


Figure 6.13: Diagnostic learning curves for neural network training. (a) Impact of learning rate on training loss: excessive learning rates cause gradient explosion (NaN); high rates cause premature plateaus; low rates result in sluggish linear descent; well-tuned rates achieve deep asymptotic minimization. (b) Train vs. validation accuracy: a large expanding gap indicates severe overfitting (requiring stronger regularization or data augmentation); a negligible gap indicates underfitting (requiring larger models or longer schedules).

- **Loss Explodes to** NaN **or** $+\infty$: Learning rate α is too high, or weight initialization variance is too large. Gradients blow up, throwing weights outside numerical floating-point bounds.
- **Loss Completely Flat**: Learning rate is too small, gradients vanished to zero (e.g. saturated Sigmoid/Tanh), or all hidden ReLU neurons died during the first step.
- **Loss Decreasing but Suddenly Plateaus**: Learning rate was decayed too early while loss was still falling steeply.
- **Large Train/Validation Gap**: The model has memorized training idiosyncrasies; variance dominates. **Remedy**: increase L_2 weight decay, increase Dropout probability, inject Cutout/Mixup, or collect more data.
- **No Train/Validation Gap (Both Low)**: The model is underfitting; bias dominates. **Remedy**: increase network capacity (add layers or width), train for more epochs, or relax regularization.

The Update-to-Weight Ratio Rule of Thumb (Slide 116): During training, Justin Johnson recommends tracking the dimensionless ratio between the parameter update norm and the parameter magnitude:

$$r = \frac{\|\Delta \mathbf{w}\|_2}{\|\mathbf{w}\|_2} = \frac{\alpha \|\mathbf{g}\|_2}{\|\mathbf{w}\|_2}. \tag{6.118}$$

Listing 6.1: Tracking update-to-weight magnitude ratio in Python.

```
# Assume parameter vector W and its gradient dW
param_scale = np.linalg.norm(W.ravel())
update = -learning_rate * dW
update_scale = np.linalg.norm(update.ravel())
W += update
print(f"Update/Weight Ratio: {update_scale / param_scale:.6f}")
```

The target heuristic is $r \approx 10^{-3}$ (0.001). If $r \ll 10^{-4}$, updates are too timid, requiring an order of magnitude increase in α. If $r \gg 10^{-2}$, updates cause violent parameter shockwaves that corrupt existing feature representations.

Drawback of Constant Learning Rates and Motivation for Scheduling: Diagnostic curve inspection exposes a fundamental contradiction inherent to any **constant learning rate**:

- Early in training, parameters are situated far from the optimal basin. A large learning rate is indispensable to traverse flat plateaus, bypass high-curvature saddle points, and make rapid progress.
- Late in training, the parameters approach the bottom of a narrow, delicate loss basin. A large learning rate is now catastrophic: the optimizer lacks sufficient precision to settle into the minimum, bouncing chaotically between the valley walls and stagnating at high loss. Conversely, keeping the learning rate permanently small makes early exploration agonizingly slow.

This limitation makes dynamic **Learning Rate Scheduling and Warmup** mathematically mandatory.

6.6.3 Schedules and warmup dynamics

To resolve the constant-learning-rate dilemma, modern optimizers adjust the step size α_t over the course of training, decaying it according to principled mathematical trajectories (Figure 6.14):

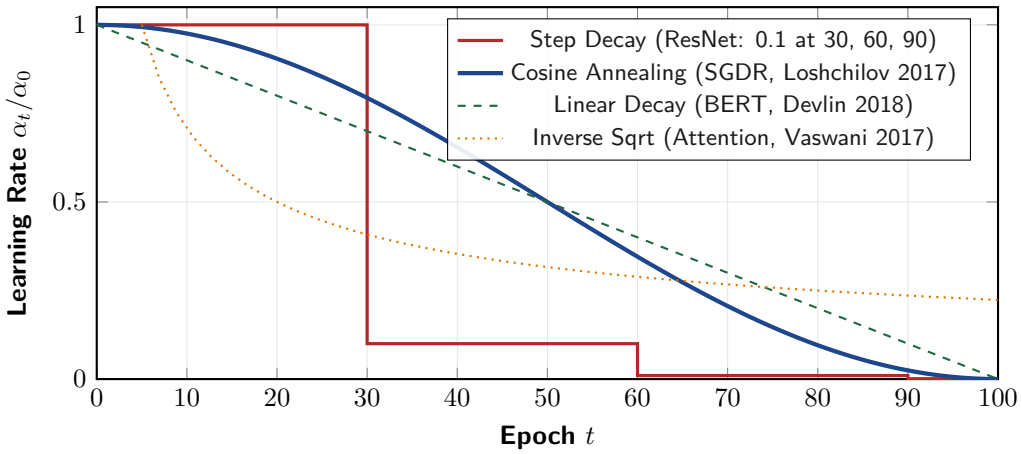


Figure 6.14: Comparison of modern learning rate decay schedules over a 100-epoch training horizon. Step decay imposes abrupt discrete drops; Cosine Annealing decreases smoothly without discrete milestones; Linear decay declines steadily to zero; Inverse Square Root decays aggressively early and flattens out.

The primary mathematical formulations include:

1. **Step Decay (Multi-Step Milestones)**: Reduces the learning rate by a multiplicative factor γ (typically $\gamma = 0.1$) at fixed milestone epochs:

$$\alpha_t = \alpha_0 \cdot \gamma^{\sum_k \mathbb{I}(t \geq M_k)}. \tag{6.119}$$

   Standard in original ResNet training: $\alpha_0 = 0.1$, multiplied by 0.1 at epochs 30, 60, and 90.

2. **Cosine Annealing (Loshchilov & Hutter, 2017)** [60]: Decays smoothly following a half-period cosine curve:

$$\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_0 - \alpha_{\min}) \left(1 + \cos\left(\frac{\pi t}{T_{\max}}\right)\right). \tag{6.120}$$

   Cosine annealing requires zero milestone tuning and provides a gentle, continuous deceleration. Widely adopted across modern vision (RegNet, SlowFast) and foundation models (GPT-1 [74], LLaMA).

3. **Linear Decay (Devlin et al., 2018)** [20]:

$$\alpha_t = \alpha_0 \left(1 - \frac{t}{T_{\max}}\right). \tag{6.121}$$

   Standard in Transformer fine-tuning (BERT, RoBERTa, XLNet).

4. **Inverse Square Root Decay (Vaswani et al., 2017)** [95]:

$$\alpha_t = \alpha_0 \cdot \frac{1}{\sqrt{\max(t, t_{\text{warmup}})}}. \tag{6.122}$$

5. **Constant Learning Rate**: Maintains $\alpha_t = \alpha_0$. Standard in adversarial GAN training (BigGAN [7]) where shifting learning rates can destabilize the Nash equilibrium between generator and discriminator.

6. **Linear Warmup Dynamics**: At epoch 0, parameters are random, and gradient vectors are noisy and large. In adaptive optimizers (Adam), second-moment accumulators v_t are initialized to zero, causing early update steps to be uncalibrated. **Warmup linearly ramps the learning rate from 0 to** α_0 over the initial T_{warmup} steps:

$$\alpha_t = \alpha_0 \cdot \frac{t}{T_{\text{warmup}}} \quad \text{for } t \leq T_{\text{warmup}}. \tag{6.123}$$

This protects early representations from destructive shocks before engaging decay schedules.

Drawback of Manual Tuning and Motivation for Systematic Search: Even with sophisticated schedules and warmup dynamics, the engineer is confronted with a critical dilemma: **how do we set the initial learning rate** α_0**, the weight decay** λ**, the warmup length** T_{warmup}**, and the momentum coefficient** β**?** Tuning these hyperparameters by hand is painfully slow, subjective, and prone to human cognitive bias. This motivates the necessity of a principled, automated discipline: **Hyperparameter Optimization and Experiment Tracking**.

6.7 Hyperparameter optimization

In deep learning, we must distinguish between two fundamental classes of parameters: **model parameters** (weights **W** and biases **b**), which are updated automatically through backpropagation and gradient descent, and **hyperparameters** (learning rate α, weight decay λ, batch size B, Dropout keep probability p, architecture depth and width), which govern the operation of the learning algorithm itself and define the model's inductive capacity.

Hyperparameters cannot be learned directly via gradient descent on training data because minimizing empirical loss would lead to trivial, catastrophic solutions (for example, setting the regularization weight $\lambda \to 0$ or selecting an infinite model width to memorize the data).

Furthermore, the performance of deep neural networks is exceptionally sensitive to hyperparameter configurations:

- A sub-optimal learning rate α can degrade accuracy by 30%–50% or cause complete numerical divergence (NaN).
- Evaluating a single hyperparameter configuration is computationally expensive, requiring hours or days of GPU compute across an entire training run.
- Because the hyperparameter space is continuous and multi-dimensional, an exhaustive brute-force search is mathematically and economically impossible.

Therefore, machine learning requires a rigorous, disciplined methodology to explore hyperparameter spaces efficiently within a finite compute budget, ensuring sound validation without data leakage.

6.7.1 Random search versus grid search

In classical engineering, practitioners frequently deploy **Grid Search**: laying out an equidistant mesh across all hyperparameter axes. James Bergstra and Yoshua Bengio [4] mathematically proved that **Random Search is strictly superior to Grid Search** (Figure 6.15):

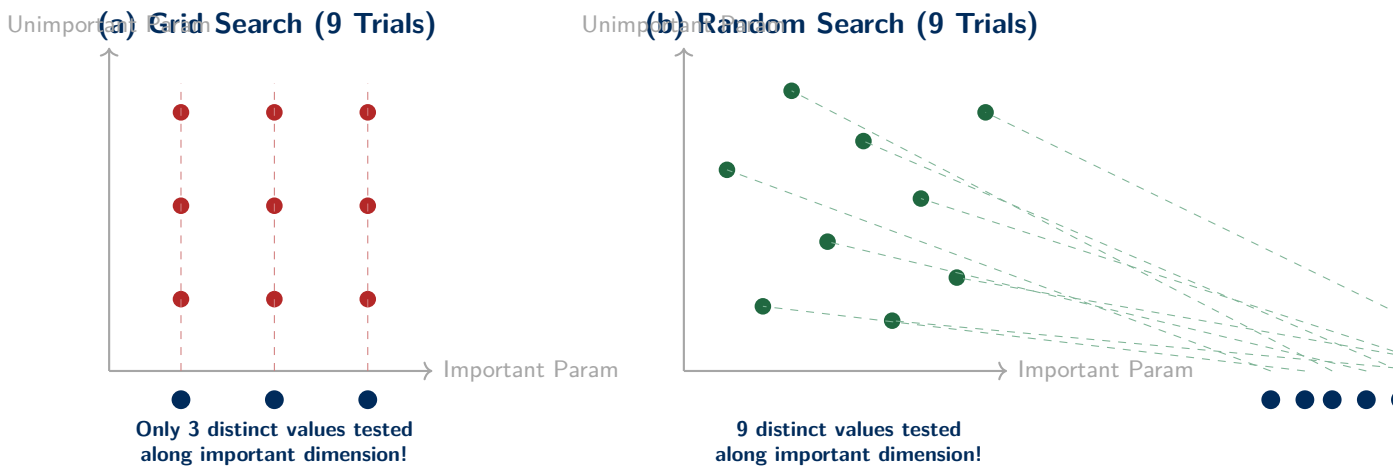


Figure 6.15: Grid Search versus Random Search for hyperparameter optimization (Bergstra & Bengio, 2012). Deep neural network functions exhibit low effective dimensionality: only a small subset of hyperparameters significantly affects performance. For an identical budget of 9 evaluation trials, Grid Search tests only 3 distinct values along the critical dimension (wasting 3× redundancy per row). Random Search samples 9 completely distinct values, achieving 3× higher exploration resolution along the important axis.

Mathematical Rationale. Let D denote the total number of hyperparameters being optimized, and let $d \ll D$ denote the effective dimensionality (the small subset of parameters that truly impact generalization error). Suppose the computational budget allows evaluating exactly M models.

Under Grid Search with G steps per axis, the total evaluations equal $M = G^D$. Along any single hyperparameter projection, grid search tests only $G = M^{1/D}$ distinct values. For $D = 4$ and $M = 81$, grid search tests only $G = 3$ values along the critical learning rate dimension!

Under Random Search, M parameter configurations are drawn independently from continuous distributions. Along any 1D projection, **all M tested values are distinct**. The probability that random search misses an optimal ϵ-interval of length δ along the important dimension decays exponentially:

$$P(\text{failure}) = (1 - \delta)^M. \tag{6.124}$$

For $M = 60$ trials, the probability of finding a configuration within the top 5% ($\delta = 0.05$) of performance exceeds $1 - (0.95)^{60} \approx 95.4\%$, regardless of how many irrelevant hyperparameters D are included in the search space! □

Logarithmic Space Sampling: Parameters that control multiplicative scales—such as learning rate α and weight decay λ—must be sampled in **logarithmic space**:

$$r \sim U[a, b] \implies \alpha = 10^r. \tag{6.125}$$

Sampling uniformly in linear space across $[10^{-4}, 10^{-1}]$ would waste 90% of trials testing values between 0.01 and 0.1, virtually ignoring the delicate 10^{-4} to 10^{-3} regime.

Drawback of Pure Random Search and Motivation for Intelligent Sweeps: While Random Search solves the grid redundancy problem, it suffers from a major efficiency drawback: **it operates completely open-loop without memory**. Random search samples configurations independently, failing to learn from previously evaluated points, and it runs every single trial for the full duration of training. If a randomly sampled learning rate is catastrophic, Random Search will waste an entire 100-epoch GPU budget training a hopelessly broken model. This

limitation motivates **Automated Distributed Sweeps** with adaptive pruning and Bayesian optimization.

6.7.2 Cross-validation and sweep tracking

In standard deep learning, k-fold cross-validation is avoided due to extreme computational costs (training k separate 100-epoch models). Instead, practitioners enforce a fixed split: 70% Training, 15% Validation, 15% Test.

To automate exploration across GPU clusters efficiently, modern production environments deploy **Weights & Biases (W&B Sweeps)** and adaptive scheduling frameworks:

- **Bayesian Optimization**: Rather than drawing samples blindly, Bayesian optimization fits a probabilistic surrogate model (such as a Gaussian Process or Tree-structured Parzen Estimator, TPE) mapping hyperparameters to validation performance. It dynamically selects the next evaluation point by maximizing an acquisition function (e.g., Expected Improvement), balancing exploration of uncharted regimes with exploitation of promising basins.

- **Hyperband and Early Pruning of Unpromising Runs**: Solves the wasted-compute bottleneck of running bad configurations to completion. By dynamically allocating resources across an initial large pool of runs and terminating the worst-performing fraction after 1–2 epochs, Hyperband concentrates GPU resources on the most promising configurations.

Drawback of Single-Model Optimization and Transition to Ensembles: Even after conducting comprehensive Bayesian sweeps and identifying the global best hyperparameter configuration, the resulting trained model remains a **single, isolated point estimate** in parameter space. It is trapped in a single local minimum and inevitably makes errors on specific, ambiguous test samples. Can we push generalization and calibration beyond what any single model can achieve? This directly motivates **Model Ensembling**.

6.8 Model ensembling and weight averaging

Once hyperparameter sweeps identify a collection of well-performing models, the final strategy in competitive deep learning to maximize accuracy and prediction calibration is **Model Ensembling**.

In non-convex optimization, stochastic gradient descent runs initialized with different random seeds, architectures, or mini-batch orderings settle into widely separated, diverse local minima across the loss surface. Although these models may achieve nearly identical overall validation accuracies, **they do not make the same mistakes**: each individual network learns idiosyncratic decision boundaries and possesses decorrelated error distributions.

By combining the predictions of multiple diverse models, individual misclassifications are outvoted by the collective consensus. Mathematically, ensembling reduces the variance of the estimator without increasing its bias, producing an almost guaranteed 1.5%–2.0% **top-1 accuracy boost** on challenging visual benchmarks like ImageNet. In safety-critical applications (such as autonomous vehicle perception and medical imaging diagnostics) and competitive leaderboards (Kaggle), ensembling is the universally recognized gold standard for peak performance.

6.8.1 Model ensembles

The classical approach to ensembling trains M completely independent neural networks (varying random seeds, architectures, or hyperparameter samples) and averages their predicted softmax

class probability distributions at test time:

$$P_{\text{ens}}(y = c \mid \mathbf{x}) = \frac{1}{M} \sum_{m=1}^{M} P_m(y = c \mid \mathbf{x}). \tag{6.126}$$

Because individual models converge into distinct local minima with decorrelated errors, the variance of the averaged prediction scales inversely with the effective ensemble size M. Independent consensus voting significantly dampens the effect of outliers and individual model pathologies.

Drawback of Independent Ensembles: Prohibitive Computational Cost: While independent ensembles provide superior accuracy, they suffer from a severe practical limitation: **computational training and storage overhead**. Training M independent deep neural networks requires:

$$\text{Total Training Compute} = M \times T_{\text{train}}, \qquad \text{Disk Storage} = M \times S_{\text{model}}. \tag{6.127}$$

For massive models that take days or weeks to train on GPU clusters, multiplying training costs by $M = 5$ or 10 is economically and computationally impossible for most research labs and industrial teams. This drawback motivates the search for an ensemble method that incurs **zero additional training cost**: **Snapshot Ensembles**.

6.8.2 Snapshot ensembles

Gao Huang, Yixuan Li, Geoff Pleiss et al. [43] introduced **Snapshot Ensembles**, an ingenious technique to construct an ensemble of M diverse neural networks **at the cost of a single standard training run** (Figure 6.16):

Snapshot Ensembles via Cyclic Cosine Learning Rate Schedule (Huang et al., 2017)

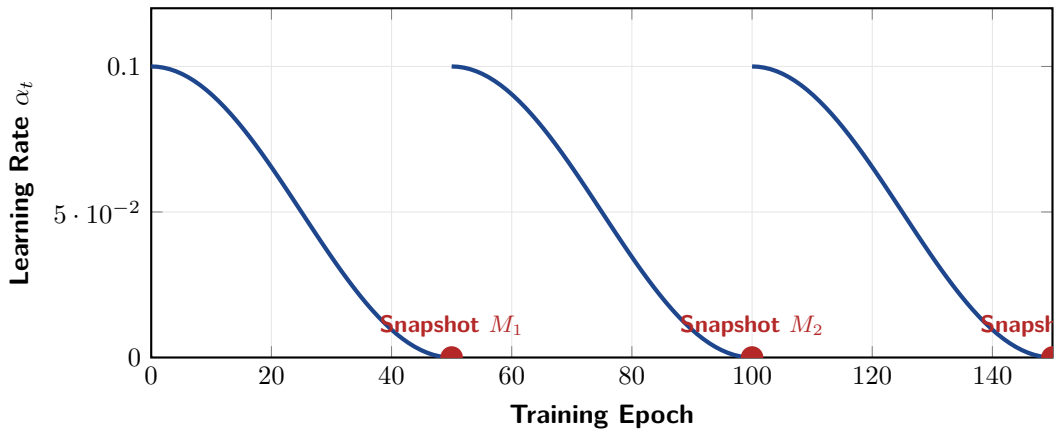


Figure 6.16: Snapshot Ensembles mechanics. A cyclic learning rate schedule executes M optimization cycles. During each cycle, the learning rate anneals to zero, allowing the network to settle into a local minimum. Right before resetting the learning rate to its maximum value (which ejects the model from the current basin), a parameter snapshot M_k is saved. Averaging predictions across the M snapshots yields full ensemble gains without extra training compute.

The mechanism relies on a **Cyclic Learning Rate Schedule with Warm Restarts** (Loshchilov & Hutter, 2017 [60]). The training duration is split into M cycles of length T.

In each cycle, the learning rate starts at a high value α_0 and decays to $\alpha_{\min} \approx 0$ via cosine annealing. As $\alpha \to 0$, the model settles into a local minimum. Right before the learning rate abruptly jumps back to α_0, the weights are saved as snapshot M_k. The abrupt jump to α_0 provides enough kinetic gradient energy to escape the current basin and explore an entirely different local minimum.

Drawback of Snapshot Ensembles: The Test-Time Inference Bottleneck: While Snapshot Ensembles eliminate the $M\times$ training compute penalty, they **do not resolve the test-time deployment bottleneck**:

- At inference time, the system must retain all M model checkpoints in memory.
- To classify a single incoming input $\mathbf{x}$, the system must execute M full forward passes across all M models and average the resulting softmax vectors.
- This multiplies **test-time computational latency and energy consumption by** $M\times$, making snapshot ensembles impractical for real-time edge devices, mobile applications, and low-latency production microservices.

Can we achieve the generalization benefits of ensembling **within a single model, with strictly zero inference latency penalty**? The definitive answer is **Parameter Trajectory Averaging**.

6.8.3 Exponential moving average (EMA)

To eliminate the $M\times$ inference latency bottleneck of classical ensembles, modern deep learning leverages **Exponential Moving Average (EMA) of Weights**, rooted in classical stochastic approximation theory by Boris Polyak and Anatoli Juditsky (1992) [72].

Rather than averaging model outputs in prediction space at test time, Polyak-Juditsky averaging averages the network's weights in **parameter space** during training.

Mathematical Formulation: During standard training, the optimizer updates the raw parameter vector $\mathbf{w}^{(t)}$ via stochastic gradient descent. In parallel, an auxiliary weight vector $\mathbf{w}_{\text{EMA}}^{(t)}$ maintains an exponential running average:

$$\mathbf{w}_{\text{EMA}}^{(t)} = \beta \mathbf{w}_{\text{EMA}}^{(t-1)} + (1-\beta)\mathbf{w}^{(t)}, \tag{6.128}$$

where $\beta \in [0.99, 0.9999]$ is a high momentum coefficient (typically $\beta = 0.999$ or 0.995).

Listing 6.2: Implementing Polyak parameter averaging (EMA) during training.

```
# Polyak EMA implementation
beta = 0.995
w_ema = np.copy(w)

for t, (data, targets) in enumerate(dataloader):
    loss = model.forward(data, targets)
    loss.backward()
    optimizer.step()

    # Update running exponential average of weights
    w_ema = beta * w_ema + (1.0 - beta) * model.get_weights()

# At inference time, load EMA weights for evaluation
model.set_weights(w_ema)
```

Theoretical Mechanism and Geometric Intuition: Because Stochastic Gradient Descent samples mini-batches, the parameter vector $\mathbf{w}^{(t)}$ oscillates noisily around the floor of the loss basin. Polyak averaging smooths out these high-frequency stochastic fluctuations, effectively computing:

$$\mathbf{w}_{\text{EMA}} \approx \frac{1}{K} \sum_{k=0}^{K-1} \mathbf{w}^{(t-k)}. \tag{6.129}$$

Geometrically, this temporal average converges to the **center of wide, flat minima** (the foundational principle behind Stochastic Weight Averaging / SWA).

At inference time, the noisy raw weights $\mathbf{w}^{(t)}$ are discarded, and $\mathbf{w}_{\text{EMA}}$ is loaded directly into the network. The result is a single, robust, regularized model that generalizes significantly better with **strictly $1\times$ parameters, zero extra FLOPs, and zero extra inference latency at test time**.

Chapter 7

Transfer learning and adaptation

In Chapter 6, we established the mathematical mechanisms required to train a deep neural network from scratch: variance-preserving weight initialization, batch normalization dynamics, gradient descent babysitting, and stochastic noise regularization. Yet, an overwhelming empirical challenge remains: **what happens when the target task possesses only a few dozen or hundred labeled training samples?**

Training a modern deep architecture (such as a 50-layer ResNet with 25.6 million parameters) on a tiny dataset violates the classical sample complexity bound: the model will immediately traverse the interpolation threshold into pathological overfitting. Historically, this led to the widespread dogma that deep learning is exclusively viable when millions of labeled training instances are available.

This chapter dismantles that myth through the paradigm of **Representation Reuse and Transfer Learning**: transferring inductive visual and semantic representations learned from massive upstream data reservoirs (such as ImageNet) to data-scarce downstream tasks, navigating the delicate trade-offs between feature extraction, fine-tuning, and out-of-distribution feature distortion, and distilling overparameterized models into lightweight edge architectures.

—

7.1 Transfer learning principles

The classical supervised learning framework assumes that training and testing data are drawn independently and identically from the exact same underlying joint probability distribution:

$$(\mathbf{x}_{\text{train}}, y_{\text{train}}) \sim \mathcal{D}_{\text{source}}, \qquad (\mathbf{x}_{\text{test}}, y_{\text{test}}) \sim \mathcal{D}_{\text{source}}. \tag{7.1}$$

In real-world engineering, this assumption rarely holds. Gathering hundreds of thousands of expert-annotated samples for every novel domain (e.g., rare pathological histopathology, underwater robotics, specialized industrial defect detection) is economically and logistically impossible.

Transfer Learning relaxes the identical distribution assumption. Following the formal statistical framework of Goodfellow, Bengio, and Courville [32], we define a **Domain** $\mathcal{D} = \{\mathcal{X}, P(X)\}$ as a tuple of a feature space $\mathcal{X}$ and a marginal probability distribution $P(X)$, and a **Task** $\mathcal{T} = \{\mathcal{Y}, P(Y \mid X)\}$ as a tuple of a label space $\mathcal{Y}$ and an objective conditional probability distribution $P(Y \mid X)$.

Definition 7.1 (Transfer Learning)**.** Given a source domain $\mathcal{D}_{\mathcal{S}}$ with source task $\mathcal{T}_{\mathcal{S}}$ and a target domain $\mathcal{D}_{\mathcal{T}}$ with target task $\mathcal{T}_{\mathcal{T}}$, transfer learning aims to improve the learning of the target conditional distribution $P_{\mathcal{T}}(Y_{\mathcal{T}} \mid X_{\mathcal{T}})$ using the statistical strength learned from $\mathcal{D}_{\mathcal{S}}$ and $\mathcal{T}_{\mathcal{S}}$, where:

$$\mathcal{D}_{\mathcal{S}} \neq \mathcal{D}_{\mathcal{T}} \qquad \text{or} \qquad \mathcal{T}_{\mathcal{S}} \neq \mathcal{T}_{\mathcal{T}}. \tag{7.2}$$

This definition establishes the clear tripartite taxonomy distinguishing transfer learning from related paradigms:

1. **Transfer Learning**: The tasks themselves differ ($\mathcal{T}_\mathcal{S} \neq \mathcal{T}_\mathcal{T}$). For instance, $\mathcal{Y}_\mathcal{S}$ comprises 1,000 everyday object classes in ImageNet, while $\mathcal{Y}_\mathcal{T}$ comprises 5 stages of diabetic retinopathy. The label spaces and conditional functions $P(Y \mid X)$ are distinct.

2. **Domain Adaptation (Covariate Shift)**: The task remains identical ($\mathcal{T}_\mathcal{S} = \mathcal{T}_\mathcal{T}$, meaning $\mathcal{Y}_\mathcal{S} = \mathcal{Y}_\mathcal{T}$ and the true underlying conditional mapping $P(Y \mid X)$ is invariant), but the input marginal distribution shifts ($\mathcal{P}_\mathcal{S}(X) \neq \mathcal{P}_\mathcal{T}(X)$). Examples include training a sentiment classifier on Amazon book reviews and deploying it on consumer electronics reviews, or training autonomous driving perception on sunny California highways and deploying it in snowy Nordic blizzards.

3. **Concept Drift**: A non-stationary data stream where the joint distribution $\mathcal{P}_t(X, Y)$ evolves continuously over temporal horizon t, requiring continual adaptation of internal representations.

7.1.1 Factors of variation

Why does transfer learning work across seemingly unrelated visual problems? Goodfellow et al. [32] explain this through the hypothesis of **Shared Factors of Variation**.

High-dimensional sensory data (such as natural images) is generated by underlying physical, geometric, and semantic latent causes: illumination directions, surface reflectance, object poses, edge configurations, and 3D volumetric topologies. When a deep network trains on millions of natural images, its hidden layers learn to disentangle these latent factors. Because these physical factors are shared across virtually all optical phenomena, the learned internal representations remain highly informative for downstream domains.

This insight gives rise to an **architectural duality** in how representations are shared across tasks (Figure 7.1):

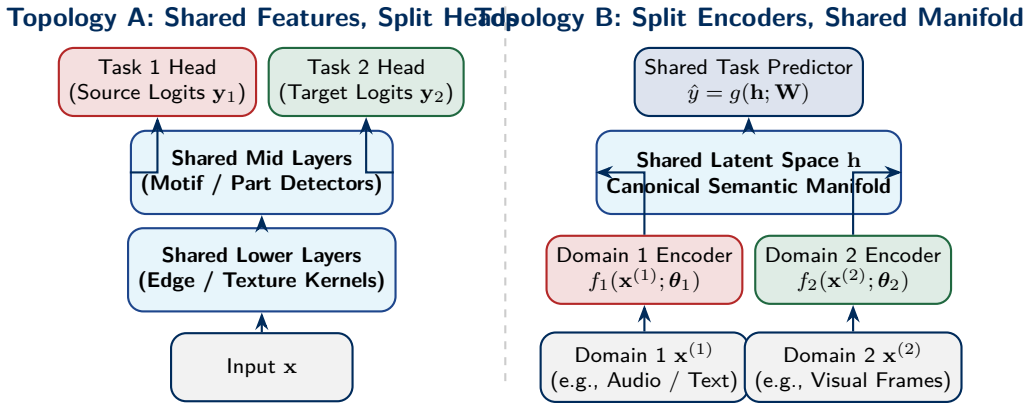


Figure 7.1: The architectural duality of shared representations (Goodfellow et al., 2016). In Topology A (standard visual transfer), inputs share identical pixel statistics; lower feature extractors are shared while task-specific heads branch into distinct label spaces. In Topology B (domain adaptation and multimodal alignment), disparate input spaces are transformed by domain-specific encoders into a shared, canonical latent manifold $\mathbf{h}$, which feeds into a unified task predictor.

- **Topology A (Shared Lower Features, Task-Specific Upper Layers)**: The dominant configuration in computer vision. The input space $\mathcal{X}$ has identical dimensionality and pixel statistics across tasks. The early and intermediate layers are shared, capturing general visual

primitives, while specialized classification heads branch out to predict task-dependent labels $y^{(1)}, y^{(2)}$.

- **Topology B (Task-Specific Lower Encoders, Shared Canonical Representation)**: Occurs when the output variable y has the identical semantics across domains, but the input spaces $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}$ differ in dimension or sensory modality. For example, in multi-speaker speech recognition, speaker-specific acoustic front-ends map disparate vocal tracts into a shared phonetic representation. Similarly, in multimodal vision-language models (e.g., CLIP), disparate image and text encoders project into a unified metric embedding space.

7.1.2 Hierarchical representations in CNNs

Why can weights trained to classify 1,000 everyday object categories in ImageNet (e.g., golden retrievers, tea kettles, freight cars) assist in detecting metastatic tumor cells in microscopic lymph node biopsies?

The answer lies in the **hierarchical nature of deep visual representations**, first uncovered empirically by Matthew Zeiler and Rob Fergus [101] and analyzed theoretically by Jason Yosinski et al. [99]:

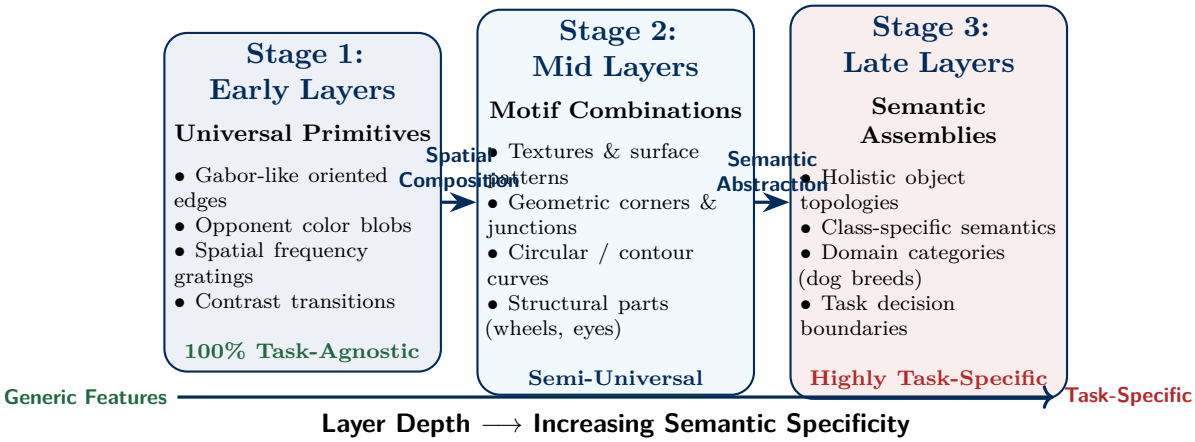


Figure 7.2: The hierarchical representation spectrum in deep convolutional neural networks (Zeiler & Fergus, 2014). Early convolutional layers learn universal spatial primitives (oriented Gabor edges, color opponency) that transfer seamlessly to virtually any natural visual domain. Intermediate layers compose edges into motifs, textures, and object parts. Late fully connected and global average pooled layers compute high-level semantic assemblies tuned specifically to the source task categories.

As formalized in Figure 7.2, a deep network does not learn monolithic representations. Instead, it computes an ascending hierarchy of abstractions:

1. **Early Layers (Generic Primitives)**: The initial convolutional kernels (Conv_1, Conv_2) consistently converge to localized oriented edge detectors, color blobs, and spatial gratings. These filters are mathematically analogous to the receptive fields of simple cells in mammalian primary visual cortex (V_1). Because all natural optical images—whether medical CT scans, microscopic biology, or satellite panoramas—share identical low-level edge and contrast statistics, **early layers are universal and task-agnostic**.

2. **Intermediate Layers (Motifs and Parts)**: Mid-level layers (Conv_3, Conv_4) compose primitive edges into geometric corners, repetitive textures, honeycombs, and component parts (e.g., circular contours, mesh grids).

3. **Deep Layers (Task-Specific Semantics)**: The final stages (Conv_5, fully connected layers, or class logits) assemble mid-level parts into complete class configurations. These representations encode high-level source domain semantics (distinguishing an Afghan Hound from a Golden Retriever) and become increasingly non-transferable to distant downstream problems.

7.1.3 Off-the-shelf features and DeCAF

In 2014, Jeff Donahue et al. [22] and Ali Sharif Razavian et al. [86] established that deep convolutional networks trained on ImageNet act as **universal off-the-shelf feature extractors**.

By stripping off the final 1,000-way classification softmax layer and passing target images through the frozen convolutional backbone, the intermediate activation vector (e.g., the 4,096-dimensional output of layer `fc7` in AlexNet) serves as an exceptionally rich generic descriptor:

$$\boldsymbol{\phi}(\mathbf{x}) = f_{\text{frozen}}(\mathbf{x}; \boldsymbol{\theta}_{\text{ImageNet}}) \in \mathbb{R}^{4096}. \tag{7.3}$$

Training a simple linear Support Vector Machine (SVM) or logistic regressor on these extracted vectors $\boldsymbol{\phi}(\mathbf{x})$ obliterated decades of hand-crafted visual feature engineering (SIFT, HOG, GIST) across diverse benchmarks including Caltech-101, PASCAL VOC object detection, and fine-grained bird classification.

Bishop's Linear Separability Engine and Forward Feature Caching: In their 2024 treatise, Bishop and Bishop [6] provide a clarifying geometric perspective: successive layers of a deep network act as a **linear separability engine**. In the raw input pixel space $\mathcal{X}$, data classes are hopelessly intertwined along complex nonlinear manifolds. The hidden layers unroll and untangle these manifolds, mapping the inputs into an embedding space $\boldsymbol{\phi}(\mathbf{x}) \in \mathbb{R}^d$ where the target classes become linearly separable by a simple hyperplane.

Furthermore, Bishop highlights a crucial computational efficiency principle: **Forward Feature Caching**. When performing linear probing, running target mini-batches through the entire network on every optimization epoch is computationally wasteful. Because the backbone parameters $\boldsymbol{\theta}_{\text{frozen}}$ are invariant:

$$\mathcal{D}_{\text{cached}} = \{(\boldsymbol{\phi}(\mathbf{x}_i), y_i)\}_{i=1}^{N_{\mathcal{T}}}, \qquad \boldsymbol{\phi}(\mathbf{x}_i) = f_{\text{frozen}}(\mathbf{x}_i). \tag{7.4}$$

We pass the target dataset **once** through the frozen network, save the compact feature matrix $\boldsymbol{\Phi} \in \mathbb{R}^{N_{\mathcal{T}} \times d}$ in system memory, and discard the heavy convolutional backbone during training. Fitting the target linear classifier reduces to a strictly convex optimization problem (such as Ridge classification or L-BFGS logistic regression) that converges in seconds without GPU backpropagation.

—

7.2 Linear probing versus fine-tuning

When deploying a pre-trained network on a downstream task, the practitioner faces a fundamental architectural continuum between two primary protocols:

1. Linear Probing (Feature Extraction): The entire pre-trained backbone parameter vector $\boldsymbol{\theta}_{\text{backbone}}$ is kept strictly **frozen** ($\nabla_{\boldsymbol{\theta}} \mathcal{L} \leftarrow \mathbf{0}$). The source classification head is discarded and replaced with a newly initialized target head $\mathbf{W}_{\text{head}} \in \mathbb{R}^{C_{\text{target}} \times D}$ and bias $\mathbf{b}_{\text{head}} \in \mathbb{R}^{C_{\text{target}}}$. Optimization updates strictly the head parameters:

$$\hat{y} = \text{softmax}\left(\mathbf{W}_{\text{head}} \boldsymbol{\phi}(\mathbf{x}; \boldsymbol{\theta}_{\text{frozen}}) + \mathbf{b}_{\text{head}}\right). \tag{7.5}$$

Because the feature representations are fixed, linear probing reduces to a convex optimization problem: it is computationally rapid, possesses zero risk of corrupting pre-trained features, and cannot overfit through internal representation shifts.

2. Full Fine-Tuning: The target classification head is initialized randomly, but the backbone parameters $\boldsymbol{\theta}_{\text{backbone}}$ are **unfrozen**. Gradients backpropagate through the entire network, updating all parameters end-to-end:

$$\boldsymbol{\theta}_{\text{backbone}} \leftarrow \boldsymbol{\theta}_{\text{backbone}} - \eta_{\text{backbone}} \nabla_{\boldsymbol{\theta}} \mathcal{L}, \qquad \mathbf{W}_{\text{head}} \leftarrow \mathbf{W}_{\text{head}} - \eta_{\text{head}} \nabla_{\mathbf{W}} \mathcal{L}. \tag{7.6}$$

7.2.1 Regularization and differential learning rates

Why does fine-tuning work so effectively compared to random initialization? Simon Prince [73] formalizes transfer learning as an **implicit parameter regularization and initialization prior**. In an overparameterized deep loss landscape, the global objective possesses countless local minima and saddle points. Initializing with pre-trained weights places the parameters within a favorable basin of attraction—a region characterized by well-conditioned curvature and high generalization capacity.

Prince categorizes transfer learning as part of a four-fold data-centric regularization continuum:

1. **Data Augmentation**: Expands the existing dataset via label-preserving spatial and photometric transformations.
2. **Multitask Learning**: Leverages additional concurrent supervisory signals to share statistical strength across tasks.
3. **Transfer Learning**: Sequentially transfers inductive bias learned from a plentiful secondary dataset to a scarce primary dataset.
4. **Self-Supervised Learning**: Exploits massive unlabeled corpora to pre-train generalizable representations via generative masking or contrastive instance discrimination.

The D2L Differential Learning Rate Protocol: In *Dive into Deep Learning*, Zhang et al. [102] formulate the definitive engineering protocol for fine-tuning. The model consists of two fundamentally asymmetrical components:

- **The Backbone**: Contains millions of parameters carefully pre-trained on massive data. These weights reside in a delicate, well-conditioned minimum. Exposing them to large learning rates causes catastrophic gradient shock, destroying the pre-trained feature detectors.
- **The Output Head**: Newly added and initialized with random Gaussian noise. It requires substantial gradient updates to establish initial alignment with target labels.

To balance these conflicting demands, Zhang et al. prescribe the **differential learning rate rule**:

$$\eta_{\text{head}} = 10 \cdot \eta_{\text{backbone}}, \qquad \text{e.g.,} \quad \eta_{\text{backbone}} = 10^{-4}, \quad \eta_{\text{head}} = 10^{-3}. \tag{7.7}$$

Furthermore, Zhang et al. emphasize that downstream inputs **must strictly preserve the source normalization constants**:

$$\boldsymbol{\mu}_{\text{ImageNet}} = [0.485, 0.456, 0.406], \qquad \boldsymbol{\sigma}_{\text{ImageNet}} = [0.229, 0.224, 0.225]. \tag{7.8}$$

Using the target dataset's own empirical mean and variance shifts the coordinate origin, forcing early convolutional filters to operate outside their calibrated activation distributions.

7.2.2 Transfer learning decision matrix

How should a machine learning engineer choose between Linear Probing, partial fine-tuning, and full fine-tuning? As formalized by Fei-Fei Li and Justin Johnson, the choice is governed by a 2×2 **Decision Matrix** balancing **Target Dataset Size** against **Domain Similarity** (Figure 7.3):

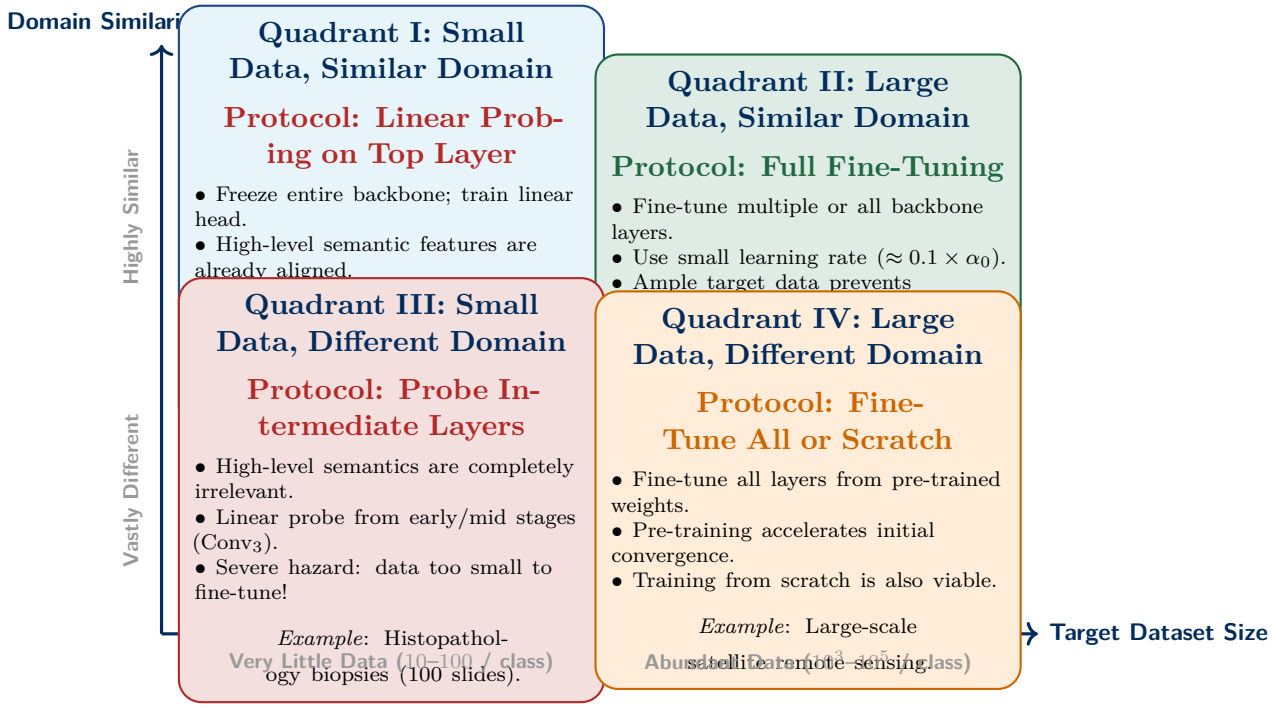


Figure 7.3: The Transfer Learning Decision Matrix. Optimal adaptation protocols depend systematically on the interplay between target data availability and the semantic divergence between source and target domains.

1. **Small Dataset, Similar Task (Quadrant I)**: Because target data is scarce, fine-tuning large parameter matrices guarantees overfitting. Because the target task is visually similar to ImageNet (e.g., classifying dog breeds), the pre-trained top-layer features already encode the relevant semantic discriminants. **Prescription**: Freeze the backbone completely; train strictly a linear classifier on top.

2. **Large Dataset, Similar Task (Quadrant II)**: With abundant data, overfitting is less threatening. The pre-trained weights provide an exceptional initialization. **Prescription**: Fine-tune the entire network with a small learning rate ($\alpha_{\text{FT}} \approx 0.1 \times \alpha_{\text{base}}$), allowing representations to deform gently to optimal target boundaries.

3. **Small Dataset, Different Task (Quadrant III)**: The most treacherous quadrant in applied machine learning. High-level ImageNet features are irrelevant or misleading, yet the dataset is too small to fine-tune millions of weights. **Prescription**: Linear probing from intermediate stages (e.g., pooling features from Conv_3 or Conv_4, which retain generic textures and edge combinations rather than semantic natural object assemblies).

4. **Large Dataset, Different Task (Quadrant IV)**: Because data is plentiful, the network can afford to reconfigure all feature layers. Starting from pre-trained weights accelerates optimization compared to random initialization, though training from scratch remains competitive.

7.2.3 Transfer learning in modern vision and AI

Transfer learning is not an auxiliary trick; it is the universal backbone of modern applied artificial intelligence:

- **Object Detection (Fast and Faster R-CNN)**: Ross Girshick [29] built the modern object detection framework by taking a pre-trained ImageNet visual backbone (VGG-16 or ResNet), attaching a Region of Interest (RoI) pooling layer to intermediate convolutional features, and fine-tuning the dual outputs for multi-task loss (classification cross-entropy plus bounding box smooth L_1 regression).
- **Multimodal Vision-Language (Karpathy & Fei-Fei, 2015)** [49]: Andrej Karpathy and Li Fei-Fei pioneered end-to-end image captioning by transferring visual features from an ImageNet-pretrained CNN into an unrolled Recurrent Neural Network (RNN/LSTM) initialized with Word2Vec word embeddings, bridging visual and linguistic semantic spaces.
- **Unified Vision-Language Pre-training (Zhou et al., 2020)** [106]: Modern foundation pipelines train a CNN on ImageNet, fine-tune on Visual Genome for object and attribute grounding, train a BERT language model on massive text corpora, and jointly fine-tune the multimodal encoder-decoder for Visual Question Answering (VQA) and caption generation.

—

7.3 Rethinking pre-training

For years, the universal assumption across computer vision was that pre-training on ImageNet provided an indispensable regularizer that yielded superior final accuracy that could never be matched by training from scratch.

In 2019, Kaiming He, Ross Girshick, and Piotr Dollár [34] published their landmark paper *"Rethinking ImageNet Pre-training"*, executing a rigorous empirical deconstruction that transformed the theoretical understanding of transfer learning:

COCO Object Detection: Pre-training vs. Training from Scratch (He et al., 2019)

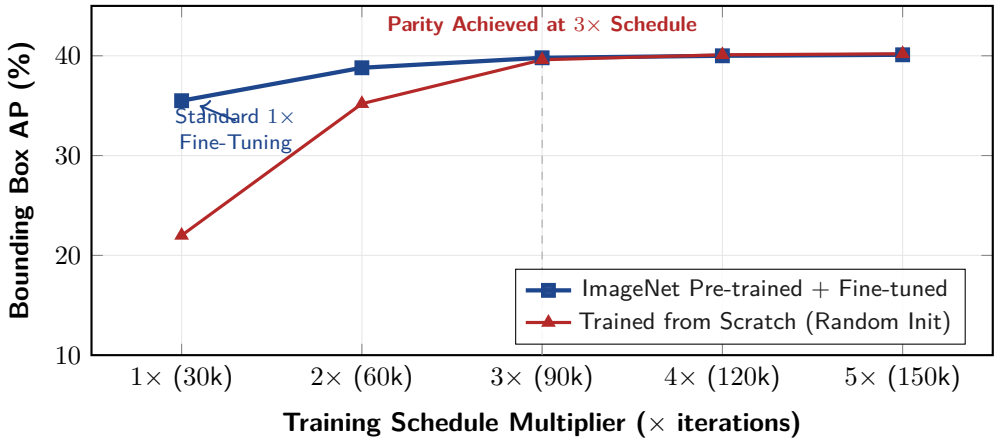


Figure 7.4: Empirical verification of the Rethinking Pre-training phenomenon (He et al., ICCV 2019). On COCO object detection, standard fine-tuning schedules (1×) suggest that pre-training is indispensable. However, extending the schedule to 3× enables a randomly initialized network trained strictly from scratch to achieve identical asymptotic performance ($\approx 40.1\%$ AP). Pre-training accelerates initial learning dynamics but does not alter final model capacity when target data is abundant.

The seminal findings of He et al. [34] include:

1. **Parity Under Extended Optimization**: When trained on the standard 1× baseline schedule (30k iterations), ImageNet pre-training vastly outperforms random initialization (35.5% vs. 22.0% AP). However, this gap is an artifact of premature termination. If the model trained from scratch is allowed to train for a 3× longer schedule (90k iterations), **it achieves identical asymptotic accuracy to the pre-trained model** (40.1% vs. 39.8% AP).

2. **Pre-training as Convergence Acceleration**: ImageNet pre-training does not automatically improve the model's fundamental representational limit or regularize against asymptotic overfitting. Its primary benefit is **computational acceleration**: it provides a highly conditioned parameter initialization that saves training wall-clock time.

3. **The Data Volume Dependency**: When target data is abundant (e.g., the full COCO dataset with 118k images), collecting more target data is far more effective than pre-training on out-of-domain images. However, **when target training data is scarce (1k–10k images), pre-training remains strictly indispensable**: models trained from scratch on tiny datasets fail to converge or overfit catastrophically.

—

7.4 Distribution shifts and the LP-FT protocol

While fine-tuning is universally regarded as superior to linear probing on in-distribution data, Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang [55] uncovered a profound theoretical paradox: **Fine-tuning can severely distort pre-trained representations and dramatically underperform linear probing out-of-distribution**.

7.4.1 The OOD performance paradox

Consider a pre-trained foundation model evaluated across two distinct testing distributions:

- **In-Distribution (ID)**: Test data drawn from the identical geographic or sensor distribution as the target training set (e.g., satellite imagery over North America).
- **Out-of-Distribution (OOD)**: Test data subject to natural distribution shifts (e.g., satellite imagery over Africa or Europe).

Adaptation Protocol	ID Test Accuracy (%)	OOD Test Accuracy (%)	OOD vs. ID Gap
Linear Probing (LP)	82.9%	**66.2**%	−16.7%
Standard Fine-Tuning (FT)	**85.1**%	59.3%	**−25.8**% (Severe Collapse!
LP-FT (Kumar et al., 2022)	**85.7**%	**68.8**%	**−16.9**% (+9.5% over FT

Table 7.1: Empirical performance comparison of adaptation protocols across 10 benchmark distribution shift datasets (Kumar et al., ICLR 2022). While standard fine-tuning improves in-distribution accuracy by +2.2%, it causes an catastrophic −6.9% drop on out-of-distribution generalization compared to linear probing. The 2-step LP-FT protocol resolves this dilemma, simultaneously outperforming FT in-distribution and crushing FT out-of-distribution.

As documented in Table 7.1, on 8 out of 10 distribution shift datasets, **Linear Probing substantially outperforms Full Fine-Tuning under distribution shift** (66.2% vs. 59.3%). Why does fine-tuning destroy out-of-distribution generalization?

7.4.2 Feature distortion mechanics

The vulnerability of fine-tuning originates from the **random initialization of the target classification head**:

1. At iteration $t = 0$, the backbone $\boldsymbol{\phi}(\mathbf{x})$ contains well-formed, pre-trained feature representations that generalize well across diverse domains. However, the newly appended linear head $\mathbf{W}_{\text{head}}$ contains completely random Gaussian weights.

2. During the initial gradient descent steps, the classification error $\mathcal{L}$ is enormous. The gradient backpropagated from the head into the backbone features is:

$$\nabla_{\boldsymbol{\phi}} \mathcal{L} = \mathbf{W}_{\text{head}}^{\top} \nabla_{\mathbf{z}} \mathcal{L}. \tag{7.9}$$

3. Because $\mathbf{W}_{\text{head}}$ is uncalibrated, $\nabla_{\boldsymbol{\phi}} \mathcal{L}$ is a massive, noisy vector. Backpropagating this large gradient through the pre-trained weights $\boldsymbol{\theta}_{\text{backbone}}$ causes **Feature Distortion**: it violently distorts the pre-trained feature representations during the very first epochs of training.

4. As analyzed by Simon Prince [73] and proven by Bubeck and Sellke [8], overparameterized networks have the capacity to fit arbitrary associations. Under the massive gradient shocks of an unaligned head, the overparameterized backbone distorts its manifold specifically to satisfy in-distribution samples through spurious shortcuts, destroying its out-of-distribution invariance.

7.4.3 The 2-step LP-FT algorithm

To retain the expressive feature adaptation of fine-tuning while completely immunizing the network against feature distortion, Kumar et al. [55] formulated the **Linear Probing then Fine-Tuning (LP-FT)** protocol:

Algorithm 4 The 2-Step LP-FT Adaptation Protocol

Require: Pre-trained backbone $f(\cdot; \boldsymbol{\theta}_{\text{pre}})$, target dataset $\mathcal{D}_{\mathcal{T}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, epochs E_1, E_2, learning rates $\eta_{\text{probe}}, \eta_{\text{FT}}$.

Initialize linear classification head $\mathbf{W}_{\text{head}} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}), \mathbf{b}_{\text{head}} = \mathbf{0}$.
Phase 1: Linear Probing (Head Alignment)
Freeze backbone: $\nabla_{\boldsymbol{\theta}} \mathcal{L} \leftarrow \mathbf{0}$.
for epoch $= 1$ to E_1 **do**
    Train **strictly** the head parameters $(\mathbf{W}_{\text{head}}, \mathbf{b}_{\text{head}})$ to convergence using η_{probe}.
end for
    ▷ The classification head is now perfectly aligned with the pre-trained feature manifold.
Phase 2: Gentle End-to-End Fine-Tuning
Unfreeze all backbone parameters: enable $\nabla_{\boldsymbol{\theta}} \mathcal{L}$.
for epoch $= 1$ to E_2 **do**
    Train the full network end-to-end using a scaled learning rate η_{FT} with cosine decay.
end for
return Fully adapted model $(\boldsymbol{\theta}^*, \mathbf{W}_{\text{head}}^*, \mathbf{b}_{\text{head}}^*)$.

Why LP-FT Prevents Feature Distortion: In Phase 1, training only the head aligns $\mathbf{W}_{\text{head}}$ with the pre-trained feature space without altering a single weight in the backbone. When Phase 2 begins, the initial error gradient $\nabla_{\boldsymbol{\phi}} \mathcal{L}$ is already small and meaningful: there are no violent initial gradient shocks. The features are refined smoothly to maximize target margin without distorting the underlying universal manifold.

As confirmed in Table 7.1, LP-FT achieves the highest accuracy across both domains: 85.7% **on ID** (beating standard fine-tuning) and 68.8% **on OOD** (+9.5% **over fine-tuning**).

—

7.5 Knowledge distillation

Up to this point, representation transfer has focused on adapting a single model to a new task. A complementary paradigm concerns transferring the knowledge of an overparameterized, high-capacity model (or model ensemble) into a **compact, lightweight student network** suitable for edge deployment.

The roots of model compression date back to Cristian Buciluǎ, Rich Caruana, and Alexandru Niculescu-Mizil (2006) [9], who demonstrated that massive ensembles could be compressed into a single neural network by matching pseudo-labeled predictions on large unlabeled data pools. This methodology was modernized into its foundational theoretical form by Geoffrey Hinton, Oriol Vinyals, and Jeff Dean (2015) [38] as **Knowledge Distillation**.

7.5.1 Dark knowledge and soft targets

Standard supervised classification trains networks using **hard one-hot ground-truth targets**:

$$\mathbf{y} = [0, \dots, 0, 1, 0, \dots, 0]^\top \in \mathbb{R}^C. \tag{7.10}$$

A one-hot label provides an impoverished supervisory signal: it treats all non-ground-truth classes as equally impossible. For instance, in an animal classification task:

- An image of a *Tiger* may have ground-truth label $y_{\text{tiger}} = 1$.
- Hard cross-entropy penalizes a model equally whether it assigns small probability to a *Tabby Cat* or to a *Boeing 747*.

However, a well-trained, high-capacity **Teacher Model** computes continuous pre-softmax logit scores $\mathbf{z} \in \mathbb{R}^C$. These logits encode rich geometric information about the semantic relationships among categories: the teacher recognizes that a tiger shares extensive structural traits with a domestic cat, but virtually none with an airliner.

Hinton et al. [38] termed this hidden structural similarity **Dark Knowledge**. Transferring dark knowledge enables a compact student network to learn the smooth decision manifolds discovered by the cumbersome teacher.

7.5.2 Temperature-scaled Softmax

In a standard trained network, the softmax probabilities:

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \tag{7.11}$$

are heavily peaked: the correct class probability is typically $p_k > 0.999$, while non-target probabilities are crushed into floating-point underflow (10^{-6} to 10^{-12}). This masks the dark knowledge.

To expose the relative class affinities, Hinton et al. introduced a scalar **Temperature Parameter** $T > 1$ into the softmax function:

$$p_i^{(T)} = \frac{\exp\left(\frac{z_i}{T}\right)}{\sum_{j=1}^{C} \exp\left(\frac{z_j}{T}\right)}. \tag{7.12}$$

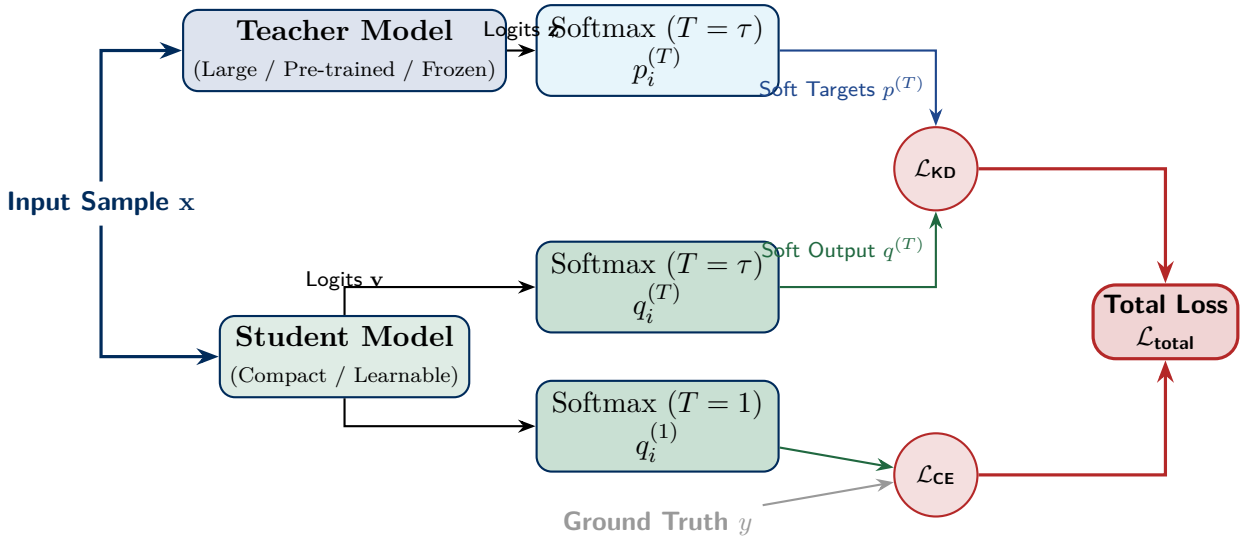


Figure 7.5: The Knowledge Distillation architectural framework (Hinton et al., 2015). A high-capacity pre-trained teacher emits softened class probabilities $p^{(T)}$ at temperature $T > 1$. The student network matches these soft targets via Kullback-Leibler divergence $\mathcal{L}_{\text{KD}}$ while simultaneously minimizing standard hard cross-entropy $\mathcal{L}_{\text{CE}}$ at $T = 1$.

- When $T = 1$, Equation (7.12) recovers standard softmax.
- When $T \to \infty$, the logits are compressed toward zero, and the probabilities approach a uniform distribution: $p_i \to \frac{1}{C}$. In this high-temperature limit, the relative logit differences $z_i - z_j$ are preserved linearly without exponential exaggeration.

7.5.3 Distillation loss objective

As illustrated in Figure 7.5, the student network (with trainable weights $\mathbf{w}$ producing logits $\mathbf{v}$) is trained on a linear combination of two objectives:

1. **Soft Target Loss ($\mathcal{L}_{\text{KD}}$)**: Minimizes the Kullback-Leibler (KL) divergence between the softened teacher probabilities $p^{(T)}$ and the softened student probabilities $q^{(T)}$:

$$\mathcal{L}_{\text{KD}}(\mathbf{v}, \mathbf{z}; T) = D_{\text{KL}}\left(\mathbf{p}^{(T)} \parallel \mathbf{q}^{(T)}\right) = \sum_{i=1}^{C} p_i^{(T)} \log\left(\frac{p_i^{(T)}}{q_i^{(T)}}\right). \tag{7.13}$$

2. **Hard Target Loss ($\mathcal{L}_{\text{CE}}$)**: Standard cross-entropy between the student's unsoftened predictions ($T = 1$) and the true one-hot ground-truth label y:

$$\mathcal{L}_{\text{CE}}(\mathbf{v}, y) = -\sum_{i=1}^{C} y_i \log q_i^{(T=1)}. \tag{7.14}$$

Combining both terms yields the total Knowledge Distillation objective:

$$\mathcal{L}_{\text{total}} = (1 - \lambda)\mathcal{L}_{\text{CE}}(\mathbf{v}, y) + \lambda \cdot T^2 \cdot D_{\text{KL}}\left(\mathbf{p}^{(T)} \parallel \mathbf{q}^{(T)}\right), \tag{7.15}$$

where $\lambda \in [0, 1]$ is a balancing hyperparameter (typically $\lambda = 0.5$ or 0.7).

7.5.4 Derivation of the T^2 scaling factor

A common oral examination trap is to ask: **why is the distillation loss multiplied by T^2 in Equation (7.15)?**

To see why, we compute the gradient of the KL divergence loss with respect to a single student logit v_i. Using the standard cross-entropy gradient identity $\frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_i} = q_i - y_i$, the derivative of the soft loss with respect to v_i is:

$$\frac{\partial \mathcal{L}_{\text{KD}}}{\partial v_i} = \frac{\partial}{\partial v_i}\left[-\sum_{j=1}^{C} p_j^{(T)} \log q_j^{(T)}\right] = \frac{1}{T}\left(q_i^{(T)} - p_i^{(T)}\right). \tag{7.16}$$

Now, evaluate the high-temperature Taylor series expansion ($T \to \infty$) for the softened probabilities:

$$q_i^{(T)} = \frac{\exp(v_i/T)}{\sum_j \exp(v_j/T)} \approx \frac{1 + \frac{v_i}{T}}{C + \frac{1}{T}\sum_j v_j}. \tag{7.17}$$

Assuming pre-activations are zero-mean centered across classes ($\sum_j v_j = 0$ and $\sum_j z_j = 0$):

$$q_i^{(T)} \approx \frac{1}{C} + \frac{v_i}{CT}, \qquad p_i^{(T)} \approx \frac{1}{C} + \frac{z_i}{CT}. \tag{7.18}$$

Substituting these approximations into the gradient expression Equation (7.16):

$$\frac{\partial \mathcal{L}_{\text{KD}}}{\partial v_i} = \frac{1}{T}\left[\left(\frac{1}{C} + \frac{v_i}{CT}\right) - \left(\frac{1}{C} + \frac{z_i}{CT}\right)\right] = \frac{1}{CT^2}(v_i - z_i). \tag{7.19}$$

Theorem 7.1 (T^2 Gradient Normalization)**.** *As temperature T increases, the magnitude of the gradient $\frac{\partial \mathcal{L}_{KD}}{\partial v_i}$ scales inversely as $\frac{1}{T^2}$.*

Multiplying the distillation loss by T^2 in the objective function ensures that the gradient magnitude remains invariant to the temperature parameter:

$$\nabla_{\mathbf{v}}\left[T^2 \mathcal{L}_{KD}\right] \approx \frac{1}{C}(\mathbf{v} - \mathbf{z}). \tag{7.20}$$

This keeps the relative gradient contributions of the hard ground-truth loss and the soft distillation loss properly balanced, preventing the soft dark knowledge signal from vanishing as T increases.

7.5.5 Attention transfer

In complex computer vision tasks, distilling strictly the final pre-softmax logits is often insufficient: two deep architectures can output identical class probabilities while routing information through entirely disparate internal representational mechanisms.

As reviewed by Simon Prince [73], Sergey Zagoruyko and Nikos Komodakis (2017) [100] introduced **Attention Transfer (AT)** to supervise intermediate layers directly. They defined spatial attention maps that capture where a convolutional network concentrates its representational energy:

Definition 7.2 (Spatial Attention Map)**.** Let $\mathbf{A} \in \mathbb{R}^{C \times H \times W}$ denote the 3D activation tensor at an intermediate convolutional stage with C feature channels and spatial dimensions $H \times W$. The spatial attention mapping operator $\mathcal{F}_{\text{AT}} : \mathbb{R}^{C \times H \times W} \to \mathbb{R}^{H \times W}$ is computed by summing the absolute channel energies:

$$\mathcal{F}_{\text{AT}}(\mathbf{A}) = \sum_{c=1}^{C} |\mathbf{A}_c|^p, \tag{7.21}$$

where $p = 2$ provides optimal empirical sensitivity to salient visual motifs.

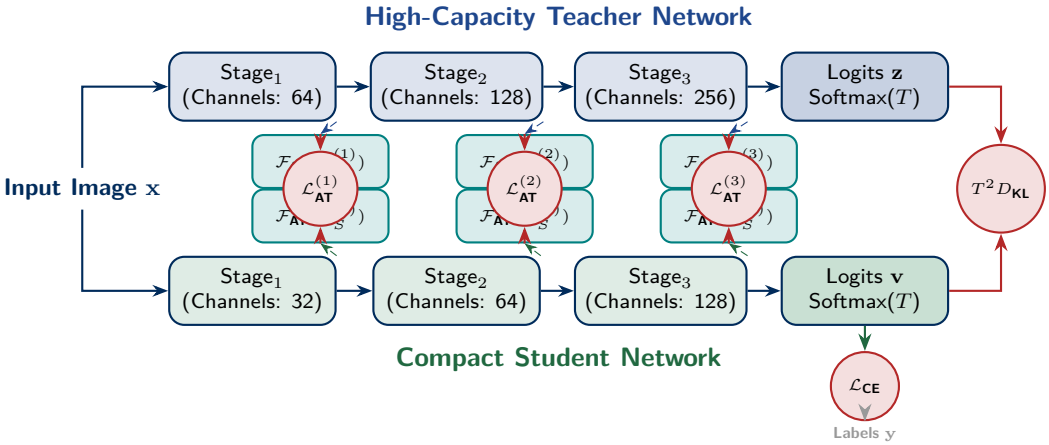


Figure 7.6: Multi-level knowledge distillation combining Hinton's softened logit matching with Zagoruyko & Komodakis (2017) intermediate Attention Transfer (AT). Beyond mimicking output probabilities via $T^2 D_{\mathrm{KL}}$, the compact student is explicitly supervised at intermediate stages $(\mathcal{L}_{\mathrm{AT}}^{(1)}, \mathcal{L}_{\mathrm{AT}}^{(2)}, \mathcal{L}_{\mathrm{AT}}^{(3)})$ to replicate the 2D spatial attention heatmaps of the overparameterized teacher.

As illustrated in Figure 7.6, normalized attention vectors $\hat{\mathbf{a}} = \frac{\mathrm{vec}(\mathcal{F}_{\mathrm{AT}}(\mathbf{A}))}{\|\mathrm{vec}(\mathcal{F}_{\mathrm{AT}}(\mathbf{A}))\|_2}$ are matched across a set of paired intermediate stages $\mathcal{I}$:

$$\mathcal{L}_{\mathrm{AT}} = \frac{1}{2} \sum_{j \in \mathcal{I}} \left\| \frac{\mathrm{vec}(\mathcal{F}_{\mathrm{AT}}(\mathbf{A}_S^{(j)}))}{\|\mathrm{vec}(\mathcal{F}_{\mathrm{AT}}(\mathbf{A}_S^{(j)}))\|_2} - \frac{\mathrm{vec}(\mathcal{F}_{\mathrm{AT}}(\mathbf{A}_T^{(j)}))}{\|\mathrm{vec}(\mathcal{F}_{\mathrm{AT}}(\mathbf{A}_T^{(j)}))\|_2} \right\|_2^2. \tag{7.22}$$

By matching these 2D spatial attention heatmaps, the student model is forced not only to produce identical class logits, but to look at the exact same salient anatomical or visual parts (e.g., eyes, wheels, tumor borders) as the heavy teacher network.

7.5.6 Distillation versus overparameterization

An enduring theoretical question in deep learning is whether distillation can compress an overparameterized model down to an *underparameterized* regime without sacrificing generalization accuracy.

Simon Prince [73] notes that while distillation can successfully approximate a 34-layer ResNet (~ 63 million parameters) with an 18-layer ResNet (~ 11 million parameters) on ImageNet, the distilled student is **still strictly overparameterized** relative to the training set size (1.28 million images).

Recent mathematical theory explains why underparameterized models encounter fundamental boundaries:

Theorem 7.2 (Universal Law of Robustness, Bubeck & Sellke, 2021)**.** *In a D-dimensional feature space, smooth interpolation of N data points with a Lipschitz-bounded neural network requires at least:*

$$P \gtrsim N \cdot D \tag{7.23}$$

parameters [8]. Simple interpolation without smoothness requires only $P \approx N$, but maintaining a smooth, non-oscillating decision boundary requires an additional factor of D in model capacity.

Consequently, compressing a network below this threshold impairs its Lipschitz continuity, leading to fragile decision boundaries and degraded out-of-distribution robustness. Distillation transfers dark knowledge and smooths optimization landscapes, but it cannot override the geometric capacity bounds governing smooth function approximation.

—

7.6 Transfer, multitask, and meta-learning

Transfer learning does not exist in isolation. It forms a coherent theoretical spectrum with **Multitask Learning** and **Meta-Learning**, as synthesized across Bishop [6], Goodfellow et al. [32], and Prince [73]:

1. **Transfer Learning (Sequential Transfer)**: Knowledge is acquired sequentially on an upstream source domain $\mathcal{D}_{\mathcal{S}}$ and subsequently transferred to a downstream target domain $\mathcal{D}_{\mathcal{T}}$. The primary challenge is preventing *catastrophic forgetting* of generic representations while adapting to target specificities.

2. **Multitask Learning (Simultaneous Joint Learning)**: Rich Caruana (1997) [11] pioneered multitask learning, where a network is trained to optimize K related tasks simultaneously:

$$\mathcal{L}_{\text{multitask}}(\boldsymbol{\theta}_{\text{shared}}, \boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_K) = \sum_{k=1}^{K} \alpha_k \mathcal{L}_k \left(f_k(\mathbf{x}; \boldsymbol{\theta}_{\text{shared}}, \boldsymbol{\theta}_k), \mathbf{y}_k \right). \tag{7.24}$$

   Because the shared parameters $\boldsymbol{\theta}_{\text{shared}}$ must satisfy gradient constraints from multiple objectives concurrently, the tasks act as mutual regularizers. Caruana demonstrated that representations learned under multitask objectives generalize significantly better than single-task models because task-specific noise cancels out.

3. **Meta-Learning (Learning to Learn)**: While multitask learning solves a fixed set of known tasks, meta-learning (Finn et al., 2017) [25] trains across a distribution of tasks $p(\mathcal{T})$ to discover an optimal parameter initialization $\boldsymbol{\theta}^*$ that can adapt to entirely novel, unseen tasks within K gradient descent steps (few-shot learning):

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \sum_{\mathcal{T}_i \sim p(\mathcal{T})} \mathcal{L}_{\mathcal{T}_i} \left(\boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathcal{T}_i}(\boldsymbol{\theta}) \right). \tag{7.25}$$

Paradigm	Primary Objective	Temporal Pipeline	Parameter Sharing	Primary Failure Mode
Transfer Learning	Transfer source knowledge to boost scarce target task accuracy	Sequential ($\mathcal{T}_{\mathcal{S}} \to \mathcal{T}_{\mathcal{T}}$)	Backbone reused; head re-initialized	Negative transfer; feature distortion on OOD
Domain Adaptation	Minimize degradation under covariate shift	Simultaneous or 2-stage adaptation	Shared canonical latent representation	Target domain over-fitting; manifold rupture
Multitask Learning	Improve generalized representations across K concurrent tasks	Simultaneous joint optimization	Hard/soft layer sharing across heads	Gradient conflict; negative task transfer
Self-Supervised Learning	Pre-train representations without human manual annotations	Sequential (Pretext → Downstream)	Entire encoder reused	Pretext task misalignment (shortcut learning)
Meta-Learning	Learn optimal initialization for few-shot adaptation	Bi-level optimization across task distribution	Initialization $\boldsymbol{\theta}^*$ adapted per task	Meta-overfitting to task distribution

Table 7.2: The representation learning continuum: comparative taxonomy of transfer learning, domain adaptation, multitask learning, self-supervised pre-training, and meta-learning (Bishop, 2024; Goodfellow et al., 2016; Prince, 2023; Caruana, 1997; Finn et al., 2017).

Chapter 8

Landmark CNN architectures

The history of modern computer vision is fundamentally defined by the transition from hand-crafted geometric feature engineering to end-to-end representation learning optimized directly from data via backpropagation. For decades, visual recognition relied on multi-stage pipelines where human experts designed specialized mathematical operators—such as SIFT, HOG, and Fisher Vectors—to capture local scale, orientation, and gradient distributions. However, because these engineered descriptors were fundamentally decoupled from the downstream classification objective, their empirical accuracy stalled at an stubborn performance plateau. Unlocking the true expressive potential of visual models required an end-to-end paradigm where multi-layer hierarchical representations were learned jointly with decision boundaries.

This paradigm shift materialized through the ImageNet Large Scale Visual Recognition Challenge (ILSVRC). Over five transformative years (2012–2015), the annual competition served as the empirical crucible that validated the foundational principles of deep convolutional neural networks. From AlexNet's dual-GPU breakthrough in 2012, to ZFNet's diagnostic deconvolutional visualization in 2013, the modular homogeneous stacking of VGG and the multi-scale Inception topology in 2014, and Kaiming He's residual learning in 2015, researchers systematically explored the dimensions of depth, width, parameter efficiency, and optimization dynamics. Analyzing these landmark architectures illuminates not only the historical evolution of deep learning, but also the universal structural principles governing modern neural network design.

8.1 The ImageNet challenge

The theoretical principles governing convolutional networks found their decisive historical validation through the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) [19, 80]. Constructed by Fei-Fei Li and colleagues in 2009, ImageNet organized over 14 million natural images mapped systematically to the WordNet lexical hierarchy. The annual competition established a standardized benchmark comprising 1.28 million training images, 50,000 validation images, and 100,000 test images across $K = 1,000$ fine-grained object categories.

8.1.1 Dataset structure and evaluation

Evaluating classification performance on ImageNet introduces subtle conceptual challenges distinct from standard small-scale benchmarks. In natural visual scenes, images are inherently complex and often contain multiple co-occurring objects. Furthermore, the 1,000 WordNet synsets include over 120 fine-grained breeds of dogs (e.g., distinguishing an Alaskan Malamute from a Siberian Husky), alongside categories with subtle semantic overlap.

To account for inherent label ambiguities where multiple categories may be visually plausible, the benchmark evaluates models using two distinct criteria:

1. *Top-1 Error*: The fraction of test images for which the model's single most confident categorical prediction does not match the ground-truth target label:

$$\text{Top-1 Error} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}\left(y_i \neq \hat{y}_i^{(1)}\right), \tag{8.1}$$

where $\hat{y}_i^{(1)} = \arg\max_{k\in\{1,\ldots,K\}} \hat{P}(y = k \mid \mathbf{X}_i)$ denotes the class assigned the highest posterior probability by the Softmax output, and $\mathbb{I}(\cdot)$ is the indicator function.

2. *Top-5 Error*: The primary competitive metric of ILSVRC. A prediction is considered correct if the ground-truth category appears anywhere among the model's top 5 most confident predictions:

$$\text{Top-5 Error} = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}\left(y_i \notin \left\{\hat{y}_i^{(1)}, \hat{y}_i^{(2)}, \hat{y}_i^{(3)}, \hat{y}_i^{(4)}, \hat{y}_i^{(5)}\right\}\right). \tag{8.2}$$

Because an image of a dog sitting in a park might legitimately contain evidence for both "grass", "park bench", and "Siberian Husky", Top-5 Error ensures that a model is not unfairly penalized for recognizing a secondary, visually prominent entity.

8.1.2 The deep learning transition

Prior to 2012, state-of-the-art visual recognition was dominated by classical multi-stage pipelines: dense handcrafted feature extraction (SIFT, HOG), vector quantization and spatial aggregation (Bag-of-Visual-Words, Spatial Pyramid Matching, Fisher Vectors), and supervised kernel Support Vector Machines [16, 61]. Despite intense mathematical engineering, these handcrafted pipelines stalled at an empirical plateau: the winning top-5 error was 28.2% in 2010 and 25.8% in 2011.

While Yann LeCun's LeNet-5 [57] had demonstrated successful gradient-based convolutional feature learning on MNIST digits as early as 1998 (28×28 grayscale inputs, 60,000 parameters, sigmoid activations, average pooling), scaling convolutional networks to complex, high-resolution natural images remained computationally impossible for over a decade.

The arrival of deep convolutional neural networks in October 2012 shattered the handcrafted ceiling. As illustrated in Figure 8.1, the transition initiated an extraordinary acceleration in visual performance, cutting classification error by more than half within three years and ultimately surpassing estimated human visual performance ($\approx 5.1\%$ top-5 error).

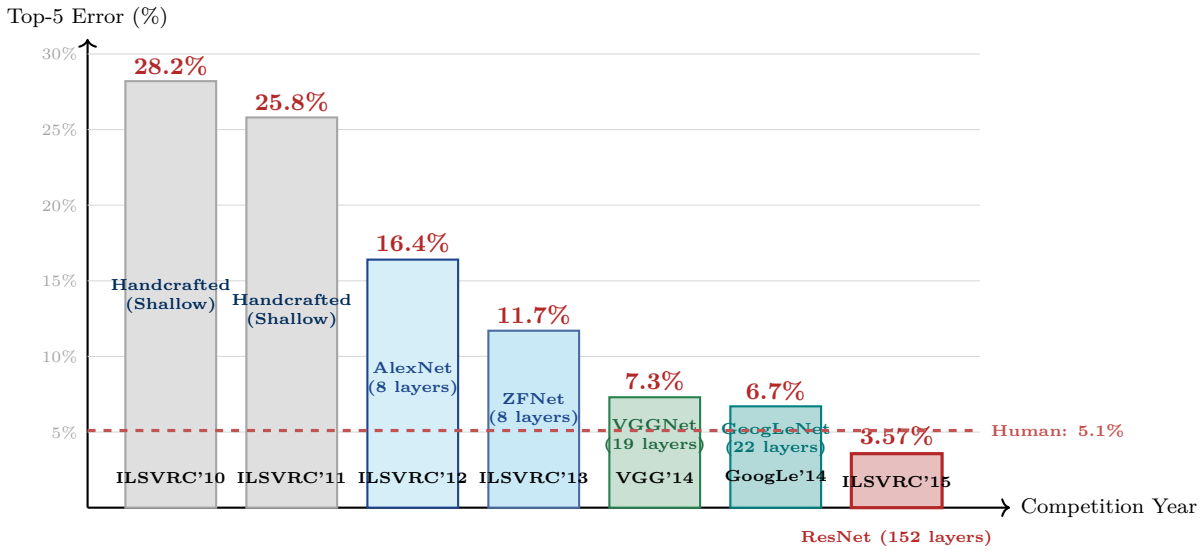


Figure 8.1: Historical evolution of winning Top-5 error rates on the ImageNet Large Scale Visual Recognition Challenge (ILSVRC 2010–2015). The arrival of AlexNet in 2012 sparked the deep learning revolution, culminating in ResNet (2015) surpassing estimated human visual classification accuracy.

8.2 AlexNet

In October 2012, Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton presented AlexNet [53], winning ILSVRC 2012 with a top-5 error of 16.4% and outpacing the second-place handcrafted entry (26.2%) by an unprecedented margin of 9.8 percentage points. AlexNet demonstrated that deep convolutional networks could be trained effectively on massive visual datasets through the convergence of high-capacity architectures, non-saturating activations, and commodity parallel graphics hardware.

8.2.1 Hardware constraints and dual-GPU parallelism

In 2012, state-of-the-art computational hardware was severely limited by on-board GPU memory. The primary accelerator utilized by Krizhevsky was the NVIDIA GeForce GTX 580, which provided only **3 GB of VRAM**.

To understand why 3 GB represented an insurmountable bottleneck for a single device, we must analyze the memory footprint of training an 8-layer network with 61 million parameters under stochastic gradient descent with momentum:

- *Parameter Storage*: 61 million 32-bit floating-point parameters require $61 \times 10^6 \times 4$ bytes $\approx$ 244 MB.
- *Gradient Storage*: Backpropagation requires storing an exact replica of the gradient tensor $\nabla_{\boldsymbol{\theta}} \mathcal{L}$, consuming an additional 244 MB.
- *Optimizer Buffers*: First-order momentum buffers duplicate parameter memory, consuming another 244 MB.
- *Forward Activation Maps*: Forward activations must be retained in memory throughout forward propagation to compute analytical Jacobians during the backward pass. For a mini-batch of 128 images, storing multi-channel intermediate feature tensors across high spatial resolutions requires several gigabytes of VRAM.

The cumulative memory requirement substantially exceeded 3 GB, making single-GPU training impossible.

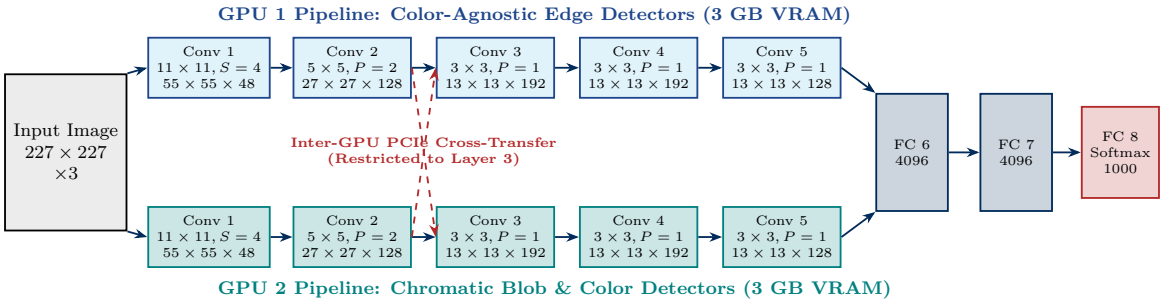


Figure 8.2: Dual-GPU model parallelism in AlexNet (2012). To circumvent the 3 GB physical VRAM ceiling of the NVIDIA GTX 580, convolutional filters were split equally across two GPUs (e.g., 48 filters each in Layer 1). Because inter-GPU PCIe bandwidth was slow, communication was restricted strictly to Layer 3 and the dense fully connected layers.

To resolve this limitation, Krizhevsky implemented custom CUDA kernels executing **model parallelism across two GPUs** (Figure 8.2):

1. *Channel Partitioning*: The feature channels of every convolutional layer were split symmetrically across both devices. For example, in Layer 1, GPU 1 hosted 48 filters while GPU 2 hosted 48 filters (totaling 96 output channels).

2. *Restricted PCIe Communication*: Inter-device communication across the host PCIe bus was a severe latency bottleneck. To minimize synchronization overhead, the GPUs operated independently without exchanging activations in Layers 1, 2, 4, and 5. Cross-GPU data communication was permitted **strictly at Layer 3** (where CONV3 took input from the concatenation of CONV2 outputs from both GPUs) and at the dense fully connected layers ($\text{FC}_6, \text{FC}_7, \text{FC}_8$).

3. *Spontaneous Specialization*: This restricted communication induced a remarkable emergent phenomenon: GPU 1 consistently specialized in high-frequency, orientation-selective, color-agnostic edge detectors, whereas GPU 2 specialized in low-frequency, chromatic blob and color-texture filters.

8.2.2 Layer anatomy and resource allocation

AlexNet comprises 8 learned layers: 5 convolutional layers followed by 3 dense fully connected layers. Operating on input images normalized to $227 \times 227 \times 3$, each layer transforms the activation volume according to rigorous spatial, memory, and computational rules:

- *Spatial Output Dimension*:

$$W' = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1, \qquad H' = \left\lfloor \frac{H - K + 2P}{S} \right\rfloor + 1. \tag{8.3}$$

- *Activation Memory (KB)*: For single-precision (32-bit float, 4 bytes per element):

$$\text{Memory (KB)} = \frac{C_{\text{out}} \cdot H' \cdot W' \cdot 4}{1024}. \tag{8.4}$$

- *Parameter Count*: For convolutional layers with kernel size $K \times K$ and additive bias:

$$N_{\text{params}} = C_{\text{out}} \cdot C_{\text{in}} \cdot K^2 + C_{\text{out}}. \tag{8.5}$$

- *FLOPs (Floating-Point Multiply-Adds)*:

$$\text{FLOPs} = (C_{\text{out}} \cdot H' \cdot W') \cdot (C_{\text{in}} \cdot K^2). \tag{8.6}$$

Table 8.1 details the exact layer-by-layer quantitative metrics across the entire AlexNet architecture.

Table 8.1: Detailed layer-by-layer architectural, memory, parameter, and FLOP analysis of AlexNet operating on $227 \times 227 \times 3$ images.

Layer	Input Shape	Filters	Kernel	Stride	Pad	Output Shape	Mem (KB)	Params	MFLOPs
Input	$3 \times 227 \times 227$	—	—	—	—	$3 \times 227 \times 227$	604	0	0
Conv 1	$3 \times 227 \times 227$	64*	11×11	4	2	$64 \times 56 \times 56$	784	23,296	72.9
Pool 1	$64 \times 56 \times 56$	—	3×3	2	0	$64 \times 27 \times 27$	182	0	0
Conv 2	$64 \times 27 \times 27$	192*	5×5	1	2	$192 \times 27 \times 27$	547	307,392	224.1
Pool 2	$192 \times 27 \times 27$	—	3×3	2	0	$192 \times 13 \times 13$	127	0	0
Conv 3	$192 \times 13 \times 13$	384	3×3	1	1	$384 \times 13 \times 13$	254	663,936	112.2
Conv 4	$384 \times 13 \times 13$	256*	3×3	1	1	$256 \times 13 \times 13$	169	885,024	144.9
Conv 5	$256 \times 13 \times 13$	256	3×3	1	1	$256 \times 13 \times 13$	169	590,080	99.8
Pool 5	$256 \times 13 \times 13$	—	3×3	2	0	$256 \times 6 \times 6$	36	0	0
Flatten	$256 \times 6 \times 6$	—	—	—	—	9216×1	36	0	0
FC 6	9216×1	4096	—	—	—	4096×1	16	37,752,832	37.7
FC 7	4096×1	4096	—	—	—	4096×1	16	16,781,312	16.8
FC 8	4096×1	1000	—	—	—	1000×1	4	4,097,000	4.1
Total							$\approx$ **2.3** MB	$\approx$ **60.9**M	$\approx$ **727**M

*Standard modern unified channel representation (equivalent to Krizhevsky's $48 \times 2, 128 \times 2, 192 \times 2$ split across two GPUs).

Analyzing Table 8.1 reveals three fundamental architectural realities that govern classical CNNs:

1. *Activation Memory Concentration*: Forward activation memory is heavily concentrated in the **early convolutional layers**. CONV1 and CONV2 alone consume 784 KB + 547 KB = 1.33 MB out of the total 2.3 MB ($> 58\%$). Because spatial dimensions are large (56×56 and 27×27), storing forward activations for large mini-batches represents the primary bottleneck for GPU training VRAM.

2. *Parameter Concentration*: Parameters are overwhelmingly concentrated in the **late fully connected layers**. Out of 60.9 million parameters, the dense layers ($\text{FC}_6, \text{FC}_7, \text{FC}_8$) account for over **58.6**M parameters ($\approx$ **96.1**%). The transition layer FC_6 alone absorbs 37.75 million weights (62% of the entire network!).

3. *FLOP Concentration*: Floating-point operations are concentrated almost entirely in the **convolutional backbone**. Sliding filters across large 2D grids requires over 654 MFLOPs ($> 90\%$ of total compute), while the massive dense layers contribute less than 10% of all multiplications.

8.2.3 Architectural innovations

Beyond model parallelism and layer scaling, AlexNet introduced three algorithmic mechanisms that established standard practice across modern deep learning:

Rectified Linear Units (ReLU). Prior to 2012, neural networks parameterized hidden activations using smooth, bounded saturating functions such as the logistic sigmoid $\sigma(z) = (1 + e^{-z})^{-1}$ or hyperbolic tangent $\tanh(z)$. As derived in Chapter 6, saturating functions exhibit

vanishing derivatives ($\sigma'(z) \to 0$ as $|z| \to \infty$), causing gradients to extinguish exponentially across multi-layer backward propagation.

AlexNet replaced saturating activations with the non-saturating **Rectified Linear Unit (ReLU)** [67]:

$$f(z) = \max(0, z), \qquad f'(z) = \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{if } z < 0. \end{cases} \tag{8.7}$$

Because the derivative is strictly 1.0 for all positive pre-activations, gradients flow through active units without attenuation. In their experiments on CIFAR-10, Krizhevsky demonstrated that a 4-layer convolutional network equipped with ReLUs reached a 25% training error rate **six times faster** than an identical network using tanh activations.

Overlapping max pooling. Standard pooling layers partition input grids into non-overlapping adjacent windows by setting stride equal to kernel size ($S_p = F_p$). AlexNet introduced **overlapping pooling**, setting filter size $F_p = 3$ with stride $S_p = 2$. By sliding a 3×3 window with stride 2, adjacent pooling windows overlap by one pixel. Empirically, Krizhevsky reported that overlapping pooling reduced top-1 error by 0.4% and top-5 error by 0.5% while making the network marginally less susceptible to overfitting.

Dropout and data augmentation. Because the 60.9 million parameters of AlexNet vastly exceeded the sample diversity of ImageNet, training unregularized models led to catastrophic memorization. To prevent overfitting, AlexNet introduced two complementary regularizers:

- *Dropout ($p = 0.5$)*: Applied to hidden activations in FC_6 and FC_7 [88]. During each training step, each neuron is independently zeroed out with probability 0.5. This prevents complex co-adaptations where neurons rely on the exact presence of specific peer units, forcing every unit to learn robust, self-contained visual representations. At test time, all neurons remain active, and their outgoing weights are scaled by 0.5 to match the expected activation magnitude.

- *Data Augmentation*: To expand the training distribution without acquiring new data, AlexNet generated augmented patches on the CPU in real-time during GPU processing:

    1. Extracting random 224×224 crops from rescaled 256×256 images alongside horizontal mirror reflections (2048× expansion).
    2. PCA-based color jittering: Performing Principal Component Analysis on RGB pixel covariance across ImageNet, adding multiples of the principal components with magnitudes proportional to the corresponding eigenvalues multiplied by random Gaussian variables drawn from $\mathcal{N}(0, 0.1)$. This altered illumination and color intensity while leaving object identities strictly invariant.

8.3 ZFNet

Despite AlexNet's empirical triumph in 2012, the machine learning community widely viewed deep convolutional networks as inscrutable "black boxes". Architectural hyperparameters—such as using 11×11 filters in Layer 1 with stride 4—had been selected via ad-hoc trial and error without qualitative interpretability.

In 2013, Matthew Zeiler and Rob Fergus introduced **ZFNet** [101], winning ILSVRC 2013 by reducing top-5 error from 16.4% to **11.7%**. Rather than designing an entirely new architecture from scratch, Zeiler and Fergus developed a novel visualization technique that diagnosed why AlexNet succeeded and revealed specific structural flaws in its hyperparameter choices.

8.3.1 Feature visualization with deconvolution

To peer inside the hidden layers of AlexNet, Zeiler and Fergus designed the **Deconvolutional Network (Deconvnet)**. Operating as an inverse feature projector, a Deconvnet maps intermediate activation maps back to the input pixel space, exposing exactly which visual patterns trigger strong activations in specific neurons.

A Deconvnet inverts standard convolutional operations through three sequential steps:

1. *Unpooling*: Standard max pooling is non-invertible because non-maximal values are permanently discarded. To approximate an inverse, the forward max pooling layer records the argmax spatial coordinate within each pooling window as a set of **switch variables**. The unpooling operator uses these switch coordinates to place the reconstructed activations back into their original locations, padding all remaining positions with zeros.

2. *Rectification*: Reconstructed signals are passed through a ReLU activation to enforce non-negative feature activations.

3. *Transposed Filtering*: The rectified maps are convolved with the transposed (horizontally and vertically flipped) kernel weights $\mathbf{W}^\top$ of the corresponding convolutional layer, projecting activations back down to the preceding layer's resolution.

8.3.2 Diagnostic refinement of hyperparameters

Applying Deconvnet visualization to AlexNet revealed two critical architectural pathologies:

- *First Layer Over-Aggression*: Visualizing Layer 1 of AlexNet demonstrated that the large 11×11 filters operating with a coarse stride of $S = 4$ were excessively aggressive. The filters captured only extreme high-frequency and low-frequency patterns, showing dead filters with zero activation and missing intermediate spatial frequencies.

- *Second Layer Aliasing Artifacts*: The large stride of 4 in Layer 1 induced severe high-frequency aliasing artifacts in Layer 2, manifesting as prominent checkerboard and grid distortion patterns in reconstructed visualizations.

Guided by these visual diagnostics, Zeiler and Fergus introduced two targeted modifications to AlexNet's topology:

1. *CONV1 Kernel and Stride Reduction*: Filter size was reduced from 11×11 to 7×7, and stride was reduced from 4 to 2. This captured a much smoother, richer spectrum of edge and orientation frequencies while halving the spatial downsampling rate, eliminating aliasing artifacts.

2. *Deeper Feature Capacity in Middle Layers*: To exploit the richer low-level spatial features preserved by the refined stem, the channel depths of layers 3, 4, and 5 were widened from $(384, 384, 256)$ to $(512, 1024, 512)$.

Without altering the underlying 8-layer macro-structure, these diagnostic adjustments caused ILSVRC top-5 error to plummet from 16.4% to 11.7%. ZFNet proved that deep learning was not an uninterpretable black box, and demonstrated that principled hyperparameter design grounded in feature visualization yielded substantial accuracy gains.

8.4 VGGNet

At the ILSVRC 2014 competition, Karen Simonyan and Andrew Zisserman from Oxford's Visual Geometry Group introduced **VGGNet** [87], securing second place with a top-5 error of **7.3%**.

While preceding models used heterogeneous filter sizes ($11 \times 11, 7 \times 7, 5 \times 5$), VGG demonstrated that state-of-the-art representation learning could be achieved by enforcing extreme structural simplicity and modular depth.

8.4.1 The small-kernel stacking principle

VGG discarded large convolutional kernels entirely, establishing a universal design law: **every convolutional kernel is fixed to 3×3 with stride 1 and padding 1**.

As derived in Chapter 5, the effective receptive field r_L of a sequence of L convolutional layers with kernel size K and unit stride is governed by:

$$r_L = 1 + L(K - 1). \tag{8.8}$$

Applying Equation (8.8) reveals that stacking two 3×3 convolutions ($L = 2, K = 3$) yields an effective receptive field of $1 + 2(2) = 5 \times 5$. Stacking three 3×3 convolutions ($L = 3, K = 3$) covers an effective receptive field of $1 + 3(2) = 7 \times 7$.

Replacing a single 7×7 convolution with a stack of three 3×3 convolutions yields two profound mathematical advantages:

1. *Parameter Efficiency*: Consider an input volume with C channels producing C output channels. A single 7×7 convolutional layer requires:

$$N_{\text{single}} = 7 \times 7 \times C^2 = 49C^2 \text{ weights.} \tag{8.9}$$

   In contrast, a stack of three 3×3 convolutional layers requires:

$$N_{\text{stacked}} = 3 \times (3 \times 3 \times C^2) = 27C^2 \text{ weights.} \tag{8.10}$$

   Stacking small kernels achieves the identical 7×7 receptive field while securing a **45% parameter reduction**!

2. *Non-Linear Expressiveness*: A single 7×7 layer applies only one non-linear activation. In contrast, three stacked 3×3 layers incorporate three successive ReLU activations. This allows the subnetwork to partition the latent space using three non-linear decision boundaries rather than one, creating substantially richer discriminative representations.

8.4.2 Modular design and computational invariance

VGG structured its architecture around three elegant, recursive design rules:

1. All convolutions are 3×3 with stride $S = 1$ and padding $P = 1$ ("same" padding).
2. All pooling layers are 2×2 max pooling with stride $S = 2$.
3. After every pooling layer, the number of feature channels is doubled ($64 \to 128 \to 256 \to 512$).

Mathematical proof of constant compute across stages. A remarkable theoretical property emerges from the coupling of spatial halving with channel doubling. Consider a convolutional stage operating at resolution $2H \times 2W$ with C channels. Applying a 3×3 convolution producing C channels incurs a computational cost of:

$$\text{FLOPs}_{\text{stage 1}} = (2H \cdot 2W) \cdot (3 \cdot 3 \cdot C) \cdot C = 36HWC^2. \tag{8.11}$$

At the next stage, spatial dimensions are halved to $H \times W$, while channels double to $2C$. Applying a 3×3 convolution from $2C$ to $2C$ channels incurs:

$$\text{FLOPs}_{\text{stage 2}} = (H \cdot W) \cdot (3 \cdot 3 \cdot 2C) \cdot (2C) = (HW) \cdot 9 \cdot 4C^2 = 36HWC^2. \tag{8.12}$$

Because spatial area decreases by a factor of 4 while channel complexity increases by a factor of 4 ($2C \times 2C = 4C^2$), the total FLOPs per convolutional layer remain **strictly invariant** across all spatial resolutions! This guarantees that computational effort is distributed evenly throughout the entire depth of the network rather than collapsing onto a single stage.

8.4.3 Architecture profiles and scaling limits

Simonyan and Zisserman evaluated multiple configurations, standardizing on **VGG-16** (13 conv + 3 dense) and **VGG-19** (16 conv + 3 dense).

While VGG demonstrated the power of depth, comparing VGG-16 with AlexNet highlights severe computational bottlenecks:

- *Memory Footprint*: Forward activation maps in VGG-16 consume **48.6 MB per image**—a **25-fold expansion** over AlexNet's 1.9 MB. The high channel count in early high-resolution layers ($224 \times 224 \times 64$) imposes massive VRAM memory bandwidth demands.
- *Parameter Overhead*: VGG-16 contains **138 million parameters** (2.3× AlexNet). Over 123M parameters reside in the dense fully connected layers ($\text{FC}_6, \text{FC}_7, \text{FC}_8$), making the network memory-intensive and prone to overfitting.
- *Computational Burden*: Processing a single image through VGG-16 requires **13.6 GFLOPs**—a **19.4-fold increase** over AlexNet's 0.7 GFLOPs.

VGG proved that extreme depth was essential for visual accuracy, but established an unsustainable trajectory for hardware resource consumption.

8.5 GoogLeNet

Concurrently at ILSVRC 2014, Christian Szegedy and colleagues at Google introduced **GoogLeNet (Inception-v1)** [90], winning the competition with a top-5 error of **6.7%**. While VGG pursued accuracy through brute-force depth (scaling parameters to 138M), GoogLeNet was engineered around a radically different philosophy: **scaling depth while maintaining strict computational efficiency**. GoogLeNet attained 22 learned layers while containing only **5 million parameters**—over 12× fewer than AlexNet and 27× fewer than VGG-16!

8.5.1 Computational efficiency and aggressive stems

In VGG-16, the earliest convolutional stages operating at resolutions 224×224 and 112×112 absorbed 42.9 MB of activation memory and $7,485$ MFLOPs of computation.

To eliminate this early bottleneck, GoogLeNet introduced an **aggressive stem network** that downsamples input imagery by a factor of 8 ($224 \times 224 \to 28 \times 28$) within the first few operations:

1. Conv $7 \times 7, S = 2, P = 3 \implies 112 \times 112 \times 64$.
2. MaxPool $3 \times 3, S = 2, P = 1 \implies 56 \times 56 \times 64$.
3. Conv $1 \times 1, S = 1, P = 0 \implies 56 \times 56 \times 64$.
4. Conv $3 \times 3, S = 1, P = 1 \implies 56 \times 56 \times 192$.
5. MaxPool $3 \times 3, S = 2, P = 1 \implies 28 \times 28 \times 192$.

Table 8.2 quantifies the efficiency of GoogLeNet's stem compared to VGG-16 across the transition from 224×224 to 28×28.

Table 8.2: Quantitative comparison of early-stage computational resource consumption from input 224×224 down to spatial resolution 28×28: GoogLeNet stem vs. VGG-16.

Architecture Stem	Activation Memory	Parameters	Compute (FLOPs)
VGG-16 (Layers 1–10)	42.9 MB	1,100K	7,485 MFLOPs
GoogLeNet (Aggressive Stem)	**7.5** MB	**124**K	**418** MFLOPs
Efficiency Gain	**5.7** × reduction	**8.9** × reduction	**17.8** × reduction

By slashing early compute by 17.8×, GoogLeNet conserved virtually its entire parameter and computational budget for deep, semantic feature extraction.

8.5.2 The Inception module and bottleneck projections

Visual features in natural imagery occur across wildly varying spatial scales: a prominent foreground object may occupy the entire canvas, while a critical diagnostic cue occupies only a few pixels. Standard CNNs force practitioners to choose a single fixed kernel size (3×3 vs. 5×5), compromising either local detail or spatial context.

GoogLeNet resolved this by introducing the **Inception Module** (Figure 8.3): a multi-branch parallel block that extracts features simultaneously at multiple spatial scales and concatenates the resulting activation tensors along the channel dimension.

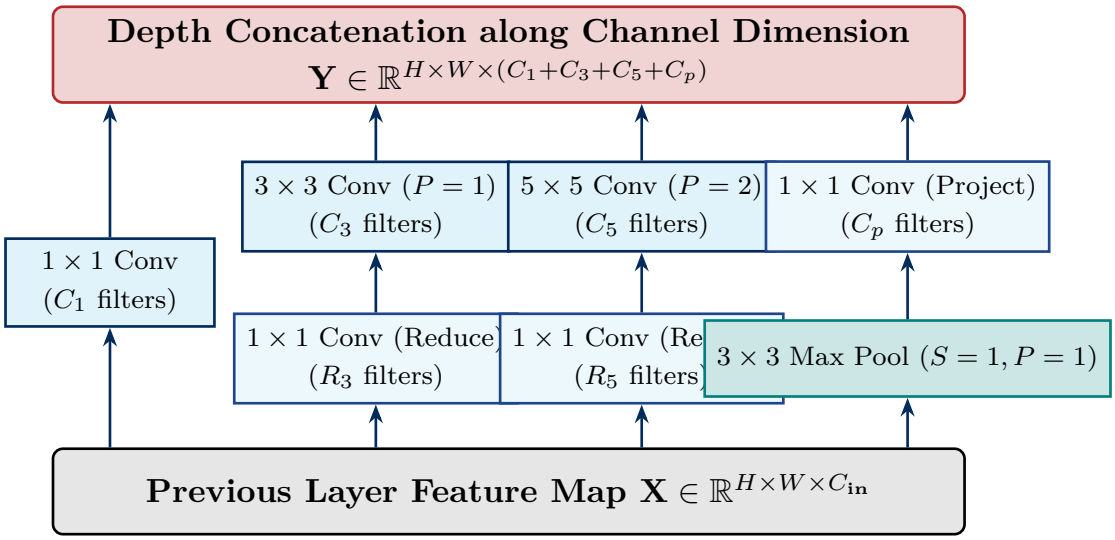


Figure 8.3: The Inception Module with 1×1 bottleneck projections (GoogLeNet / Inception-v1). Parallel multi-scale paths (1×1, 3×3, 5×5, and max pooling) process features concurrently. Learnable 1×1 convolutions compress channel dimensions prior to spatial filtering, preventing computational explosion while enabling 22-layer depth.

Dimensionality reduction via 1×1 bottleneck convolutions. Naively concatenating 3×3 and 5×5 convolutions across deep feature maps triggers an unsustainable computational explosion. If an input tensor with $C_{\text{in}} = 256$ channels is convolved directly with 128 filters of size 5×5, the layer requires:

$$N_{\text{naive}} = 128 \times 256 \times 5 \times 5 = 819,200 \text{ parameters.} \tag{8.13}$$

Operating on a 28×28 feature map, this single operation consumes over 642 MFLOPs.

To resolve this, GoogLeNet introduced 1×1 **bottleneck convolutions** immediately preceding the 3×3 and 5×5 filters (Figure 8.3). Operating as a point-wise linear combination across

channels, a 1×1 convolution reduces channel depth ($256 \to 32$) before applying spatial filtering ($32 \to 128$ with 5×5):

- 1×1 Bottleneck: $32 \times 256 \times 1 \times 1 = 8,192$ parameters (6.4 MFLOPs).
- 5×5 Spatial Filtering: $128 \times 32 \times 5 \times 5 = 102,400$ parameters (80.3 MFLOPs).
- Total Parameters: $8,192 + 102,400 = 110,592$ parameters (**86.5% parameter reduction**).
- Total Compute: $6.4 + 80.3 = 86.7$ MFLOPs (**86.5% FLOP reduction**).

This bottleneck projection enables the network to maintain rich multi-scale representational capacity while keeping computational complexity strictly manageable.

8.5.3 Global average pooling and auxiliary heads

GoogLeNet introduced two structural innovations that reshaped network heads and optimization dynamics:

Global Average Pooling (GAP). In AlexNet and VGG-16, flattening the final convolutional volume into dense layers accounted for over 89% of all network parameters (123.6M in VGG), creating severe memory bottlenecks and demanding aggressive Dropout regularization.

GoogLeNet eliminated massive fully connected layers entirely, replacing them with **Global Average Pooling (GAP)** [59]. Given the terminal convolutional volume $\mathbf{F} \in \mathbb{R}^{7\times7\times1024}$, GAP computes the spatial mean across each individual feature map:

$$h_c = \frac{1}{7 \times 7} \sum_{i=1}^{7} \sum_{j=1}^{7} F_{i,j,c}, \qquad \forall c \in \{1, \ldots, 1024\}. \tag{8.14}$$

This collapses the $7 \times 7 \times 1024$ tensor into a compact 1D vector $\mathbf{h} \in \mathbb{R}^{1024}$, which is connected directly to a single linear layer producing the 1,000 class logits:

$$N_{\text{head}} = 1024 \times 1000 + 1000 = 1,025,000 \text{ parameters.} \tag{8.15}$$

Comparing the head parameter counts:

- VGG-16 Dense Head: $123,640,000$ parameters.
- GoogLeNet GAP Head: $\mathbf{1,025,000}$ parameters (**120-fold reduction!**).

Because GAP contains zero learnable parameters, it enforces structural spatial invariance and dramatically suppresses overfitting.

Auxiliary classification heads. In 2014, deep learning lacked robust normalization techniques: **Batch Normalization had not yet been invented**. In a network with 22 non-linear layers, gradients backpropagating from the final loss suffered from severe attenuation, causing early layers to train unacceptably slowly.

To combat vanishing gradients during training, Szegedy attached two **auxiliary classification heads** at intermediate stages of the network (after Inception 4a and Inception 4d). Each auxiliary head consisted of a small 5×5 average pool, a 1×1 conv with 128 channels, a dense layer of 1024 units with ReLU, dropout (70%), and a 1,000-way Softmax.

During training, the total loss was computed as a weighted sum:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{main}} + 0.3 \cdot \mathcal{L}_{\text{aux1}} + 0.3 \cdot \mathcal{L}_{\text{aux2}}. \tag{8.16}$$

The auxiliary losses injected fresh, unattenuated supervisory gradient signals directly into the middle layers, accelerating convergence. At test time, both auxiliary branches were completely discarded, incurring zero inference cost. With the subsequent invention of Batch Normalization in 2015, auxiliary classifiers became obsolete and were removed in later Inception variants.

8.6 ResNet

By 2015, the empirical trajectory of the ImageNet challenge pointed toward an unequivocal conclusion: increasing network depth yielded richer semantic hierarchies. However, attempting to scale standard "plain" feedforward networks beyond 20 layers collided with an unexpected and catastrophic optimization pathology.

8.6.1 The degradation problem

When Kaiming He and colleagues at Microsoft Research scaled plain convolutional networks from 20 to 56 layers, test error increased substantially. Crucially, this failure was **not caused by overfitting**: training error on the training dataset was also dramatically higher for the 56-layer network than for the 20-layer network (Figure 8.4).

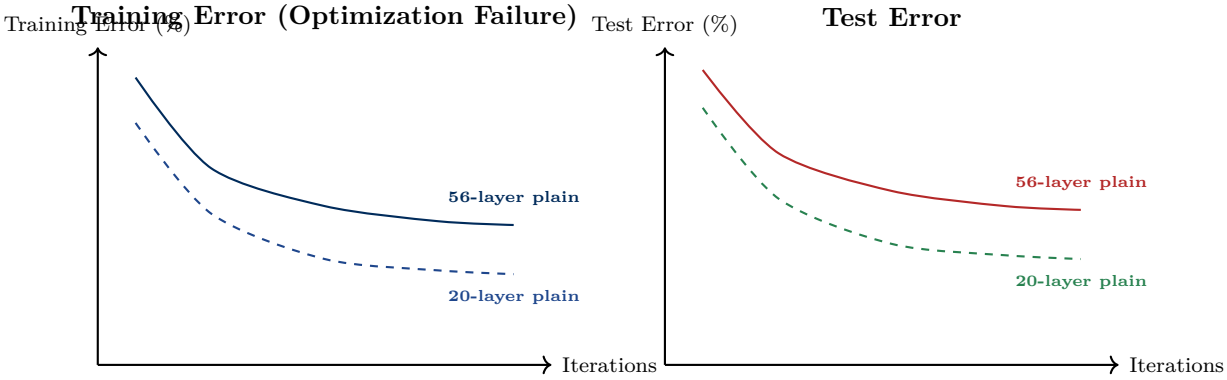


Figure 8.4: The Degradation Problem in deep plain networks (He et al., 2016). When scaling a plain convolutional architecture from 20 to 56 layers, both training error and test error degrade. Because the deeper model possesses strictly higher representational capacity, this degradation reflects an optimization pathology (underfitting) rather than overfitting.

The degradation problem exposed a profound conceptual paradox:

1. *The Overfitting Misconception*: Standard overfitting is characterized by very low training error coupled with high test error. Here, the deeper model performed worse on the training set itself, proving that the model was **underfitting**.

2. *The Identity Emulation Paradox*: A 56-layer network has strictly greater representational capacity than a 20-layer network. In principle, the 56-layer network could achieve training error at least as low as the 20-layer network by simply copying the learned weights of the 20-layer model into its first 20 layers and setting the remaining 36 layers to compute the **identity mapping**:
$$\mathcal{H}(\mathbf{x}) = \mathbf{x}. \tag{8.17}$$

The fact that stochastic gradient descent converged to a significantly worse training optimum revealed that **plain multi-layer networks cannot optimize identity mappings**. Fitting an identity function through stacked parameterized transformations $\sigma(\mathbf{W}_2\sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2)$ is mathematically difficult in non-convex optimization terrains.

8.6.2 Residual formulation and the gradient highway

In December 2015, Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun published their seminal work on **Deep Residual Learning** [36], winning ILSVRC 2015 with a top-5 error of **3.57%** and scaling networks to an astonishing **152 layers**.

The residual objective. He et al. hypothesized that if an underlying mapping $\mathcal{H}(\mathbf{x})$ can be approximated by stacked non-linear layers, it is asymptotically equivalent to hypothesize that the **residual mapping** $\mathcal{F}(\mathbf{x}) \triangleq \mathcal{H}(\mathbf{x}) - \mathbf{x}$ can be approximated. Rather than forcing stacked layers to approximate $\mathcal{H}(\mathbf{x})$ directly, ResNet reformulates the block to fit $\mathcal{F}(\mathbf{x})$, casting the overall operation as:

$$\mathcal{H}(\mathbf{x}) = \mathcal{F}(\mathbf{x}, \{\mathbf{W}_i\}) + \mathbf{x}, \tag{8.18}$$

where $\mathbf{x}$ and $\mathcal{H}(\mathbf{x})$ are input and output vectors, and $\mathcal{F}(\mathbf{x})$ is the residual function learned by convolutional layers (Figure 8.5).

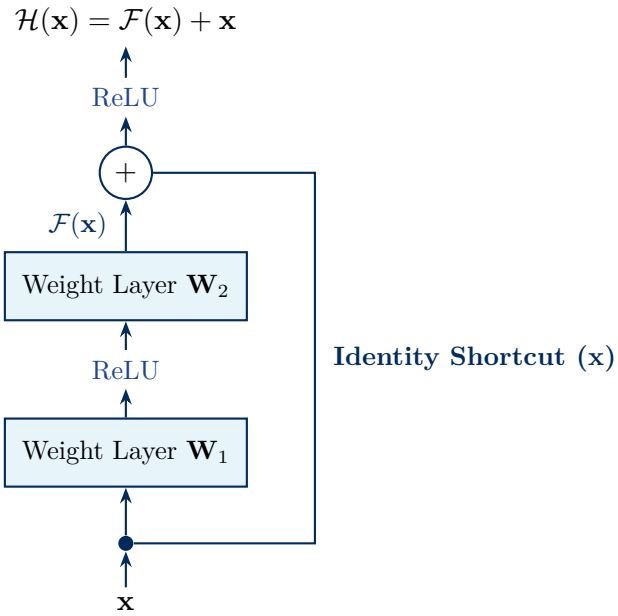


Figure 8.5: The Residual Building Block with identity shortcut connection (He et al., 2016). The additive shortcut creates an uninterrupted gradient highway, allowing optimization to learn perturbations around identity.

The mathematical brilliance of Equation (8.18) lies in its optimization dynamics:

- If identity mapping is optimal, standard gradient descent with weight decay simply drives the convolutional weights toward zero ($\mathbf{W}_i \to \mathbf{0}$), causing $\mathcal{F}(\mathbf{x}) \to \mathbf{0}$ and collapsing the block trivially to identity $\mathcal{H}(\mathbf{x}) = \mathbf{x}$. Driving weights to zero is natural and effortlessly discovered by SGD with L_2 regularization.
- In practice, residual functions $\mathcal{F}(\mathbf{x})$ exhibit small response variances, confirming that identity mappings provide an exceptionally strong inductive bias.

Mathematical proof of the unattenuated gradient highway. The structural reason ResNet scales to hundreds of layers without vanishing gradients is revealed by analyzing back-propagation through stacked residual units.

Theorem 8.1 (ResNet Gradient Flow)**.** *For any shallow residual unit l and deep unit L ($L > l$) connected by identity shortcuts $\mathbf{x}_{i+1} = \mathbf{x}_i + \mathcal{F}(\mathbf{x}_i, \mathcal{W}_i)$, the forward activation satisfies:*

$$\mathbf{x}_L = \mathbf{x}_l + \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i, \mathcal{W}_i). \tag{8.19}$$

Consequently, for any scalar loss objective $\mathcal{E}$, the backpropagated error gradient satisfies:

$$\frac{\partial \mathcal{E}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{E}}{\partial \mathbf{x}_L} \left(\mathbf{I} + \frac{\partial}{\partial \mathbf{x}_l} \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i, \mathcal{W}_i) \right). \tag{8.20}$$

Proof. Applying the forward recursion $\mathbf{x}_{i+1} = \mathbf{x}_i + \mathcal{F}(\mathbf{x}_i)$ recursively from unit l to L produces Equation (8.19). By the multivariate chain rule:

$$\frac{\partial \mathcal{E}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{E}}{\partial \mathbf{x}_L} \frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_l}. \tag{8.21}$$

Differentiating $\mathbf{x}_L$ with respect to $\mathbf{x}_l$ directly:

$$\frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_l} = \frac{\partial}{\partial \mathbf{x}_l} \left(\mathbf{x}_l + \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i, \mathcal{W}_i) \right) = \mathbf{I} + \frac{\partial}{\partial \mathbf{x}_l} \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i, \mathcal{W}_i). \tag{8.22}$$

Substituting this Jacobian directly yields Equation (8.20). □

Equation (8.20) reveals the gradient highway:

1. In plain feedforward networks, the gradient is governed by a multiplicative product of Jacobians:

$$\frac{\partial \mathcal{E}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{E}}{\partial \mathbf{x}_L} \prod_{i=l}^{L-1} \frac{\partial \mathbf{x}_{i+1}}{\partial \mathbf{x}_i} = \frac{\partial \mathcal{E}}{\partial \mathbf{x}_L} \prod_{i=l}^{L-1} \mathbf{W}_i. \tag{8.23}$$

   If the spectral norms of weight matrices satisfy $\|\mathbf{W}_i\| < 1$, the gradient decays exponentially to zero ($\prod \to \mathbf{0}$), causing catastrophic vanishing gradients.

2. In ResNet, the identity matrix $\mathbf{I}$ guarantees that the loss gradient $\frac{\partial \mathcal{E}}{\partial \mathbf{x}_L}$ flows **directly, additively, and unattenuated** to any shallow layer l, completely independent of how many layers separate them!

8.6.3 Basic and bottleneck block topologies

ResNet deploys two micro-architectural block designs depending on network depth:

Basic Residual Block (ResNet-18, ResNet-34). Used in shallower variants, the basic block consists of two stacked 3×3 convolutions with stride 1 and padding 1, each followed by Batch Normalization and ReLU (Figure 8.6, left). Operating on an activation map of spatial size $H \times W$ with C channels:

- Conv 1 $(3 \times 3, C \to C)$: $(H \cdot W) \cdot (3 \cdot 3 \cdot C) \cdot C = 9HWC^2$ FLOPs.
- Conv 2 $(3 \times 3, C \to C)$: $(H \cdot W) \cdot (3 \cdot 3 \cdot C) \cdot C = 9HWC^2$ FLOPs.
- Total Basic Block FLOPs: $\mathbf{18HWC^2}$.

Bottleneck Residual Block (ResNet-50, ResNet-101, ResNet-152). When scaling beyond 50 layers, basic blocks become computationally prohibitive. He et al. introduced the **Bottleneck Block** (Figure 8.6, right), adopting a three-layer sandwich of 1×1, 3×3, and 1×1 convolutions:

1. 1×1 Conv (Channel Reduction $4C \to C$): $(H \cdot W) \cdot (1 \cdot 1 \cdot 4C) \cdot C = 4HWC^2$ FLOPs.
2. 3×3 Conv (Spatial Filtering $C \to C$): $(H \cdot W) \cdot (3 \cdot 3 \cdot C) \cdot C = 9HWC^2$ FLOPs.
3. 1×1 Conv (Channel Expansion $C \to 4C$): $(H \cdot W) \cdot (1 \cdot 1 \cdot C) \cdot 4C = 4HWC^2$ FLOPs.

Total Bottleneck Block FLOPs:

$$\text{FLOPs}_{\text{bottleneck}} = 4HWC^2 + 9HWC^2 + 4HWC^2 = \mathbf{17HWC^2}. \tag{8.24}$$

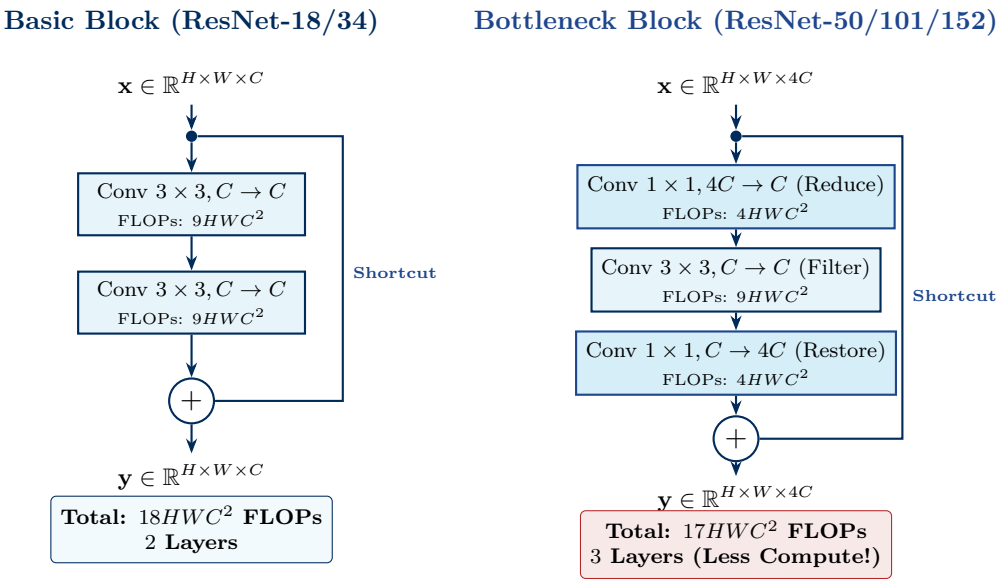


Figure 8.6: Comparison of Basic Residual Block vs. Bottleneck Residual Block in ResNet. The bottleneck design incorporates 1×1 channel reduction and restoration around the 3×3 convolution, allowing the network to insert three non-linear layers while consuming strictly less computation ($17HWC^2$ vs. $18HWC^2$).

This algebraic result is profound: **the bottleneck block achieves three layers of depth and non-linearity with strictly less computation than the two-layer basic block** ($17HWC^2 < 18HWC^2$)!

The ResNet family. Table 8.3 summarizes the structural configurations across the canonical ResNet family.

Table 8.3: Architectural specifications, layer counts across four computational stages, parameter counts, GFLOPs, and Top-5 error on ImageNet across the ResNet family.

Model	Block Type	Stage 1	Stage 2	Stage 3	Stage 4	FC	GFLOPs	Top-5 Error
ResNet-18	Basic	$2\times[2]$	$2\times[2]$	$2\times[2]$	$2\times[2]$	1	1.8	10.92%
ResNet-34	Basic	$3\times[2]$	$4\times[2]$	$6\times[2]$	$3\times[2]$	1	3.6	8.58%
ResNet-50	Bottleneck	$3\times[3]$	$4\times[3]$	$6\times[3]$	$3\times[3]$	1	3.8	7.13%
ResNet-101	Bottleneck	$3\times[3]$	$4\times[3]$	$23\times[3]$	$3\times[3]$	1	7.6	6.44%
ResNet-152	Bottleneck	$3\times[3]$	$8\times[3]$	$36\times[3]$	$3\times[3]$	1	11.3	**5.94**%

Single-crop 224×224 test error reported by TorchVision. In 10-crop testing, ResNet-152 achieved 4.49%, and an ensemble achieved 3.57%.

ResNet-50 is the identical macro-structure as ResNet-34, but replaces each 2-layer Basic block with a 3-layer Bottleneck block. This expands depth from 34 to 50 layers with virtually negligible compute increase (3.6 $\to$ 3.8 GFLOPs) while driving top-5 error down from 8.58% to 7.13%. At ILSVRC and COCO 2015, ResNet swept 1st place in all five competitive tracks: ImageNet Classification, ImageNet Detection, ImageNet Localization, COCO Detection, and COCO Segmentation.

8.7 Complexity and design guidelines

The rapid succession of architectures on ImageNet established an extensive dictionary of topological models. To select, deploy, and adapt these architectures for real-world downstream applications, machine learning engineers must evaluate them through multi-dimensional efficiency trade-offs.

8.7.1 Empirical complexity benchmarking

In their landmark benchmark study, Alfredo Canziani, Adam Paszke, and Eugenio Culurciello (2017) conducted a comprehensive empirical comparison of deep architectures on ImageNet [10]. Rather than evaluating models strictly on classification accuracy, they mapped the Pareto frontiers balancing:

- Top-1 Accuracy (%) vs. Computational Operations (GFLOPs).
- Top-1 Accuracy (%) vs. Parameter Size (M).
- Maximum Memory Footprint during inference (MB).

Figure 8.7 illustrates the empirical landscape discovered by Canziani et al.

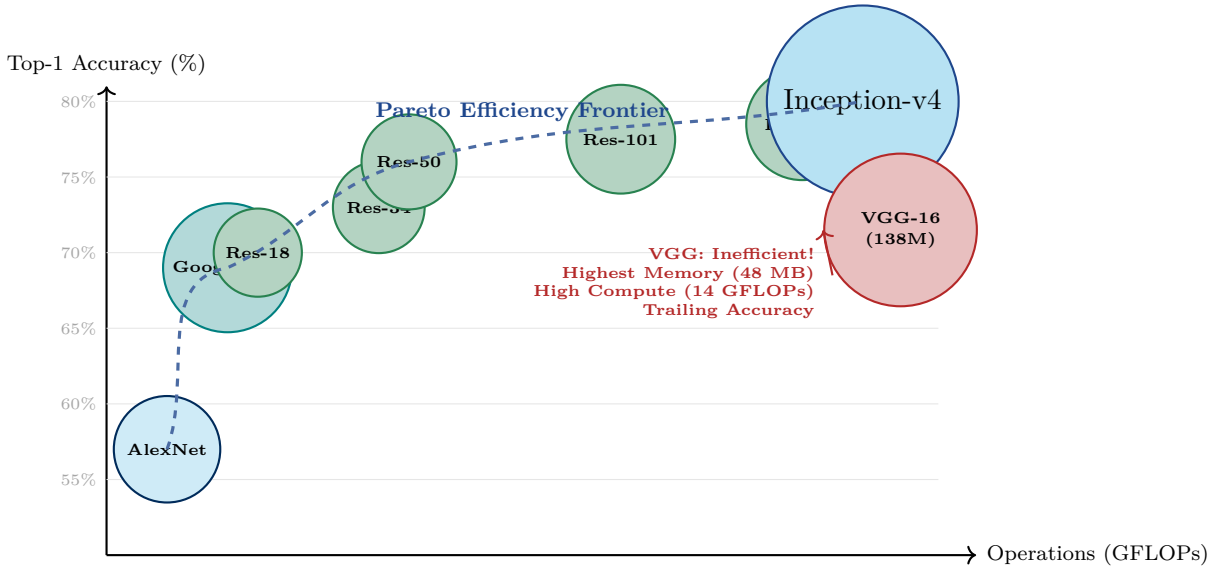


Figure 8.7: Empirical Pareto efficiency frontier comparing Top-1 Accuracy against Computational Cost (GFLOPs), adapted from Canziani et al. (2017). Node area represents relative parameter count. While VGG incurs massive parameter and computational costs, GoogLeNet and the ResNet family define the optimal Pareto frontier.

The Canziani benchmark revealed critical architectural trade-offs:

1. *The Inefficiency of VGG*: VGG occupies a highly suboptimal position in the trade-off space. Despite consuming 138 million parameters, 48.6 MB of activation memory, and nearly 14 GFLOPs, its accuracy (71.5% top-1) is decisively surpassed by ResNet-18 (1.8 GFLOPs, 11M params) and ResNet-50 (3.8 GFLOPs, 25M params).

2. *The Efficiency of GoogLeNet*: GoogLeNet represents an exceptional compute-to-accuracy ratio, achieving nearly 70% top-1 accuracy with only 5M parameters and 1.5 GFLOPs.

3. *ResNet as the Modern Industry Standard*: The ResNet family defines the optimal Pareto frontier across diverse operational regimes. In particular, **ResNet-50** represents the universal

industry sweet spot: delivering state-of-the-art accuracy with moderate parameter size (25.6M) and low compute (3.8 GFLOPs).

4. *Inception-v4 and Inception-ResNet*: Szegedy et al. (2017) demonstrated that the Inception and ResNet paradigms are fully complementary [89]. In **Inception-ResNet**, residual shortcut connections are introduced into multi-scale Inception blocks, accelerating training convergence while setting new benchmarks on ImageNet.

8.7.2 Model ensembles in competitive vision

While individual architectures advanced representational depth, winning competitive benchmarks routinely demanded **model ensembling**.

At ILSVRC 2016, the winning entry by Shao et al. [85] deployed a multi-scale, multi-architecture ensemble combining:

- Inception-v3 (4.20% error).
- Inception-v4 (4.01% error).
- Inception-ResNet-v2 (3.52% error).
- ResNet-200 (4.26% error).
- Wide ResNet (WRN-68-3, 4.65% error).

By averaging softmax posterior distributions across diverse architectural topologies and multiple image crops ($224 \times 224, 299 \times 299, 331 \times 331$), the fused ensemble reduced validation top-5 error to 2.92% and achieved a winning test error of **2.99%**—the first system to break the 3% error threshold on ImageNet.

8.7.3 Practical architectural selection

For researchers and practitioners applying deep learning to novel downstream problems, the ImageNet challenge yields vital engineering principles:

"Don't be a hero." In applied machine learning, a common temptation is to design a custom convolutional neural network from scratch. Decades of empirical architecture search demonstrate that designing high-capacity architectures that optimize smoothly without vanishing gradients or rank collapse requires enormous validation compute. For the overwhelming majority of visual tasks, practitioners should **never design an architecture from scratch**. Instead, adopt an off-the-shelf, well-validated architecture pre-trained on ImageNet.

Regime-dependent selection rules. The choice of backbone architecture is governed by deployment constraints:

1. *Maximum Accuracy Regime (Cloud / Server / Offline)*:
   - **ResNet-50** or **ResNet-101**: The undisputed workhorses of modern computer vision. Stable optimization, predictable gradient dynamics, and universally supported across deep learning frameworks.
   - For extreme performance where compute is unconstrained: Inception-ResNet-v2, ResNeXt, or modern Vision Transformers (ViT) [23].
2. *Resource-Constrained / Edge / Mobile Deployment*:
   - Standard ResNets ($>$ 25M params, 3.8 GFLOPs) exceed the thermal and memory constraints of embedded IoT chips, smartphones, and robotics platforms.

- **MobileNets** [42]: Replaces standard 3×3 convolutions with **depthwise separable convolutions** (a depthwise spatial convolution applied independently per channel, followed by a 1×1 pointwise convolution across channels), slashing compute and parameters by $8\times$ to $9\times$.
- **ShuffleNets** [105]: Deploys pointwise group convolutions combined with a channel shuffle operator, enabling fast feature exchange across channel groups with minimal memory latency.

3. *Automated Architectural Discovery*:

- **Neural Architecture Search (NAS)** [107]: Uses reinforcement learning or differentiable gradient search (DARTS) to automate the design of micro-blocks, discovering non-intuitive topologies optimized directly for specific hardware latency and accuracy targets.

Bibliography

[1] Julia Angwin, Jeff Larson, Surya Mattu, and Lauren Kirchner. Machine bias: There's software used across the country to predict future criminals. and it's biased against blacks. *ProPublica*, 23:2016, 2016.

[2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[3] Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences (PNAS)*, 116(32):15849–15854, 2019.

[4] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research (JMLR)*, 13:281–305, 2012.

[5] Christopher M. Bishop. Training with noise is equivalent to Tikhonov regularization. *Neural Computation*, 7(1):108–116, 1995.

[6] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024.

[7] Andrew Brock, Jeff Donahue, and Karen Simonyan. Large scale GAN training for high fidelity natural image synthesis. In *International Conference on Learning Representations (ICLR)*, 2019.

[8] Sébastien Bubeck and Mark Sellke. A universal law of robustness via isoperimetry. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 28811–28822, 2021.

[9] Cristian Buciluǎ, Rich Caruana, and Alexandru Niculescu-Mizil. Model compression. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 535–541, 2006.

[10] Alfredo Canziani, Adam Paszke, and Eugenio Culurciello. An analysis of deep neural network models for practical applications. *arXiv preprint arXiv:1605.07678*, 2016.

[11] Rich Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997.

[12] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.

[13] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (elus). *arXiv preprint arXiv:1511.07289*, 2015.

[14] Richard T. Cox. Probability, frequency and reasonable expectation. *American Journal of Physics*, 14(1):1–13, 1946.

[15] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.

[16] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 1, pages 886–893, 2005.

[17] Yann N. Dauphin, Razvan Pascanu, Caglar Gulcehre, Kyunghyun Cho, Surya Ganguli, and Yoshua Bengio. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 2933–2941, 2014.

[18] Bruno de Finetti. La prévision: ses lois logiques, ses sources subjectives. *Annales de l'Institut Henri Poincaré*, 7(1):1–68, 1937.

[19] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009.

[20] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 4171–4186, 2019.

[21] Terrance DeVries and Graham W. Taylor. Improved regularization of convolutional neural networks with Cutout. *arXiv preprint arXiv:1708.04552*, 2017.

[22] Jeff Donahue, Yangqing Jia, Oriol Vinyals, Judy Hoffman, Ning Zhang, Eric Tzeng, and Trevor Darrell. DeCAF: A deep convolutional activation feature for generic visual recognition. In *International Conference on Machine Learning (ICML)*, pages 647–655, 2014.

[23] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[24] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research (JMLR)*, 12:2121–2159, 2011.

[25] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *International Conference on Machine Learning (ICML)*, pages 1126–1135, 2017.

[26] Kunihiko Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36(4):193–202, 1980.

[27] Stephen I. Gallant. Perceptron-based learning algorithms. *IEEE Transactions on Neural Networks*, 1(2):179–191, 1990.

[28] Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand, and Victor Lempitsky. Domain-adversarial training of neural networks. *Journal of Machine Learning Research (JMLR)*, 17(59):1–35, 2016.

[29] Ross Girshick. Fast r-cnn. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1440–1448, 2015.

[30] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010.

[31] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. Deep sparse rectifier neural networks. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 315–323, 2011.

[32] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[33] Benjamin Graham. Fractional max-pooling. *arXiv preprint arXiv:1412.6071*, 2014.

[34] Kaiming He, Ross Girshick, and Piotr Dollár. Rethinking ImageNet pre-training. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4918–4927, 2019.

[35] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.

[36] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[37] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.

[38] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[39] Sepp Hochreiter and Jürgen Schmidhuber. Flat minima. *Neural Computation*, 9(1):1–42, 1997.

[40] Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.

[41] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

[42] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

[43] Gao Huang, Yixuan Li, Geoff Pleiss, Zhuang Liu, John E. Hopcroft, and Kilian Q. Weinberger. Snapshot ensembles: Train 1, get M for free. In *International Conference on Learning Representations (ICLR)*, 2017.

[44] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q. Weinberger. Deep networks with stochastic depth. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 646–661, 2016.

[45] David H. Hubel and Torsten N. Wiesel. Receptive fields of single neurones in the cat's striate cortex. *The Journal of Physiology*, 148(3):574–591, 1959.

[46] David H. Hubel and Torsten N. Wiesel. Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. *The Journal of Physiology*, 160(1):106–154, 1962.

[47] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.

[48] Edwin T. Jaynes. *Probability theory: The logic of science*. Cambridge University Press, 2003.

[49] Andrej Karpathy and Li Fei-Fei. Deep visual-semantic alignments for generating image descriptions. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3128–3137, 2015.

[50] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. In *International Conference on Learning Representations (ICLR)*, 2017.

[51] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[52] Jon Kleinberg, Sendhil Mullainathan, and Manish Raghavan. Inherent trade-offs in the fair determination of risk scores. In *8th Innovations in Theoretical Computer Science Conference (ITCS)*, 2017.

[53] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 25, pages 1097–1105, 2012.

[54] Solomon Kullback and Richard A. Leibler. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951.

[55] Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *International Conference on Learning Representations (ICLR)*, 2022.

[56] Yann LeCun, Bernhard Boser, John S. Denker, Donnie Henderson, Richard E. Howard, Wayne Hubbard, and Lawrence D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4):541–551, 1989.

[57] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[58] Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. Visualizing the loss landscape of neural nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, pages 6389–6399, 2018.

[59] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. *arXiv preprint arXiv:1312.4400*, 2013.

[60] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017.

[61] David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision (IJCV)*, 60(2):91–110, 2004.

[62] Andrew L. Maas, Awni Y. Hannun, and Andrew Y. Ng. Rectifier nonlinearities improve neural network acoustic models. In *Proc. ICML*, volume 30, page 3, 2013.

[63] Justin Matejka and George Fitzmaurice. Same stats, different graphs: Generating datasets with varied appearance and identical summary statistics through simulated annealing. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*, pages 1290–1294, 2017.

[64] Marvin Minsky and Seymour A. Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969.

[65] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, 1997.

[66] Guido F. Montúfar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. On the number of linear regions of deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 2924–2932, 2014.

[67] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010.

[68] Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep double descent: Where bigger models and more data hurt. In *International Conference on Learning Representations (ICLR)*, 2020.

[69] Allen Newell and Herbert A. Simon. Gps, a program that simulates human thought. *Lernende Automaten*, pages 109–124, 1961.

[70] Albert B. J. Novikoff. On convergence proofs on perceptrons. In *Proceedings of the Symposium on the Mathematical Theory of Automata*, volume 12, pages 615–622, 1962.

[71] Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.

[72] Boris T. Polyak and Anatoli B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992.

[73] Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023.

[74] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. *OpenAI Technical Report*, 2018.

[75] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017.

[76] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.

[77] J. Alan Robinson. A machine-oriented logic based on the resolution principle. *Journal of the ACM*, 12(1):23–41, 1965.

[78] Frank Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.

[79] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[80] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. Imagenet large scale visual recognition challenge. *International Journal of Computer Vision (IJCV)*, 115(3):211–252, 2015.

[81] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.

[82] Kate Saenko, Brian Kulis, Mario Fritz, and Trevor Darrell. Adapting visual category models to new domains. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 213–226, 2010.

[83] Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014.

[84] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[85] Jie Shao et al. Imagenet 2016 classification winner: Multi-scale ensemble models. *ILSVRC 2016 Workshop*, 2016.

[86] Ali Sharif Razavian, Hossein Azizpour, Josephine Sullivan, and Stefan Carlsson. CNN features off-the-shelf: An astounding baseline for recognition. In *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 806–813, 2014.

[87] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[88] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

[89] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alexander A Alemi. Inception-v4, inception-resnet and the impact of residual connections on learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017.

[90] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–9, 2015.

[91] Matus Telgarsky. Benefits of depth in neural networks. In *29th Annual Conference on Learning Theory (COLT)*, pages 1517–1539, 2016.

[92] Antonio Torralba and Alexei A. Efros. Unbiased look at dataset bias. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1521–1528, 2011.

[93] Alan M. Turing. Computing machinery and intelligence. *Mind*, 59(236):433–460, 1950.

[94] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization. *arXiv preprint arXiv:1607.08022*, 2016.

[95] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[96] Li Wan, Matthew Zeiler, Sixin Zhang, Yann LeCun, and Rob Fergus. Regularization of neural networks using DropConnect. In *International Conference on Machine Learning (ICML)*, pages 1058–1066, 2013.

[97] Paul Werbos. *Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*. PhD thesis, Harvard University, 1974.

[98] Yuxin Wu and Kaiming He. Group normalization. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 3–19, 2018.

[99] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 3320–3328, 2014.

[100] Sergey Zagoruyko and Nikos Komodakis. Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer. In *International Conference on Learning Representations (ICLR)*, 2017.

[101] Matthew D. Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 818–833, 2014.

[102] Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola. *Dive into Deep Learning*. Cambridge University Press, 2023.

[103] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations (ICLR)*, 2017.

[104] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.

[105] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 6848–6856, 2018.

[106] Luowei Zhou, Hamid Palangi, Lei Zhang, Hu Houdong, Jason Corso, and Jianfeng Gao. Unified vision-language pre-training for image captioning and VQA. In *AAAI Conference on Artificial Intelligence*, volume 34, pages 13041–13049, 2020.

[107] Barret Zoph and Quoc V Le. Neural architecture search with reinforcement learning. *arXiv preprint arXiv:1611.01578*, 2016.